Hilvana — Manual de uso

Visual API Orchestrator

Hilvana 6.3.1
30 de octubre de 2026

Índice

🖥️ 01 · La pantalla principal

- La toolbar, botón a botón
- La barra lateral de iconos 🆕
- Pestañas, canvas y minimapa
- Cajitas colapsadas, nº de orden y 📌
- Pantalla dividida
- Importar OpenAPI 🆕 (5.3)
- Pantalla a pantalla 🆕 (5.38.1)

🟢 02 · El nodo Request (la cajita curl)

- Ejecutar y leer la respuesta
- Variable extractions 🆕 4.32
- Estados y colores
- Pantalla completa
- Asserts 🆕 (5.3)

🔗 03 · Conexiones y variables

- Conectar nodos
- Variables: mover datos entre cajitas
- Entornos con nombre 🆕 (5.3)
- Credenciales cifradas 🆕 (5.6 · Pro)

📝 04 · Notas, Mermaid y Capturas

- Nota de texto (Add Note, ámbar)
- Diagrama Mermaid (Add Mermaid, violeta)
- Nodo Captura (Add Screenshot, rosa) 🆕

🗄️ 05 · El nodo SQL

- Configuración
- Perfiles de conexión (SQL Conns)
- Resultado y modos de vista
- Extraer variables

🌐 06 · El nodo Web y el modo Live

- El nodo y sus botones
- El modo Live: navegar y capturar
- El flujo de trabajo para documentar un proceso
- Detalles que conviene saber

🕷️ 07 · Documentar una web — flow-explore

- Modo crawl (automático)
- Modo manual (tú navegas)
- Flags útiles

Seguridad

🧰 **08 · Paneles y utilidades**

Configuración con menú lateral 🆕 (5.22)
Imagen de fondo del flow 🆕 (4.49)
La configuración vive en cada flow 🆕 (4.48)
Vista — tamaño de los nodos, modo compacto y separación 🆕 (4.27)
Proyecto — flows/ como workspace 🆕 (4.25)
Enlaces entre flows en las notas 🆕 (4.25)
Pizarra 🆕 (4.24)
Guía de nodos 🆕 (4.24)
Ocultar conectores 🆕 (4.24)
Variables usadas en la caja (4.23)
📌 Fijar posición (4.23)
Historial
Consola
Correo de prueba — SMTP embebido 🆕 (4.36)
Batch Run
GitHub Flows
Export / Abrir / guardado
Collapse All · Expand All · Auto Layout · Alinear
Cron
Webhook entrante 🆕 (5.3)
Monitor programado 🆕 (5.4)
Colaboración en vivo 🆕 (5.5 · exclusiva Business)
Deshacer y rehacer del canvas 🆕 (5.5 · todos los planes)
Panel Proyecto compacto 🆕 (5.5)
Carpetas privadas por miembro 🆕 (5.6 · cloud)
AgentOffice 🆕

⌨️ **09 · El CLI flow-run**

Flags
Informes
Asserts, data-driven y CI 🆕 (5.3)
Todos los flags 🆕 (5.38.1)

🤖 **10 · El MCP embebido**

Conectarlo a Claude Code
Las 39 tools, por grupos
Seguridad
Conectar tu IA al espacio del cloud 🆕 (5.8)
Las tools, por grupos 🆕 (5.38.1)

🐳 **11 · Docker y despliegue**

Ejecutar

Desde el móvil
Construir
Peculiaridades dentro del contenedor
Chrome dentro de la imagen (desde la 4.5.0)
Licencia y prueba de 14 días 🆕 5.1
Actualizaciones 🆕 5.22
Versiones
Vincular esta instalación a tu cuenta cloud 🆕 (5.8)

📱 12 · La vista móvil

Llegar desde el teléfono
La lista
Documentación de un flow
El flow en PDF 🆕 (6.3.0)
Ejecutar y ver el resultado
Lo que no hace
AgentOffice en la app Android 🆕 (5.30.2)
App Android de administración 🆕 (5.32.0)
Admin del cloud en el móvil 🆕 (5.36.0)
Adjuntos, Equipo y botón atrás en AgentOffice móvil 🆕 (5.37.0)
La pestaña ✨ IA 🆕 (5.13.1)
Reportar un problema desde el móvil 🆕 (5.14)
La lista, pantalla a pantalla 🆕 (5.38.1)

13 · El asistente de IA 🆕 (5.13)

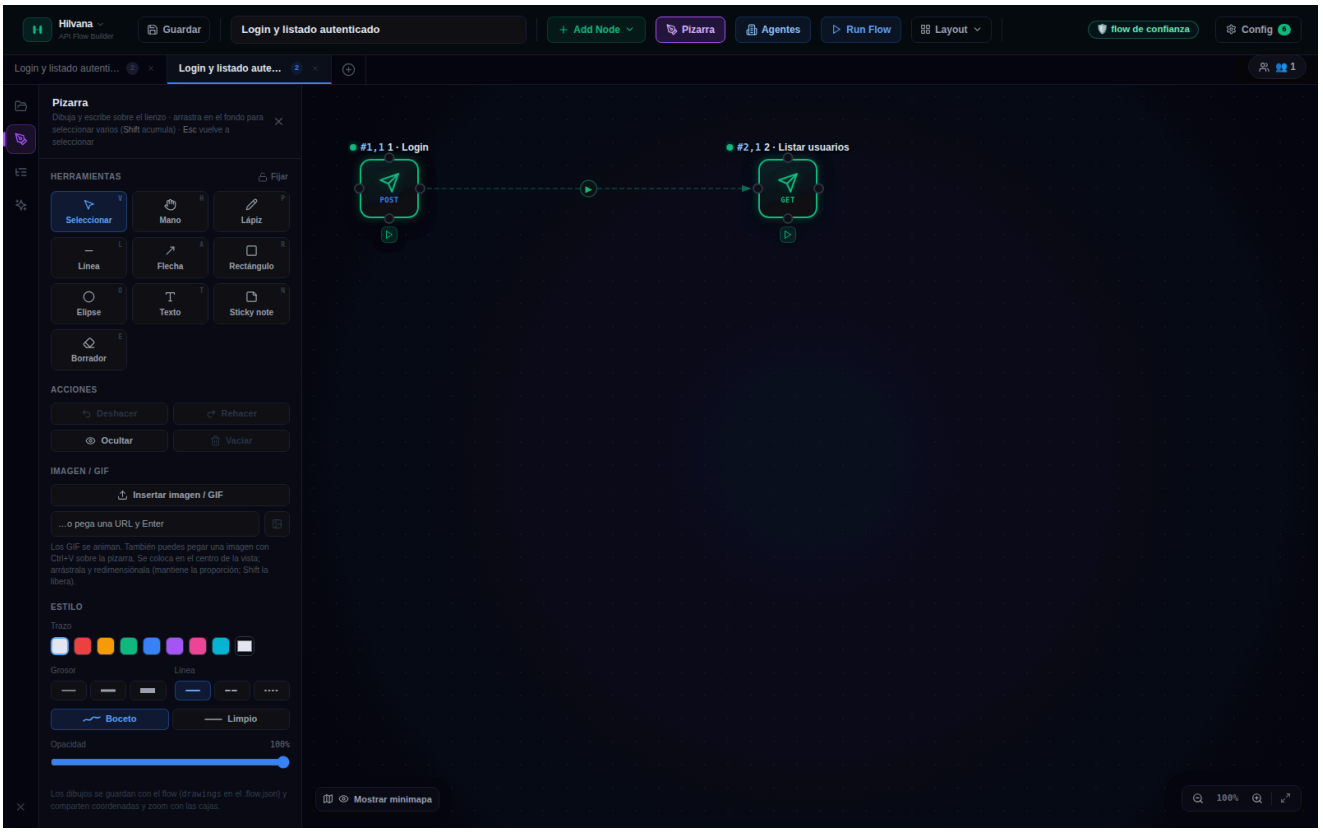
Qué sabe de serie: la skill de flows 🆕 (5.14)
Configurar: tu clave, tu proveedor
Estado de la conexión 🆕 (5.27)
Conexión desde el navegador
Cómo trabaja
Comandos rápidos
Adjuntos y contexto
Seguridad y coste
En el móvil 🆕 (5.13.1)
Límites y consejos

14 · Reportar un problema 🆕 (5.14)

Dónde está
Qué se envía
Cómo llega
Mis reportes y la respuesta
Para el administrador (hilvana-accounts)
Preguntas rápidas

01 · La pantalla principal

> Revisado para Hilvana **6.2.1** · capturas: 30.



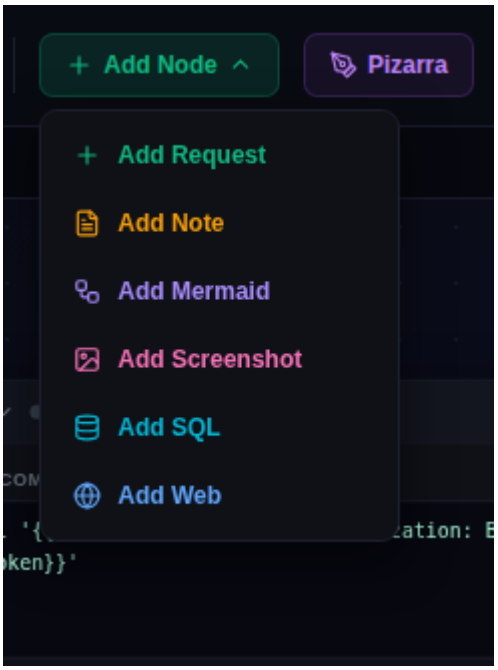
> **Consejo: Toolbar compacta (desde la 4.17) y barra lateral de iconos (4.24).** Los botones viven en desplegables (**Add Node**, **Layout**) y en el modal **Config**; los paneles laterales se eligen desde la barra de iconos de la izquierda, **uno visible a la vez**.

La toolbar, botón a botón

Botón	Qué hace
⚡ **Hilvana** ▾ (logo) 🆕	Menú del flow: **Nuevo flow**, **Abrir .flow.json…** (del disco local), **Guardar** (Ctrl+S), **Guardar como…** (Ctrl+Shift+S), **Export (descargar)**, **Exportar PDF** 🆕 4.44 y **Reset**. Un punto ámbar sobre el logo avisa de cambios sin guardar
💾 **Guardar** 🆕 5.21.1	Guarda el flow en su fichero del proyecto (lo mismo que Ctrl+S); se pone **verde con un punto ámbar** cuando hay cambios sin guardar. Si el flow aún no tiene fichero, abre «Guardar como…», cuyo botón Guardar queda siempre a la vista aunque el proyecto tenga muchas carpetas
(campo de texto)	**Nombre del flow** activo (campo ancho desde la 4.25.9). Si hay cambios sin guardar aparece el aviso ámbar «Sin guardar»

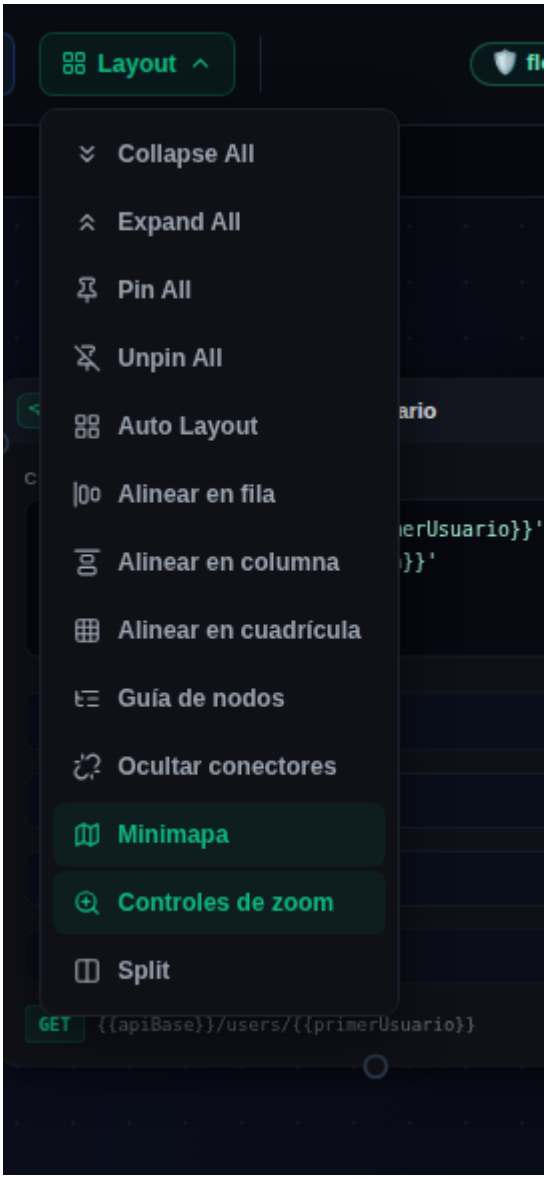
Botón	Qué hace
+ Add Node ▾ (verde)	Desplegable con los seis tipos de nodo (tabla siguiente)
Pizarra (morado) 🆕	Abre el panel de la **pizarra**: dibujar y escribir sobre el lienzo, estilo Excalidraw (08 Paneles y utilidades)
Run Flow	Ejecuta todo el flujo en orden topológico (las flechas mandan). Desde la 4.23 ejecuta antes los **scripts JS de las notas** y mete sus valores como variables
Layout ▾	Organización del canvas: colapsar, expandir, auto layout, alinear, **guía de nodos**, **ocultar conectores**, split
Config	Modal con **Batch Run**, Historial, Consola, **Correo** (SMTP de prueba, 4.36), Vista, SQL Conns, GitHub Flows, Variables y Global. El badge verde es el contador de la consola
🔒 **scripts en sandbox** / 🛡️ **flow de confianza** (chip) 🆕 5.15	Estado de los scripts de nota de la pestaña: en un sandbox sin página (flow abierto de fichero, cloud, import o galería) o en la página (flow de confianza). Clic para cambiarlo — 04

El desplegable Add Node



Opción	Qué añade
Add Request (verde)	Cajita de petición HTTP (curl). La pieza básica
Add Note (ámbar)	Nota de texto libre con scripts JS opcionales
Add Mermaid (violeta)	Nota que renderiza un diagrama Mermaid en vivo
Add Screenshot (rosa)	Nodo Captura: muestra una imagen, o hace un screenshot de una web pegando su URL
Add SQL (cian)	Consulta a Postgres / MySQL / Oracle
Add Web (azul)	Vista de una web: iframe clásico o **modo Live** navegable

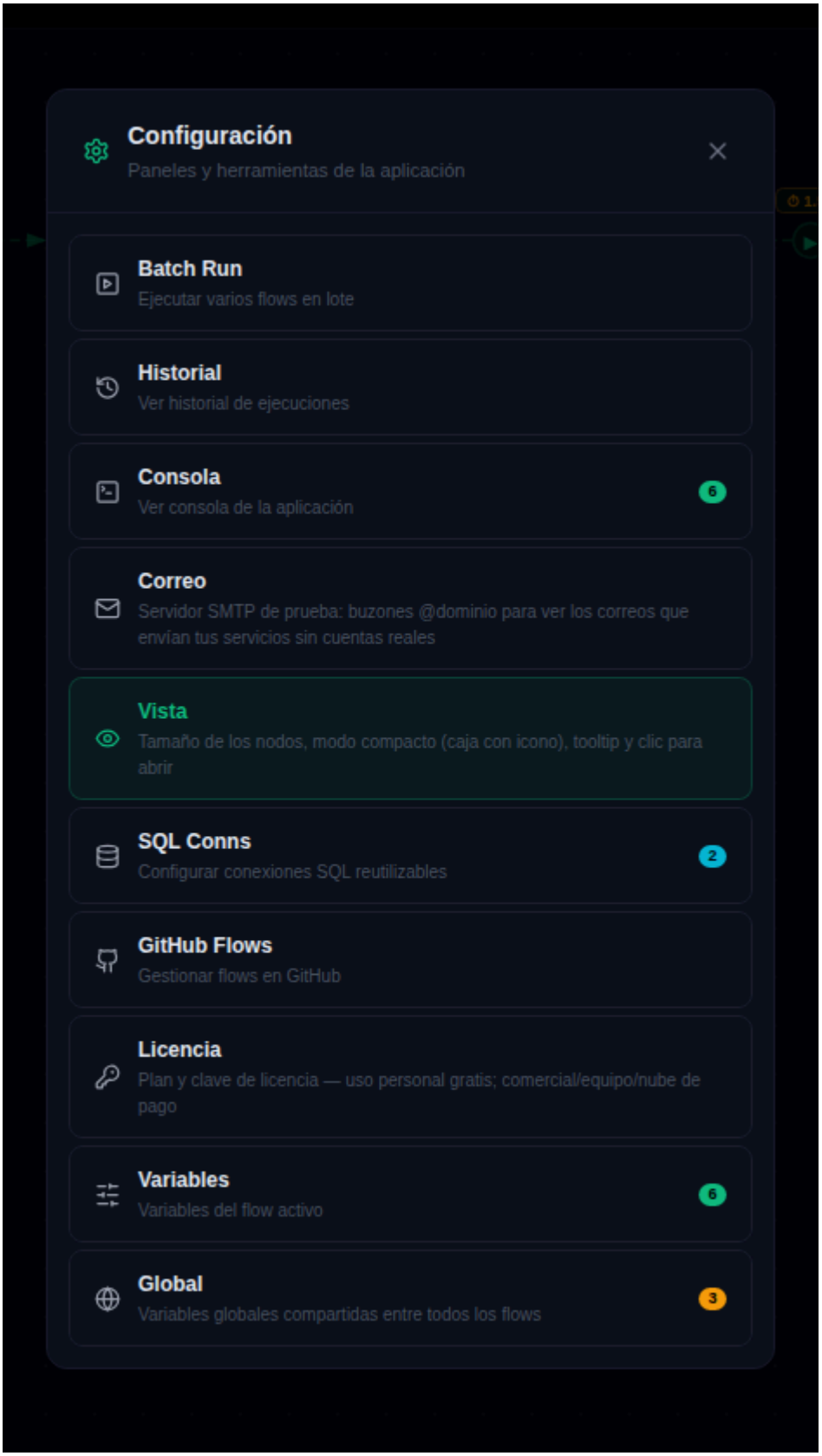
El desplegable Layout



Opción	Qué hace
Collapse All	Colapsa todas las cajitas **y las acerca** conservando tu disposición
Expand All	Expande todo y **restaura las posiciones** previas al Collapse All
Pin All / Unpin All 🆕 4.29	Fija (📌) o libera **todas** las cajas del flow visible de una vez: fijadas, ningún relayout ni el arrastre las mueve
Auto Layout	Reordena los nodos por el grafo: cadena en columnas y cajitas informativas colgando de su padre
Alinear en fila / en columna	Coloca los nodos en fila o columna según su **nº de orden** (el campo `#` de cada cabecera)
Alinear en cuadrícula 🆕 4.28	Coloca cada caja en la celda **`columna,fila`** escrita en su campo `#` (`1,1` arriba a la izquierda, `3,4` = columna 3, fila 4). Ver más abajo
Guía de nodos 🆕	Panel lateral con todos los nodos por tipo y nombre; clic en uno → el lienzo se centra en él y lo resalta

Opción	Qué hace
Ocultar / Mostrar conectores 🆕	Quita las líneas de conexión del lienzo (las conexiones siguen existiendo; útil para dibujar tus propias flechas en la pizarra)
Minimapa / **Controles de zoom** 🆕 4.25.8	Conmutadores: ocultan o muestran el botón del minimapa y la botonera ± del lienzo. Ocultar la botonera **no cambia el zoom** (Ctrl + rueda sigue funcionando). Se recuerdan entre sesiones
Split	Pantalla dividida: dos flows a la vez. Con el split activo aparece la opción para cambiar la dirección

El modal Config

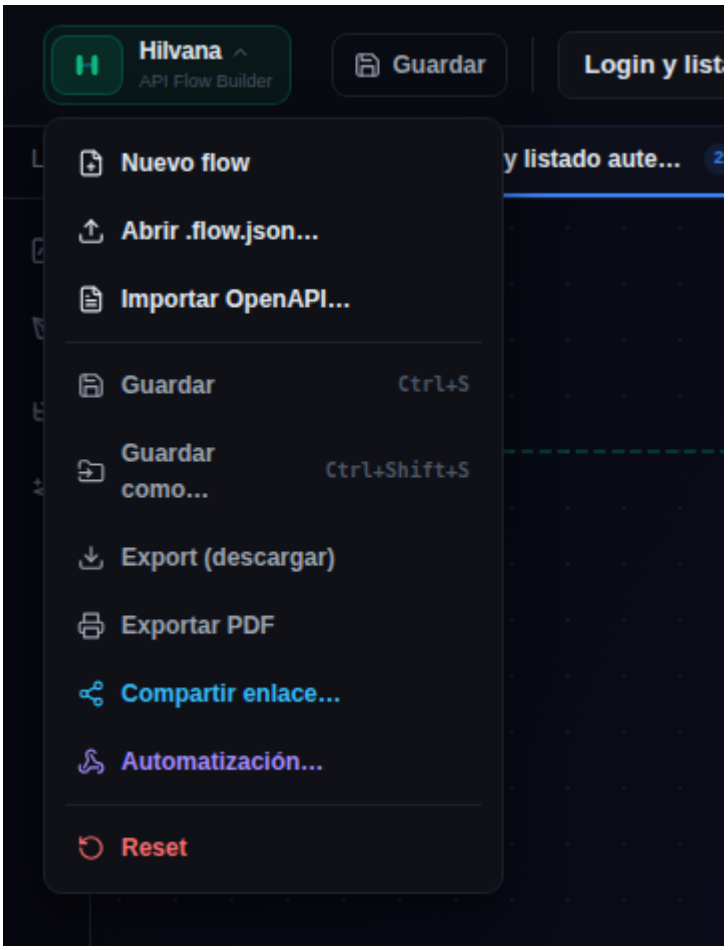


Batch Run (4.25.7), **Historial**, **Consola**, **Correo** 🆕 4.36 (servidor SMTP de prueba con buzones `@midominiotest.com`), **Vista** 🆕 4.27 (tamaño de los nodos, modo compacto y separación entre cajas), **SQL Conns**, **GitHub Flows**, **Variables** (del flow activo) y **Global** (compartidas entre flows), cada uno con su badge contador. Detalle en 08 Paneles y utilidades.

Exportar el canvas a PDF 🆕 (4.44)

> **4.51.2** — el PDF incluye la **imagen de fondo del flow** y las **imágenes/GIF de la pizarra** también cuando la app corre en Docker: el navegador incrusta las imágenes y el CSS en la petición, así el Chromium del servidor ya no necesita alcanzar tu origen (`localhost:9998` no existe dentro del contenedor — por eso el mapa salía en negro).

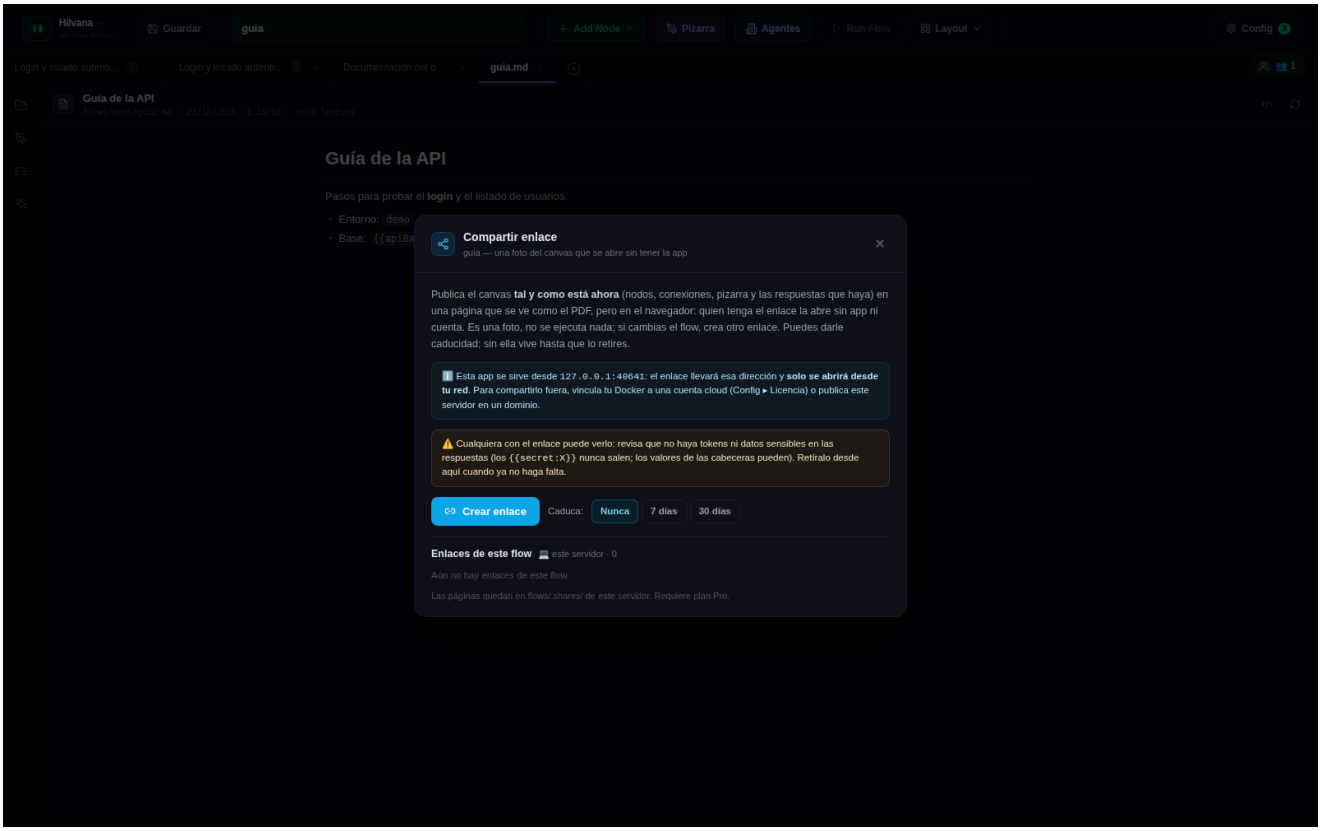
Logo → Exportar PDF descarga un PDF de **una sola página con el tamaño exacto del flujo**: los nodos con su contenido actual, las conexiones y la **pizarra**, tal y como están en ese momento — **completo aunque el flujo no quepa en la pantalla** (no es una captura del viewport: el contenido se recorta al rectángulo que de verdad ocupa, con un margen, y lo imprime el Chromium del servidor).



- El fichero se llama como la pestaña (`Mi flow.pdf`).
- Sale lo que se ve: cajas colapsadas salen colapsadas, respuestas desplegadas salen desplegadas, y los dibujos y textos de la pizarra van encima, como en el canvas.
- En pantalla dividida exporta el canvas izquierdo; los nodos Web en modo Live salen con su último fotograma.
- Límite práctico: ~19000 px por lado (el máximo de una página PDF). Errores y progreso se anotan en la Consola.

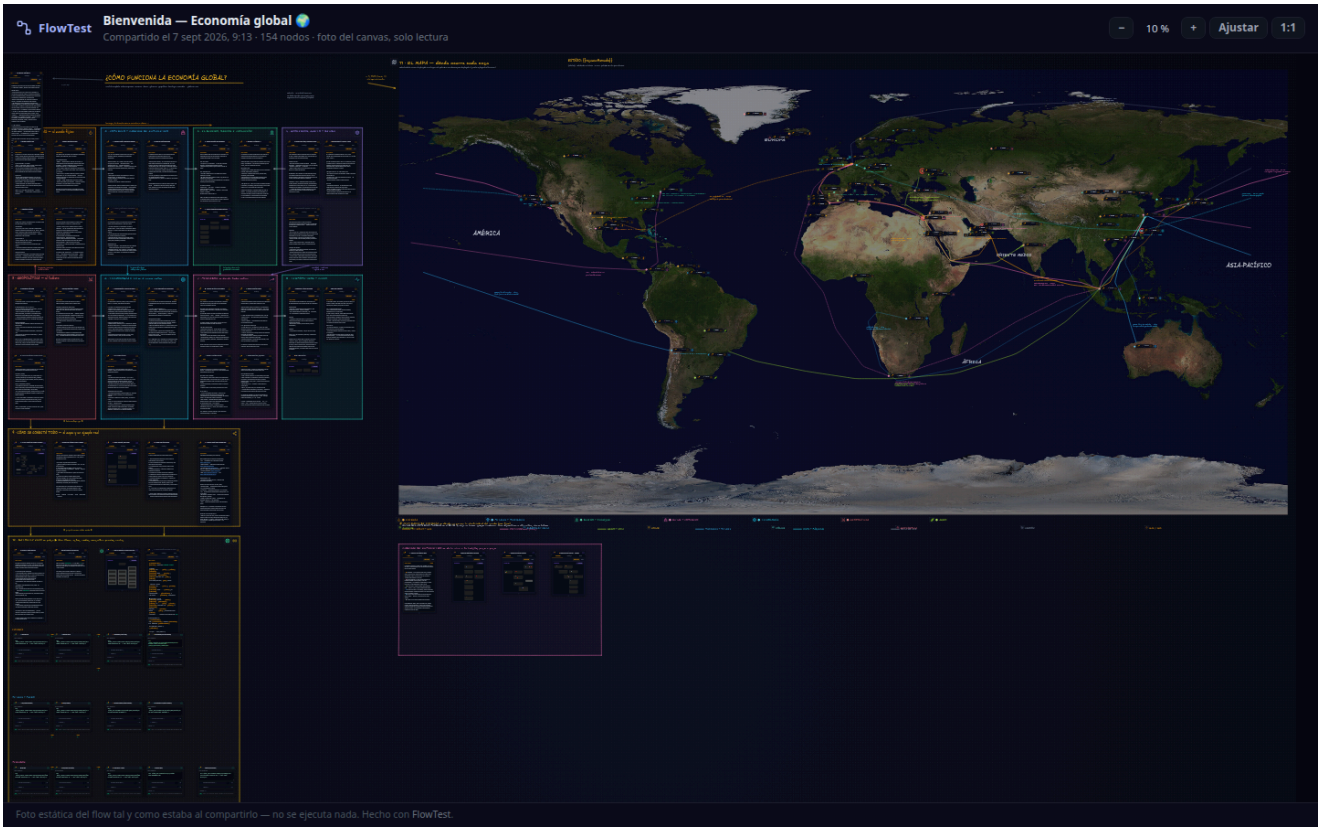
Compartir enlace público 🆕 (5.12)

Logo → Compartir enlace… publica una **foto del canvas** — nodos con su contenido actual, conexiones, pizarra y las respuestas que haya — en una **URL que abre cualquiera con el enlace, sin tener la app ni cuenta**. Se ve como el PDF, pero en el navegador: barra con el nombre, la fecha y el número de nodos, zoom (ajustar al ancho, 1:1, ±, Ctrl+rueda), solo lectura (se pueden seleccionar y copiar textos, nada se ejecuta). Es una foto: si cambias el flow, crea otro enlace.

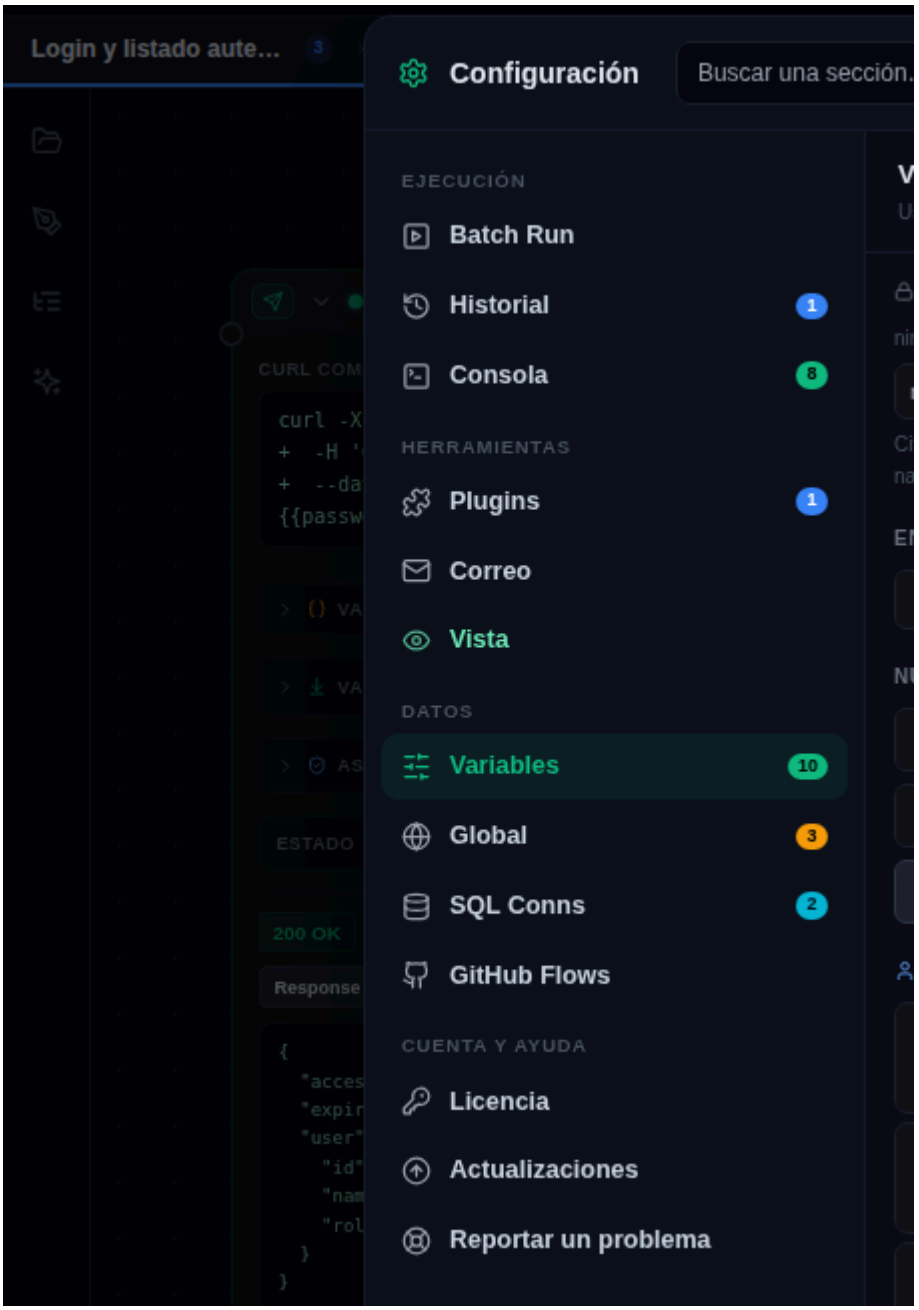


- **Dónde se publica.** En una instalación normal, en este mismo servidor: la URL lleva su dirección, y si es `localhost` o una IP privada el modal avisa de que **solo se abrirá desde tu red**. Con el Docker **vinculado a una cuenta cloud** (11) el modal arranca en ☁️ **Cloud**: la página se guarda en el espacio de tu org y el enlace es `https://<org>.app.hilvana.cloud/share/<id>`, **público en internet** aunque tu app pida sesión. «💻 Este servidor» sigue disponible.
- **Caducidad.** Nunca, 7 o 30 días. Al pasar la fecha el servidor retira el enlace solo (página «ha caducado»). Sin caducidad vive hasta que lo retires.
- **Gestión.** El modal lista los enlaces de este flow (o todos) con fecha, peso, nodos, **visitas** y caducidad: copiar, abrir, **retirar** (404 al instante). En el cloud también se ven y retiran desde **Mi cuenta → Mis flows → 🔗 Mis enlaces compartidos**, y el administrador desde su panel.
- **Seguridad.** Quien tenga la URL lo ve: el id es aleatorio (144 bits), la página lleva `noindex` y no expone la app ni el servidor. Revisa antes que no haya tokens ni datos sensibles en las respuestas (los `{{secret:X}}` nunca salen; los valores de las cabeceras sí pueden). Cada página cuenta en la cuota de disco del plan.
- **Plan.** Crear un enlace es **Pro** (en prueba, chip PRO y modal de compra); en el cloud además la org necesita suscripción activa, y si la pierde el servidor deja de servir sus enlaces. Ver y retirar no están limitados.

- Si al pegar la URL en un chat se le queda un punto, un paréntesis o una barra detrás, el servidor la limpia y abre igual.

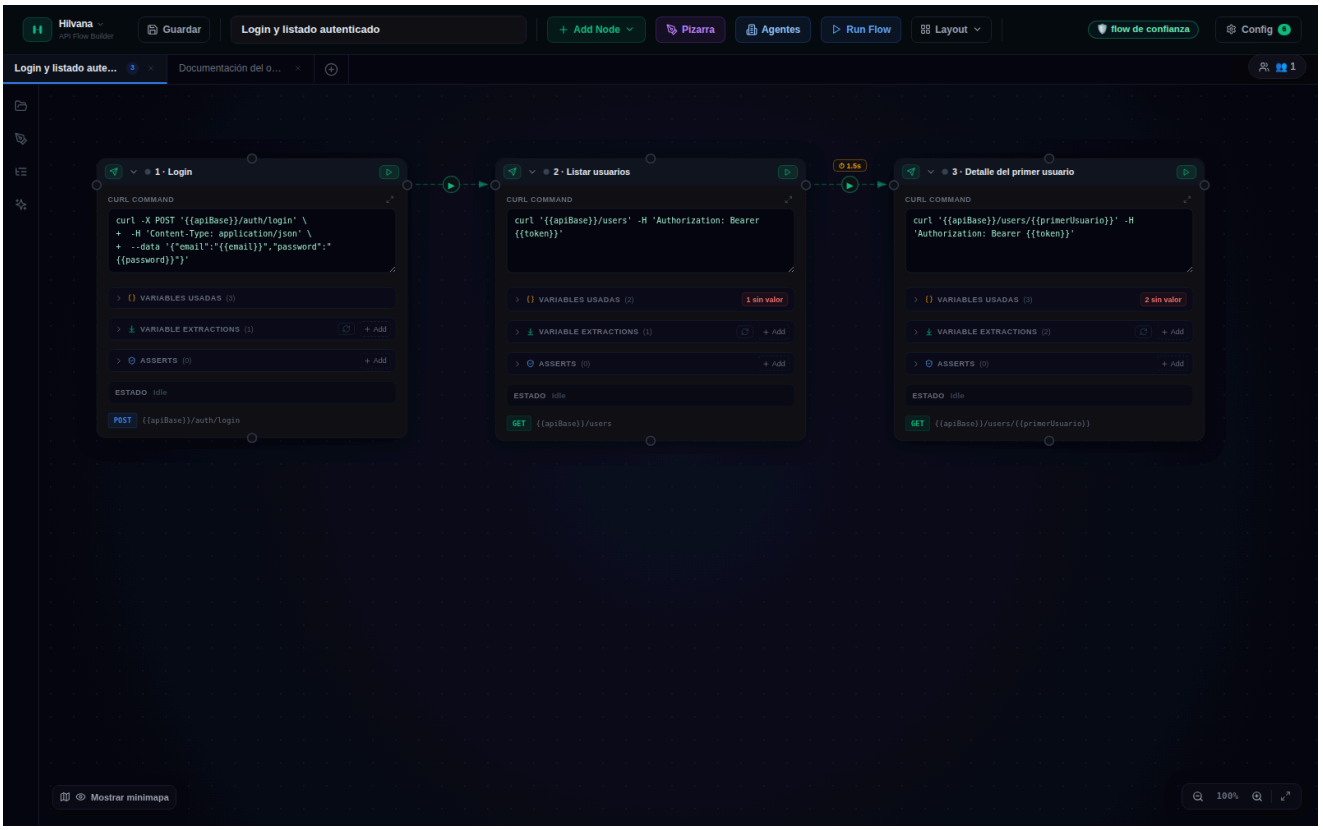


La barra lateral de iconos 🆕



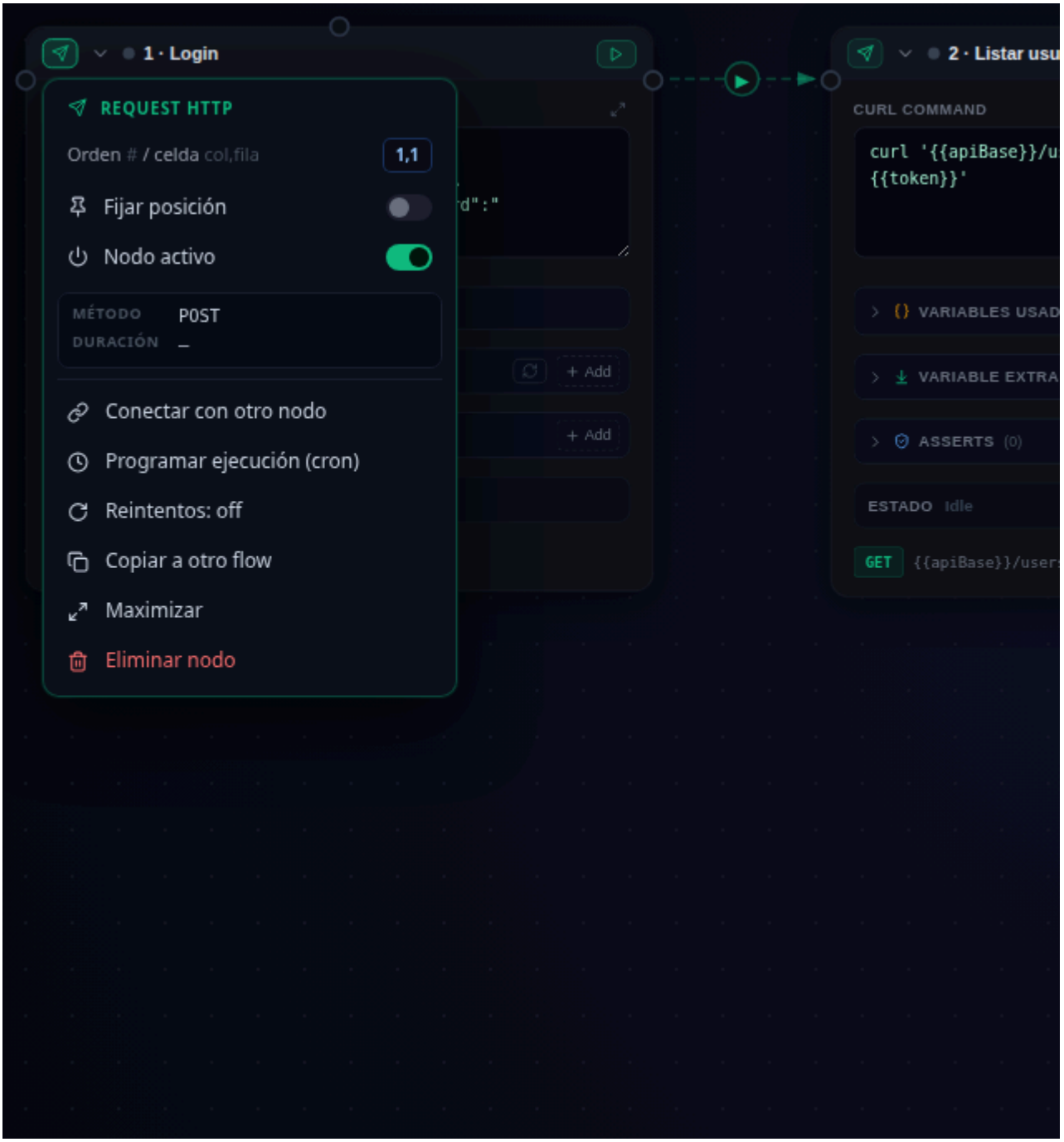
A la izquierda del lienzo hay siempre una barra con tres iconos: **Proyecto** (los flows de `flows/`, 4.25), **Pizarra** y **Guía de nodos**. Los paneles de **Config** (Variables, Globales, Conexiones SQL, GitHub) aparecen en la barra solo mientras están abiertos y desaparecen al cerrarlos. Solo se ve **un panel a la vez**: clic en un icono lo abre (y cierra el anterior), clic en el icono activo lo cierra, y la ✕ de abajo cierra el panel abierto. El panel elegido se recuerda entre recargas.

Pestañas, canvas y minimapa

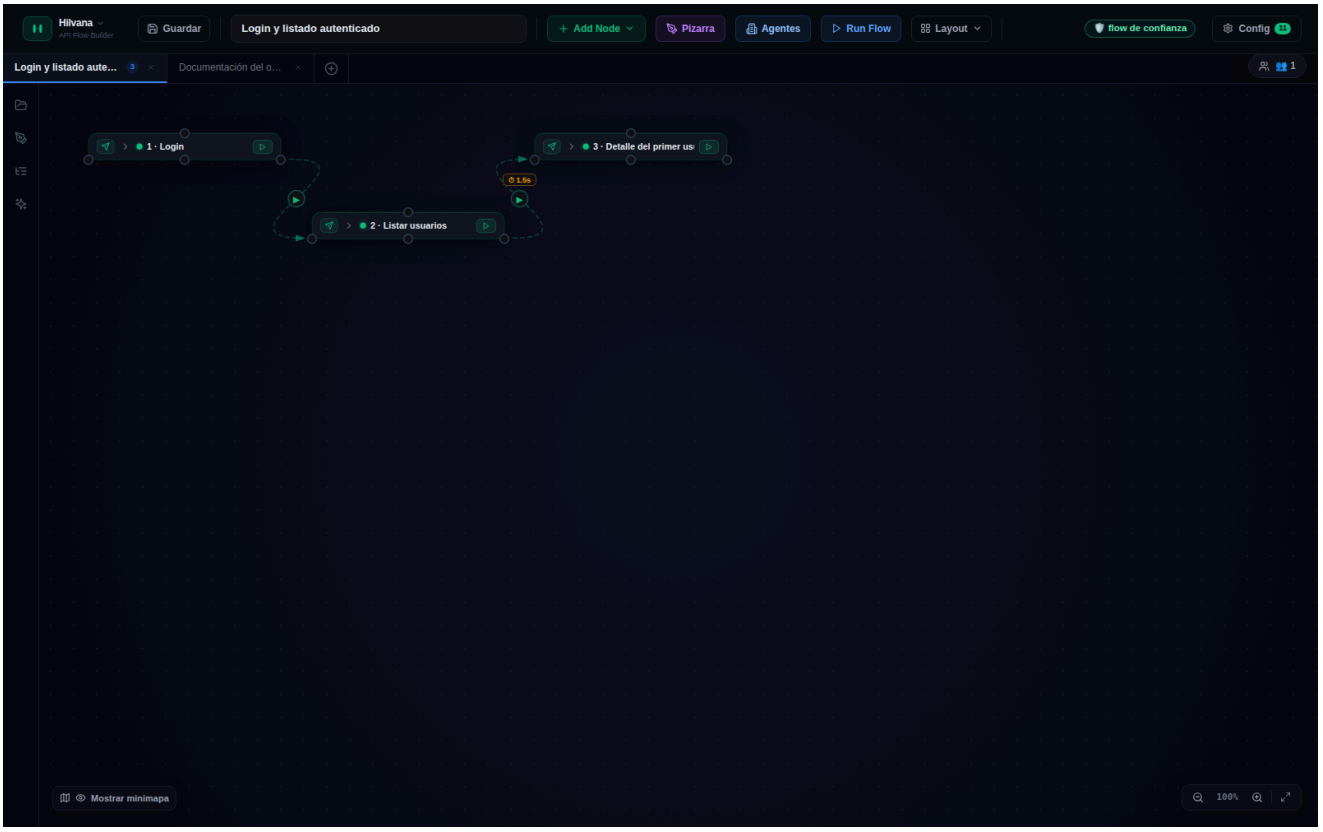


- **Pestañas**: cada pestaña es un flow independiente. El botón ⊕ de la barra crea una vacía; para importar un `.flow.json` del disco local usa ⚡ **Hilvana ▸ Abrir .flow.json…** (o abre los del proyecto desde el panel Proyecto). La ✕ cierra (avisa si hay cambios). Todo se **autoguarda en el navegador** (localStorage); además, desde la 4.25 cada pestaña abierta desde el proyecto **sabe cuál es su fichero** de `flows/` (la etiqueta muestra el nombre del fichero y un ● ámbar si hay cambios) y **Ctrl+S** lo guarda ahí. Export es para descargarlo fuera del proyecto.
- **Minimapa**: vista de pájaro del canvas (cajas **y dibujos de la pizarra**); clica para saltar a una zona. Arranca oculto — botón «Mostrar minimapa» abajo a la izquierda.
- **Zoom**: botones ± o **Ctrl + rueda**. Los dibujos de la pizarra escalan con las cajas.
- **Moverse**: botón central del ratón, o la herramienta **Mano** de la pizarra. Arrastra una cajita por su cabecera para recolocarla.
- **Escape** cancela una conexión a medias (y en la pizarra vuelve a Seleccionar).

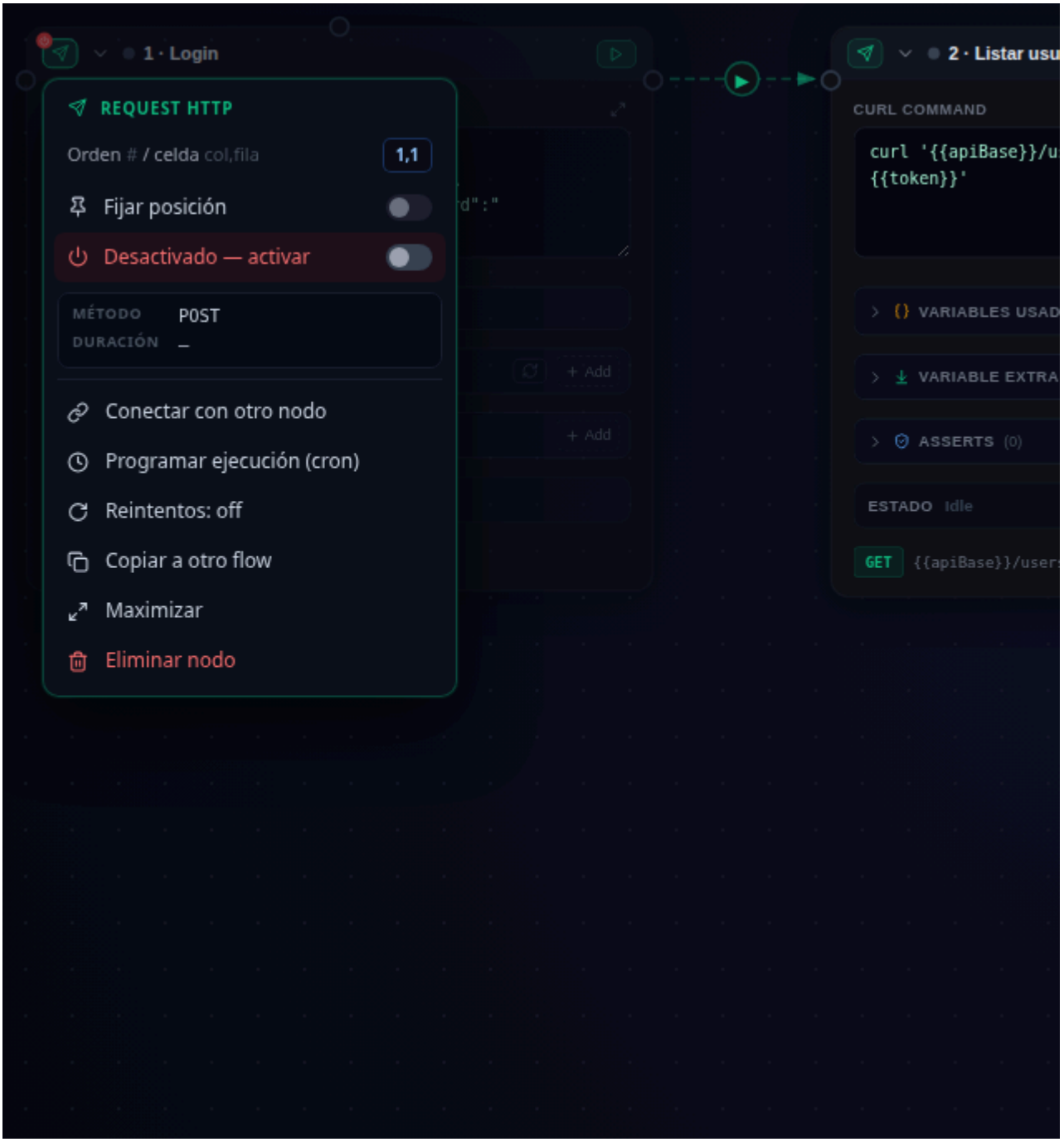
La cabecera de las cajas 🆕 4.30



Desde la 4.30 **todas las cajas** (request, SQL, nota, web) tienen la misma cabecera mínima: **icono del tipo · plegar · ● estado · título · ▶**. El **icono del tipo es un menú**: ahí están **Fijar posición** 📌, el campo `#` (nº de orden o celda `columna,fila`), **Conectar con otro nodo**, **Cron** (request) o **Configurar conexión** (SQL), **Copiar a otro flow**, **Maximizar** y **Eliminar**, más una tarjeta de información (método y duración de la última respuesta, conexión y filas del resultado SQL, URL y estado del nodo web, scripts de la nota). El ▶ ejecuta la request o la query, lanza los scripts de la nota o recarga la web. Las cajitas **colapsadas** son una sola fila con los mismos cinco elementos.



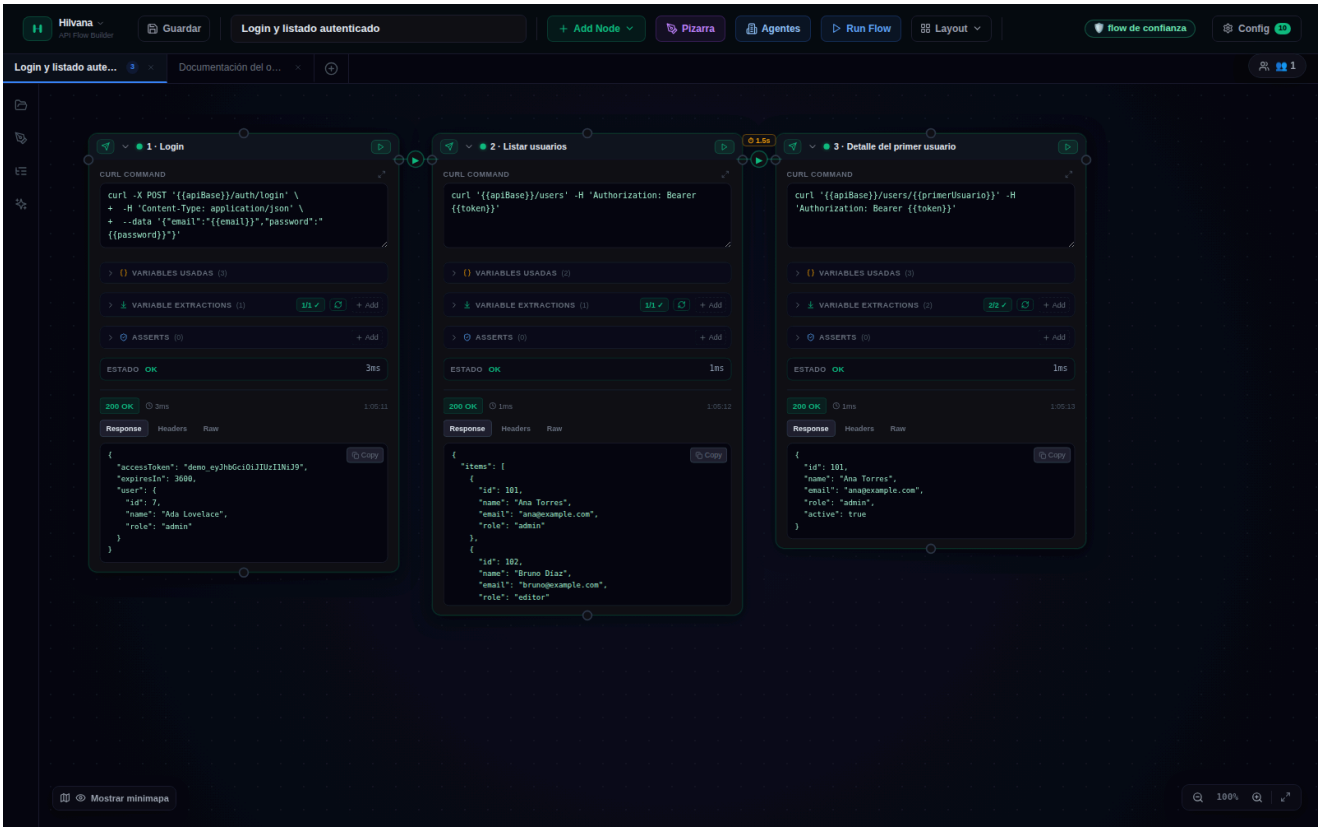
Activar / desactivar 🆕 4.31: el menú tiene también el conmutador **Nodo activo / Desactivado**. Un nodo desactivado se **salta al ejecutar** — Run Flow, las cadenas ▶, el cron, `flow_run` por MCP y el CLI (`⊘ … desactivado (saltado)`) — pero **sus conexiones se siguen recorriendo**, así que el resto del flow continúa. Se ve atenuado, con un ⏻ rojo sobre el icono y la etiqueta «OFF» en la guía de nodos; el ▶ de la propia caja sí lo ejecuta (para probarlo suelto). Se guarda en el `.flow.json` como `disabled`.



Cajitas colapsadas, nº de orden y 📌

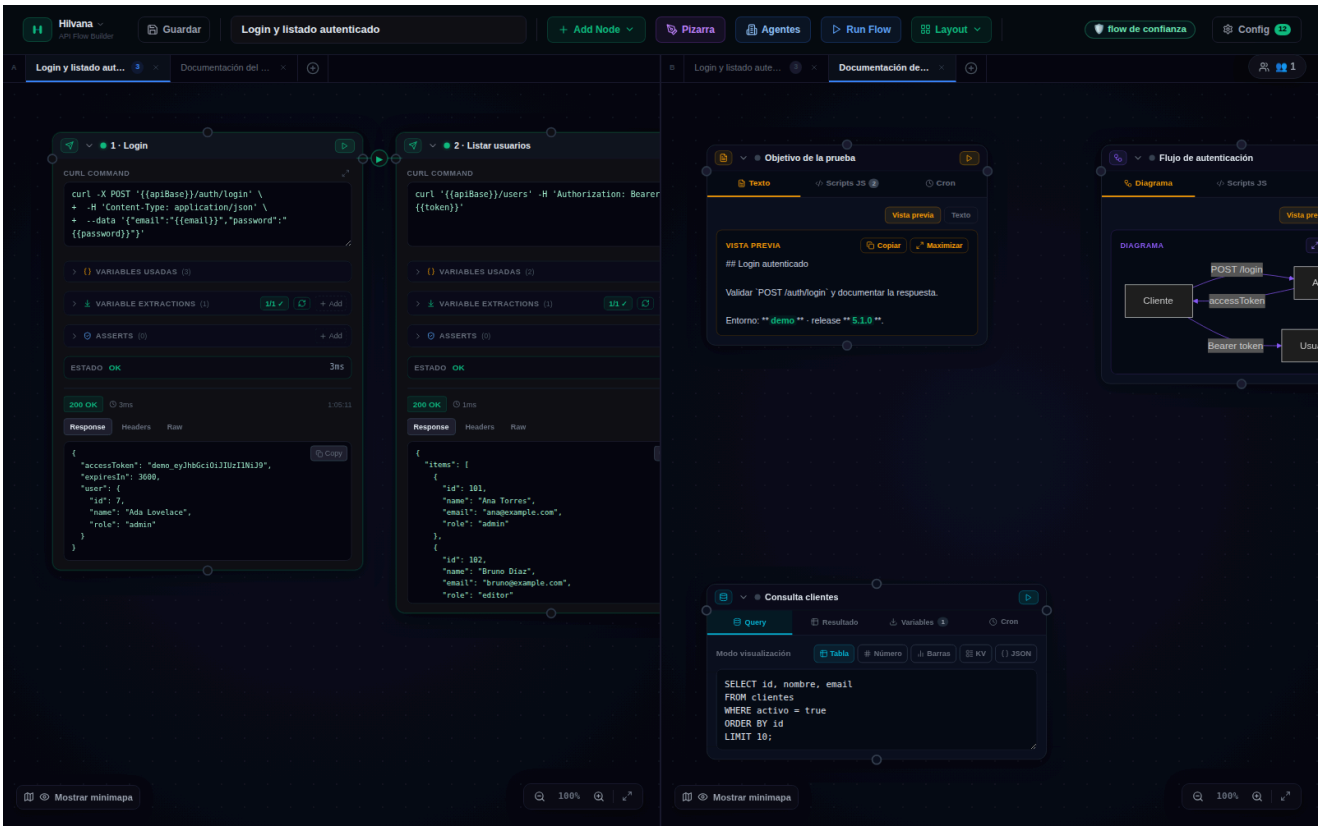
- Una cajita colapsada es una **caja compacta**: se estrecha al ancho de su contenido (190–260 px según el tipo) y la fila de botones pasa **debajo del título**.
- Cada nodo tiene un campo `#` (desde la 4.30, dentro del menú del icono de la cabecera): su **nº de orden**, que usan *Alinear en fila/columna* y la *Guía de nodos*; se guarda en el `.flow.json` (`order`). Desde la 4.28 admite también una celda `columna,fila` (`cell`), ver abajo.
- El botón 📌 **Fijar** (4.23) bloquea la posición de la caja: ni colapsar/expandir, ni Collapse/Expand All, ni alinear, ni el auto-layout, ni el arrastre la mueven. Se guarda como `pinned`.

Alinear en cuadrícula (campo `#` = `columna,fila`) 🆕 4.28



En el campo `#` de cada cabecera puedes escribir, en vez de un número, una **celda** `columna,fila`: `1,1` es arriba a la izquierda, `3,4` la columna 3, fila 4 (se pinta en azul). **Layout ▸ Alinear en cuadrícula** coloca cada caja en su celda: cada columna se hace tan ancha como su caja más ancha y cada fila tan alta como la más alta (40 px de hueco; una columna o fila sin cajas deja 80 px). Las cajas sin celda y las 📌 fijadas no se mueven. **Auto Layout** también las respeta (4.28.1): coloca primero la cuadrícula y ordena por el grafo solo las cajas sin celda, debajo. La celda se guarda en el `.flow.json` (`cell: {col, row}`), se ve en la guía de nodos y en la caja compacta, y las acciones *Alinear en fila/columna* la respetan (orden de lectura detrás de los `#` numéricos). Por MCP: `cell` en `node_add_*` / `node_update` y `canvas_layout` con `mode: "grid"`.

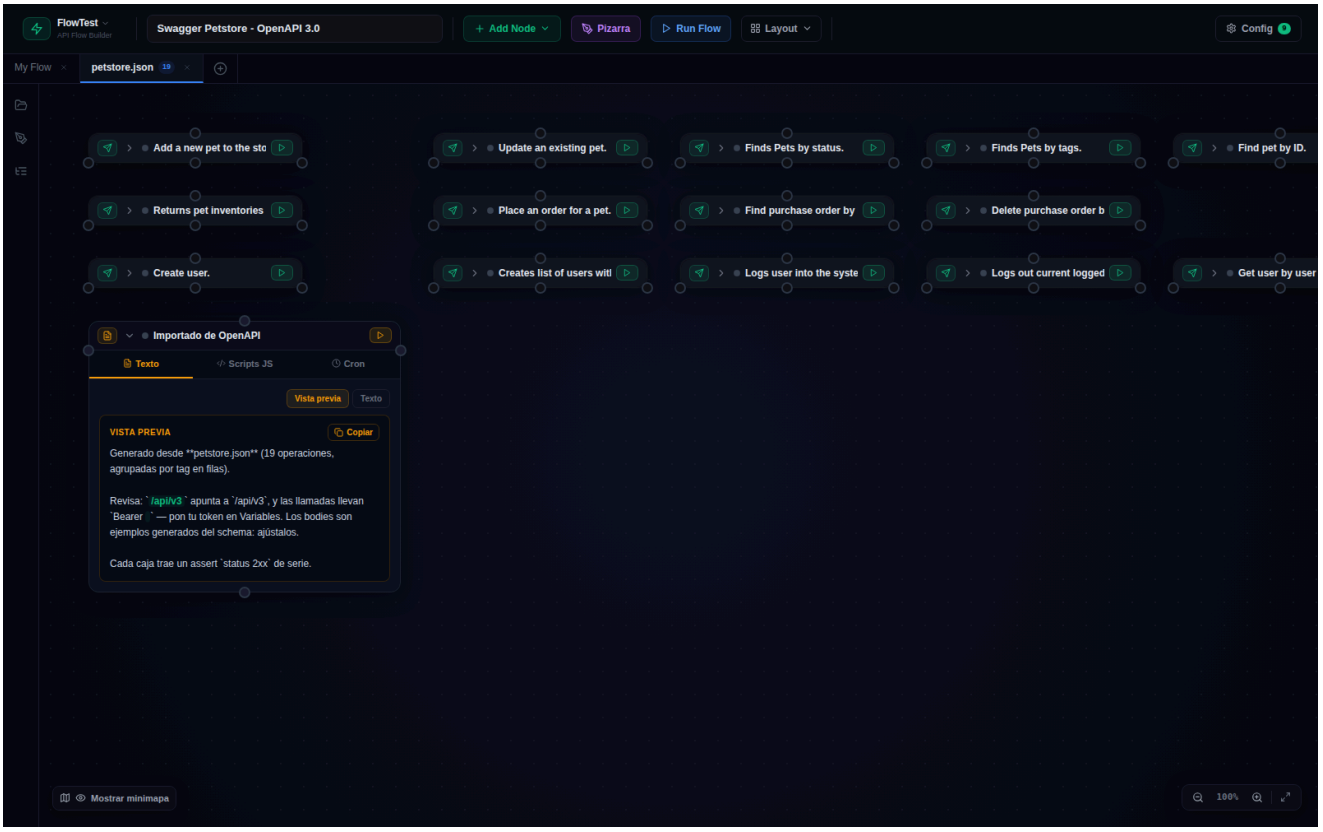
Pantalla dividida



Con **Layout ▸ Split** ves dos flows lado a lado (o arriba/abajo cambiando la dirección). Útil para comparar, o para copiar cajitas de un flow a otro con el botón **Copiar** de cada nodo. En split no se muestran los paneles laterales.

Importar OpenAPI 🆕 (5.3)

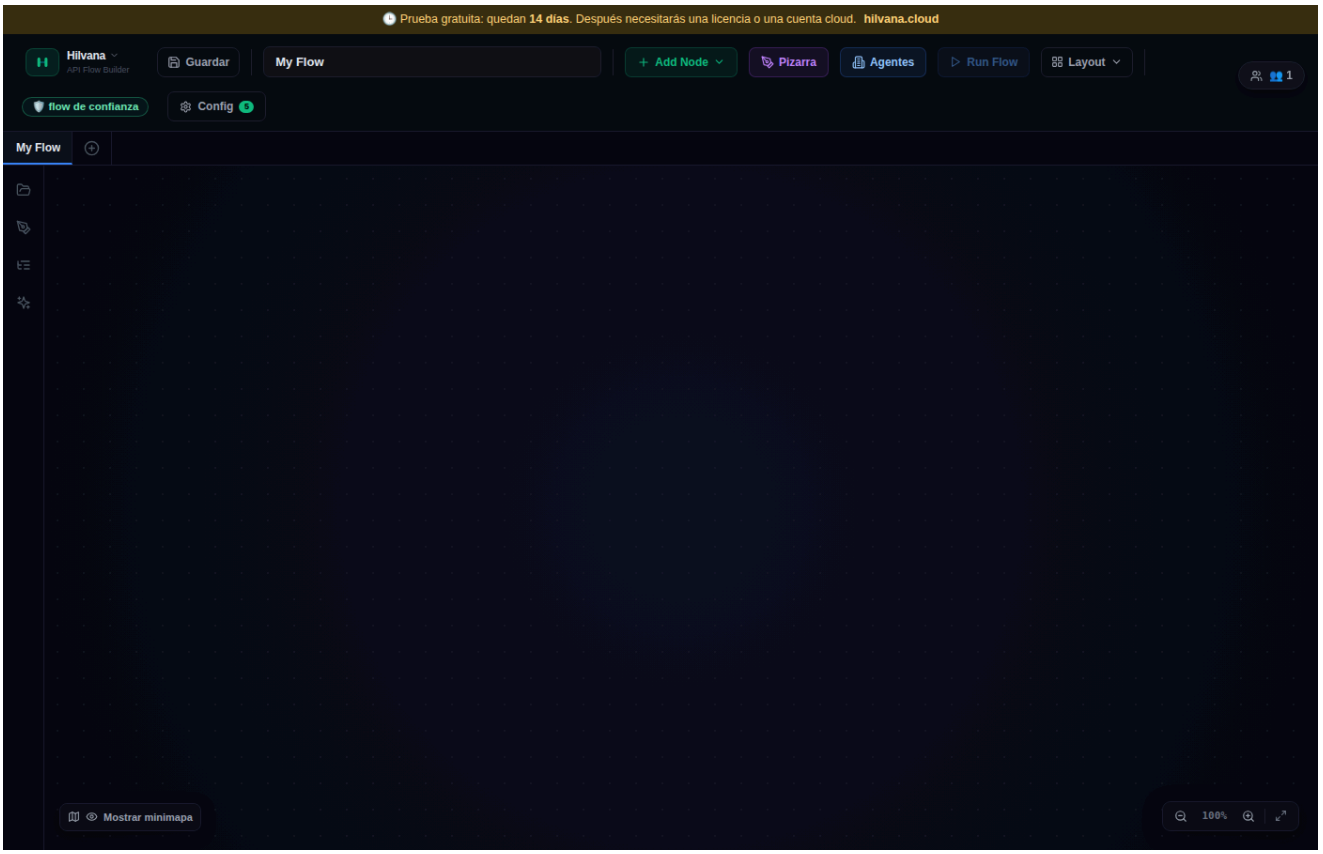
Menú del logo → **«Importar OpenAPI…»**: elige una spec **OpenAPI/Swagger** (`.json` o `.yaml`) y se convierte en un flow con **una caja Request por operación**, agrupadas por *tag* en filas de la cuadrícula.



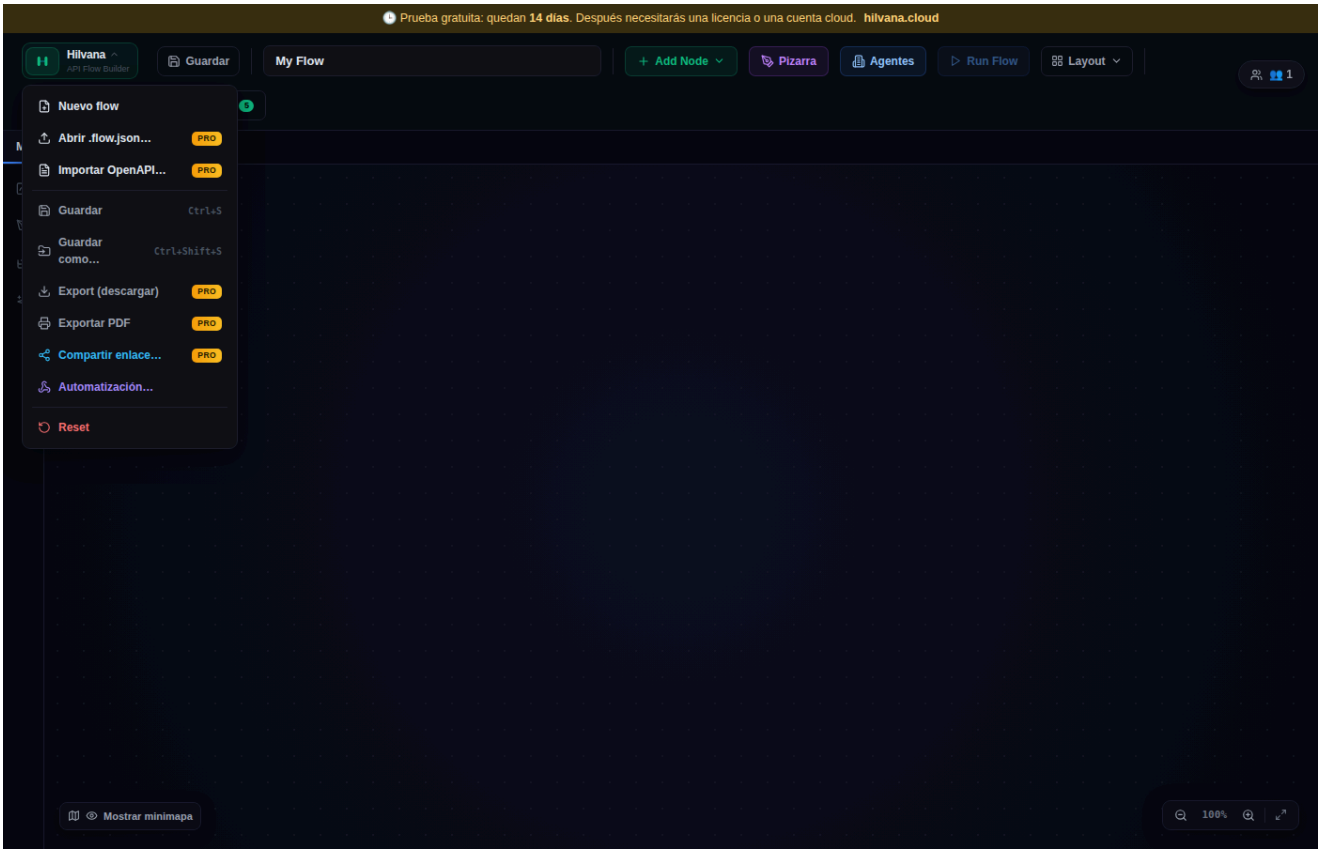
Cada caja llega lista para editar: curl contra `{{apiBase}}` (tomado de `servers`), los parámetros de ruta como `{{variables}}`, un body de ejemplo generado del schema, cabecera `Bearer {{authToken}}` si la spec declara auth, y un **assert `status 2xx`** de serie. Una nota al pie resume qué revisar. Es un *arranque* honesto para specs grandes — ajusta bodies y variables y a correr.

Pantalla a pantalla 🆕 (5.38.1)

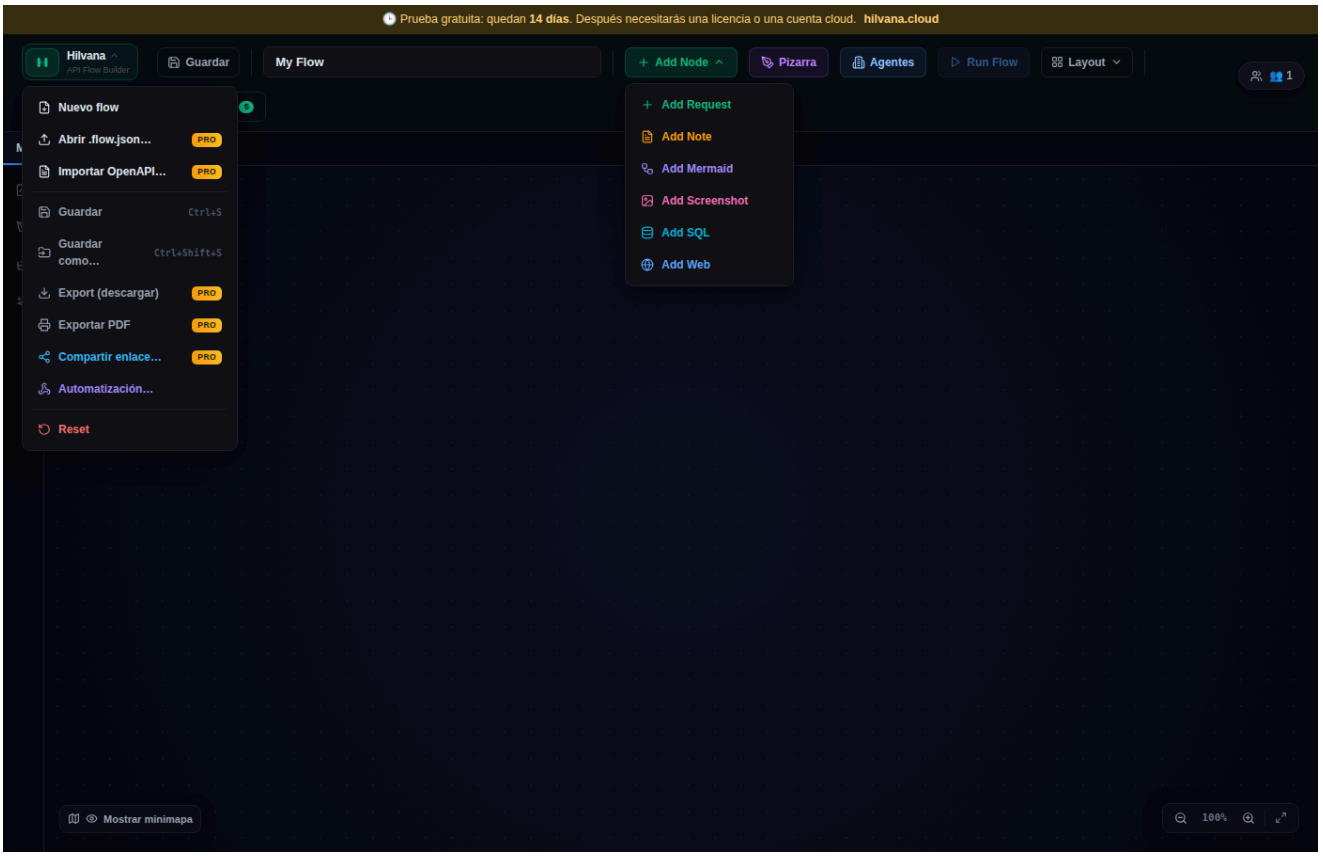
Barra superior y canvas



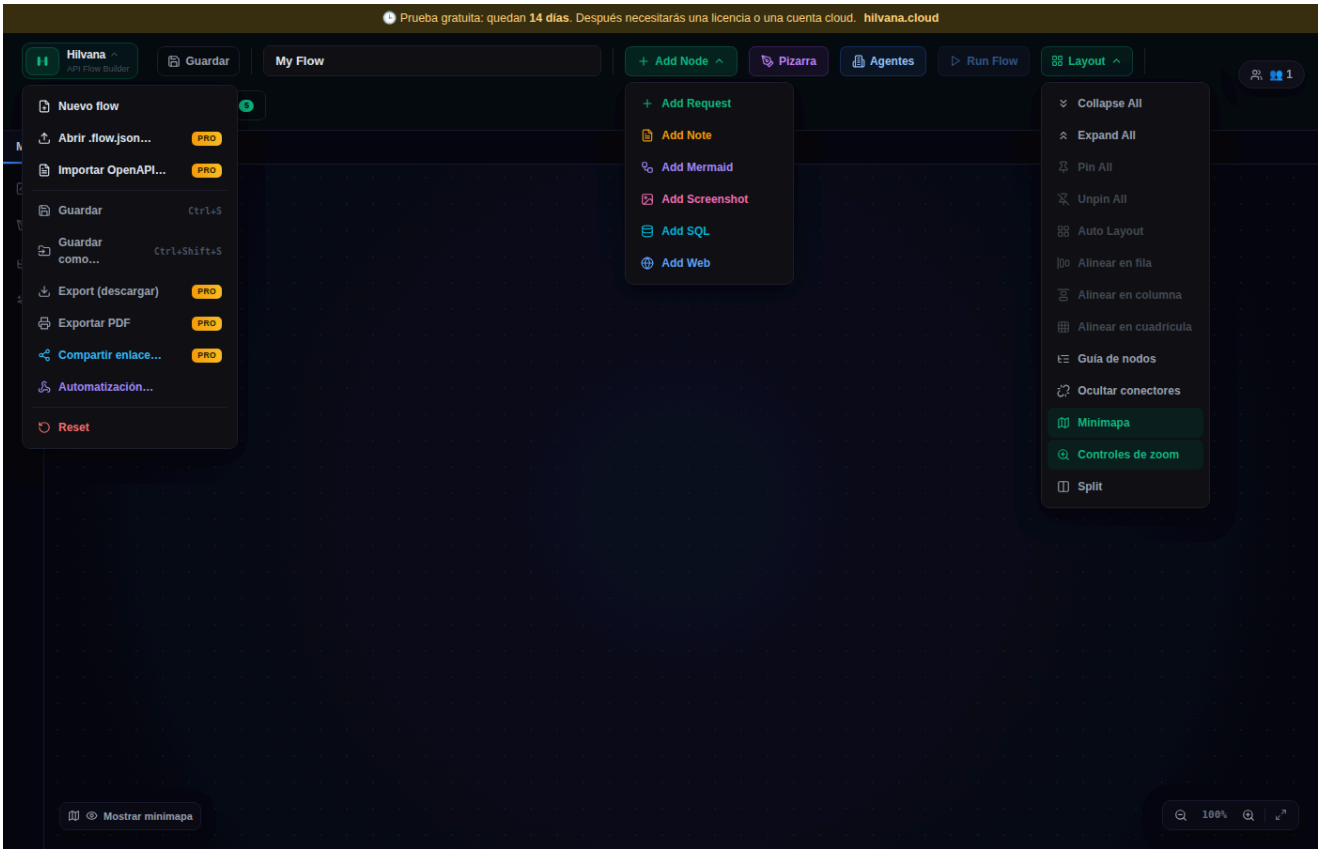
Menú del flow (botón de la esquina: nuevo, abrir, guardar Ctrl+S, exportar, reset)



Añadir un nodo (Request, SQL, Web, Info)

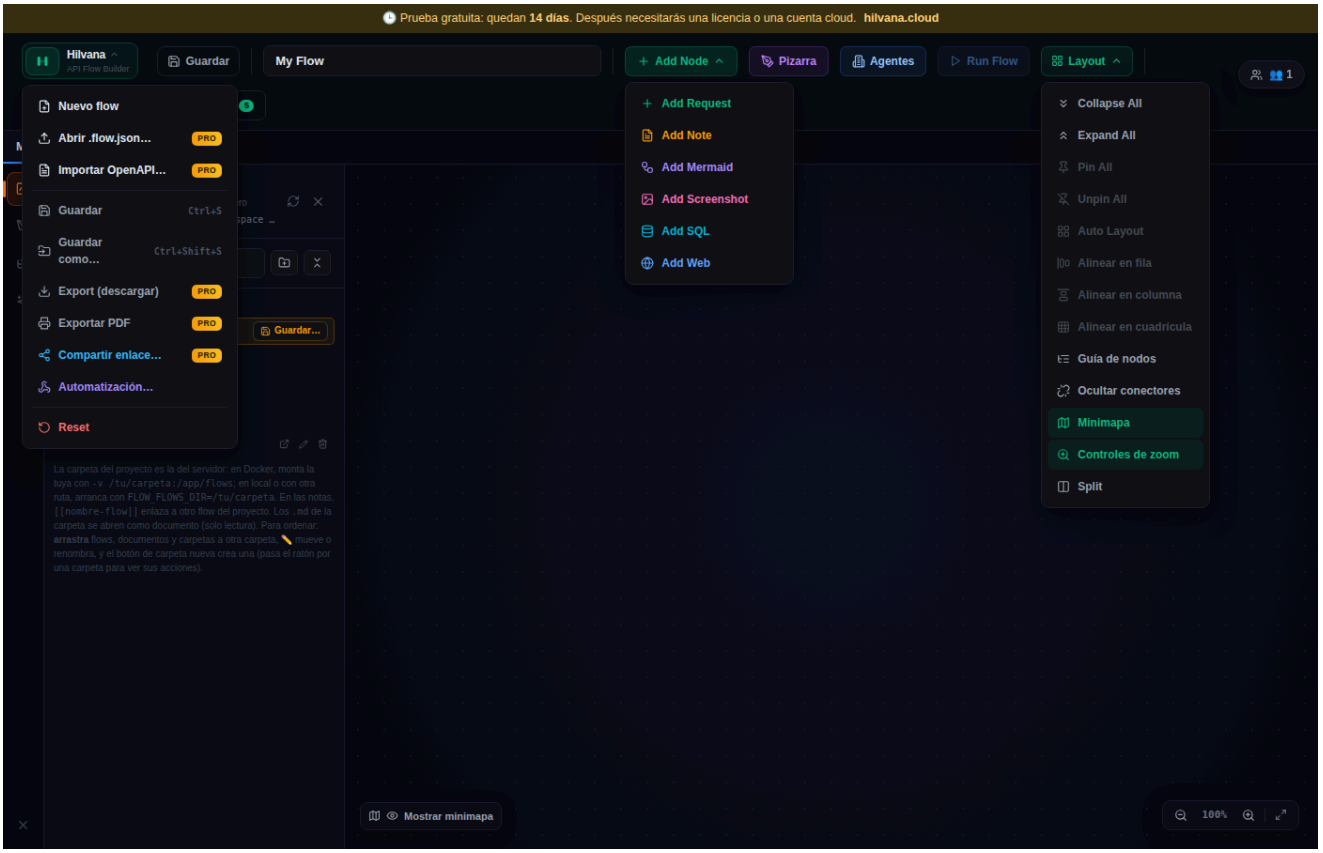


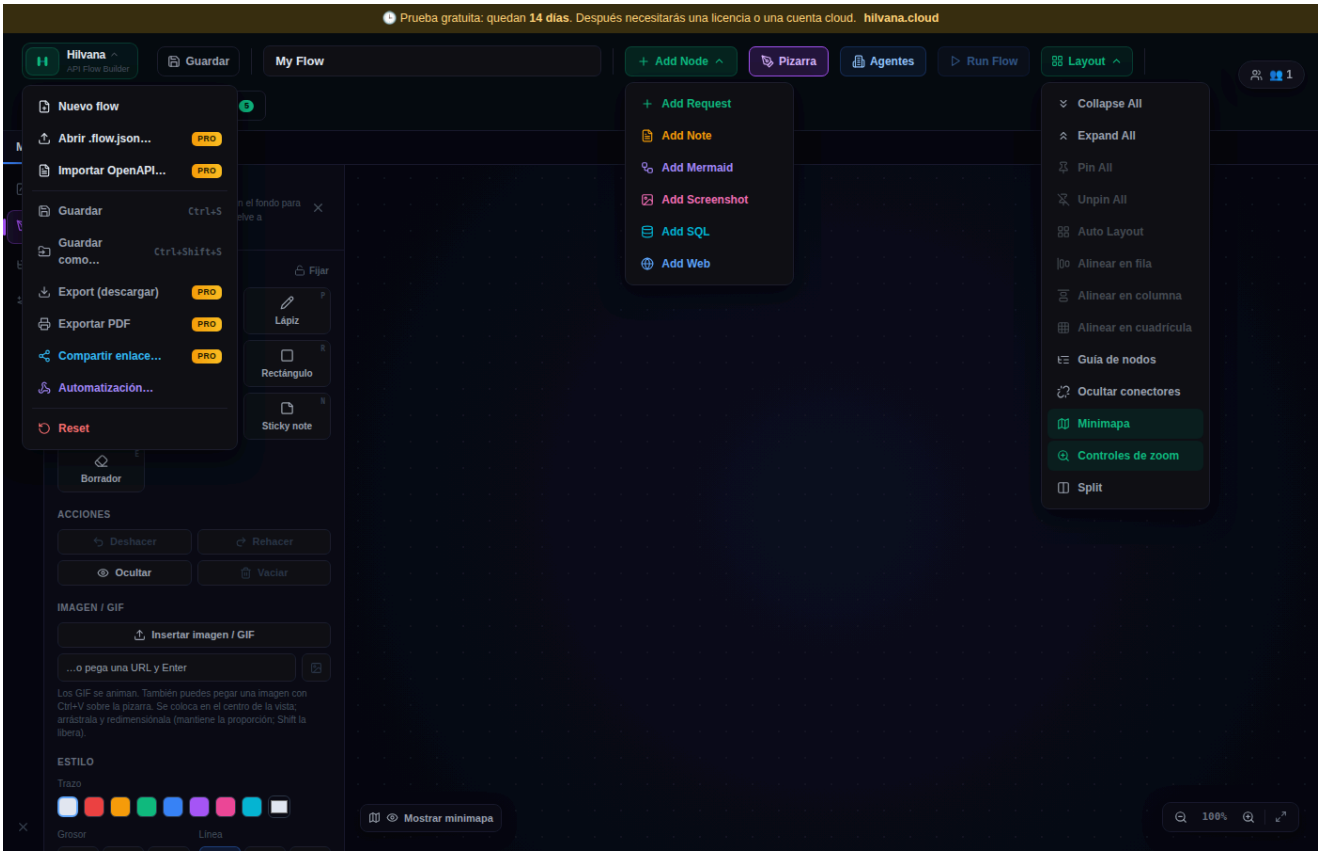
Layout (Collapse, Expand, Auto Layout y Split)

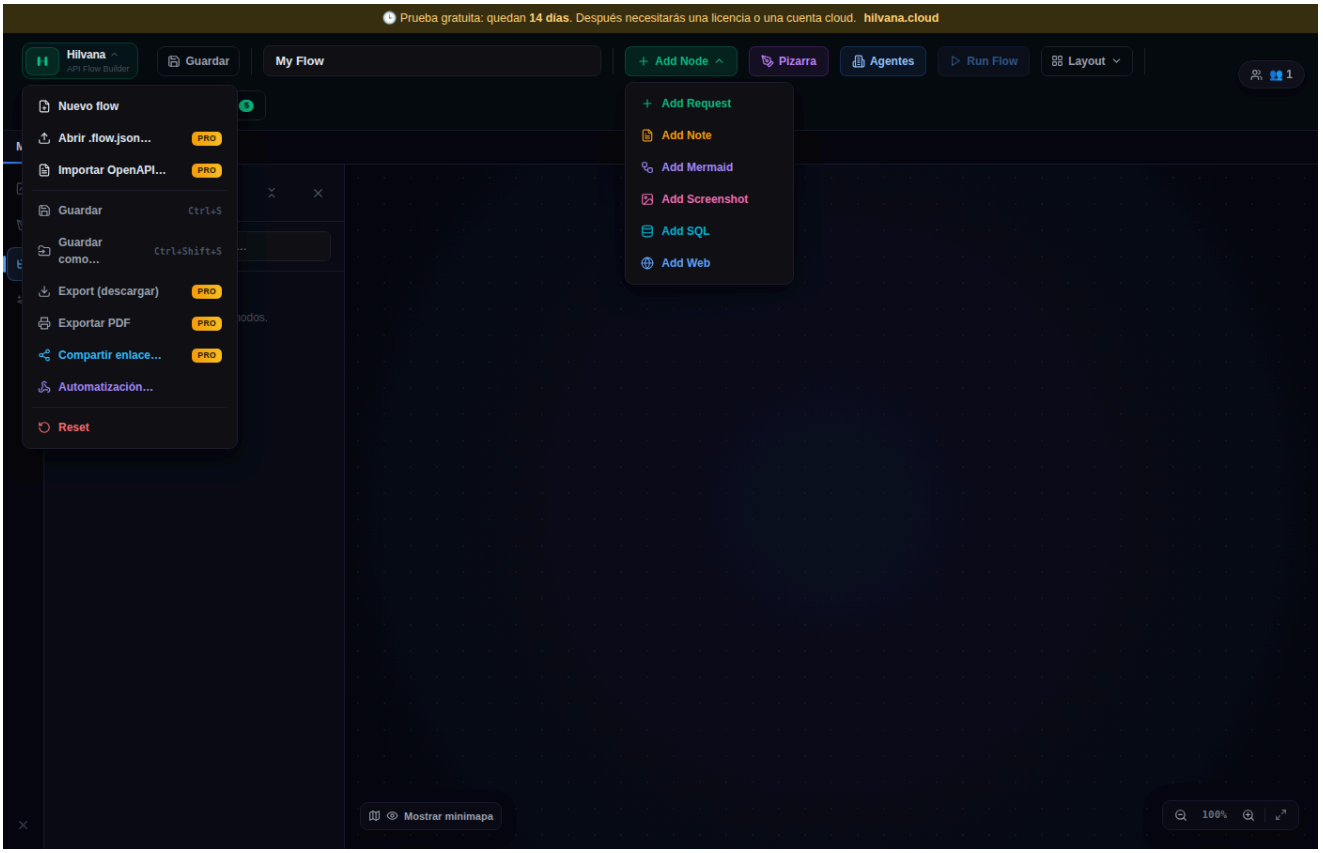


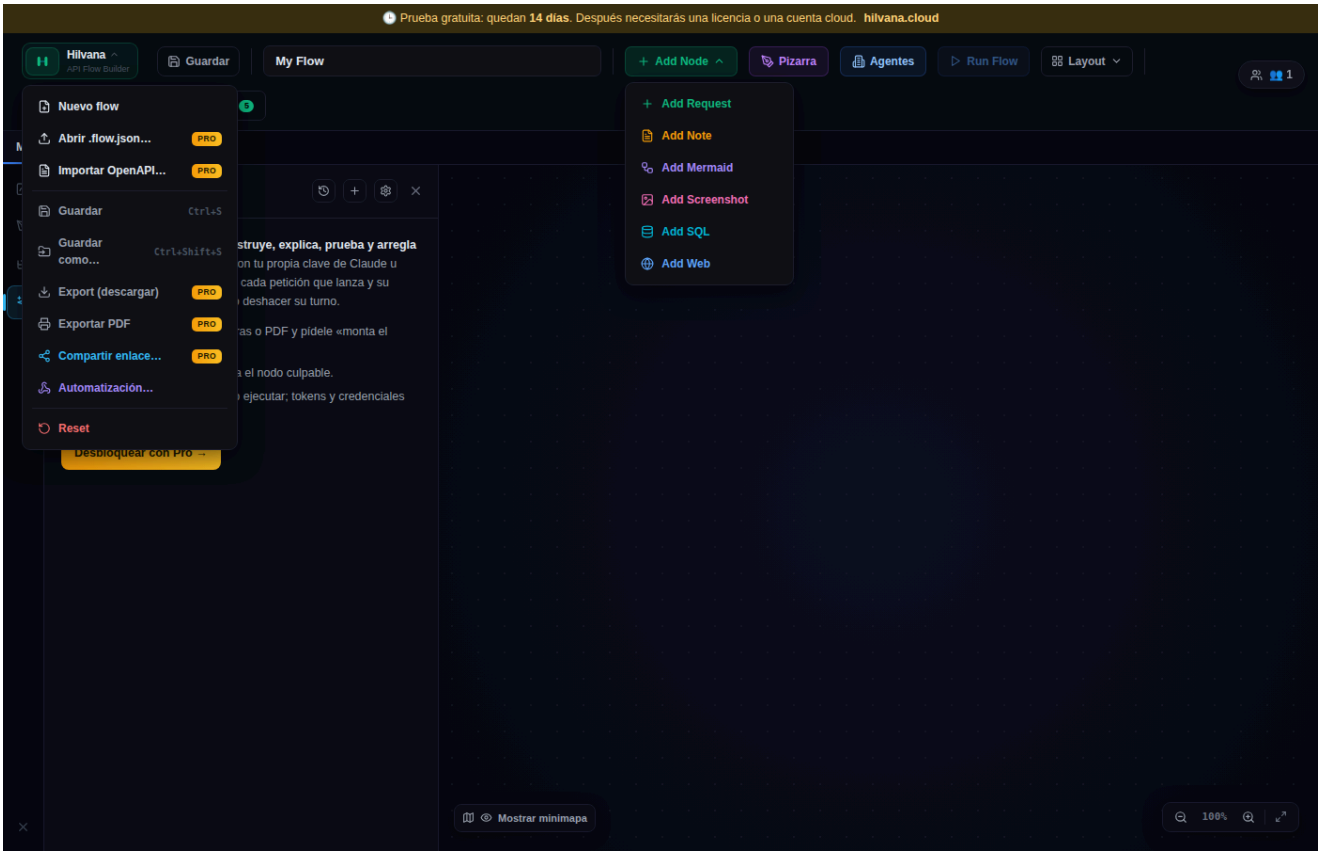
Barra lateral (rail) — iconos

Proyecto (`flows/` del servidor), Variables del flow, Variables globales, Conexiones SQL, Flows en GitHub, Pizarra, Guía de nodos y Asistente de IA. Clic abre el panel, clic otra vez lo cierra; ✕ «Cerrar panel lateral».


Prueba gratuita: quedan 14 días. Después necesitarás una licencia o una cuenta cloud. hilvana.cloud
Hilvana
API Flow Builder
Guardar
My Flow
Add Node
Pizarra
Agentes
Run Flow
Layout
Nuevo flow
Abrir .flow.json...
Importar OpenAPI...
Guardar
Guardar como...
Export (descargar)
Exportar PDF
Compartir enlace...
Automatización...
Reset
Add Request
Add Note
Add Mermaid
Add Screenshot
Add SQL
Add Web
Collapse All
Expand All
Pin All
Unpin All
Auto Layout
Alinear en fila
Alinear en columna
Alinear en cuadrícula
Guía de nodos
Ocultar conectores
Minimapa
Controles de zoom
Split
Mostrar minimapa
100%


Prueba gratuita: quedan 14 días. Después necesitarás una licencia o una cuenta cloud. hilvana.cloud
Hilvana
API Flow Builder
Guardar
My Flow
Add Node
Pizarra
Agentes
Run Flow
Layout
Nuevo flow
Abrir .flow.json...
Importar OpenAPI...
Guardar
Guardar como...
Export (descargar)
Exportar PDF
Compartir enlace...
Automatización...
Reset
Add Request
Add Note
Add Mermaid
Add Screenshot
Add SQL
Add Web
Collapse All
Expand All
Pin All
Unpin All
Auto Layout
Alinear en fila
Alinear en columna
Alinear en cuadrícula
Guía de nodos
Ocultar conectores
Minimapa
Controles de zoom
Split
Fijar
Lápiz
Rectángulo
Sticky note
Borrador
ACCIONES
Deshacer
Rehacer
Ocultar
Vaciar
IMAGEN / GIF
Insertar imagen / GIF
...o pega una URL y Enter
ESTILO
Trazo
Grosor
Línea
Mostrar minimapa
100%

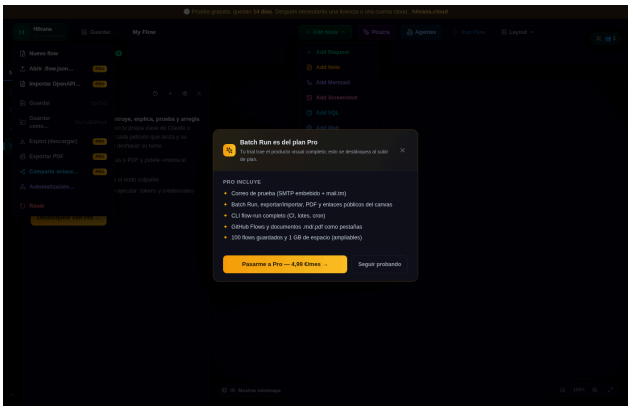
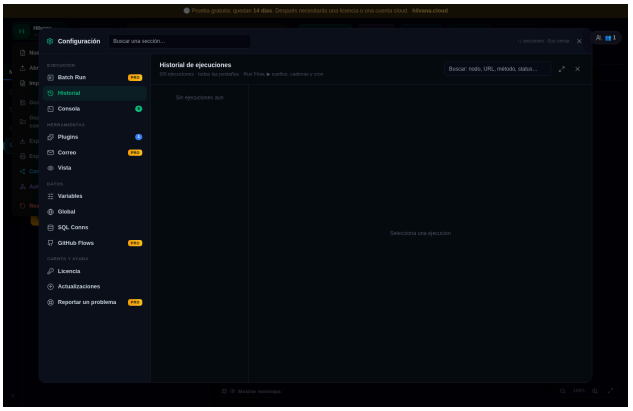
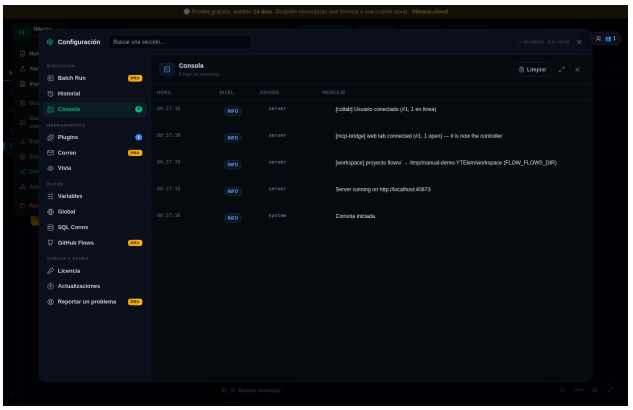
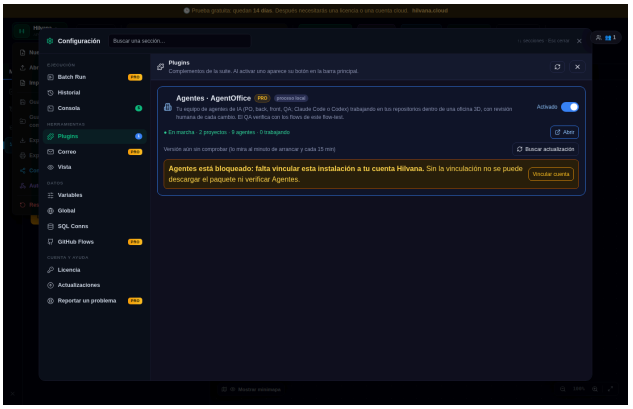


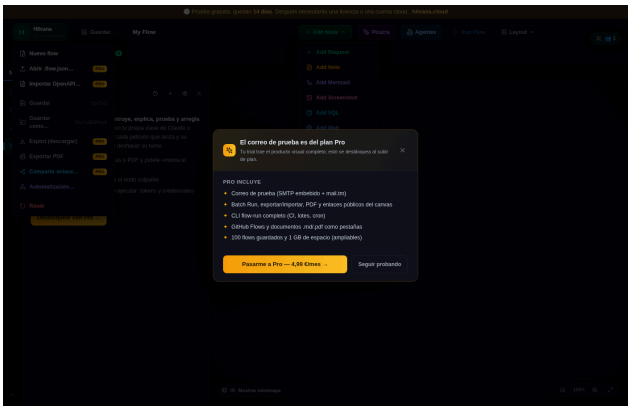
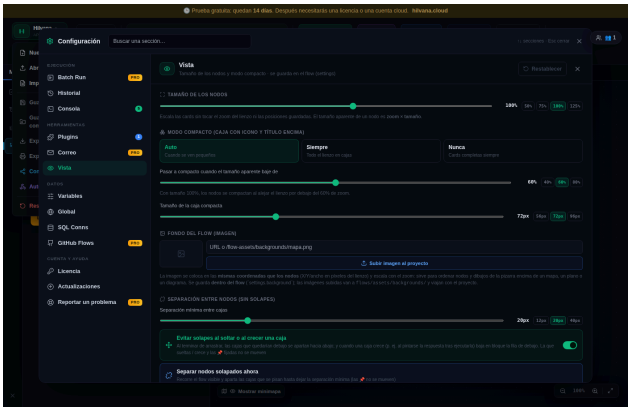
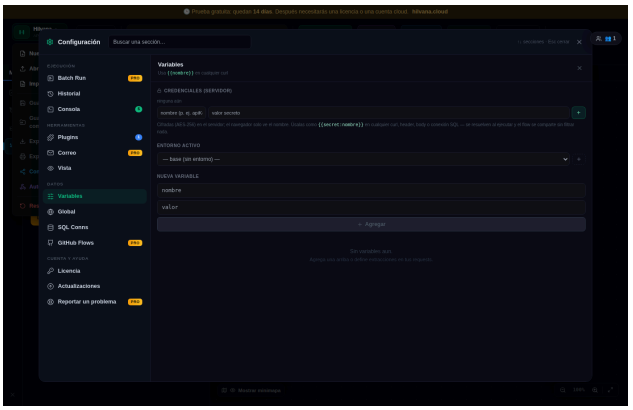
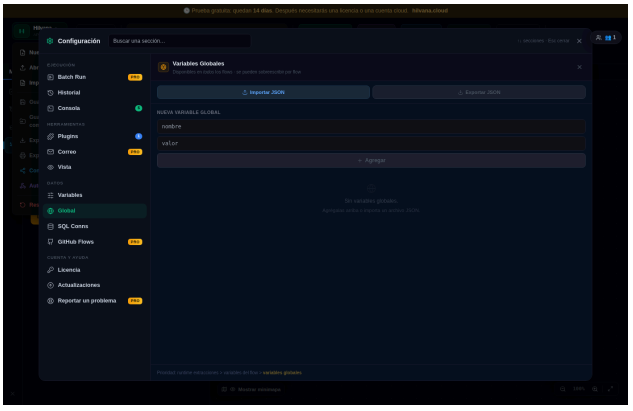


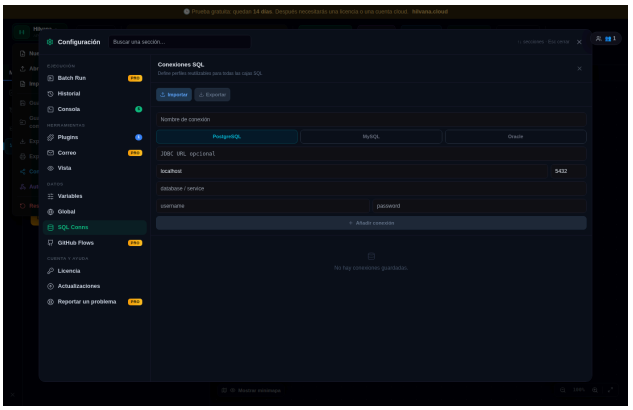
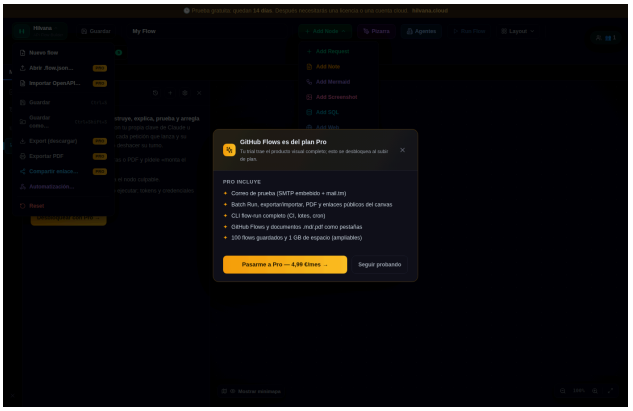
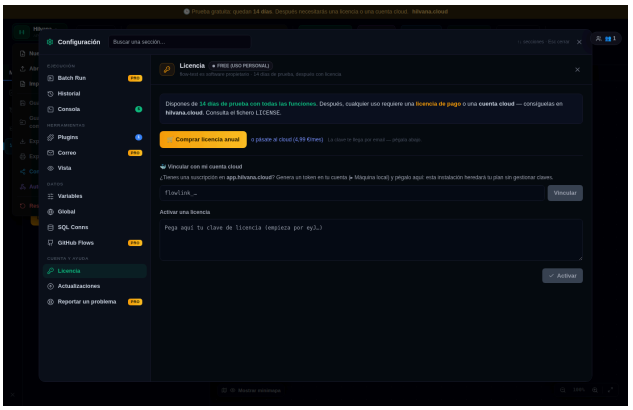
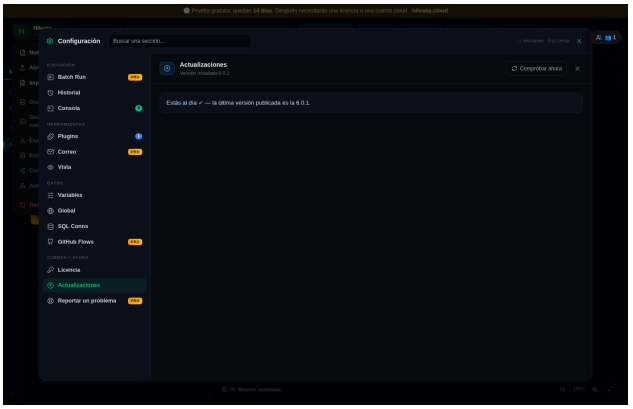
en la demo; son los mismos paneles que Config ▸ Variables / Global / SQL Conns / GitHub Flows, capturas 112–115).

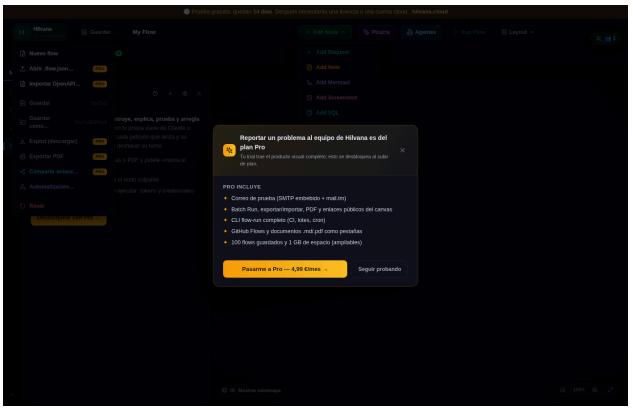
Config (botón «Historial, consola, vista, conexiones SQL y GitHub Flows»)

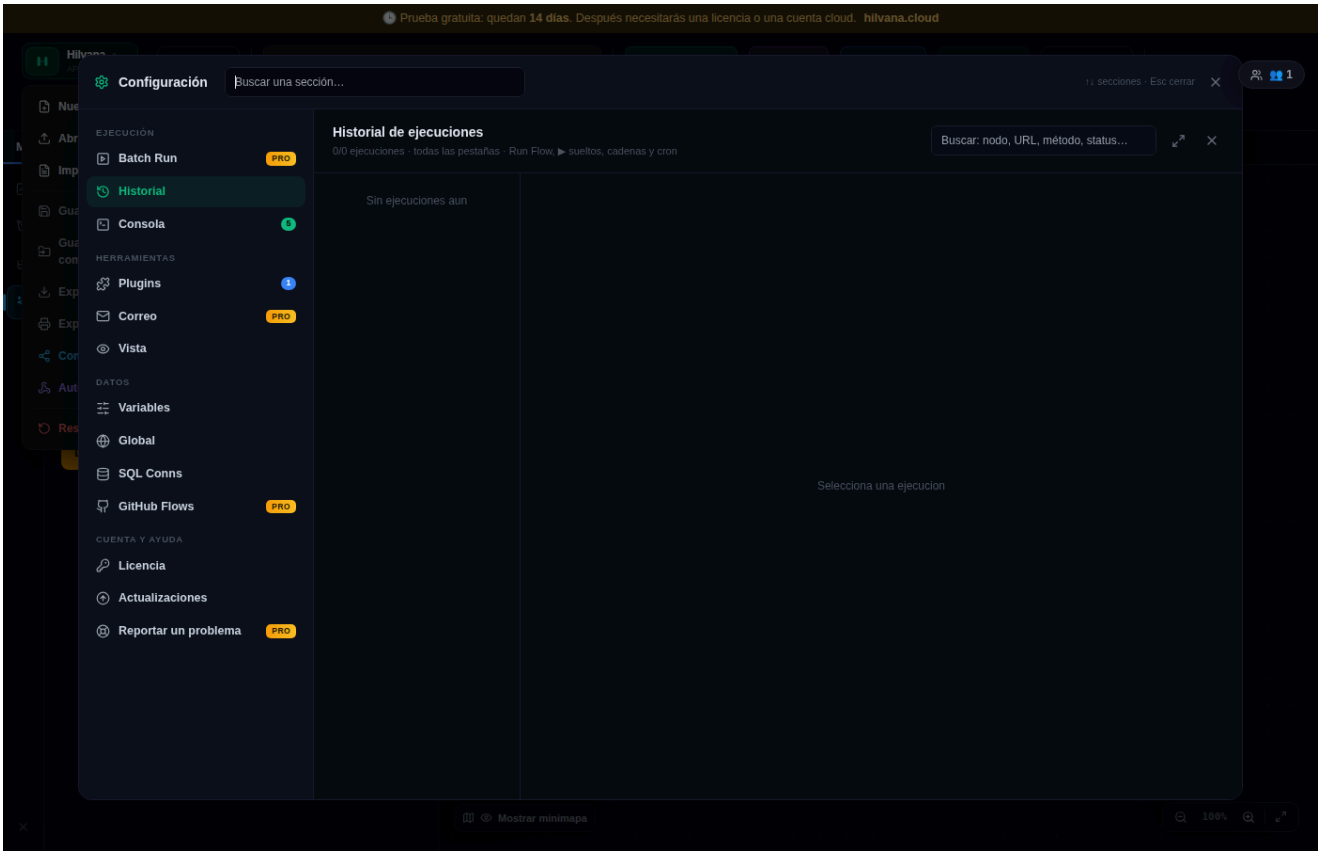
Modal con menú lateral, buscador («Buscar una sección…»), ↑↓ para moverse y Esc para cerrar. Secciones por grupo:

Grupo	Sección	Pro	Captura
Ejecución	Batch Run	sí	
Ejecución	Historial		
Ejecución	Consola		
Herramientas	Plugins		

Grupo	Sección	Pro	Captura
Herramientas	Correo (SMTP de prueba)	sí	
Herramientas	Vista (tamaño de nodos, compacto, tooltip)		
Datos	Variables		
Datos	Global		

Grupo	Sección	Pro	Captura
Datos	SQL Conns		
Datos	GitHub Flows	sí	
Cuenta y ayuda	Licencia (plan, comprar, vincular cuenta cloud `flowlink_…`, activar clave)		
Cuenta y ayuda	Actualizaciones (versión, novedades, actualizar con un clic si `FLOW_SELF_UPDATE=on`)		

Grupo	Sección	Pro	Captura
Cuenta y ayuda	Reportar un problema	sí	



En el plan de prueba, entrar en una sección Pro muestra el aviso «… es del plan Pro» con «Pasarme a Pro» / «Seguir probando».

🟢 02 · El nodo Request (la cajita curl)

> Revisado para Hilvana **6.2.1** · capturas: 97 · 112 · 113.

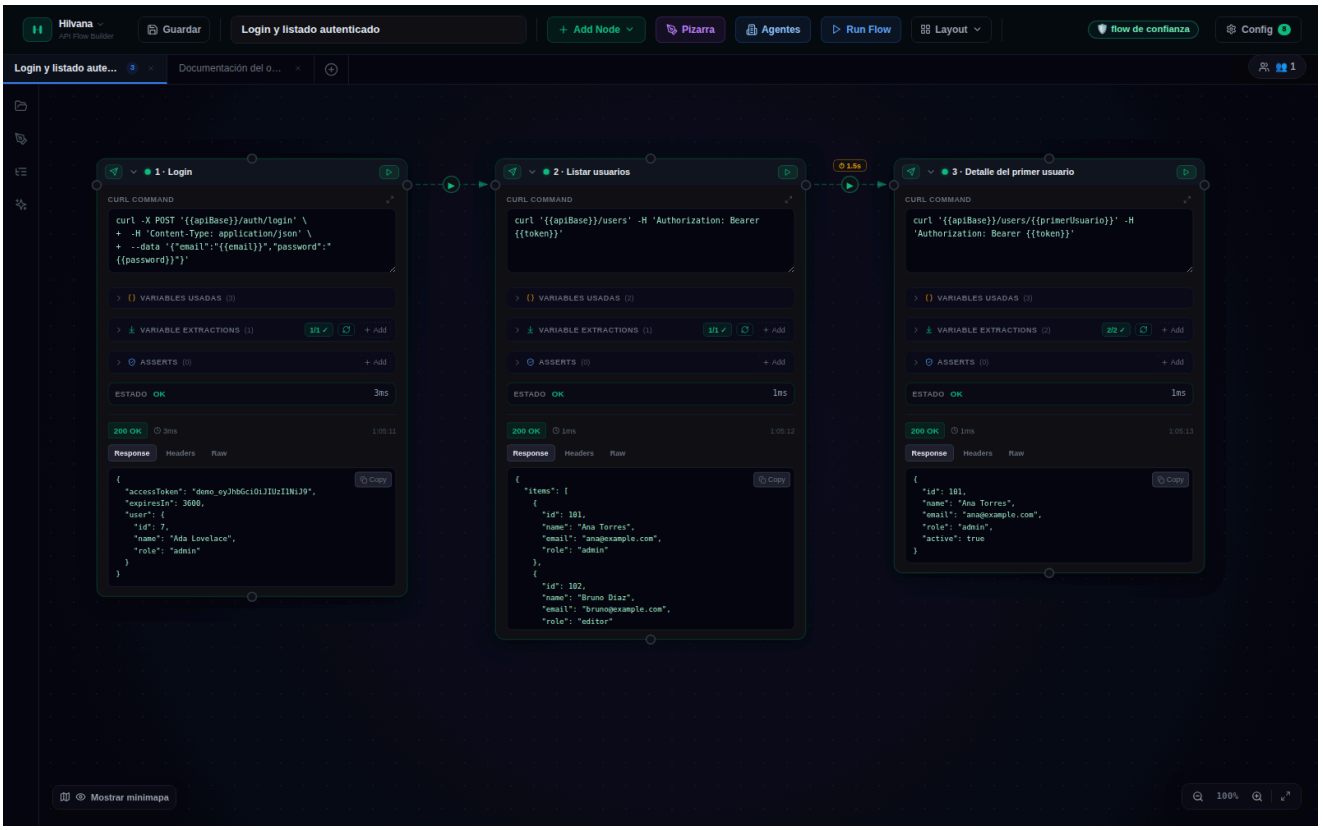
La pieza central: cada cajita verde es **una petición HTTP definida como curl**. Puedes pegar cualquier curl copiado de la pestaña Network del navegador, de Postman o de una doc de API, y funciona.

Nº	Elemento	Qué hace
①	Nombre	Editable inline. Ponle nombres descriptivos («1. Login», «2. Crear pedido»)
②	▶ Run	Ejecuta **solo este nodo** (con sus dependencias de variables ya resueltas)
③	🔗 Connect	Inicia una conexión: después clica el nodo destino
④	🕐 Cron	Programa la ejecución periódica del nodo
⑤	⿻ Copiar	Clona la cajita a otro flow (elige pestaña destino)
⑥	⤢ Maximize	Abre el nodo a pantalla completa (ver abajo)
⑦	Editor cURL	El comando; admite `{{variables}}` y el icono ⤢ lo expande a editor grande

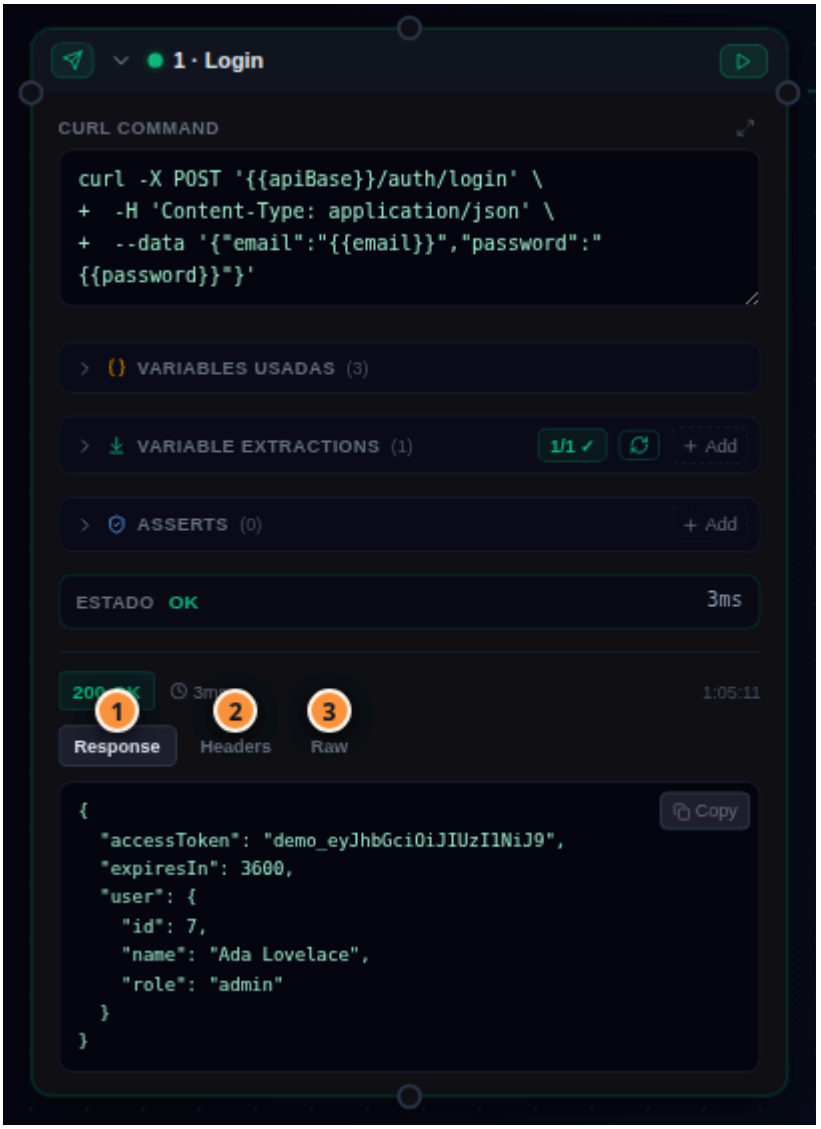
Debajo del editor: la franja **ESTADO**, el badge con **método + URL** parseados, y las **VARIABLE EXTRACTIONS** (ver 03 Conexiones y variables).

Ejecutar y leer la respuesta

Al darle a **Run Flow** (o al ▶ del nodo) las cajitas se encienden por orden: ámbar ejecutando, verde OK, rojo error, con la duración en ms.

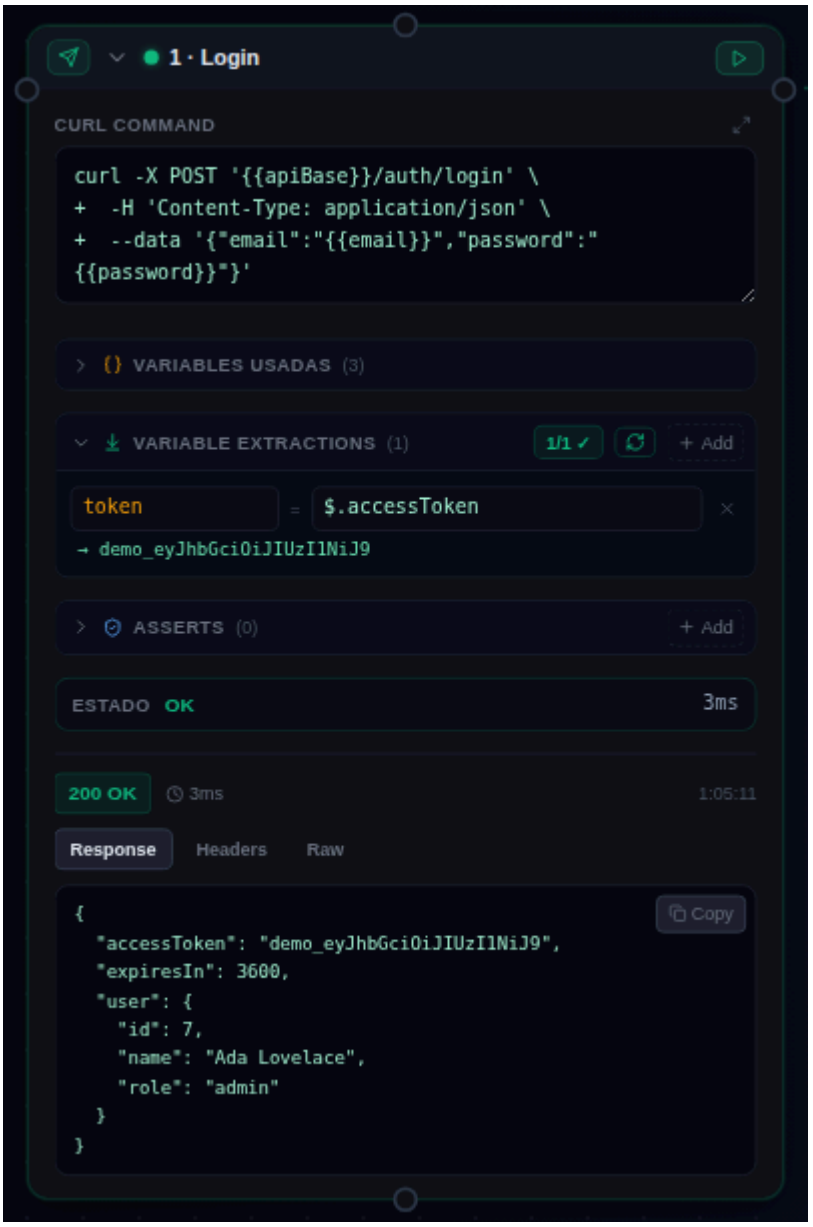


La respuesta aparece dentro de la cajita con cuatro pestañas:



Nº	Pestaña	Contenido
①	**response**	Body con JSON formateado y botón Copy
②	**headers**	Cabeceras de la respuesta
③	~~extractions~~	Desde la 4.32 las extracciones viven en la franja plegable **Variable extractions** bajo el editor (ver abajo)
④	**raw**	El curl **con las variables ya interpoladas** — lo que se envió de verdad. Perfecto para copiar y reproducir fuera

Variable extractions 🆕 4.32



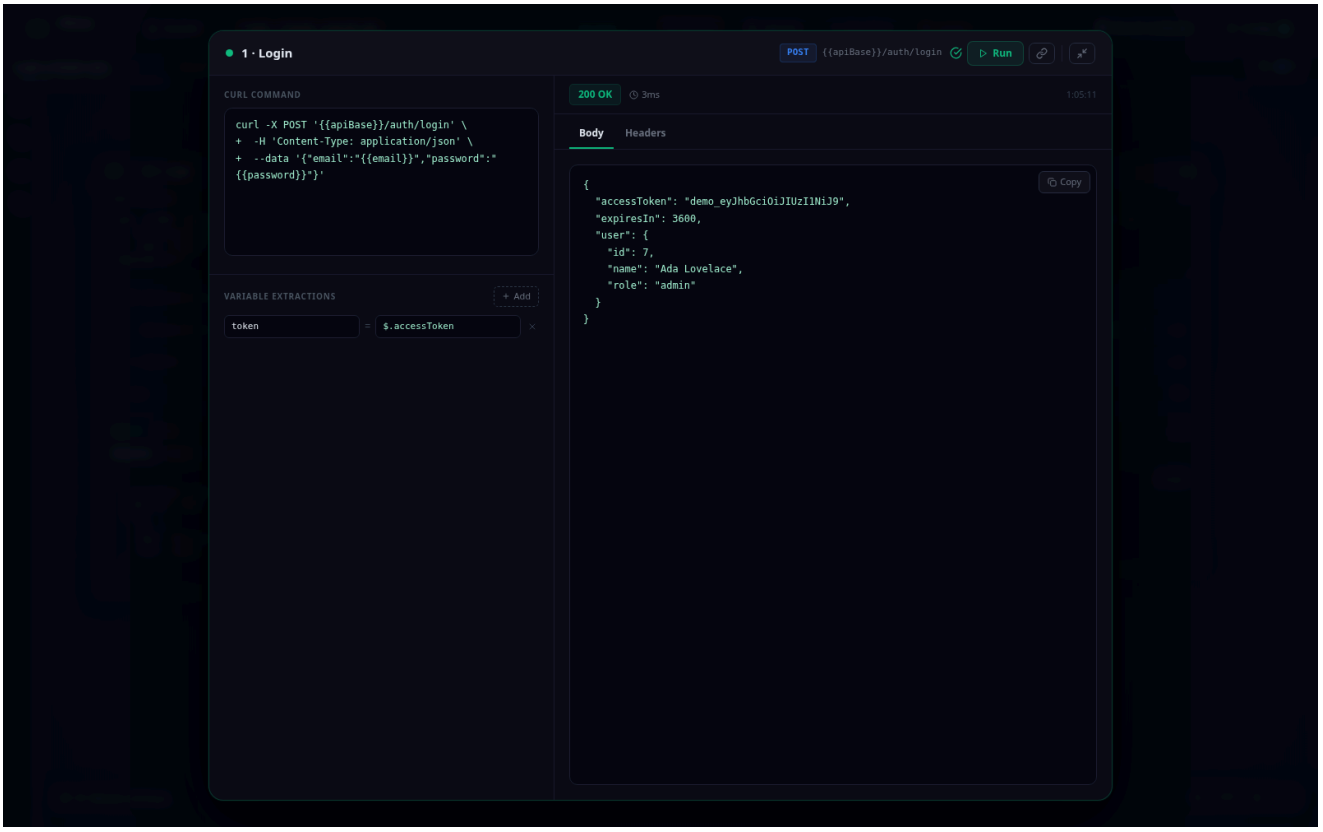
Bajo **Variables usadas** está la franja **VARIABLE EXTRACTIONS**, con el mismo diseño plegable (plegada por defecto). En su cabecera: cuántas extracciones hay, si resuelven en la **última respuesta** (`2/2 ✓` en verde o `1 sin valor` en rojo), **Add** para añadir una fila y ↻ para **volver a extraer** de la respuesta guardada **sin repetir la llamada** — si una variable no salió porque el JSONPath estaba mal, corrígelo y pulsa ↻: el valor entra en las variables del flow al momento. Desplegada, cada fila (`varName = $.ruta`) muestra debajo el valor que resuelve ahora mismo (o «sin valor en la última respuesta»). La pestaña «extractions» de la respuesta desaparece: todo vive aquí.

Estados y colores

Color	Significado
Gris	Idle, sin ejecutar
Ámbar (pulsando)	Ejecutando — muestra el tiempo transcurrido
Verde	Éxito (2xx)
Rojo	Error HTTP o de red

Pantalla completa

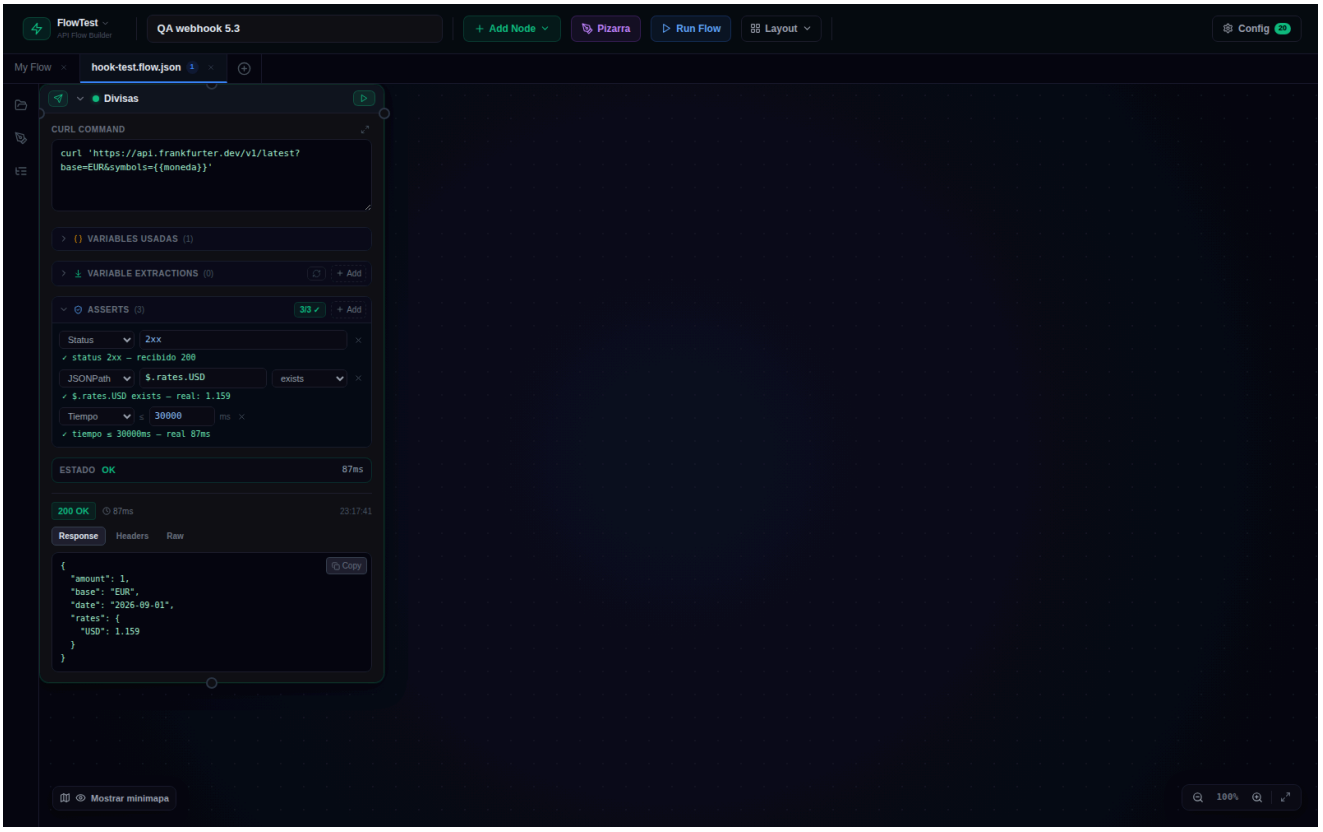
El botón ⤢ abre el nodo en un modal grande: curl y extracciones a la izquierda, respuesta con pestañas Body/Headers a la derecha. Se cierra con **Esc**.



> **Consejo: CORS** Las peticiones no salen directas del navegador: pasan por el proxy del server (`/proxy`), así que **no hay problemas de CORS** y valen URLs internas (localhost, LAN, Docker).

Asserts 🆕 (5.3)

Condiciones que deben cumplirse **después de la respuesta**. Si alguna falla, la caja queda en **rojo aunque el HTTP fuera 2xx**, y el CLI sale con código ≠ 0 — pensado para CI.



La sección **Asserts** de la caja (plegable, como las extracciones) admite tres tipos:

Tipo	Qué comprueba	Ejemplos
Status	el código HTTP	`2xx` · `200-204` · `200,201`
JSONPath	un valor del body	`$.items[0].id` con `exists`, `equals`, `not-equals`, `contains`, `gt`, `lt`
Tiempo	duración máxima	`≤ 1500 ms`

Tras ejecutar, cada assert enseña su resultado real (`✓ status 2xx — recibido 200` / `✗ $.rates.USD exists — real: (sin valor)`), la cabecera resume `3/3 ✓` o `1 KO`, y el fallo también sale en la Consola y en el historial. En el CLI, los asserts KO aparecen bajo el nodo y fuerzan `exit 1` (y en `--data`, el caso concreto queda marcado). El MCP los acepta en `node_add_request` (`asserts: [{kind, expected|path|op|value|maxMs}]`).

Errores esperados y variables 🆕 (5.19)

- **Errores esperados:** si la caja lleva un assert de **status** que pide un error (`404`, `4xx`, `400-499`…), la caja queda en **verde** cuando TODOS sus asserts se cumplen, aunque el HTTP no sea 2xx. Sirve para probar a propósito un «no encontrado» o un «sin permiso». El indicador OK de la caja y del modal de respuesta siguen la misma regla, y el CLI también.
- **Variables en los asserts:** `path`, `value` y `expected` aceptan `{{variables}}`, que se resuelven justo antes de evaluar: `$.code equals {{taskCode}}` compara con lo que extrajo un paso anterior.

🔗 03 · Conexiones y variables

> Revisado para Hilvana **6.2.1** · capturas: 97 · 112 · 113.

Los dos mecanismos que convierten cajitas sueltas en un **flujo**.

Conectar nodos

1. Clica el botón **Connect/🔗** del nodo origen (o uno de los 4 puertos circulares de sus lados).
2. Clica el nodo destino.

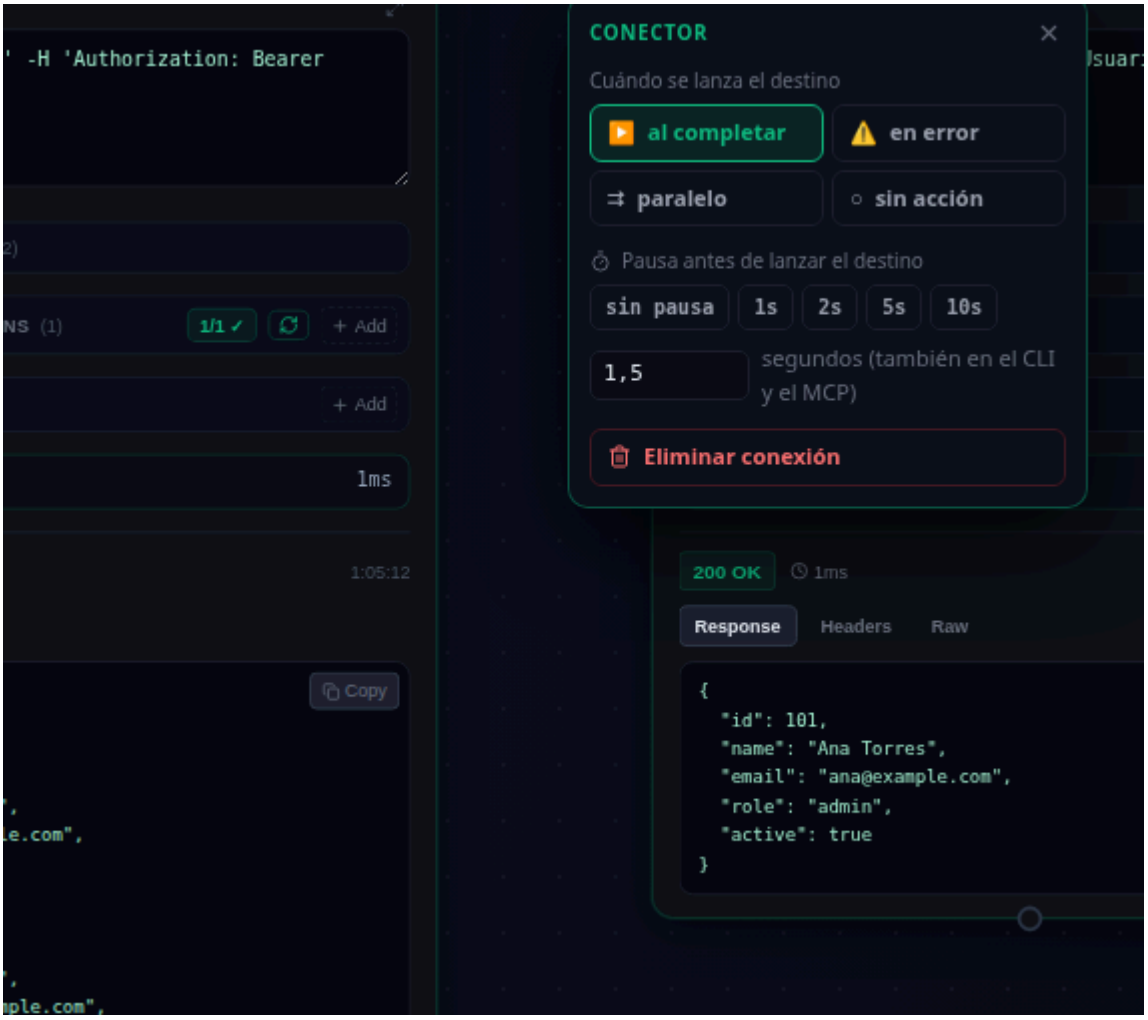
La flecha define el orden: al darle a **Run Flow**, cada nodo espera a que terminen los que le apuntan (orden topológico). Los nodos sin dependencias entre sí corren **en paralelo**.

Tipos de conexión (behavior)

Clicando sobre una conexión puedes cambiar su comportamiento:

Behavior	Significado
next (por defecto)	El destino corre cuando el origen termina **bien**
on_error	El destino corre **solo si** el origen falla (rutas de recuperación)
parallel	El destino arranca a la vez que el origen, sin esperar
none	Solo informativa: dibuja la relación pero **no afecta** a la ejecución. Es la que usan las capturas para colgar sus cajitas de llamadas 🆕

Cadenas entre cualquier tipo de nodo y pausa por conector 🆕 4.34



- El ▶ **de cualquier caja** (request, SQL, nota, web) ejecuta el nodo y **sigue sus flechas hacia nodos de cualquier tipo**: de una nota a una request, de una request a un SQL, de un SQL a una web… `▶ next` se recorre si el nodo fue bien, `⚠ on_error` si falló, `⇉ parallel` siempre (sin esperar). Una nota «ejecutada» lanza sus scripts JS y pasa las variables generadas al siguiente; una web se recarga.
- **Run Flow** ejecuta ahora también los **nodos SQL** en el mismo orden topológico que las requests (antes solo HTTP): las variables extraídas por una query llegan a las requests siguientes y al revés. Igual que el CLI.
- **Configurar un conector**: clic en el círculo del medio de la flecha → popover con el **comportamiento** (al completar / en error / paralelo / sin acción) y una **pausa antes de lanzar el destino** (sin pausa, 1 s, 2 s, 5 s, 10 s o los segundos que escribas). La flecha muestra `⏱ 1.5s`; clic derecho sigue borrando la conexión. La pausa se guarda en el flow (`delayMs`) y la respetan la web, **el CLI** (`⏱ … pausa de 2s`) y el MCP (`nodes_connect` / `connection_update` con `delayMs`). Útil para APIs con procesos asíncronos (esperar a que un webhook llegue, a que un job termine) antes de consultar.

Variables: mover datos entre cajitas

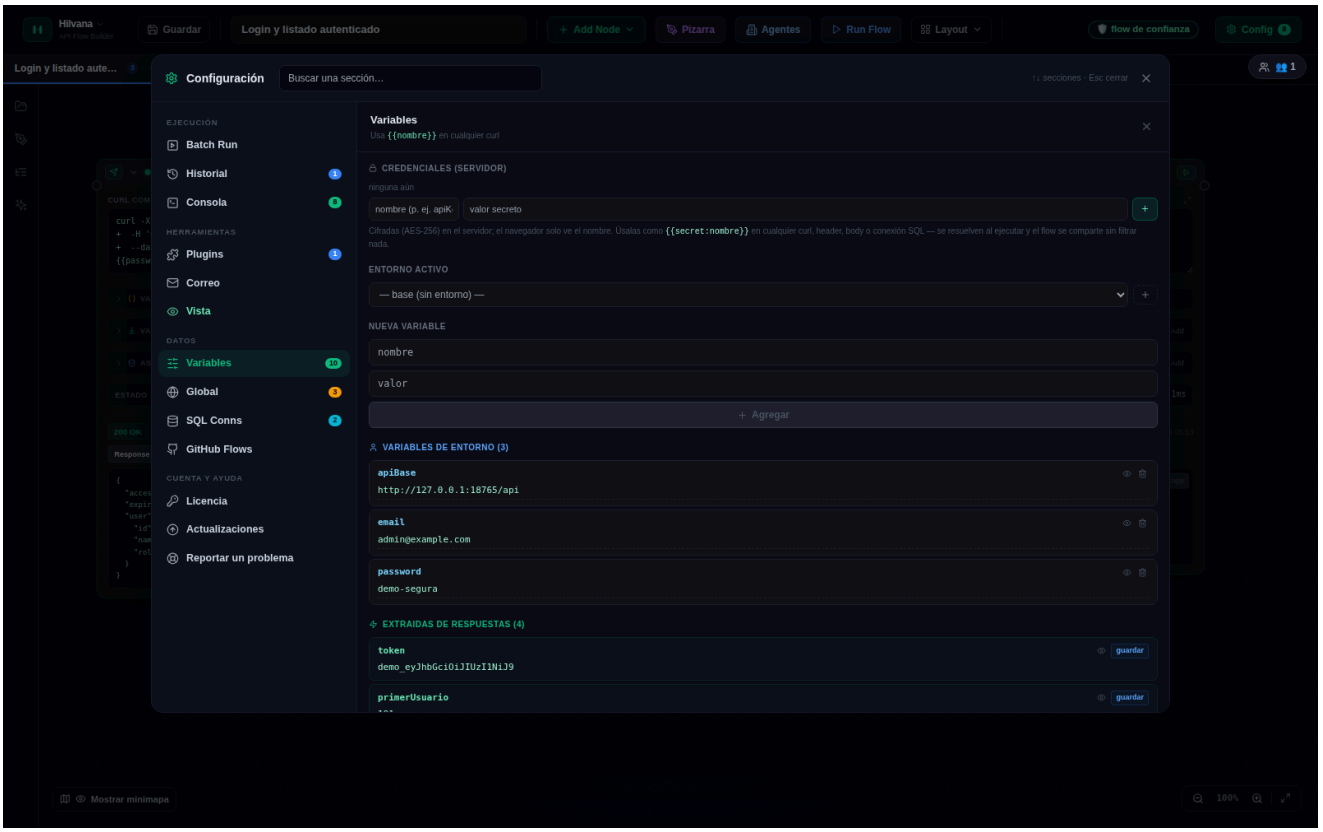
El ciclo completo, con el flujo de ejemplo:

1. **Extraer**: el nodo «1. Catálogo» define la extracción `primerProducto = $.items[0].id` — al ejecutarse, guarda en esa variable el id del primer producto de la respuesta (JSONPath sobre el body).
2. **Usar**: el nodo «2. Añadir al carrito» escribe `{{primerProducto}}` dentro de su curl (URL, headers o body, da igual) y se interpola al ejecutar.

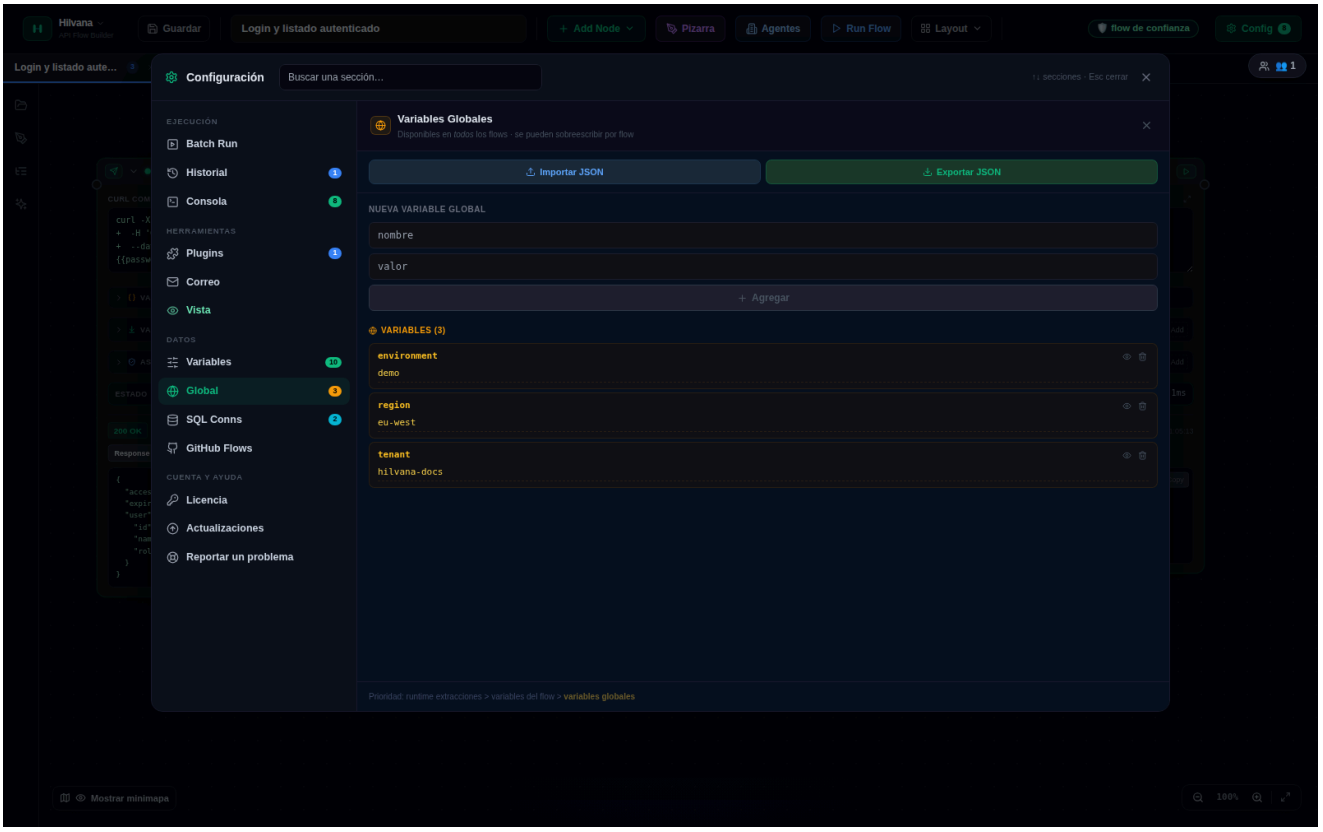
```
curl -X POST 'http://.../api/cart' \
  -H 'authorization: {{authToken}}' \
  --data '{"productId": {{primerProducto}}, "qty": 1}'
```

Los tres niveles de variables

Nivel	Dónde vive	Para qué
Entorno del flow	Panel **Variables** (se exportan con el flow)	Config del flujo: hosts, tokens, ids fijos
Runtime	Panel **Variables** (sección runtime; se limpian al recargar)	Lo extraído durante la ejecución
Globales	Panel **Global** (compartidas entre todos los flows)	Cosas transversales: `apiBase`, tokens de entorno



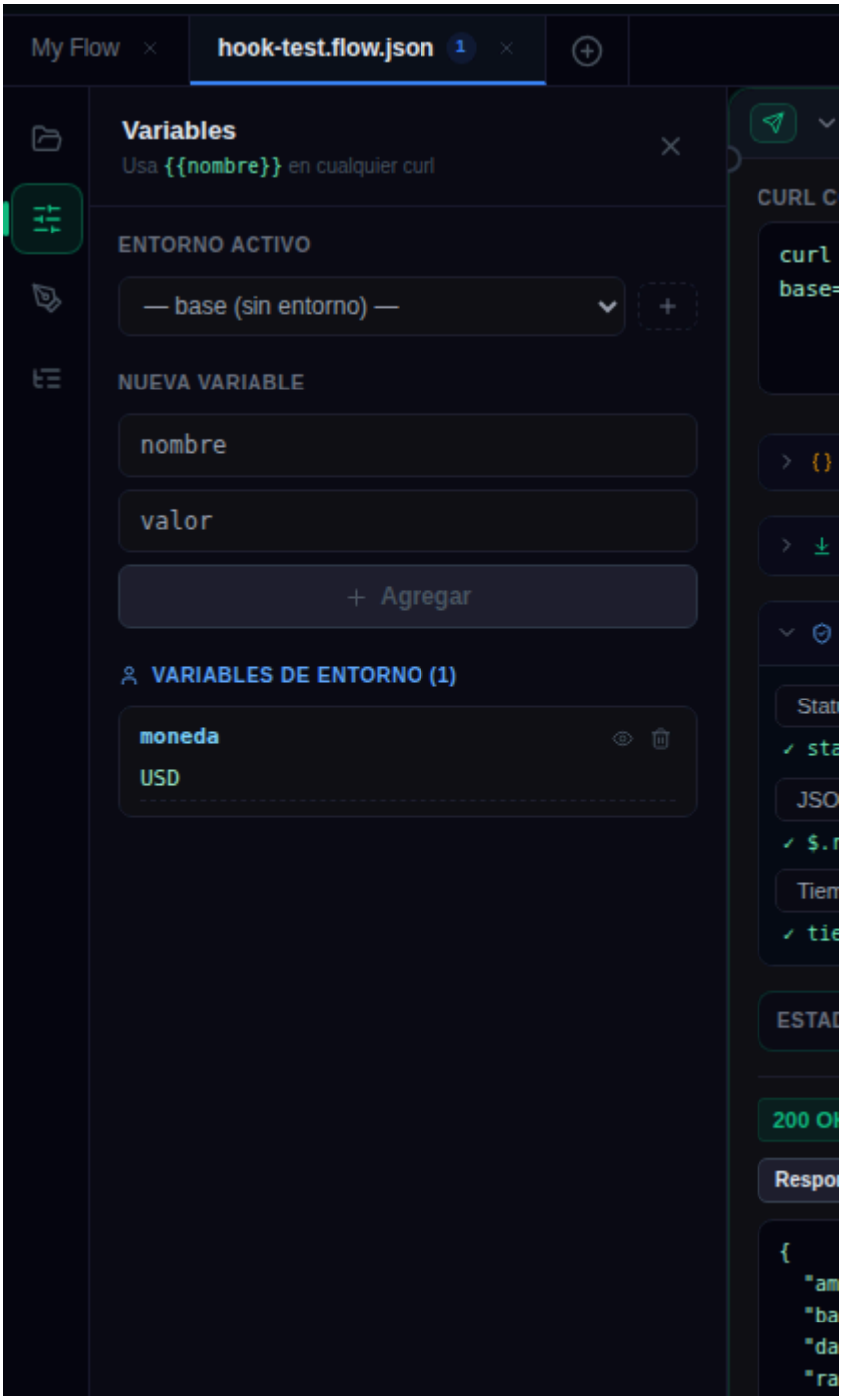
El panel **Variables** muestra las del flow activo — tras una ejecución verás también las extraídas (aquí `primerProducto`). El panel **Global**:



> **Consejo: Dónde valen las `{{variables}}`** En los curl, en las consultas y conexiones SQL, en las URLs de los nodos Web, en el contenido de notas y diagramas Mermaid, y en las rutas de imagen de los nodos Captura. En todas partes, vaya.

Entornos con nombre 🆕 (5.3)

Los mismos flows contra **dev, staging o prod** sin tocar nada: cada entorno es un set de variables que **sobrescribe** a las del flow mientras está activo.



En el panel **Variables** (Config ▸ Variables), el desplegable **Entorno activo** crea (+), selecciona o elimina entornos; con uno activo aparece su sección de *overrides*: solo defines lo que cambia (p. ej. `apiBase`), el resto se hereda de las variables base. El entorno activo se guarda dentro del flow.

En el **CLI**: `flow-run --flow api.flow.json --env prod` (si no pasas `--env`, se respeta el entorno activo guardado; un nombre inexistente da error con la lista de los disponibles). Precedencia total: `envVariables < entorno < scripts de notas < --var`. Por **MCP**, `variables_set` acepta `environment` (escribe en ese set, creándolo si falta) y `activate: true`.

Credenciales cifradas 🆕 (5.6 · Pro)

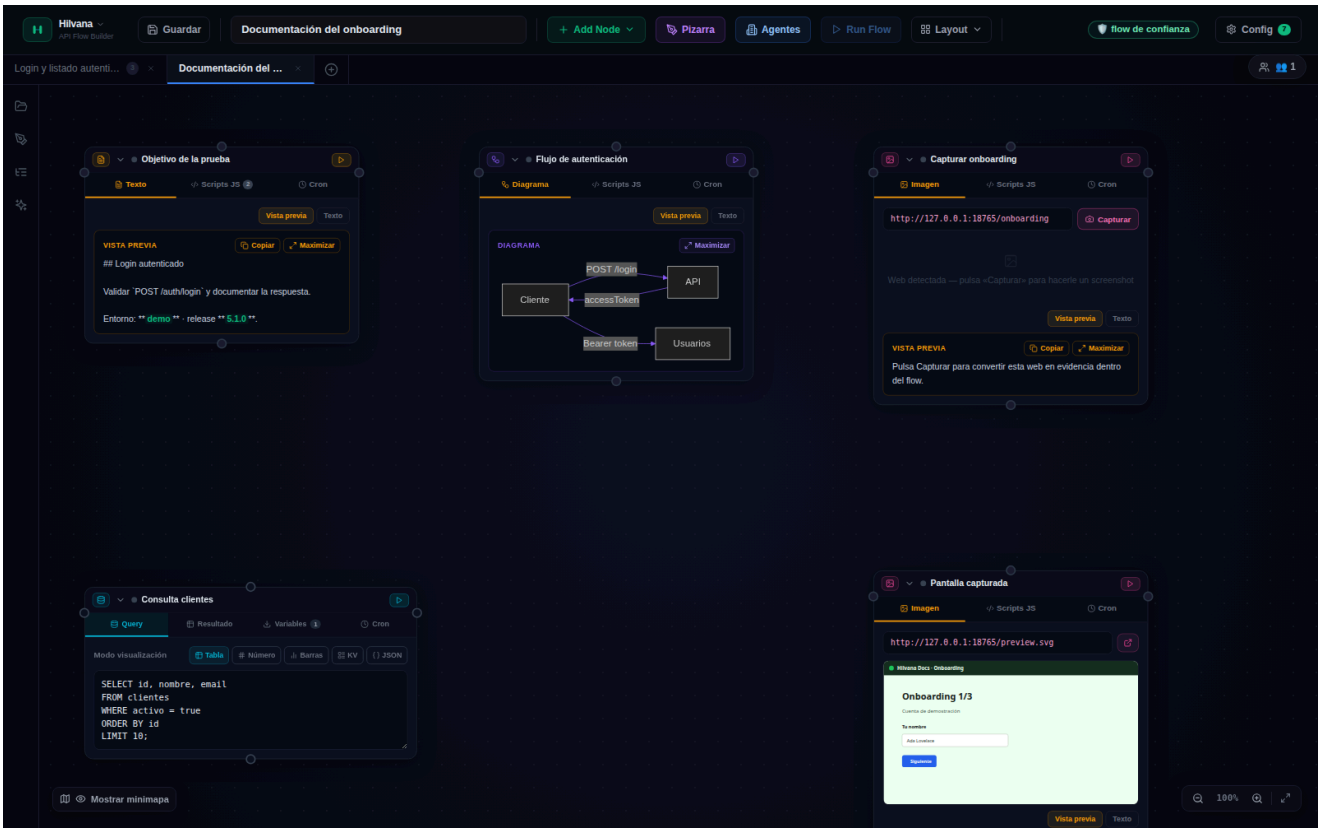
Los tokens y contraseñas no tienen por qué vivir en `envVariables` (que se comparten y se exportan con el flow). En **Variables ▸ 🔒 Credenciales (servidor)** guardas valores que se **cifran (AES-256) en el servidor** y se usan como `{{secret:nombre}}` en cualquier curl, header, body, query SQL o campo de conexión:

- El navegador **nunca ve el valor** — solo el nombre; la sustitución ocurre en el servidor al ejecutar (web, CLI, webhook entrante y monitores, todos por igual).
- El `.flow.json` guarda el placeholder tal cual: **compartir o versionar el flow no filtra nada** (la galería, un repo git, un compañero de colaboración… ven `{{secret:apiKey}}`, no la clave).
- Un secreto inexistente hace fallar el nodo con un error claro, nunca envía el placeholder al API.
- La clave maestra vive en `<flows>/.secret-key` (autogenerada) o en la variable de entorno `FLOW_SECRET_KEY` (64 hex) si prefieres inyectarla tú.

📝 04 · Notas, Mermaid y Capturas

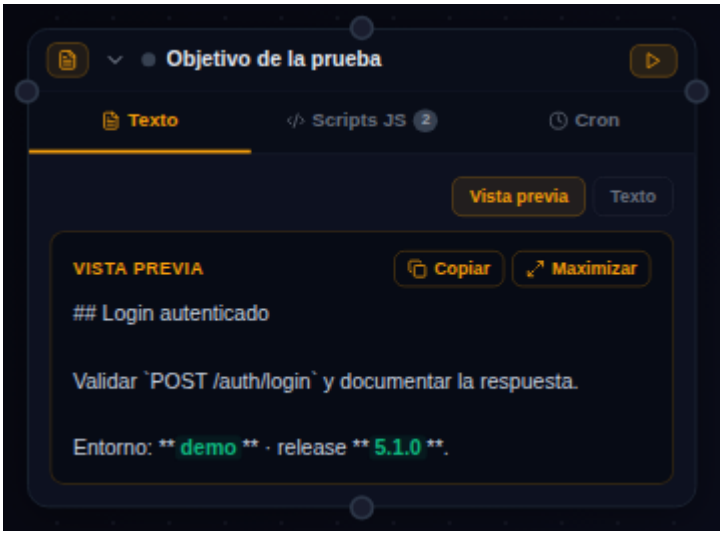
> Revisado para Hilvana **6.2.1** · capturas: 99.

Tres variantes del mismo nodo (la nota) para **documentar dentro del canvas**:



Nota de texto (Add Note, ámbar)

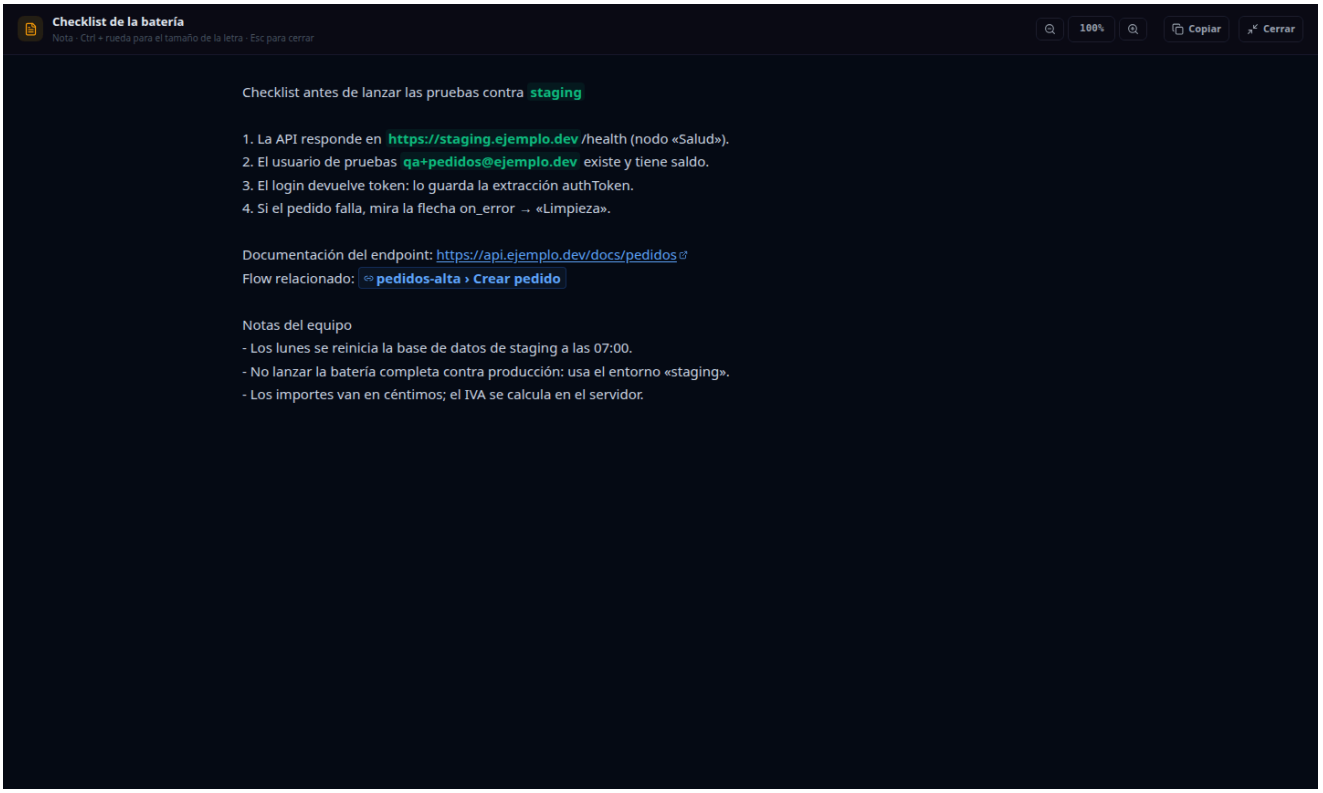
Texto libre. Si escribes `{{variables}}`, la **vista previa** resuelve los valores en verde (y en ámbar las que aún no existen). Un conmutador **Vista previa / Texto** (arriba a la derecha de la pestaña) muestra uno u otro — vista previa por defecto; en los nodos Mermaid alterna entre el diagrama y el código. Desde la 4.29 la vista previa tiene un botón **Copiar** que lleva al portapapeles **toda la vista**: texto plano con las `{{variables}}` resueltas y, a la vez, HTML con los enlaces (al pegar en Slack/Word/Obsidian se conservan).



Nota a pantalla completa 🆕 (5.16)

A la derecha de **Copiar**, el botón **Maximizar** abre la nota a pantalla completa — para leer una nota larga sin estirar la caja ni hacer zoom en el lienzo:

Checklist de la batería
Texto
Scripts JS
Cron
Vista previa
Texto
VISTA PREVIA
Copiar
Maximizar
Checklist antes de lanzar las pruebas contra staging
1. La API responde en https://staging.ejemplo.dev /health (nodo «Salud»).
2. El usuario de pruebas qa+pedidos@ejemplo.dev existe y tiene saldo.
3. El login devuelve token: lo guarda la extracción authToken.
4. Si el pedido falla, mira la flecha on_error → «Limpieza».
Documentación del endpoint:
https://api.ejemplo.dev/docs/pedidos
Flow relacionado: pedidos-alta › Crear pedido
Notas del equipo
- Los lunes se reinicia la base de datos de staging a las 07:00.
- No lanzar la batería completa contra producción: usa el entorno «staging».
- Los importes van en céntimos; el IVA se calcula en el servidor.



- Se ve **como en la vista previa**: `{{variables}}` resueltas, URLs clicables y enlaces `[[flow]]` / `[[flow#Nodo]]` — al seguir uno, la nota se cierra y la app navega al flow.
- **Tamaño de la letra**: botones − / % / + de la barra, **Ctrl + rueda** o `Ctrl +` / `Ctrl −` / `Ctrl 0` (de 60 % a 300 %).
- **Copiar** copia la nota entera (texto plano + HTML, igual que el botón de la caja). **Esc** o **Cerrar** vuelven al lienzo.
- En una **Captura**, la imagen sale encima del texto. Los diagramas Mermaid tienen su propio «Maximizar diagrama» con zoom.
- También está en el menú de la caja (icono del tipo ▸ **Maximizar nota**), que funciona aunque la nota esté plegada.

La pestaña **Scripts JS** define variables calculadas con JavaScript (`return "hola"` → variable disponible en el flow). **Desde la 4.23 esos scripts se ejecutan solos al pulsar Run Flow** (también con `flow_run` por MCP y en el CLI) y sus valores entran en el flow **antes de la primera petición** — ideal para ids, emails o tokens únicos por ejecución (`return 'qa+' + Date.now() + '@example.com'`). Precedencia en el CLI: `envVariables` < scripts < `--var` ; `--skip-info-scripts` los desactiva. La pestaña **Cron** los recalcula periódicamente.

Scripts «antes» y «después», con `vars` 🆕 (4.51)

Cada script tiene un conmutador **antes / después** en su cabecera:

- **antes** (por defecto, como hasta ahora): corre antes de la primera petición de Run Flow — ids únicos, fechas, tokens.
- **después**: corre **al terminar Run Flow**, cuando ya están todas las extracciones del run. Es el sitio para calcular a partir de los datos: deltas (`return (vars.wti - vars.wtiPrev).toFixed(2)`), semáforos (`return Number(vars.vix) > 30 ? '🔴 pánico' : '🟢 calma'`), textos resumen… Sus valores entran como variables del flow, así que una nota, un Mermaid o un texto de la pizarra los muestran con `{{variable}}`.
- Todo script recibe ahora **`vars`**: un objeto con las variables del flow en ese momento (entorno + runtime) más las de los scripts anteriores de la misma tanda. También al pulsar ▶ en la nota o en el script, y en las cadenas.
- El Historial registra cada tanda como `JS` («scripts JS (antes)» / «(después)»). En el CLI ocurre lo mismo (`--var` sigue ganando; `--skip-info-scripts` salta las dos fases) y el MCP acepta `when: "after"` en `scripts`.

Scripts que dibujan botones, páginas y acciones 🆕 (5.14)

Un script de nota es JavaScript que corre **dentro de la página de Hilvana** cuando ejecutas el flow (`new Function('vars', código)`), con los mismos permisos que la app — desde la 5.15, solo en los **flows de confianza**; los demás corren en un sandbox sin página (ver más abajo). Además de devolver variables, puede **dibujar interfaz**: colgar del `document.body` una barra de botones, abrir una «página» (un `<dialog>` con pestañas), abrir una pestaña nueva del navegador con un informe (`window.open` + `document.write`), descargar un fichero (`Blob` + `<a download>`), copiar al portapapeles y, al pulsar un botón, **hablar con tu propia instalación** llamando al MCP embebido desde la página:

```
const r = await fetch('/mcp', { method: 'POST',
  headers: { 'Content-Type': 'application/json', Accept: 'application/json, text/event-stream' },
  body: JSON.stringify({ jsonrpc: '2.0', id: Date.now(), method: 'tools/call',
    params: { name: 'flow_run', arguments: {} } }) });   // o variables_set, flow_open, node_update…
```

El endpoint acepta llamadas sueltas sin sesión; `Accept` debe incluir `text/event-stream`. Si la instalación lleva `FLOW_MCP_TOKEN`, añade `Authorization: Bearer …` y haz que el botón lo diga si falla, en vez de callar.

Ejemplos listos en la galería: **Scripts 04 · Divisas** (barra de botones, diálogo con conversor y pestañas, informe en pestaña nueva, CSV, copiar y ▶ que reejecuta el flow) y **Scripts 05 · Tiempo** (botones que escriben Variables y relanzan el flow).

El contrato de un script que dibuja (lo que lo convierte en herramienta y no en trampa):

1. **Guardas**: `if (typeof document === 'undefined') return 'CLI: sin interfaz'` (el CLI y los monitores no tienen página) y sin datos no dibuja nada.
2. **Una sola instancia**: id fijo y `document.getElementById(ID)?.remove()` antes de crearse; como corre «después» de cada Run Flow, se redibuja con los datos nuevos.
3. **Shadow DOM** (`host.attachShadow({ mode: 'open' })`): sus estilos no pisan los de la app ni al revés.
4. **Nada persistente ni automático**: sin `localStorage`, sin `Notification`, sin `setInterval`, sin relanzar el flow por su cuenta. Cada acción es un clic del usuario; para ejecutar solo está el Monitor.
5. **Botón ✕** siempre activo que quita el panel, y `return` de un texto de estado para que la nota diga qué pasó.
6. El panel es de la página, no de la pestaña: sigue ahí al cambiar de flow hasta que lo quitas o reejecutas.

Lo que no cambia: el `.flow.json` sigue siendo un fichero de datos y la app no se modifica. La otra cara: un flow que te pasen puede ejecutar código en tu navegador — por eso desde la 5.15 los flows que no son tuyos corren sus scripts en un **sandbox** hasta que tú decides confiar (sección siguiente). Si repartes flows a gente que no los va a leer, quita los scripts o mantén una copia sin ellos.

Cómo llegan los datos al script: las extracciones convierten lo que sacan en texto (`String(valor)`): un array queda como `"v1,v2,…"` (un array de pares como `[[ts, precio], …]` se aplana a `"ts,precio,ts,precio…"`) y un objeto como `[object Object]`. Extrae números, textos o arrays de primitivos y parsea en el script (`String(vars.x).split(',').map(Number)`).

🆕 4.25: en el texto, `[[otro-flow]]`, `[[otro-flow|texto]]` y `[[otro-flow#Nombre de nodo]]` se convierten en **enlaces a otros flows del proyecto** (abre el flow y centra el nodo), y las URLs `http(s)` son clicables — ver 08 · Enlaces entre flows.

Sandbox y flows de confianza 🆕 (5.15)

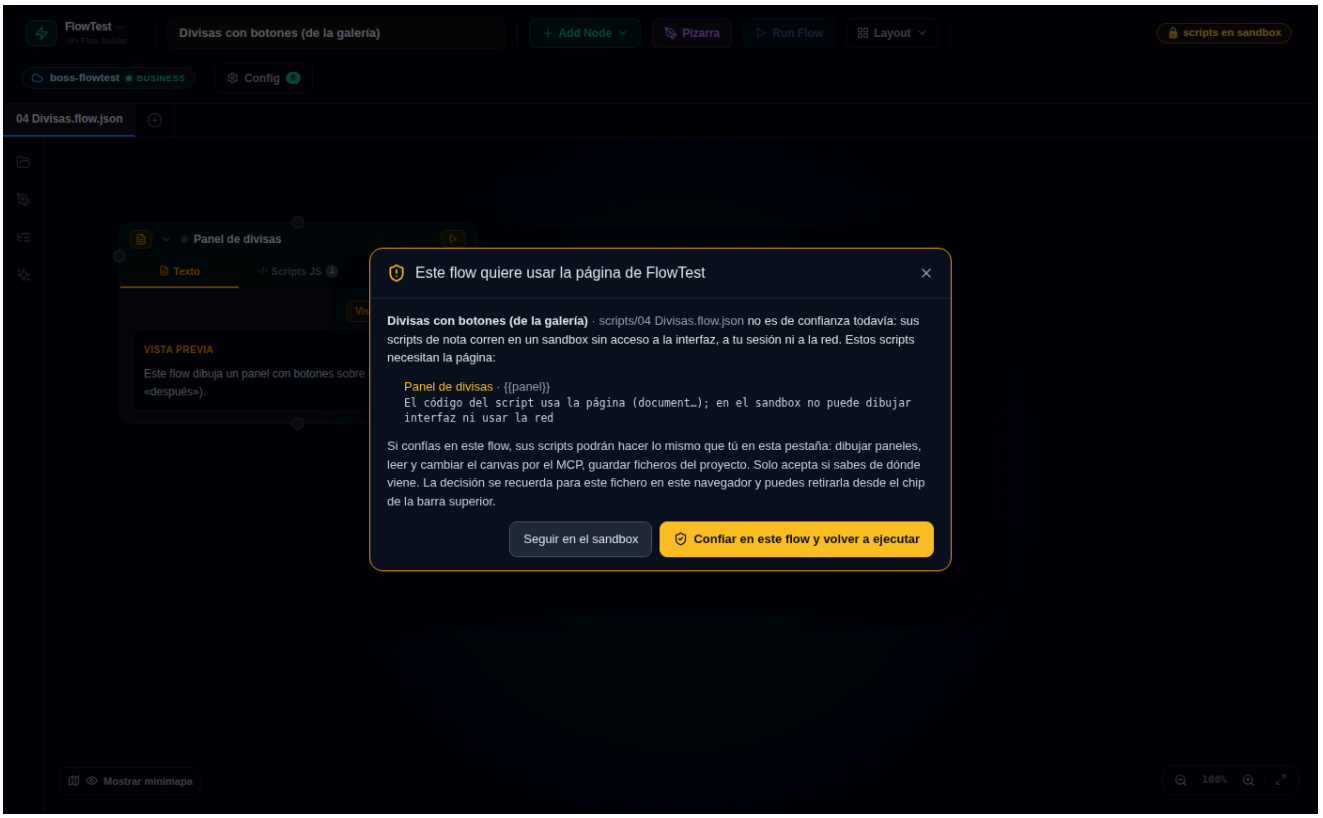
Un script de nota es código. Hasta la 5.14 corría siempre en la página, así que un flow que te pasaran podía hacer en tu navegador lo mismo que tú: leer y cambiar tu canvas por el MCP, guardar ficheros del proyecto, pintar cualquier cosa sobre la app. Desde la 5.15 la app distingue dos casos:

- 🛡️ **Flows de confianza**: los que nacen en esta pestaña (Nuevo flow, los que construye el asistente de IA o el MCP con `flow_create`) y los que marcas tú. Sus scripts corren en la página, como siempre: pueden dibujar interfaz y llamar al MCP.
- 🔒 **Todo lo demás** — abierto desde el panel Proyecto (disco o cloud), «Abrir .flow.json…», importado o de la galería: sus scripts corren en un **sandbox**: un iframe de origen opaco (sin

cookies ni sesión, sin el DOM de la app, sin `localStorage`), con una CSP sin red (ni `fetch`, ni XHR, ni WebSocket) y dentro de un Worker con tope de tiempo (5 s: un bucle infinito se corta y no congela la app). El script recibe una **copia** de `vars`, devuelve su valor y nada más. Para los scripts que solo calculan (`return Number(vars.total) > 0 ? 'OK' : 'VACÍO'`) no cambia nada.

El **chip de la barra superior** dice en qué estado está la pestaña: 🔒 **scripts en sandbox** o 🛡️ **flow de confianza**. Clic para cambiarlo (al conceder confianza pide confirmación; solo aparece en pestañas de flow, no en documentos).

Cuando un script necesita la página. Si el código de un script de un flow no confiable usa `document`, `window`, `fetch`, `localStorage`, `Notification`… la app lo detecta **antes de ejecutarlo** (5.15.1) y pregunta:



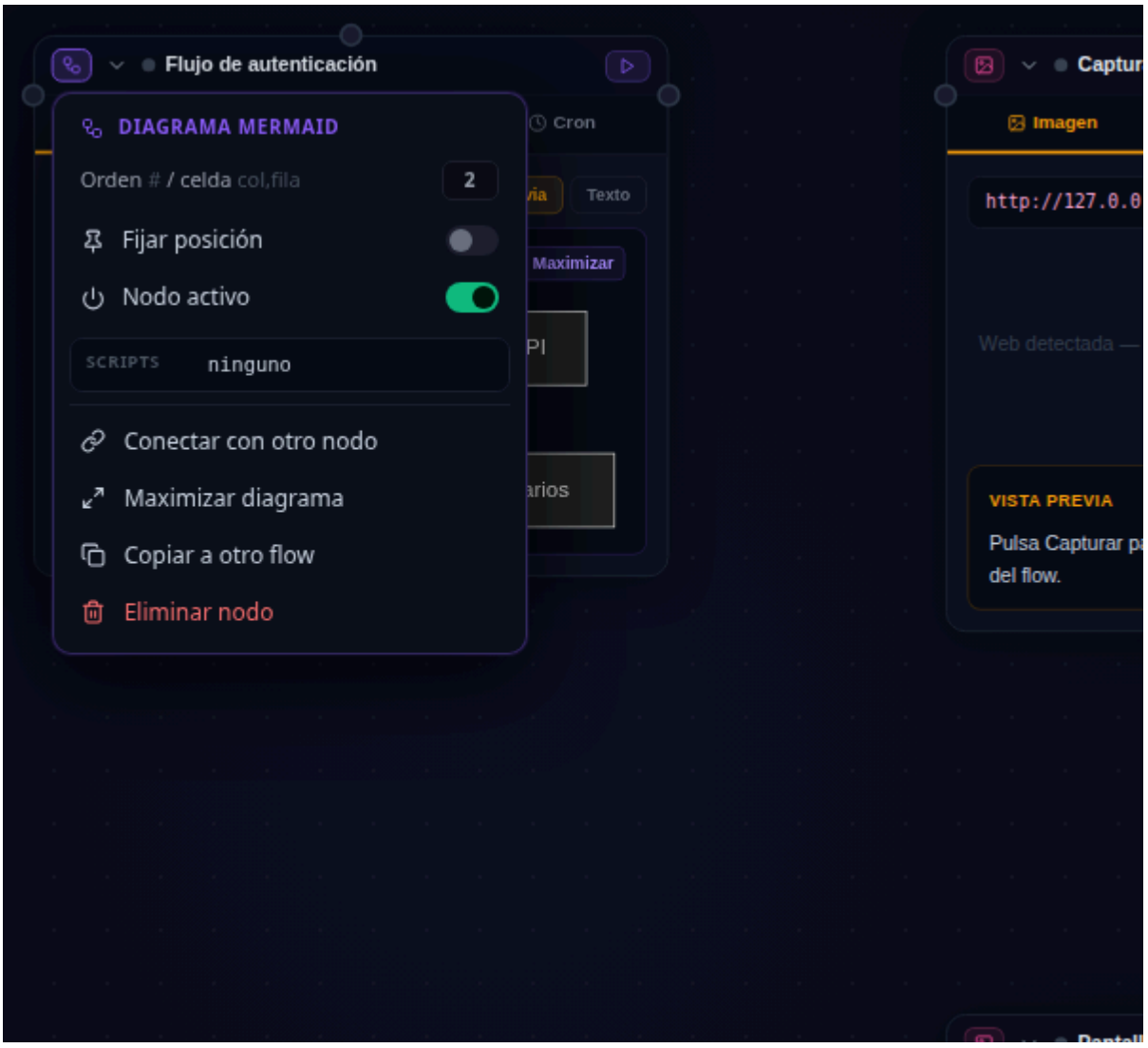
- **Confiar en este flow y volver a ejecutar**: el flow pasa a ser de confianza, el script se ejecuta en la página y a partir de ahora todos los de ese flow también.
- **Seguir en el sandbox**: ese script (y los que pidan la página en la misma pestaña) devuelven un error explicativo en la nota, y no se vuelve a preguntar hasta que reabras el flow o toques el chip. Un script que ya corría en el sandbox y falla intentando usar la red o el DOM también dispara la pregunta.

La decisión se guarda **por fichero y por navegador** (`localStorage`, clave `local:` o `cloud:` + ruta), nunca dentro del `.flow.json`: compartir el flow no comparte tu confianza, y la misma ruta en local y en el cloud se decide por separado. Retirarla es un clic en el chip.

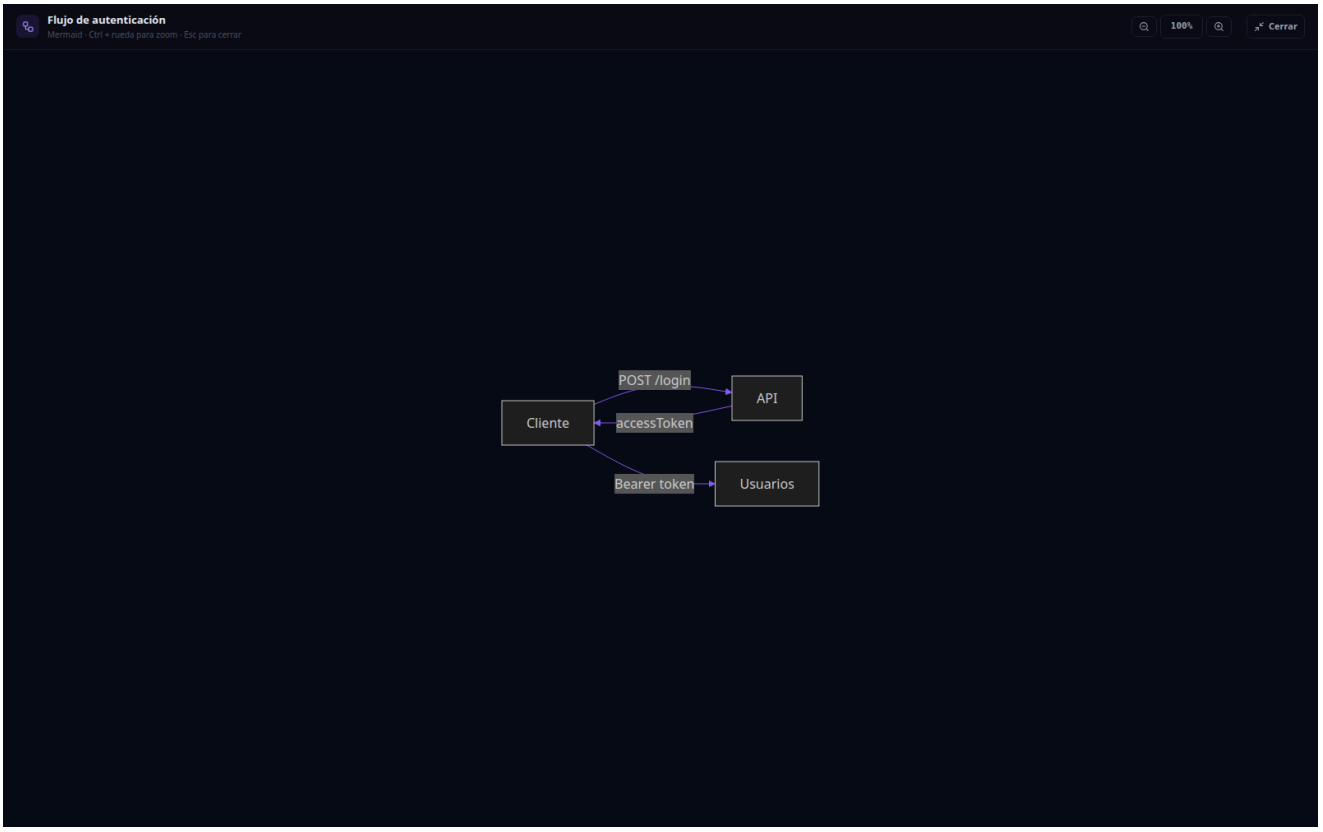
Cuándo confiar. Acepta si sabes de dónde viene el flow: lo hiciste tú, lo hizo tu equipo, o es de la galería y lo has leído. Un script que dibuja y sigue el contrato de arriba es una herramienta; el mismo script en un flow ajeno puede llevar cualquier cosa. En el servidor (CLI, móvil, webhooks, monitores) los scripts siguen corriendo en el sandbox `vm` de la 5.14.1, sin acceso a la página ni a los secretos del contenedor — y allí no se pregunta nada: un script no puede dibujar sin navegador.

Diagrama Mermaid (Add Mermaid, violeta)

Escribe código Mermaid y el diagrama se renderiza **en vivo**. Admite `{{variables}}` dentro del código — un diagrama que se actualiza con datos del flujo.



🆕 El botón **Maximizar** de la cabecera de la vista previa abre el diagrama **a pantalla completa**: zoom del 25 % al 400 % con los botones, `Ctrl` + rueda o `Ctrl` `+` / `-` / `0`, con scroll real cuando no cabe; `Esc` o **Cerrar** vuelve al canvas.

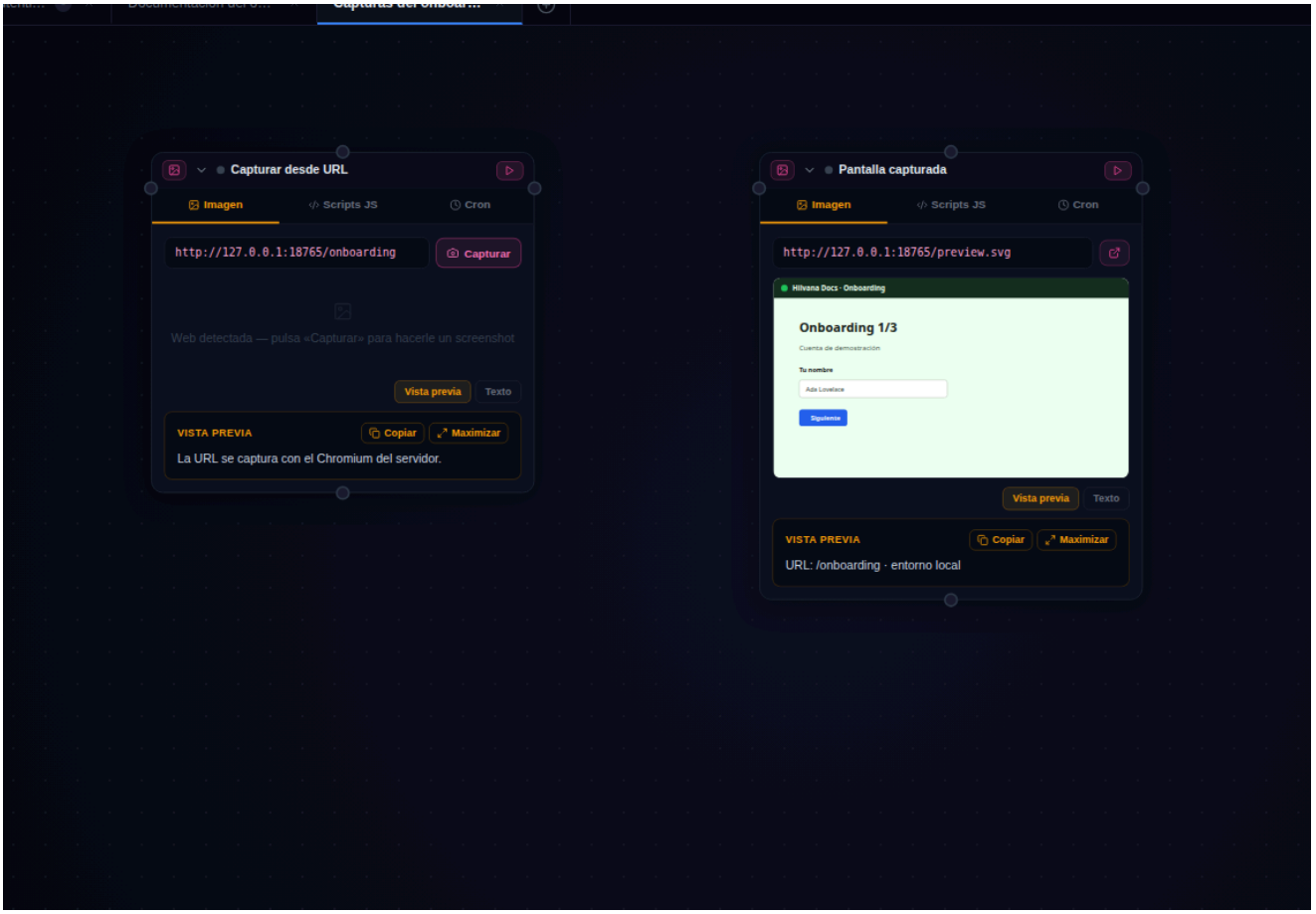



```
graph TD
  A[Cliente] --> B[API]
  B --> C[(BBDD)]
```

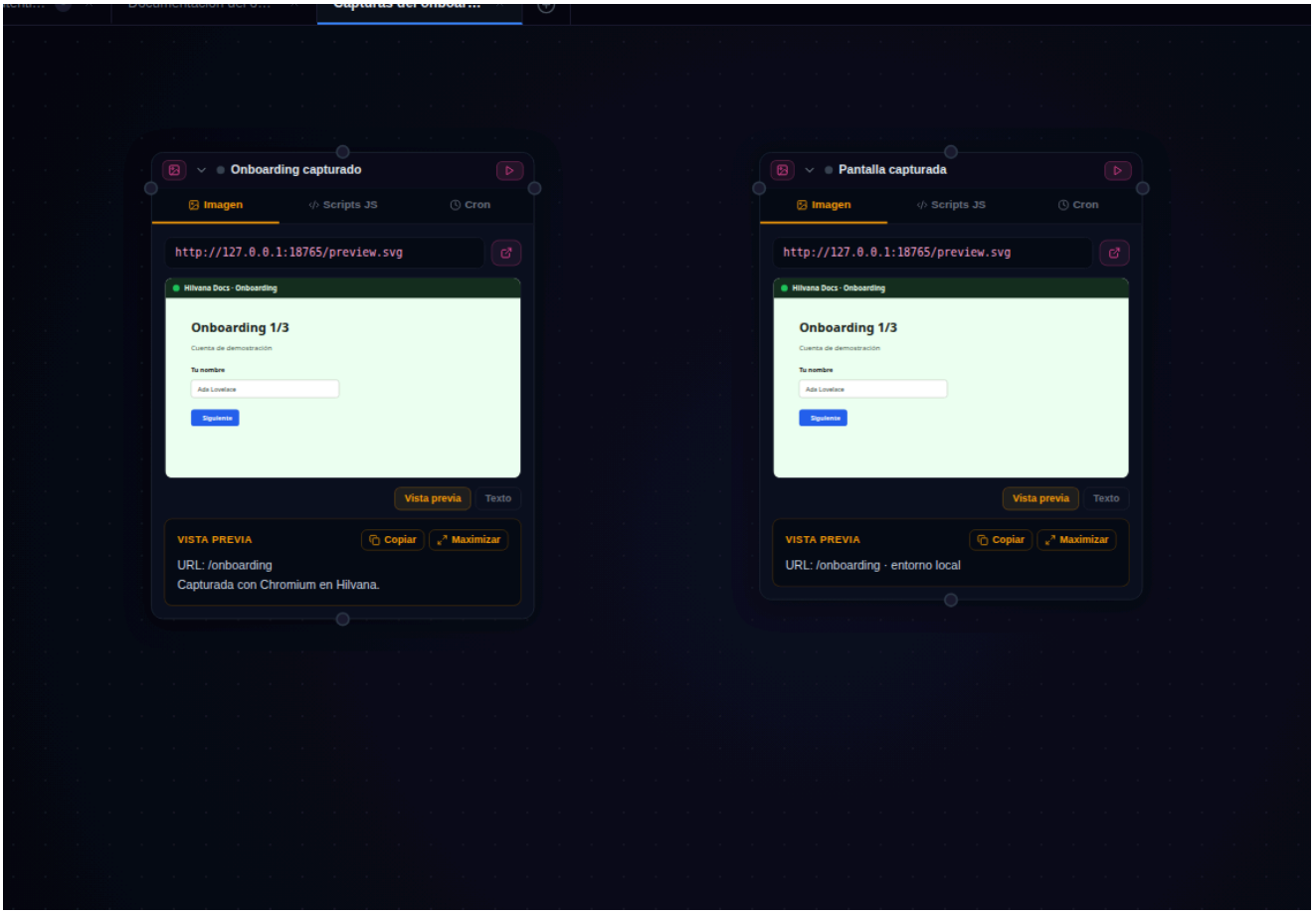
Nodo Captura (Add Screenshot, rosa) 🆕

Muestra una **imagen** en el canvas. El campo de arriba acepta:

- La ruta de un screenshot generado por la app (`/flow-assets/...`) — se muestra directamente, con enlace para abrirla a tamaño completo.
- **La URL de una web** → aparece el botón 📸 **Capturar**:



Al pulsar ① **Capturar**, el server abre esa web con Chrome (headless), le hace un screenshot, lo guarda en `flows/assets/captures/` y la cajita pasa a mostrarlo — rellenando las notas con URL, título y fecha si estaban vacías:



Las notas de la captura (URL, fecha, qué se ve) van en el cuadro de texto de abajo y se guardan con el flow.

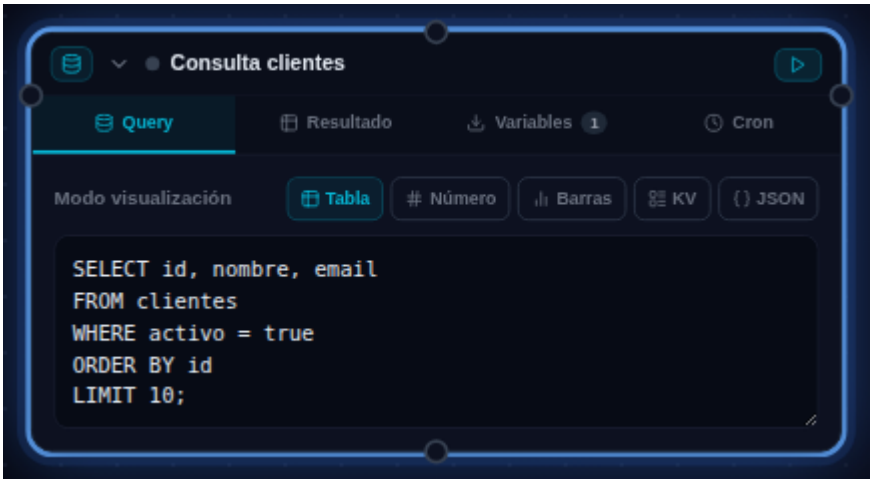
> **Nota: De dónde salen las capturas «buenas»** Este botón hace una foto puntual. Para documentar **flujos completos** (pantalla a pantalla con sus llamadas HTTP) están el modo Live y flow-explore, que crean estos nodos Captura automáticamente.

> **Nota: Chrome**: el botón Capturar usa el Chrome de la máquina donde corre el server. Desde la **4.5.0 la imagen Docker trae Chromium**, así que funciona también dentro del contenedor.

🗄 05 · El nodo SQL

> Revisado para Hilvana **6.2.1** · capturas: 114.

Consultas a base de datos **dentro del flujo**, en el mismo orden topológico que las cajitas HTTP: siembra datos antes de llamar a la API, o verifica después lo que la API escribió.

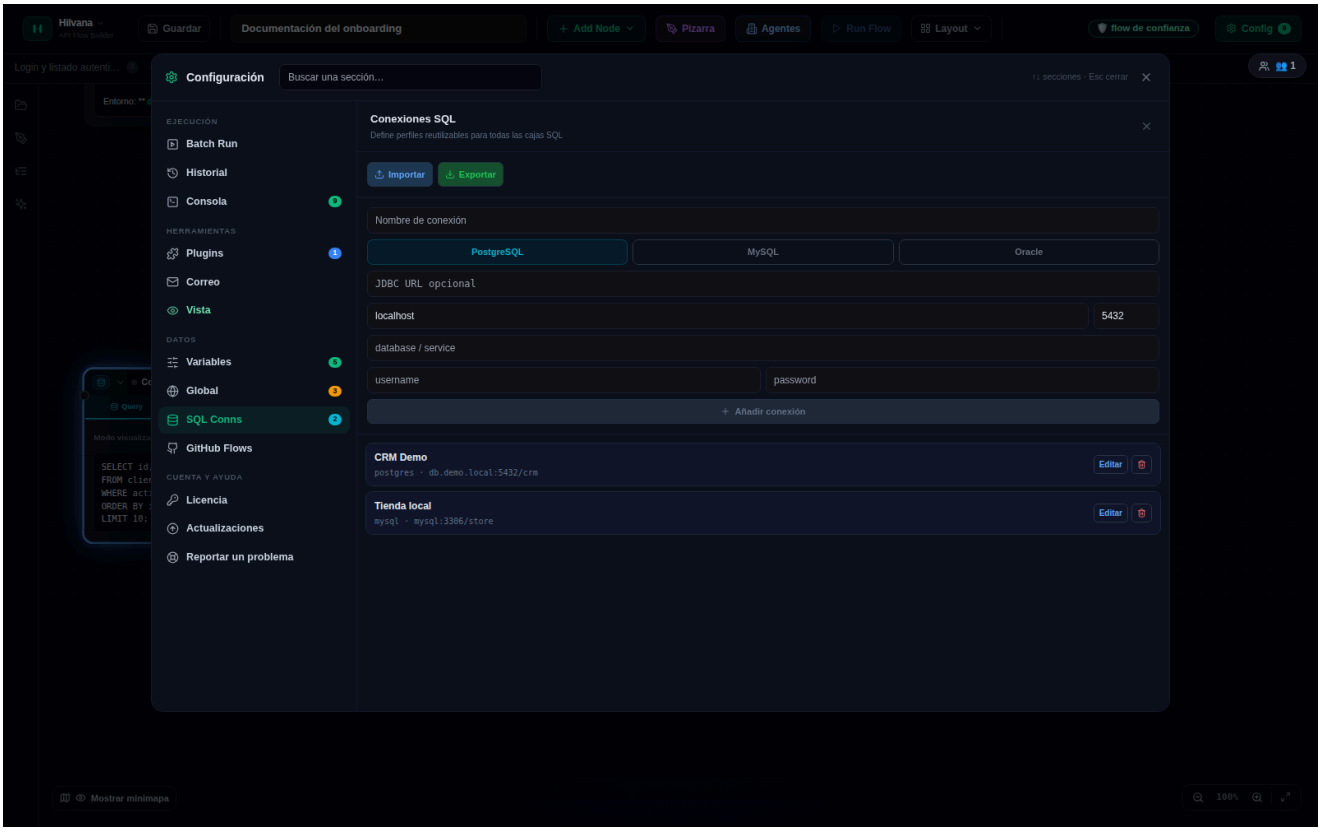


Configuración

- **Motor**: Postgres, MySQL u Oracle (drivers `pg`, `mysql2`, `oracledb` en el server — la query nunca sale del backend, sin CORS ni credenciales en el navegador).
- **Conexión**: campos host/puerto/BD/usuario/contraseña, una URL JDBC (`jdbc:postgresql://...`), o un **perfil guardado**.
- **Query**: admite `{{variables}}` tanto en la consulta como en la conexión.

Perfiles de conexión (SQL Conns)

El botón **SQL Conns** de la toolbar guarda conexiones reutilizables — defines «BBDD staging» una vez y los nodos la referencian:



En disco, los perfiles para el CLI viven en `flows/sql-connections.json` (misma estructura).

Resultado y modos de vista

El resultado se muestra según el **display mode**: tabla, número suelto (para `SELECT count(*)`), barras, clave-valor o JSON crudo.

Extraer variables

Igual que los request pero **por columna**: `varName + columna + fila` → el valor queda disponible como `{{varName}}` para los nodos siguientes. Ejemplo clásico:

1. SQL: `SELECT id FROM users WHERE email='test@x.com'` → extrae `userId`.
2. Request: `curl http://api/users/{{userId}}/orders`.

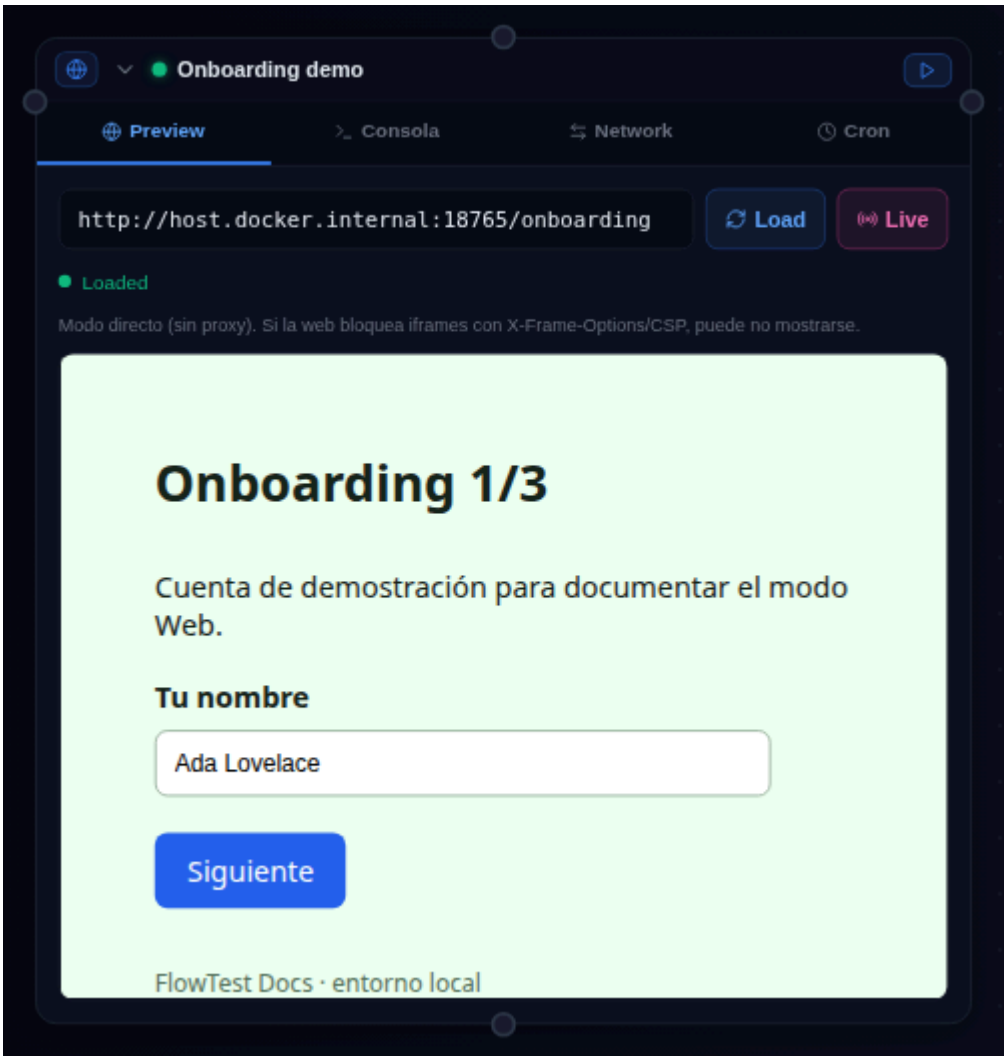
> **Consejo: También en el CLI** `flow-run` ejecuta los sqlNodes exactamente igual (perfiles vía `--sql-connections` o el `sql-connections.json` más cercano). Ojo: eso significa que un batch (`--dir flows`) **toca las BBDD de verdad** — usa `--skip-sql-nodes` si no quieres.

🌐 06 · El nodo Web y el modo Live

> Revisado para Hilvana **6.2.1** · capturas: 99.

El nodo azul muestra una web en el canvas. Tiene dos modos: **Preview** (un iframe de toda la vida) y el **modo Live** 🆕, que es la joya: navegas una web *de verdad* dentro de la cajita y documentas cada paso con un click.

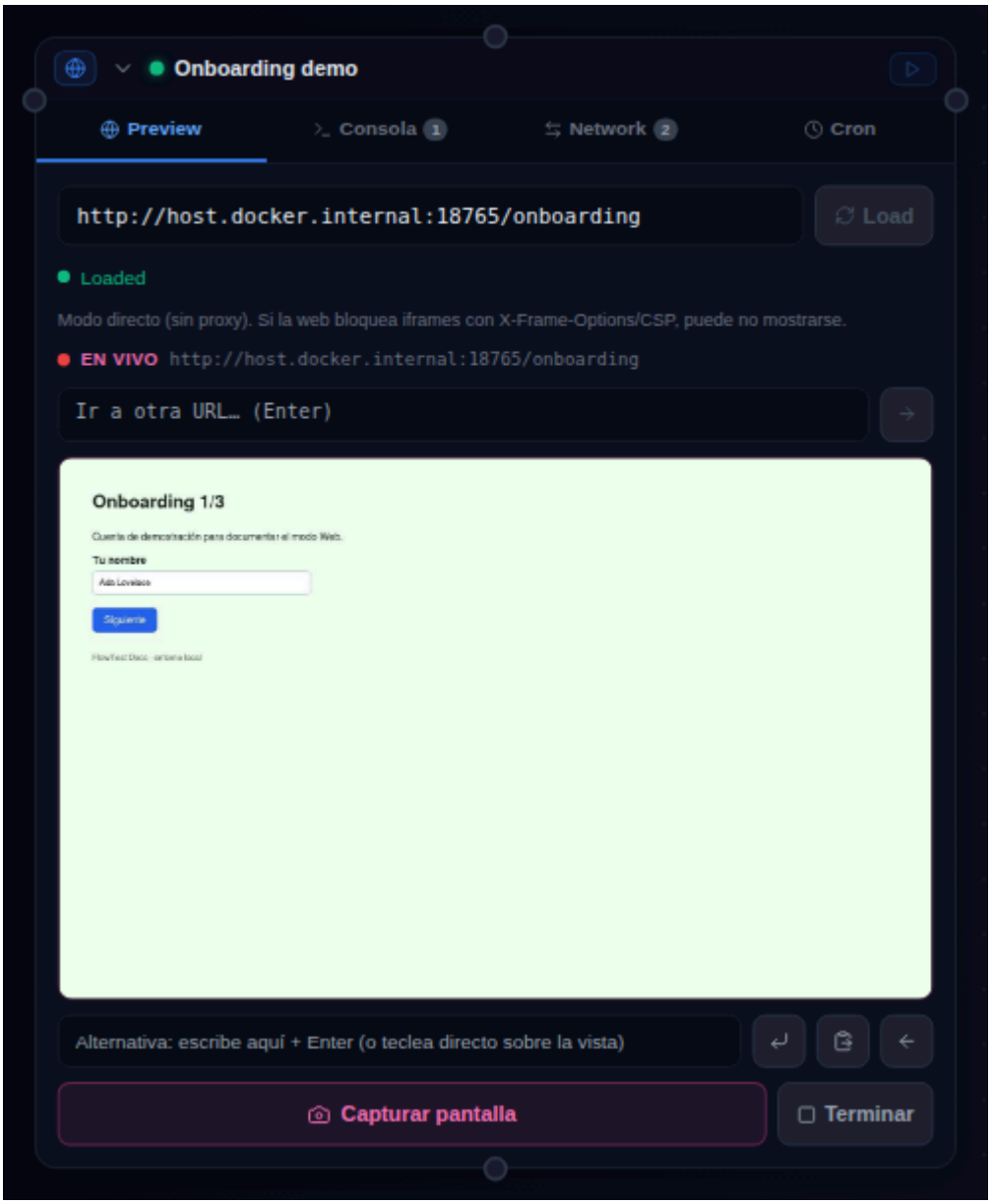
El nodo y sus botones



Nº	Elemento	Qué hace
①	URL	La web a mostrar (admite `{{variables}}`)
②	Load	Carga/recarga el **iframe** clásico. Ojo: muchas webs bloquean iframes (X-Frame-Options/CSP)
③	**Live** 🆕	Abre la web en una **sesión navegable** en el server (ver abajo)
④	⤢ Maximizar 🆕	La cajita a pantalla completa (Esc para volver) — imprescindible con Live

El modo Live: navegar y capturar

Al pulsar **Live**, el server abre la URL con su propio Chrome (headless) y la cajita se convierte en un **navegador remoto**: ves frames en vivo y todo lo que haces se reenvía a esa sesión. Como es *un* navegador real con *una* sesión, **login, cookies y estado se conservan** entre pasos — justo lo que necesita un onboarding o un wizard.



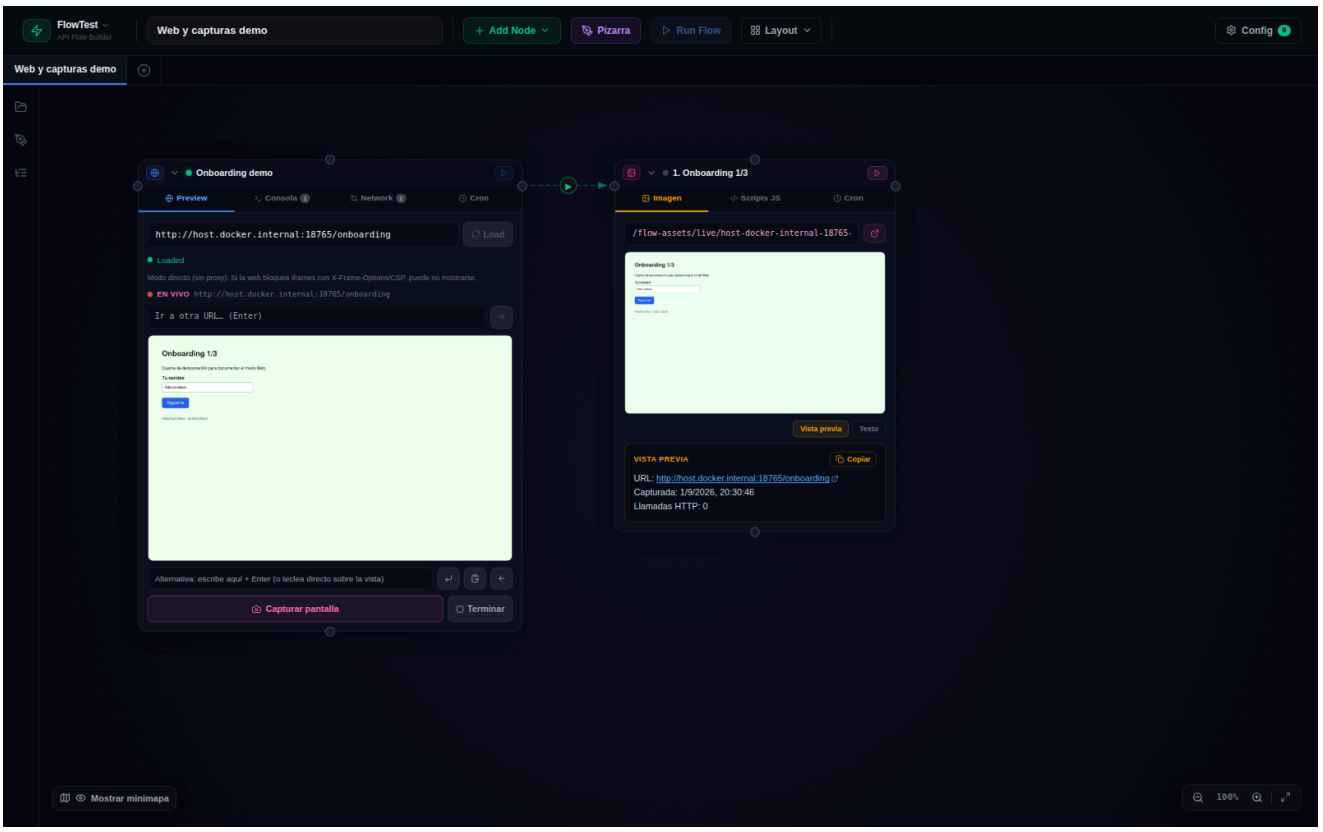
Nº	Elemento	Qué hace
①	La vista	**Clica directamente sobre ella**: el click se reproduce en la web (reescalado exacto). La rueda hace scroll. Arriba, el punto rojo EN VIVO y la URL actual
②	Campo de texto	Clica un campo en la vista para enfocarlo, escribe aquí y **Enter**: se teclea en la web
③	⏎	Pulsa Enter en la web (enviar formularios)
④	←	Volver atrás
⑤	📸 **Capturar pantalla**	La estrella: crea un **nodo Captura** con el screenshot del momento **+ las cajitas curl de las llamadas HTTP** ocurridas desde la captura anterior

Nº	Elemento	Qué hace
⑥	Terminar	Cierra la sesión del server

El flujo de trabajo para documentar un proceso

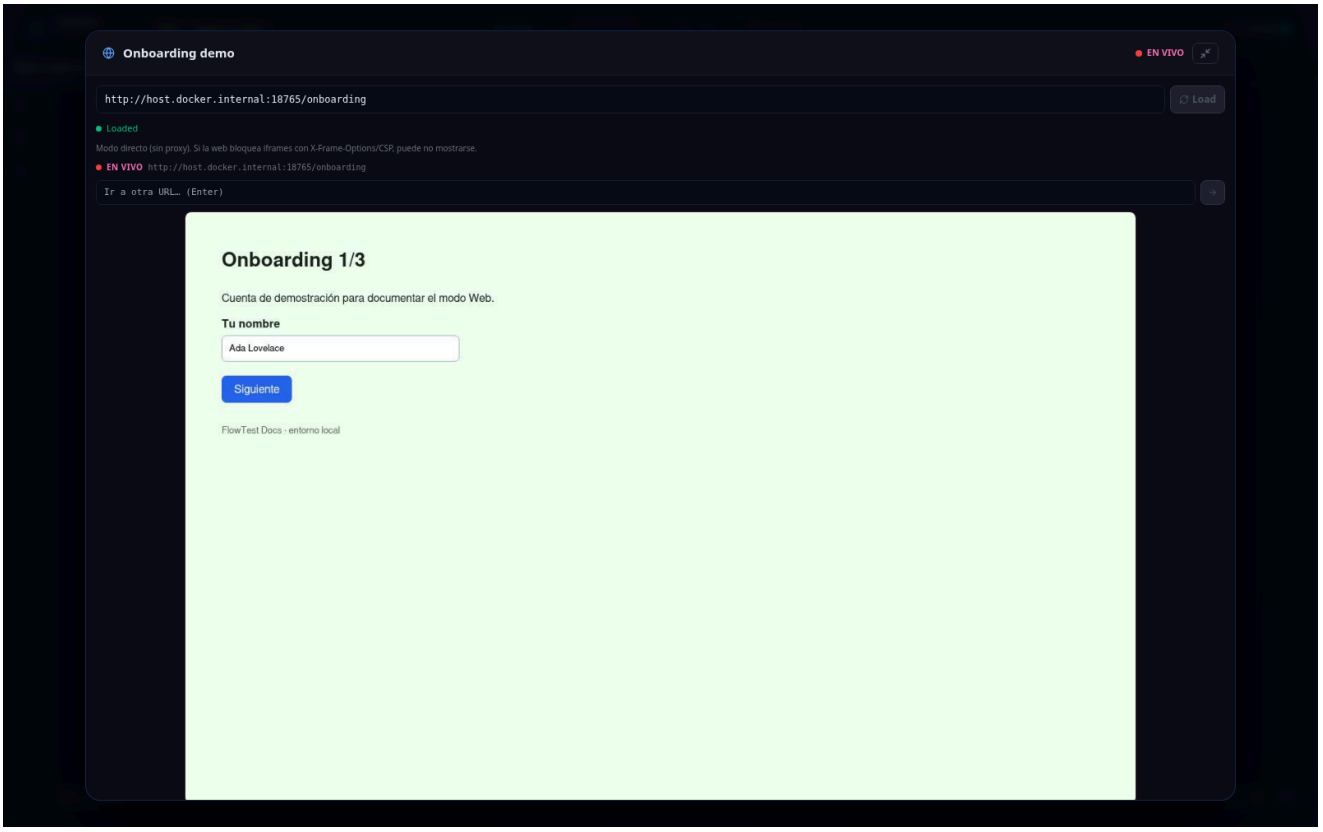
1. **Add Web** → URL → **Live** (y ⤢ Maximizar para trabajar cómodo).
2. Navega/rellena el paso: click en el campo → escribir + Enter → **cuando la pantalla muestre lo que quieres documentar** → 📸 **Capturar pantalla**.
3. Avanza (click en «Siguiente», enlace, etc.) y repite.
4. **Terminar** al acabar.

Cada captura aparece **encadenada a la anterior** (web → 1 → 2 → 3…) con sus llamadas colgando debajo como cajitas colapsadas re-ejecutables:



> **Consejo: Captura ANTES de avanzar** Si capturas después de pulsar «Siguiente», la foto sale del paso nuevo (vacío). Captura con el formulario relleno y luego avanza — la llamada del «Siguiente» caerá en la ventana de la captura siguiente.

Maximizada, la vista Live se ve a tamaño casi real:



Detalles que conviene saber

- **Seguridad**: en las cajitas generadas, el header `Authorization` se enmascara como `{{authToken}}` (variable creada en el flow) y las cookies **no se escriben**. Nunca quedan tokens reales en el `.flow.json`.
- Las sesiones Live inactivas **se cierran solas a los 10 minutos**.
- Los screenshots van a `flows/assets/live/<sesión>/` y se sirven como `/flow-assets/...`.
- Requiere Chrome en la máquina del server → desde la **4.5.0 la imagen Docker lo trae** (Chromium embebido). Para webs tras un login/SSO (Cloudflare Access, etc.), pasa las credenciales con `FLOW_CAPTURE_COOKIES` / `FLOW_CAPTURE_HEADERS` — ver 11-docker.md.
- Desde la **4.6.0** el Live usa un **perfil de Chromium persistente** (`flows/.chrome-profile`): haces el login/OTP una vez y sobrevive al cierre por inactividad e incluso a un restart del contenedor. `FLOW_CHROME_PROFILE=off` para perfil efímero. ⚠ Ese directorio guarda cookies vivas — no lo compartas.
- Desde la **4.7.0** se teclea **directamente sobre la vista**: clica un campo y escribe (Enter/Tab/flechas incluidas); **Ctrl+V pega el portapapeles** (el OTP del correo) y Ctrl+letra llega a la web como atajo. La vista marca el foco con «teclado → web». ⚠ Lo tecleado viaja en claro a tu server local y no se loguea; para contraseñas usa mejor Ctrl+V.
- También desde la 4.6.0: **barra de URL** en la sesión (ir a otra página sin cerrarla), botón de **pegar del portapapeles** (el OTP sin teclearlo carácter a carácter), y los **fallos de navegación se pintan** en la tarjeta (`errorCode` + dónde se quedó la sesión) en vez de quedarse mudos en los logs.
- Si el modo directo (iframe) es imposible (`X-Frame-Options` / `frame-ancestors`), la tarjeta lo dice **nombrando al host que bloquea** — que tras un redirect puede ser el SSO (p. ej. `*.cloudflareaccess.com`), no tu URL — y ofrece abrir en Live.

- Desde la **4.8.0** el nodo tiene pestañas **Consola** y **Network**: con la sesión Live abierta ves los `console.*` y errores de la página, y todo su tráfico (método, URL, status, tipo, duración — fallos `net::ERR_*` incluidos). Solo método/URL/status: nunca headers ni cuerpos. Contadores en vivo en las pestañas y botón Limpiar. Perfecto para depurar por qué un paso no hace la llamada que esperabas antes de capturarlo.
- Si una captura sale sin datos o sin llamadas, revisa el orden del tip de arriba.

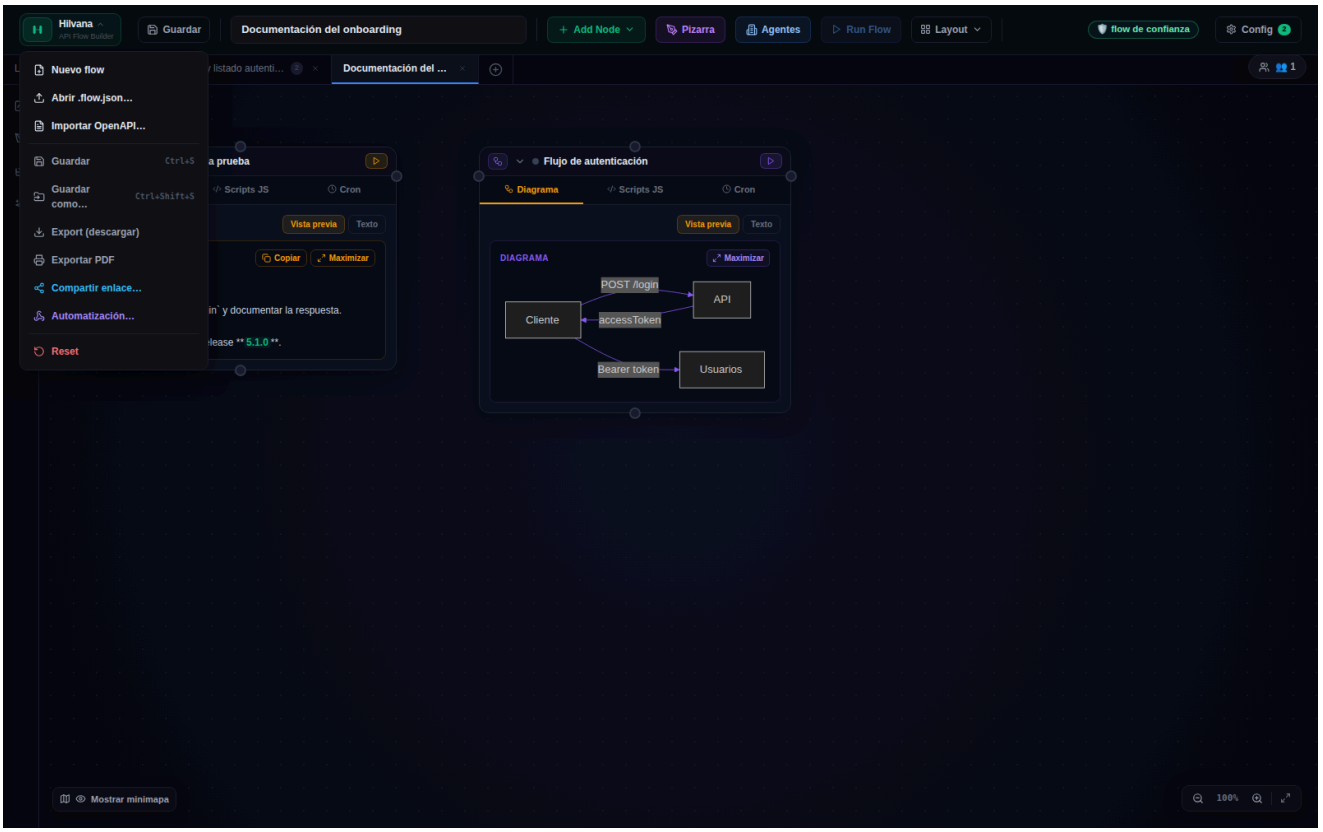
🕷 07 · Documentar una web — flow-explore

> Revisado para Hilvana **6.2.1** (herramienta de terminal, sin pantalla propia).

`flow-explore` es el CLI que documenta una web **navegándola**: abre Chrome, hace screenshot de cada pantalla, captura las llamadas HTTP que se producen en cada una, y genera la documentación en dos formatos a la vez:

- **`flows/<slug>.flow.json`** — un canvas documental: un nodo Captura por pantalla + sus cajitas curl re-ejecutables debajo. Se abre con **Abrir** en la web.
- **`flows/assets/<slug>/README.md`** — informe markdown con los screenshots, tabla de llamadas por pantalla y el detalle (curl + bodies) desplegable.

Así queda el flow documental en el canvas:



Modo crawl (automático)

Sigue los enlaces del **mismo origen** hasta el límite indicado. Ideal para el «mapa» de una web pública:

```
cd hilvana
npm run flow:explore -- https://mi-web.com
node cli/explore-web.js https://mi-web.com --max-pages 5 --depth 2 --out mi-web
```

Modo manual (tú navegas)

Para procesos con **sesión**: login, onboarding, wizards. Abre un Chrome visible y navegas tú; la herramienta lo registra todo:

```
npm run flow:explore -- https://mi-web.com/onboarding --manual --filter /api/
```

- Cada **cambio de URL** crea una pantalla automáticamente.
- Para pasos que **no cambian la URL** (formularios multi-paso, modales): botones 📸 **Capturar** / 📸 **Con nombre** de la esquina inferior derecha (atajos F8/F9). «Con nombre» te pide el título del paso.
- **Cerrar el navegador** (o ENTER en la terminal) termina y genera la documentación.

Flags útiles

Flag	Qué hace
`--manual`	Modo manual (Chrome visible, sesión única)
`--filter <substr>`	Solo documenta llamadas cuya URL contenga eso (repetible). P. ej. `--filter /api/`
`--max-pages N` / `--depth N`	Límites del crawl (defecto 8 páginas, profundidad 2)
`--all-requests`	Captura también estáticos (css/js/img), no solo xhr/fetch
`--max-calls N`	Máximo de cajitas por pantalla (defecto 15; el resto se anota)
`--full-page`	Screenshots de la página completa, no solo el viewport

Flag	Qué hace
`--out <slug>`	Nombre base de la salida (defecto: el host)
`--chrome <ruta>`	Ejecutable de Chrome (o variable `FLOW_CHROME`)
`--viewport WxH`, `--settle ms`, `--timeout ms`, `--headful`	Ajustes finos

Seguridad

Los headers sensibles se **enmascaran** antes de escribir nada a disco: `authorization` → `{{authToken}}` (variable en el flow, con placeholder), y `cookie` / `x-api-key` /etc. se eliminan. El informe markdown tampoco lleva tokens.

> **Nota: Dónde funciona** Necesita `puppeteer-core` y un Chrome. Desde la **4.5.0 la imagen Docker trae ambos**: el modo crawl funciona con `docker exec flow node cli/explore-web.js https://mi-web.com`. `--manual` / `--headful` abren un Chrome visible → necesitan pantalla, se ejecutan desde el repo (`npm install`). La carpeta `flows/assets/` está en `.gitignore`: la documentación se regenera cuando quieras.

🧰 08 · Paneles y utilidades

> Revisado para Hilvana **6.2.1** · capturas: 101 · 105 · 109 · 446 · 447 · 448 · 449 · 450 · 451 · 117.

> **Consejo: ¿Dónde está cada cosa? Proyecto**, **Pizarra** y **Guía de nodos** viven en la **barra lateral de iconos**; Batch Run, Historial, Consola, **Vista**, GitHub, Variables, Global y SQL Conns se abren desde **Config** (los paneles laterales aparecen en la barra mientras están abiertos; un panel a la vez); y las utilidades de organización desde el desplegable **Layout**. Ver 01 La pantalla principal.

Configuración con menú lateral 🆕 (5.22)

Config (arriba a la derecha) abre un panel con las secciones a la izquierda, agrupadas — **Ejecución** (Batch Run, Historial, Consola), **Herramientas** (Plugins, Correo, Vista), **Datos** (Variables, Global, SQL Conns, GitHub Flows) y **Cuenta y ayuda** (Licencia, Actualizaciones, Reportar un problema) — y la elegida se abre a la derecha, sin salir del panel. Arriba hay un buscador; ↑↓ cambia de sección y **Esc** cierra. Recuerda la última sección que abriste.

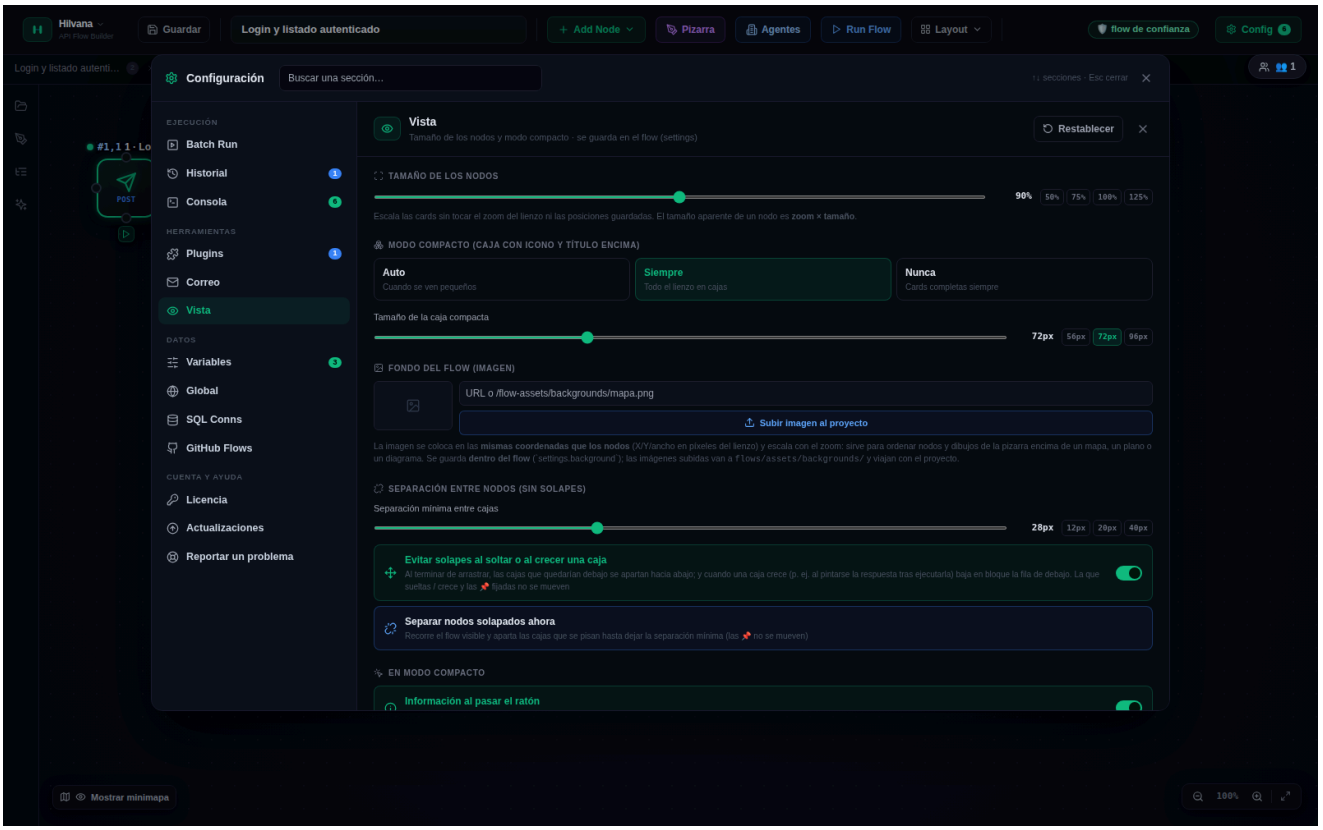
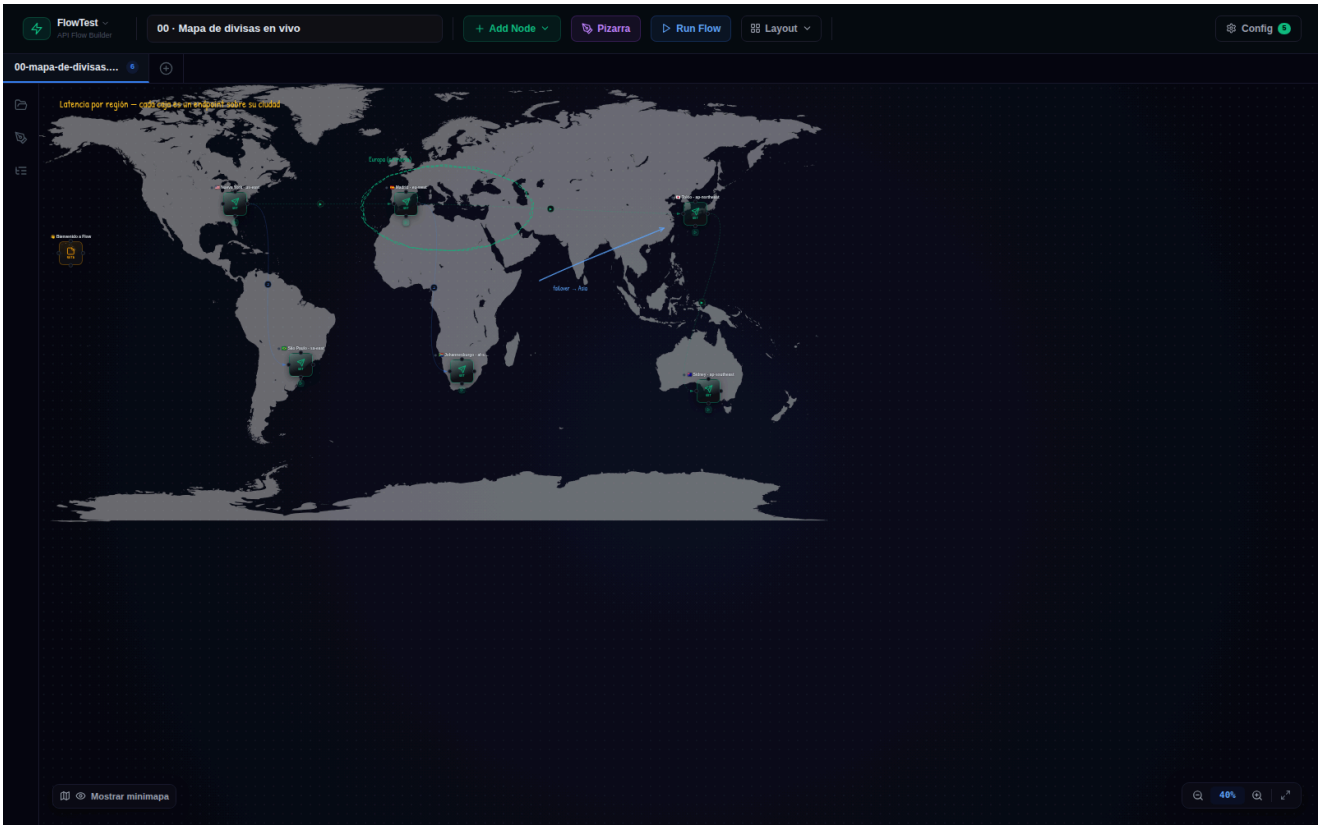
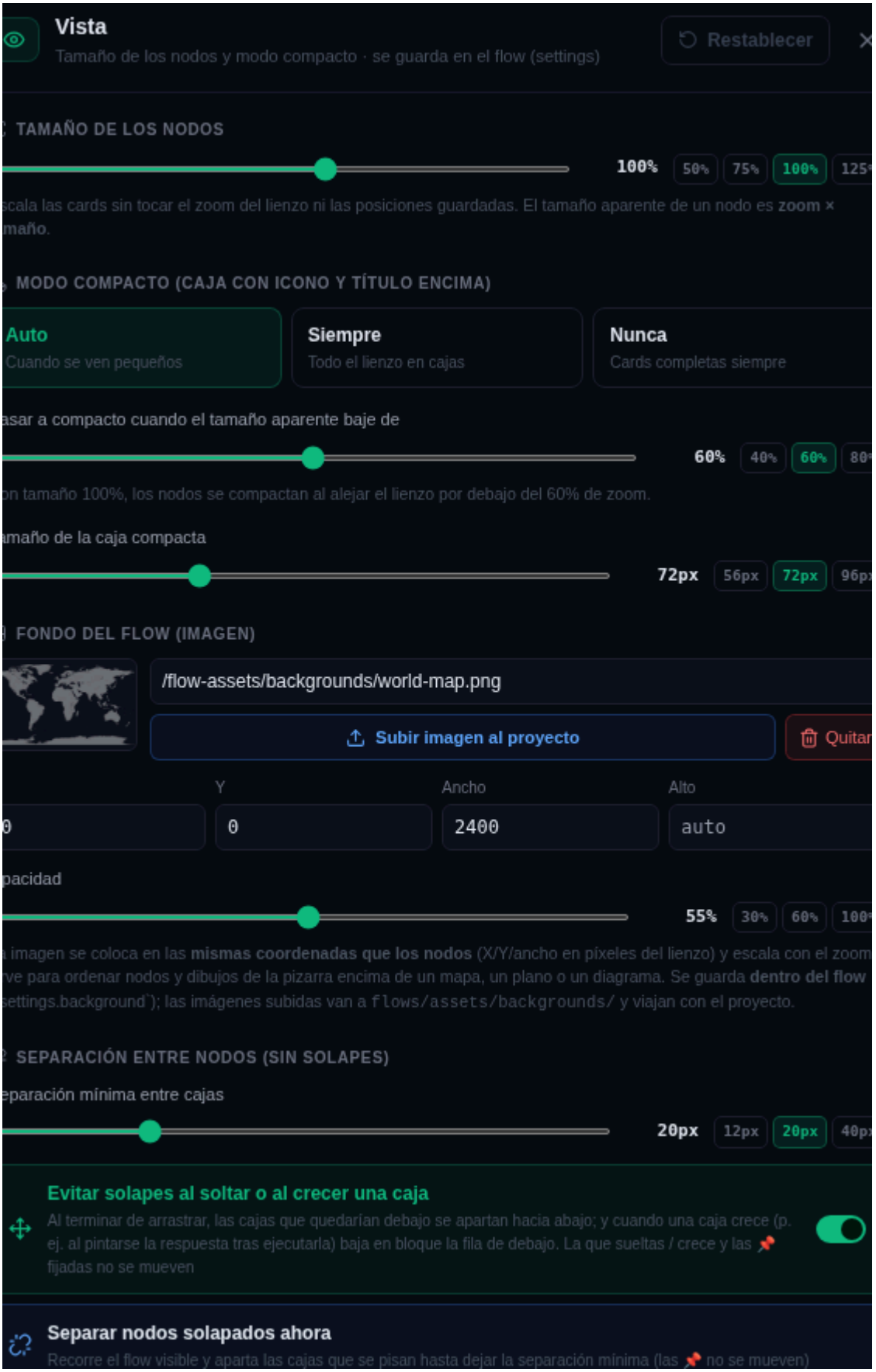


Imagen de fondo del flow 🆕 (4.49)

Config ▸ Vista ▸ Fondo del flow (imagen) pone una imagen detrás del lienzo — un mapa del mundo, el plano de una oficina, un diagrama de arquitectura… — para **ordenar nodos y dibujos de la pizarra encima de ella**.



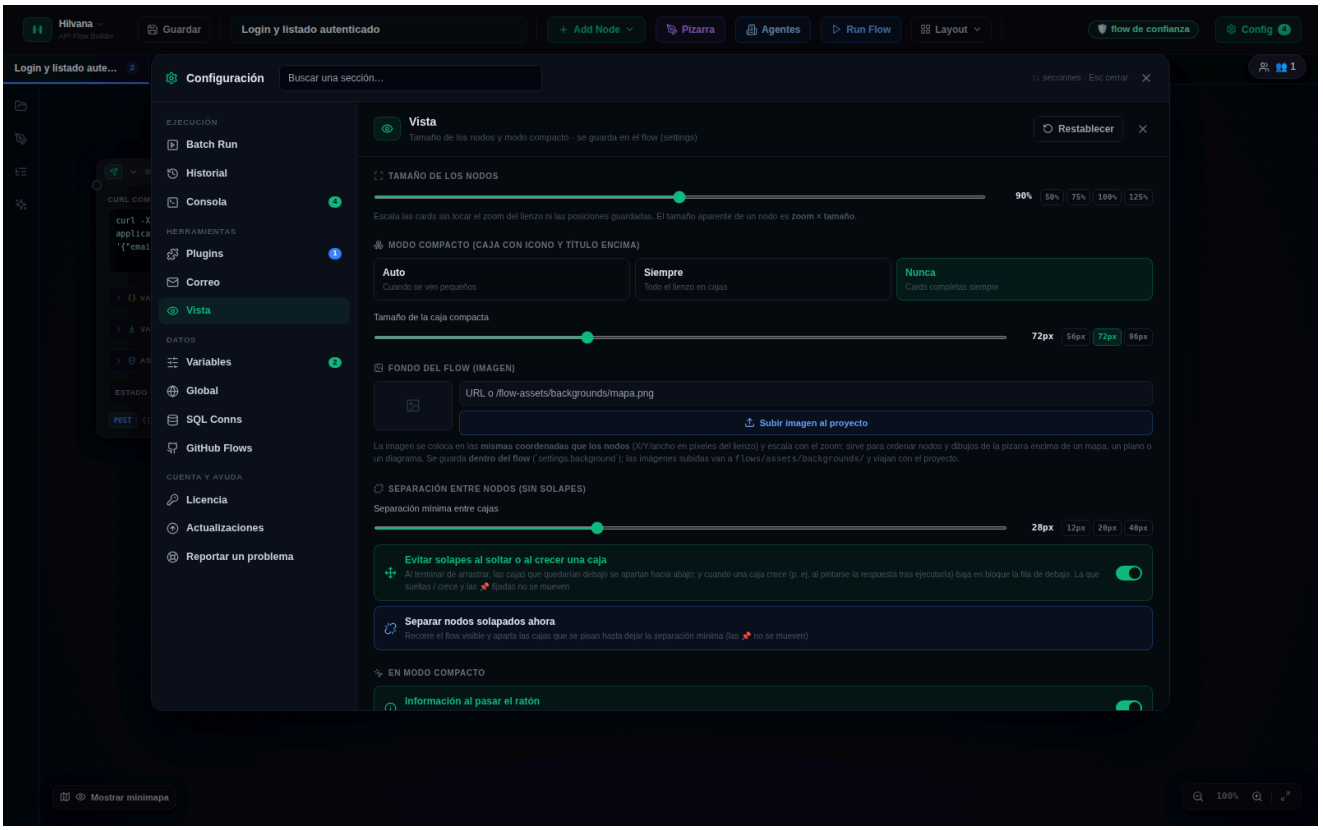
- **Subir imagen al proyecto** la guarda en `flows/assets/backgrounds/` (viaja con la carpeta del proyecto y se sirve como `/flow-assets/backgrounds/…`); también vale una URL http(s).
- La imagen se coloca en las **mismas coordenadas que los nodos** (X, Y y ancho en píxeles del lienzo; alto automático por proporción salvo que lo fijes) y **escala con el zoom**: lo que pongas sobre Madrid sigue sobre Madrid al acercar o alejar.
- Se ve también en el **minimapa** (4.50.0), en su posición y tamaño.
- **Opacidad** (10–100 %) para que las cajas se lean bien encima; **Quitar** la elimina.
- Se guarda **dentro del flow** (`settings.background`, ver La configuración vive en cada flow), cuenta para el tamaño del lienzo y sale en «Exportar PDF».
- MCP: `flow_background` (`set` con `imageSrc` / `x` / `y` / `width` / `height` / `opacity`, o `clear`); `flow_state` la devuelve en `settings.background`.



Ejemplo incluido: `mapa-mundial.flow.json` (un endpoint colapsado sobre cada región, con anotaciones de pizarra).

La configuración vive en cada flow 🆕 (4.48)

Hasta la 4.47, **Config ▸ Vista**, **Ocultar conectores** y el estilo de la pizarra eran ajustes del navegador: los mismos para todas las pestañas y perdidos al cambiar de ordenador. Desde la 4.48 **se guardan dentro del propio flow**, en el campo opcional `settings` del `.flow.json`, y **cambian al cambiar de pestaña**:

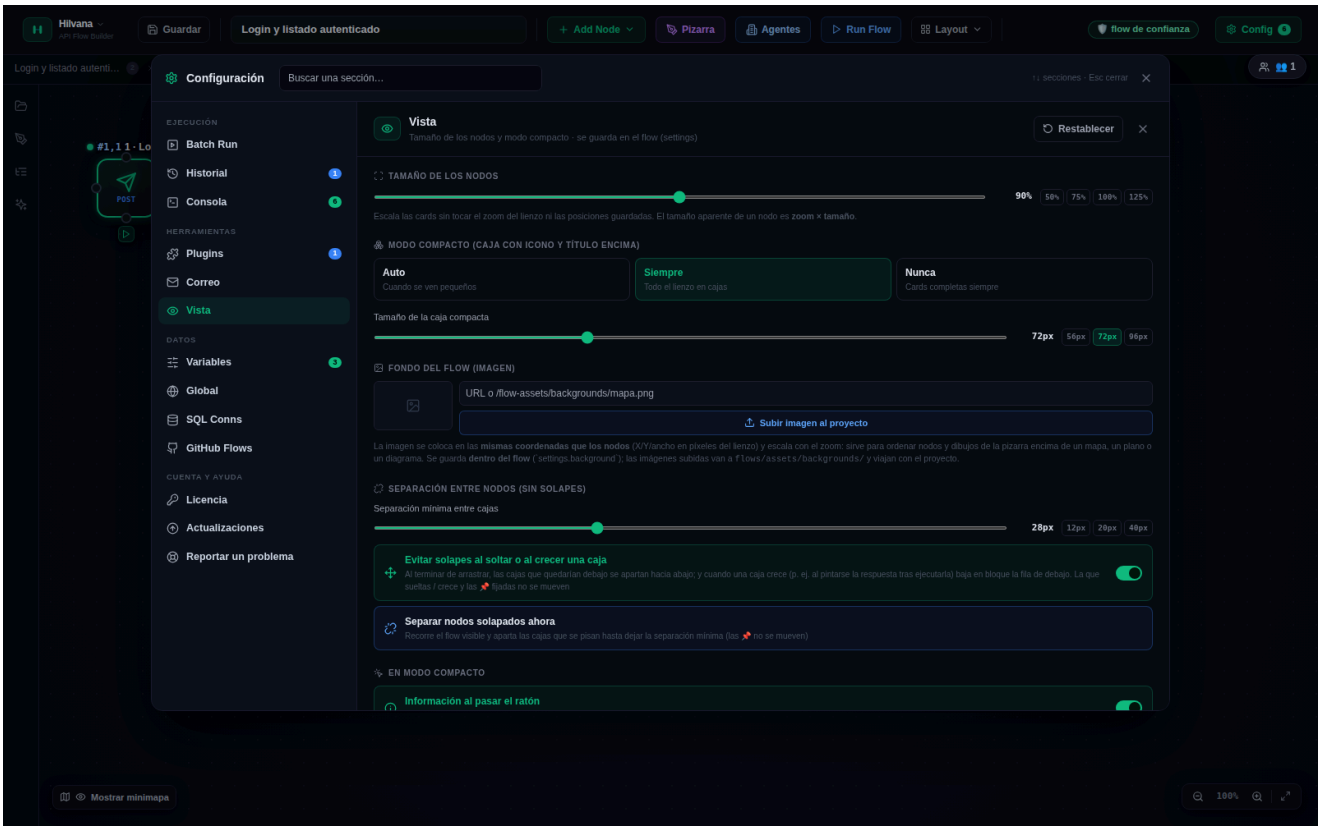


Qué	Dónde se guarda
Config ▸ Vista (tamaño de los nodos, modo compacto, umbral, tooltip, clic abre, separación mínima, evitar solapes)	`settings.view`
Layout ▸ Ocultar conectores	`settings.hideConnections`
Estilo activo de la pizarra (color, grosor, trazo, relleno, opacidad, boceto, fuente, tamaño)	`settings.whiteboardStyle`
Zoom y posición del lienzo	`settings.viewport` (`zoom`, `scrollLeft`, `scrollTop`)

- Cada flow se abre **tal y como lo dejaste**: mismo zoom, misma zona del lienzo, misma vista — también si lo abres desde otro navegador u ordenador (el fichero es la única fuente).
- Cambiar el **zoom o desplazar el lienzo no marca la pestaña como modificada** (no verás el ● ámbar por eso); Vista, conectores y estilo de pizarra sí, como cualquier otro cambio del flow.
- Los flows antiguos (sin `settings`) usan el **último valor que hayas usado en el navegador** como valor por defecto — y un flow nuevo hereda la vista que tenías en el anterior. En cuanto toques un ajuste en ese flow, se le queda grabado a él.
- En pantalla partida cada lado usa la configuración de su propio flow.

- El CLI ignora `settings`; el MCP lo devuelve en `flow_state` y lo escribe con `flow_overwrite` y `view_settings` (que ahora edita la vista del flow activo).

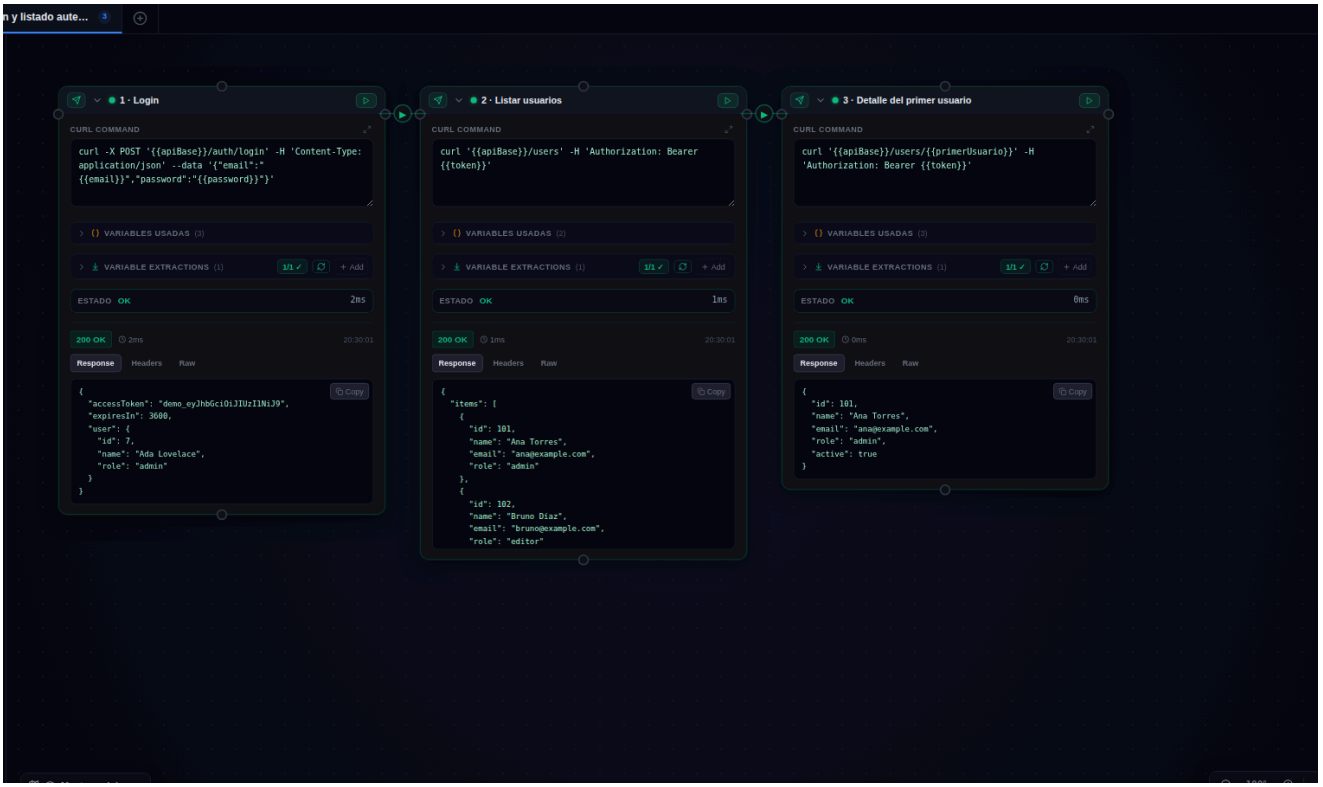
Vista — tamaño de los nodos, modo compacto y separación 🆕 (4.27)



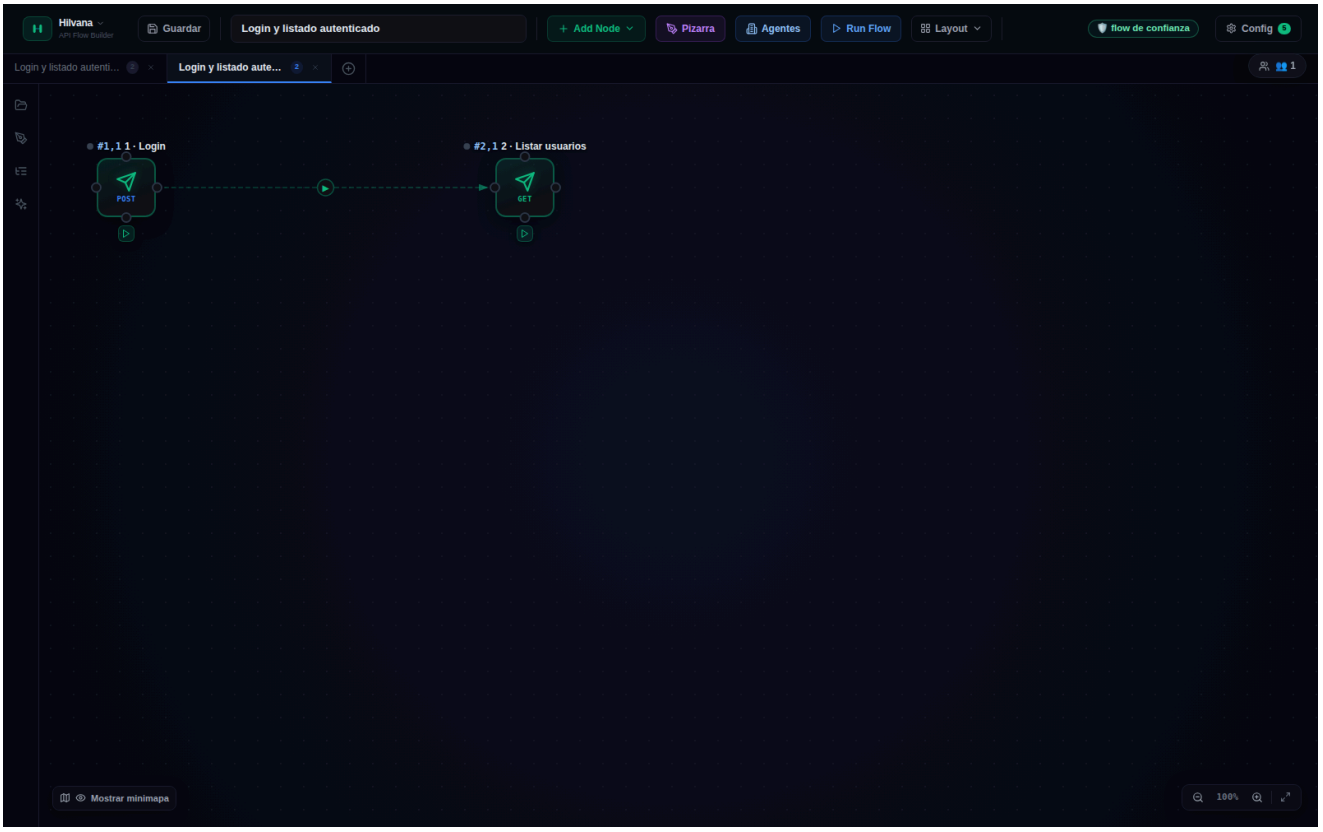
Config ▸ Vista abre un modal (como la Consola) con ajustes **solo de presentación** — desde 4.48 se guardan **dentro del flow** (`settings.view`, ver arriba); antes solo en el navegador:

Ajuste	Qué hace
Tamaño de los nodos (30–150 %)	Escala las cards sin tocar el zoom del lienzo ni las posiciones guardadas. El tamaño aparente de un nodo es `zoom × tamaño`
Modo compacto Auto / Siempre / Nunca	En **Auto**, cuando el tamaño aparente baja del umbral (60 % por defecto) cada nodo se convierte en una **caja con el icono de su tipo** (y su método HTTP, motor SQL o tipo de nota), el **título encima con la redondita de estado**, sus 4 conectores y un ▶ debajo para ejecutarlo. **Siempre** compacta todo el lienzo; **Nunca** deja siempre las cards completas. Tamaño de la caja 48–120 px
Información al pasar el ratón	Tooltip con método y URL, query, conexión, estado, respuesta, extracciones o scripts — sin abrir el nodo
Clic abre el nodo completo	Un clic en la caja abre la card entera en una ventana (editar, ejecutar, ver la respuesta; Esc cierra). Arrastrar sigue moviendo la caja
Separación mínima entre cajas (0–80 px)	Distancia que se respeta al resolver solapes (también al expandir una caja colapsada)
Evitar solapes al soltar o al crecer	Al terminar de arrastrar, las cajas que quedarían debajo se apartan hacia abajo; desde 4.35.0 también cuando una caja **crece** (p. ej. al pintarse la respuesta tras ejecutarla): baja **en bloque** la

Ajuste	Qué hace
una caja	fila de debajo y todo lo que hay bajo ella, así la fila sigue alineada y la información se ve entera; **la que sueltas y las 📌 fijadas no se mueven**
Separar nodos solapados ahora	Recorre el flow visible (los dos, en split) y aparta las cajas que se pisan hasta dejar la separación mínima; el botón indica cuántas movió

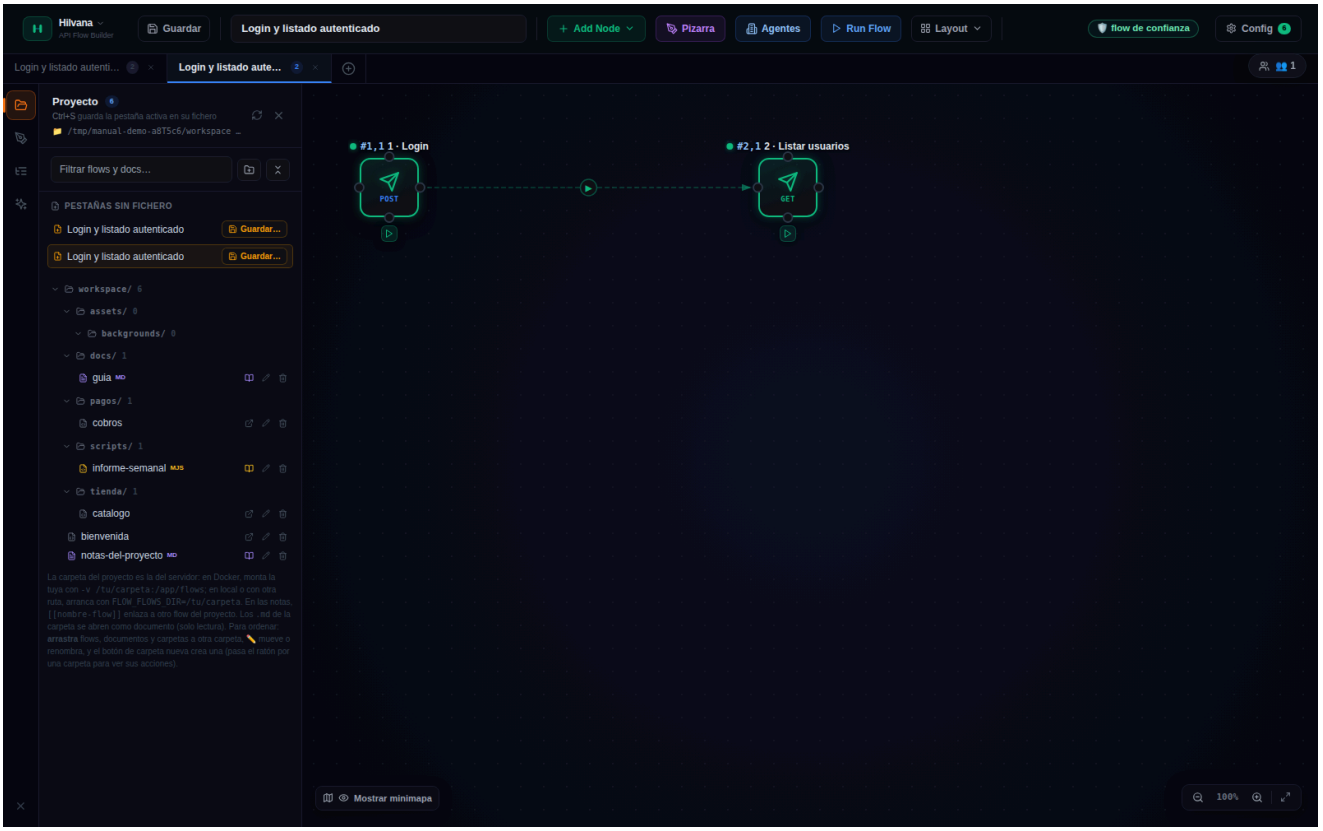


Las filas bajan cuando una card crece 🆕 (4.35). Al ejecutar una request o un SQL la caja se hace más alta al pintarse la respuesta; antes quedaba tapada por la fila siguiente. Ahora, con «Evitar solapes…» activo, las cajas que empiezan en esa fila **y todas las de debajo** bajan en bloque lo justo (separación mínima incluida), así la fila sigue alineada y se ve toda la información. La caja que creció y las 📌 fijadas no se mueven; al encoger (Reset, colapsar) nada vuelve a subir — Auto Layout o Alinear en cuadrícula recompactan.



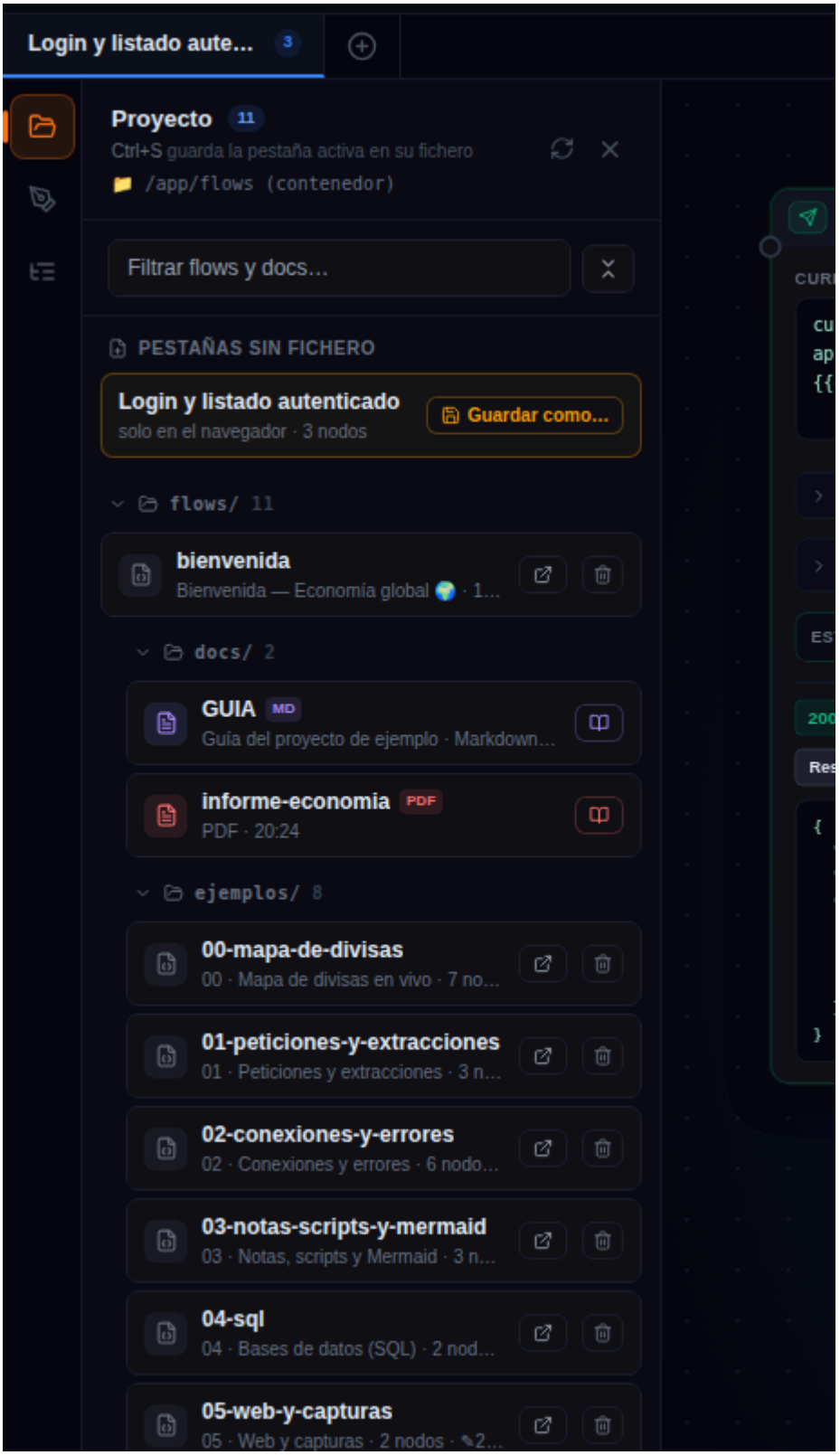
En compacto las líneas, el minimapa y la guía de nodos siguen la caja real; los relayouts del menú Layout (alinear, auto-layout, colapsar/expandir) funcionan igual.

Proyecto — `flows/` como workspace 🆕 (4.25)



La carpeta `flows/` del servidor es el **proyecto** — en Docker `/app/flows`, así que **apúntala a una carpeta tuya** con `-v /tu/carpeta:/app/flows` (o monta donde quieras y fija `FLOW_FLOWS_DIR=/esa/ruta`; en local, `FLOW_FLOWS_DIR=/tu/carpeta npm run dev`). El panel muestra la ruta efectiva en su cabecera. El primer icono de la barra lateral abre el panel **Proyecto**:

Zona	Qué hay
Pestañas sin fichero	Pestañas creadas con *Nuevo* o importadas con *Abrir* que aún no viven en `flows/` — botón **Guardar como…**
flows/ (y subcarpetas)	Cada `.flow.json` con su nombre de flow, nº de nodos y fecha. Desde la 4.41 las carpetas se muestran como **árbol anidado**: cada subcarpeta dentro de su padre, con su nombre corto e indentación (nada de rutas repetidas `flows/a/b/`). Se **pliegan** con un clic en su cabecera — plegar oculta todo el subárbol, el contador suma los ficheros del subárbol y se recuerda; botón plegar/desplegar todo junto al filtro; plegada, muestra si tiene abiertos o cambios. Marcas: azul = pestaña activa, verde = abierto, ● = cambios sin guardar, ⚠ = el fichero cambió en disco después de abrirlo (CLI, MCP, git…)
Botones por fichero	Abrir en una pestaña · ir a la pestaña si ya está abierto · **Guardar** (si tiene cambios) · **Recargar desde disco** (si cambió fuera) · ✕ **Cerrar la pestaña** (si está abierto; confirma si tiene cambios) · 🗑 **Borrar el fichero** del disco (con confirmación; cierra la pestaña si estaba abierto)



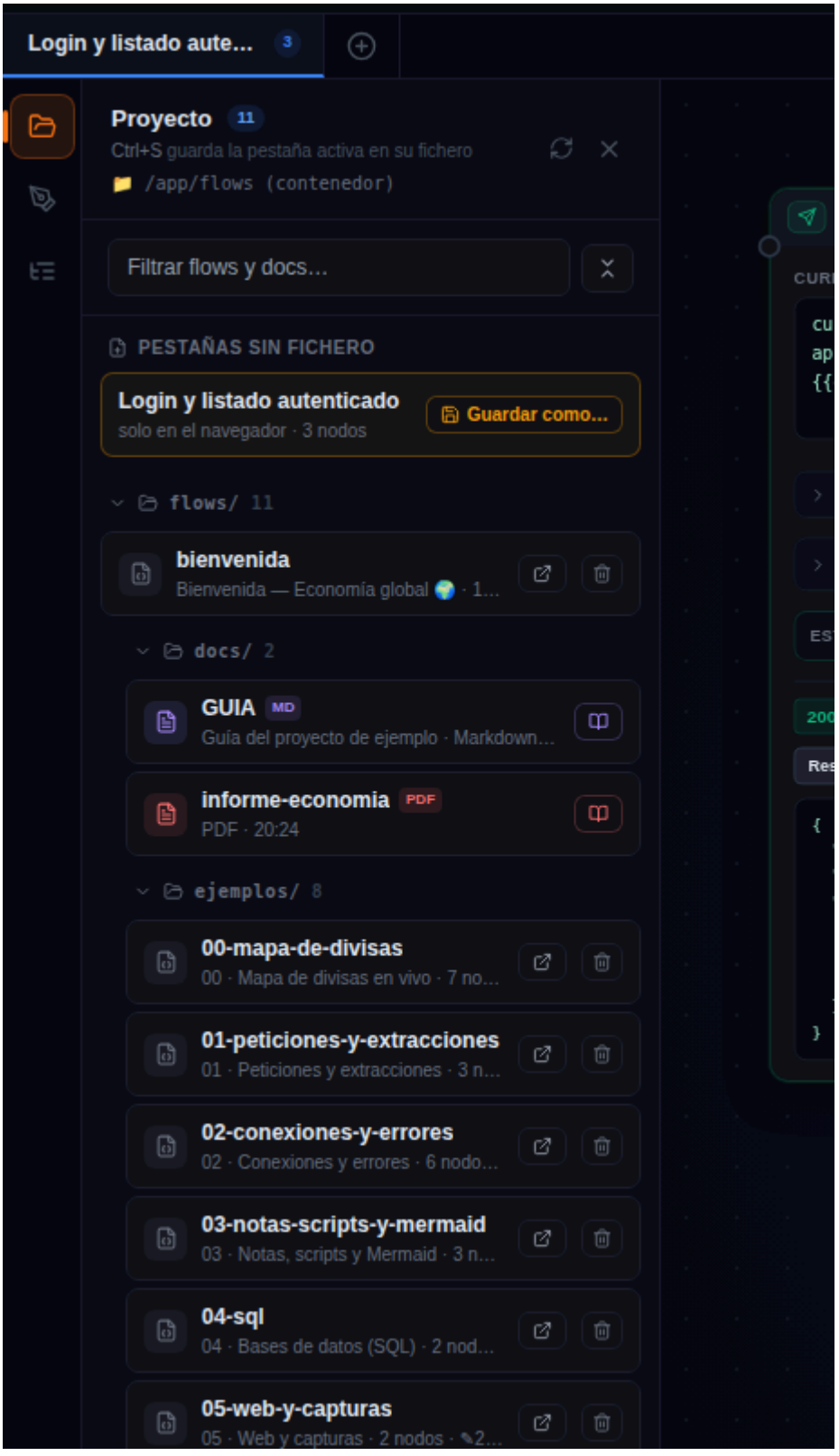
Cómo se trabaja:

- **Abrir**: clic en un fichero → pestaña con la etiqueta del fichero. Los ids de los nodos se conservan, así que al guardar el fichero cambia solo lo que tocaste (diffs limpios en git).
- **Guardar**: **Ctrl+S** (o ⚡ **Hilvana ▸ Guardar**) escribe la pestaña activa en su fichero. El aviso «Sin guardar» y el ● ámbar desaparecen.

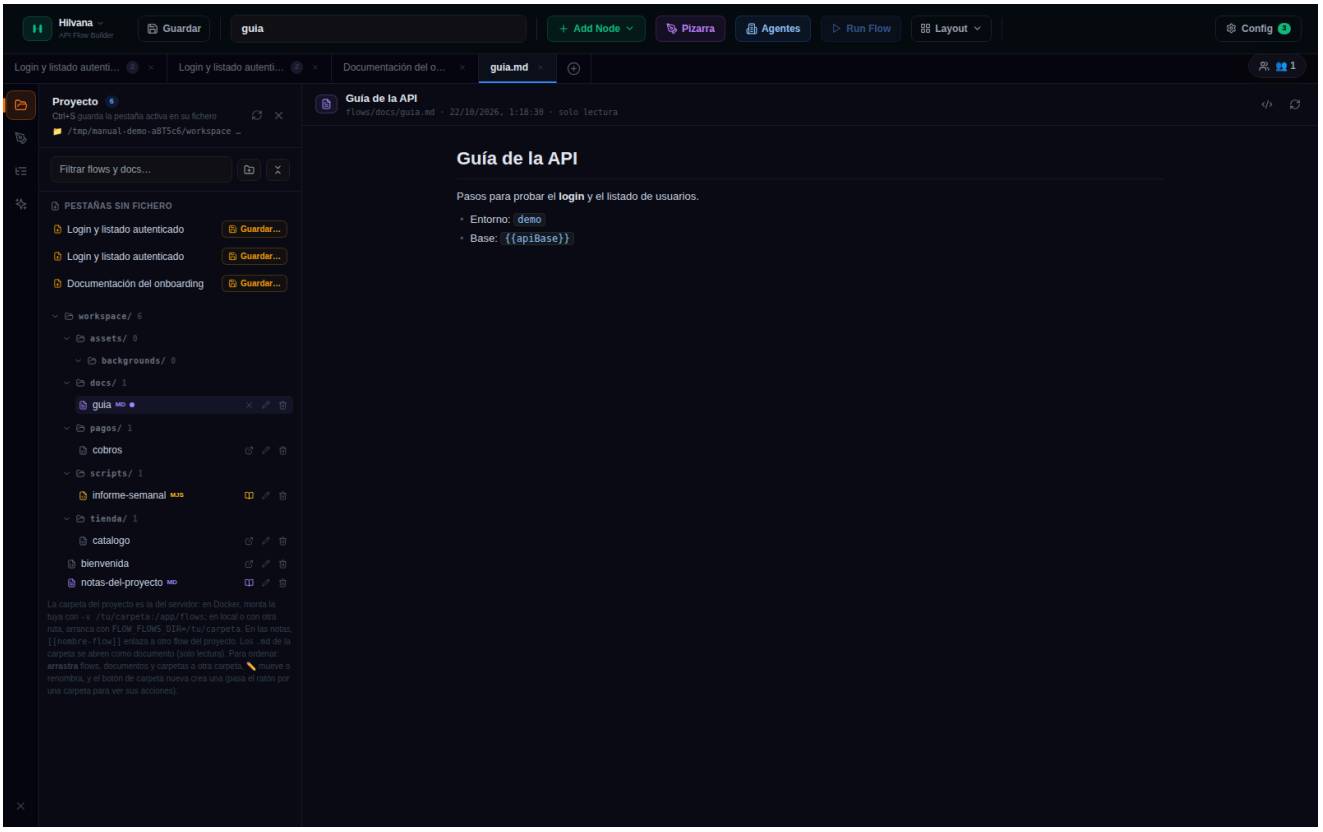
- **Guardar como…** (**Ctrl+Shift+S** o ⚡ **Hilvana ▸ Guardar como…**): nombre + carpeta dentro de `flows/` (avisa si va a sobrescribir). Es lo que pide Ctrl+S en una pestaña sin fichero.
- **Conflictos**: si el fichero cambió en disco desde que lo abriste, Ctrl+S pregunta antes de pisarlo; el panel ofrece **recargar desde disco** (pierdes los cambios de la pestaña).
- **Docker**: los ficheros que crea o sobrescribe el contenedor conservan el **dueño del host** (aunque el server corra como root), así que puedes seguir editándolos y versionándolos sin `sudo`.

> Los flows editados en la web siguen autoguardándose en el navegador como siempre; lo nuevo es que además cada pestaña sabe a qué fichero pertenece. *Export* sigue sirviendo para descargar el `.flow.json` fuera del proyecto, y las tools MCP `flow_save` / `flow_file_read` escriben y leen la misma carpeta.

Documentos Markdown en el panel 🆕 (4.38)



El panel lista también los `.md` de `flows/` (recursivo): tus notas y los informes que genera flow-explore en `assets/<slug>/README.md`. Cada uno aparece con su primer encabezado `# …` como título y un chip **MD**; el clic lo abre como **una pestaña más** (desde la 4.40 — antes era un modal): el documento ocupa el área del canvas, la pestaña lleva su chip **MD** y convive con las de flows; la fila del panel sabe si está abierto (ir a la pestaña / cerrar):

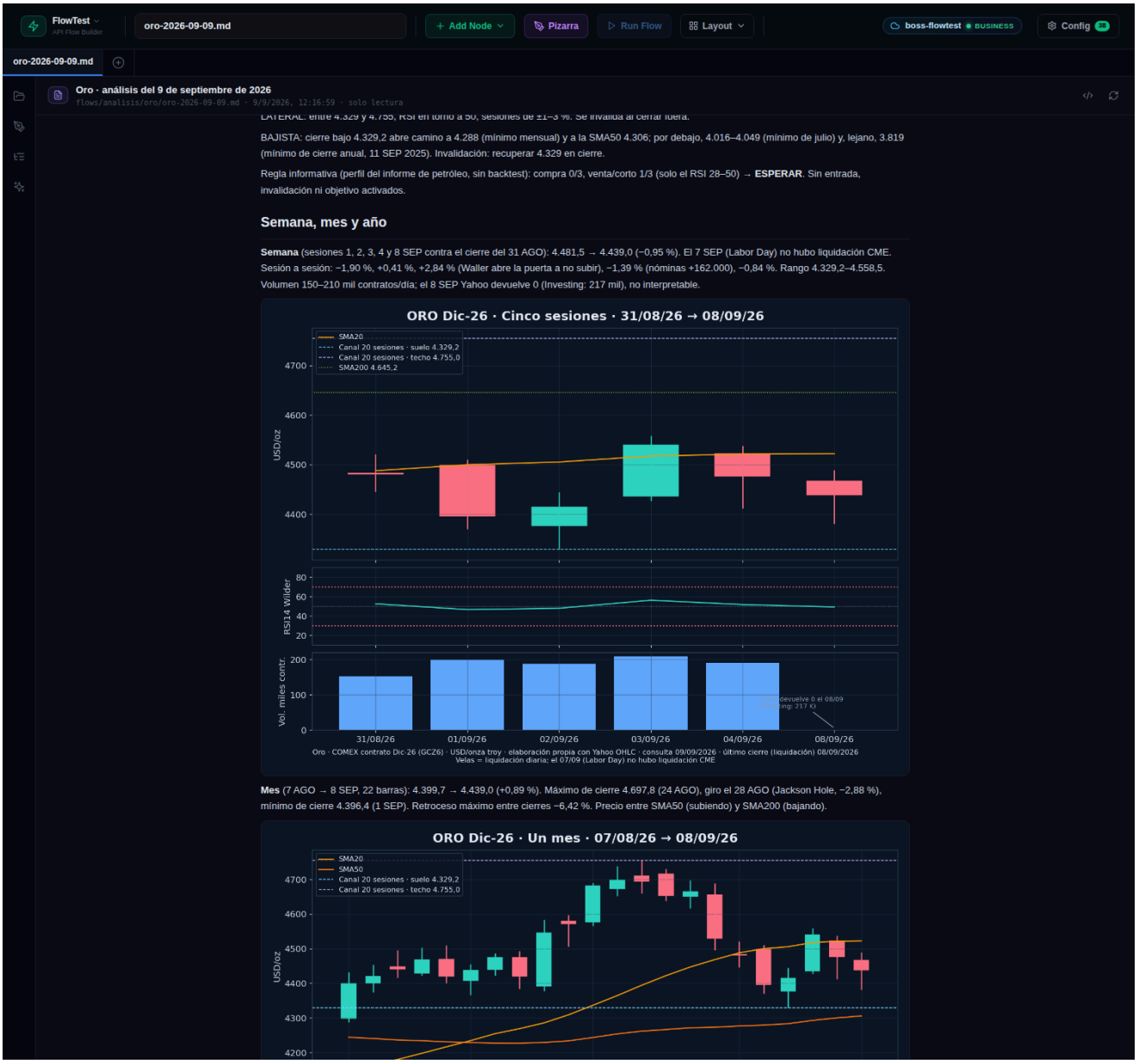


- Renderiza encabezados, listas, tablas, bloques de código, citas, desplegables `<details>` (los usan los informes del explorador) e imágenes — 🆕 5.14.2: **cualquier imagen del proyecto** se ve, con rutas relativas al documento (p. ej. `oro-2026-09-09-datos/semana.png`) o desde la raíz de `flows/`; el servidor las sirve por `GET /workspace/image?path=` sin salir de `flows/` (misma guarda que el resto del workspace, carpetas privadas respetadas, 25 MB por imagen). Las de `flows/assets/` siguen por `/flow-assets/*`.

- Los **enlaces `[[flow]]`, `[[flow|texto]]` y `[[flow#Nombre de nodo]]`** funcionan como en las notas: cierran el visor, abren ese flow del proyecto y centran el nodo. Las URLs se abren en pestaña nueva.

- **▶ Ejecutar desde el documento** 🆕 (4.39): cada enlace `[[flow]]` lleva adosado un botón verde ▶ que abre el flow **y lanza su Run Flow completo** (scripts de notas + nodos HTTP y SQL en orden topológico) — un `.md` con enlaces sirve de lanzador de baterías de prueba. En `[[flow#Nodo]]` el ▶ ejecuta el flow entero.

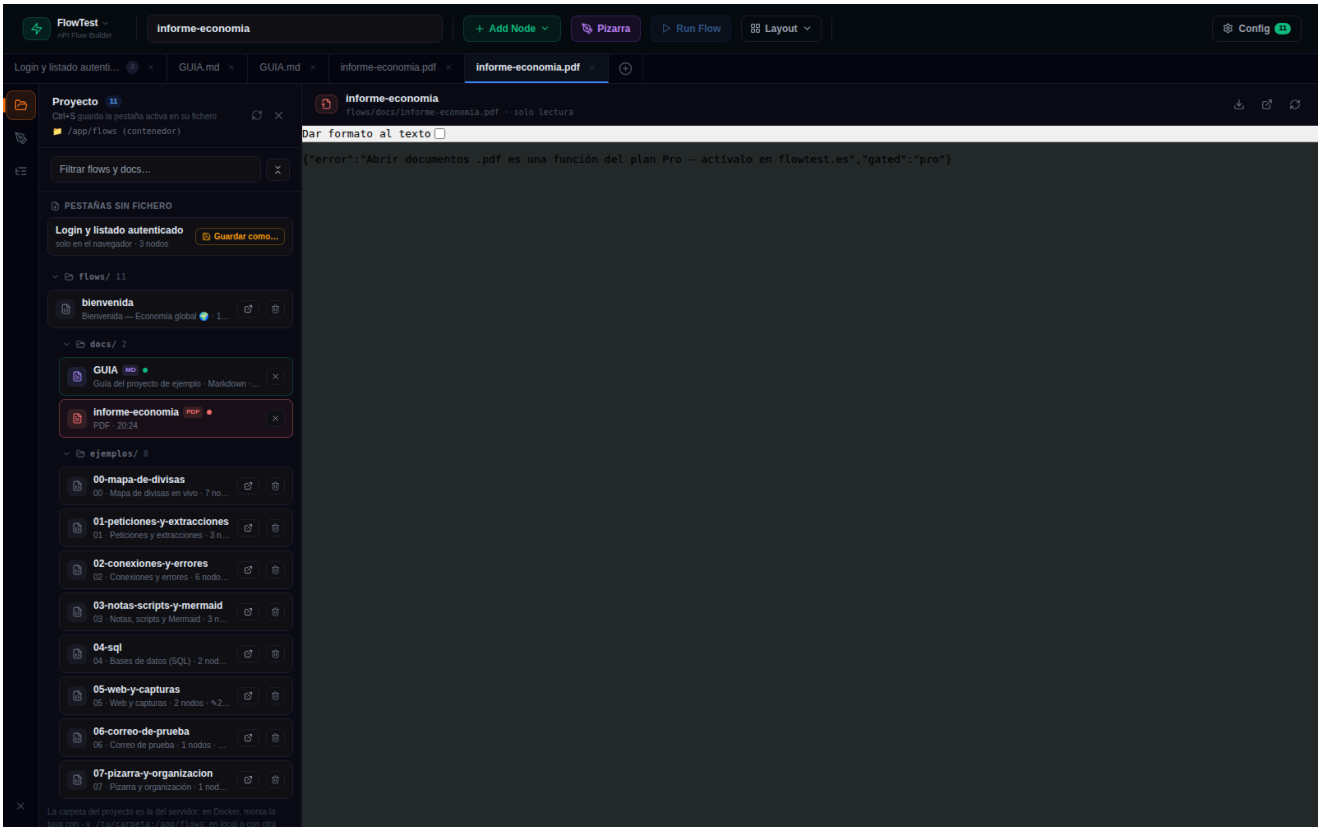
- Botones: **ver la fuente** (el Markdown original) y **recargar desde disco**; la pestaña se cierra como cualquier otra.
- **Solo lectura**: los `.md` se editan fuera (tu editor, git); la web no los guarda ni los borra. El MCP (`flow_files_list`) y la CLI siguen viendo únicamente `.flow.json`.



Un informe del proyecto con sus gráficos, tal cual en el visor (5.14.2).

Documentos PDF en el panel 🆕 (4.45)

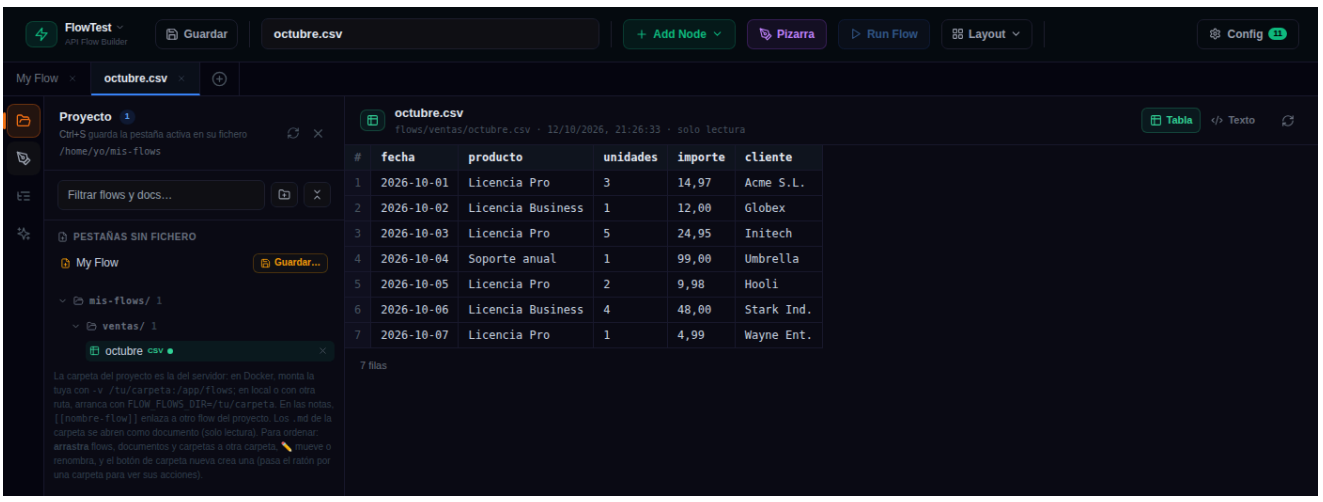
El panel Proyecto lista también los `.pdf` de `flows/` (recursivo, chip **PDF** rosa) y clicarlos abre **una pestaña de documento** de solo lectura, igual que los `.md`:



- El PDF lo pinta el **visor nativo del navegador** (zoom, buscar, imprimir); la cabecera añade **descargar**, **abrir en una pestaña del navegador** y **recargar desde disco**.
- Combina de maravilla con **Exportar PDF** (menú del logo, 4.44): el informe del canvas que exportes puede vivir en `flows/` y consultarse sin salir de la app.
- Solo lectura: Ctrl+S nunca lo pisa, y la pestaña jamás aparece como «sin guardar». Los enlaces `[[flow]]` de notas y documentos siguen resolviendo **solo contra flows** (los `.md` / `.pdf` quedan fuera).
- Servido por `GET /workspace/pdf?path=` con las mismas guardas que el resto del workspace (nada fuera de `flows/`, carpetas ocultas excluidas, máx. 50 MB).

Datos CSV y TSV en el panel 🆕 (5.28)

Los ficheros `.csv` y `.tsv` de la carpeta del proyecto salen en el panel Proyecto (chip **CSV**) y se abren como pestaña de documento, de **solo lectura**:



- **Tabla** con la cabecera fija y el número de fila; las filas largas se cargan de **1000 en 1000**.
- **Texto**: el contenido tal cual, para copiar o revisar el formato.
- Detecta el separador (`,` `;` o tabulador) y respeta las comillas.
- ↻ vuelve a leer el fichero si ha cambiado en disco. Hasta **5 MB** por fichero; no se mueven ni se borran desde el panel y no cuentan como flows. De momento solo en 💻 Local (y carpetas enlazadas), no en ☁️ Cloud.

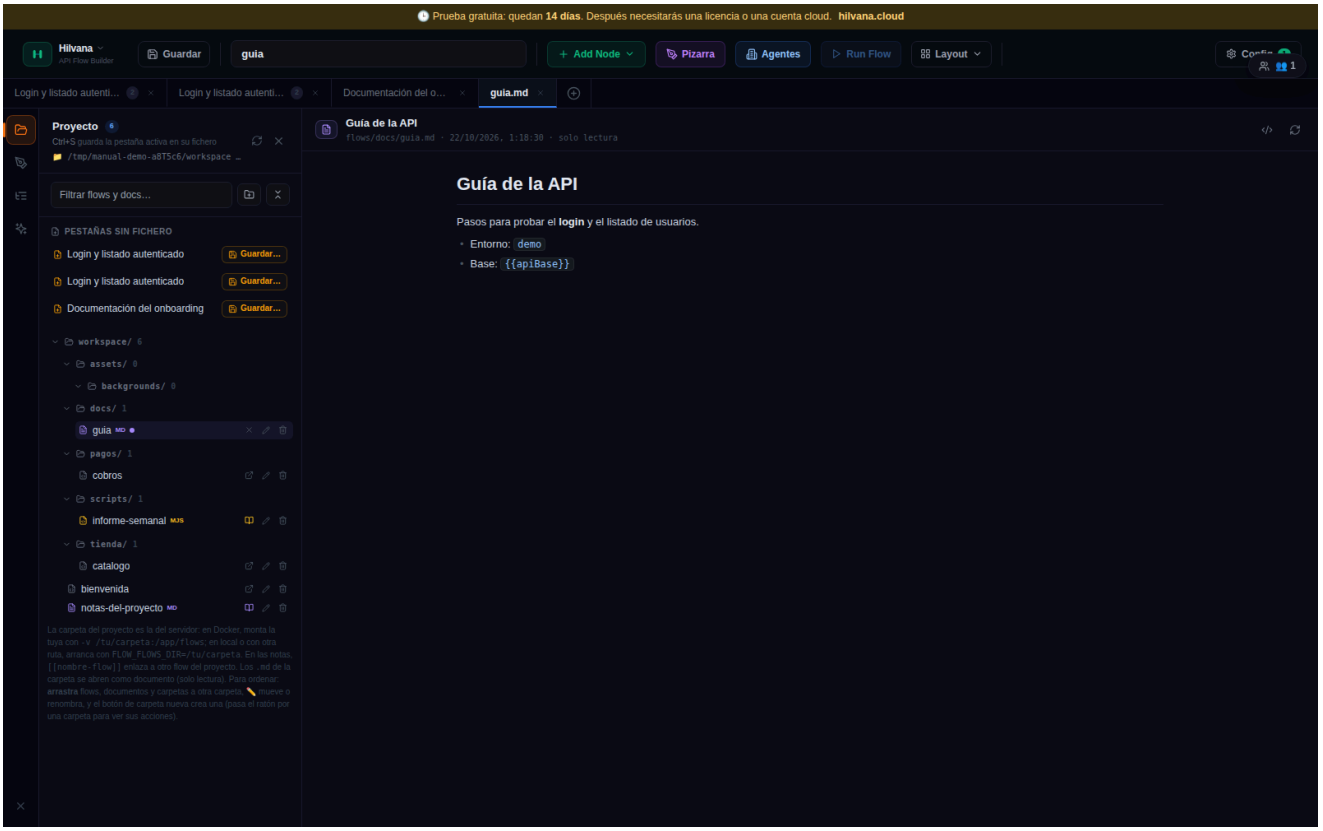
Ordenar el proyecto 🆕 (5.20)

En 💻 **Local**, el panel Proyecto sirve también para ordenar la carpeta:

- **Carpeta nueva**: botón de carpeta en la barra del panel (crea en `flows/`) o al pasar el ratón por una carpeta (crea dentro). Las carpetas **vacías** también se ven.
- **Mover**: **arrastra** un flow, un documento o una carpeta entera y suéltalo sobre otra carpeta (o sobre `flows/`). Si el flow estaba abierto, **su pestaña lo sigue**: Ctrl+S guarda ya en la ruta nueva.
- ✏️ **Mover o renombrar**: en cada fichero y en cada carpeta; escribe la ruta nueva dentro de `flows/` (si la carpeta de destino no existe, se crea). Un flow sigue acabando en `.flow.json` y un documento conserva su extensión.
- **Borrar**: 🗑 en flows y, desde la 5.20.1, también en documentos **`.md` y `.pdf`** (con confirmación; si estaba abierto, se cierra su pestaña). Una carpeta se borra cuando está **vacía**. Un **enlace** se quita sin tocar el fichero al que apunta.

Nada de esto sale de `flows/` ni toca carpetas ocultas o privadas de otro miembro. En ☁️ **Cloud** (5.24) el panel también crea carpetas, mueve, renombra y borra en tu espacio del cloud. Si tu espacio estaba dormido (tras un rato sin uso), el panel **espera a que arranque** —unos segundos— en vez de mostrar «El contenedor está despertando» (5.24.1).

Panel Proyecto a medida 🆕 (5.21)



- **Scripts visibles**: los `.js`, `.mjs`, `.cjs`, `.ts`, `.py`, `.sh` y `.sql` de la carpeta salen en el árbol (chip ámbar con la extensión, p. ej. **MJS**). Clic = pestaña de solo lectura con el código tal cual. Se mueven, renombran y borran como los documentos. El MCP y la CLI siguen viendo solo flows.
- **A dónde apunta cada enlace**: si una carpeta es un enlace simbólico (un workspace «hub» que reúne los `flows/` de varios repos), junto al contador aparece su destino en gris — `→ ~/dev/mi-repo/flows`; pasa el ratón para verlo entero. Solo en 💻 Local.
- **Ancho del panel**: arrastra el **borde derecho** del panel para ensancharlo o estrecharlo (de 220 a 900 px); **doble clic** en el borde vuelve al ancho normal. Se recuerda en el navegador.
- **Ocultar carpetas**: pasa el ratón por una carpeta y pulsa 👁‍🗨 para sacarla del árbol (y de los contadores). Arriba aparece **«👁‍🗨 N»**: púlsalo para ver las ocultas atenuadas y recupera una con 👁. Es solo una vista: los ficheros siguen en disco y el resto de la app los ve. **Desde la 5.30.3** la lista se guarda en el servidor, así que la vista móvil y otros navegadores ocultan las mismas; lo que tenías oculto en un navegador se sube la primera vez que abres el panel.

Carpetas enlazadas y copias de los agentes 🆕 (5.19.4)

Desde la 5.18 el workspace **sigue enlaces simbólicos**: una carpeta de proyecto puede ser un «hub» con enlaces a los `flows/` de varios repos y a sus documentos, y el panel los lista como propios.

Si enlazas un repo entero en el que trabajan agentes (por ejemplo con AgentOffice), sus **copias de trabajo** (git worktrees, una por tarea) repetirían todos los flows. Desde la 5.19.4 el panel Proyecto, `/workspace/flows` y la tool MCP `flow_files_list` **saltan los git worktrees enlazados**; los submódulos de git se siguen listando.

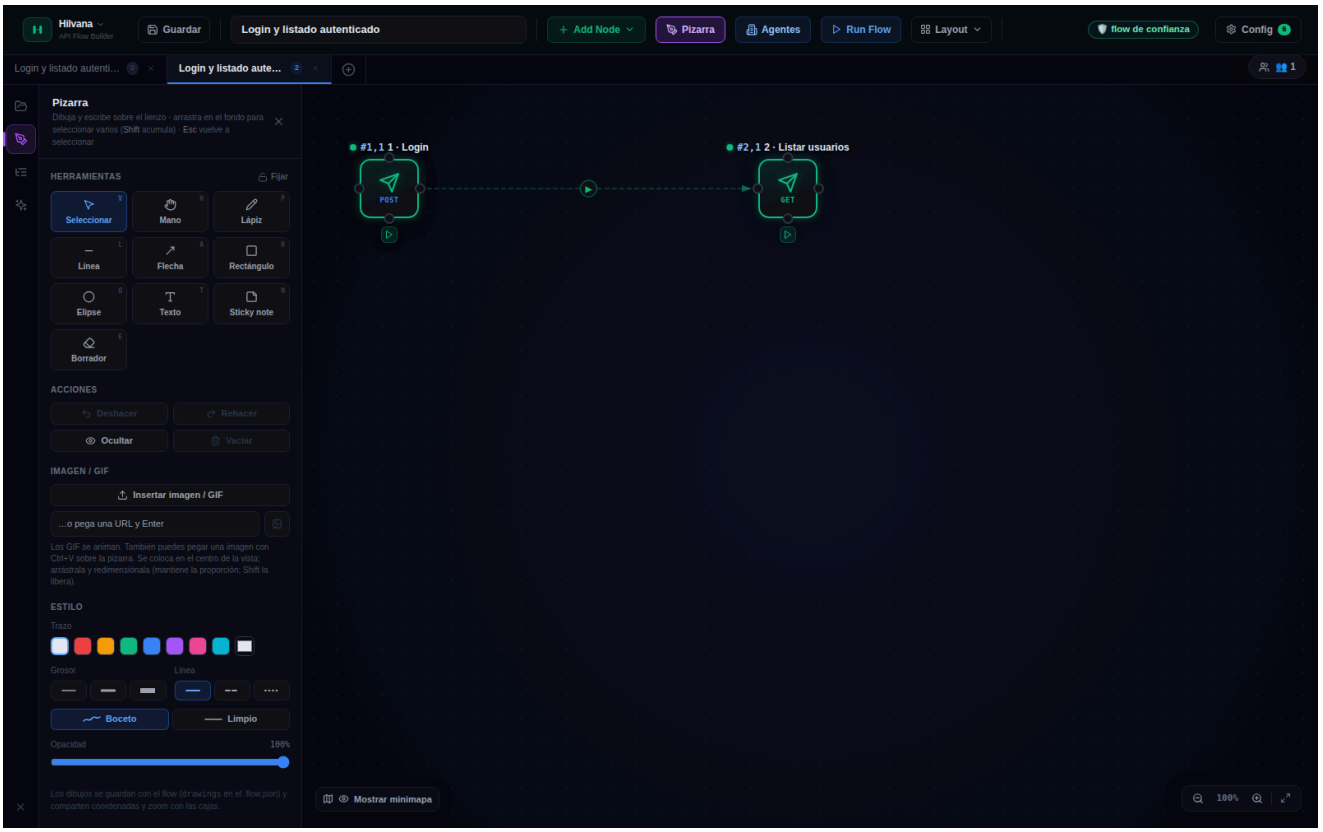
Enlaces entre flows en las notas 🆕 (4.25)

En el texto de una **nota** (Add Note), escribe:

Sintaxis	Qué hace al hacer clic en la vista previa
`[[api-store]]`	Abre (o activa) el flow `api-store` — busca por nombre de fichero o de flow, primero en las pestañas y luego en `flows/`
`[[api-store	ver la API de la tienda]]`
`[[api-store#Stripe - Webhook]]`	Abre el flow **y centra/resalta** el nodo con ese nombre
`https://…`	Las URLs se vuelven enlaces (nueva pestaña del navegador)

Combinado con la pizarra y la guía de nodos, permite montar un flow «índice» que enlaza al resto del proyecto.

Pizarra 🆕 (4.24)



Una capa de dibujo libre **sobre el lienzo**, al estilo Excalidraw: rodea grupos de cajas, escribe títulos y avisos, dibuja tus propias flechas. Se abre con el botón morado **Pizarra** de la toolbar o desde la barra lateral.

Sección del panel	Qué hay
Herramientas	Seleccionar (`V`), Mano (`H` , mueve el lienzo), Lápiz (`P`), Línea (`L`), Flecha (`A`), Rectángulo (`R`), Elipse (`O`), Texto (`T`), Sticky note (`N` , 5.29), Borrador (`E`). El candado **Fijar** mantiene la herramienta

Sección del panel	Qué hay
	tras dibujar (por defecto las formas vuelven a Seleccionar; el lápiz se queda)
Acciones	Deshacer / Rehacer (también `Ctrl+Z` / `Ctrl+Shift+Z`), Ocultar/Mostrar los dibujos, Vaciar la pizarra
Selección	Con algo seleccionado: Duplicar (`Ctrl+D`), Eliminar (`Supr`), Traer al frente, Enviar al fondo
Estilo	8 colores + personalizado, grosor, línea continua/discontinua/punteada, trazo **Boceto** (a mano alzada) o **Limpio**, relleno para rectángulos y elipses, **fuente** (10 desde la 4.35, ver abajo) y tamaño para los textos, opacidad. Con una selección, el cambio se aplica a ella; si no, al siguiente dibujo

Cómo se usa:

1. Elige una herramienta y **arrastra** sobre el lienzo. `Shift` restringe a cuadrado/círculo/ángulos de 45°.
2. **Texto**: clic donde quieras escribir, teclea (Enter hace salto de línea) y `Esc` o clic fuera para terminar. Doble clic sobre un texto existente lo edita.
3. **Seleccionar**: clic sobre un trazo (en formas sin relleno, sobre el borde) para moverlo o cambiarlo con los tiradores; `Shift` + clic suma a la selección.
4. **Selección múltiple** 🆕 (4.46): con Seleccionar activo, **arrastra sobre el fondo del lienzo** y aparece un rectángulo azul punteado que selecciona todos los dibujos que toca; con `Shift` el rectángulo **acumula** sobre lo que ya tenías seleccionado (y `Ctrl+A` sigue seleccionando todo). Duplicar, Eliminar, el estilo o traer al frente/enviar al fondo se aplican a todos a la vez. Un clic en el fondo, fuera de la selección, deselecciona. Las cajas del flow no se ven afectadas: el rectángulo solo existe mientras el panel Pizarra está abierto, y las cajas y conexiones siguen por encima y funcionan igual.
5. **Mover el grupo** 🆕 (4.47): arrastra desde cualquier trazo de la selección **o desde el hueco dentro de su caja** (p. ej. entre las piernas de un muñeco) — antes ese hueco empezaba un rectángulo nuevo y se perdía la selección. `Shift`+arrastre sobre el fondo sigue haciendo rectángulo aunque caiga dentro.
6. **Copiar y pegar** 🆕 (4.47): `Ctrl+C` copia la selección, `Ctrl+X` la corta y `Ctrl+V` la pega desplazada 24 px (cada pegado se desplaza un poco más); también con los botones **Copiar** / **Pegar** de la sección Selección del panel. Es un portapapeles propio de la pizarra (no toca el del sistema) y sobrevive al cambio de pestaña, así que sirve para llevar dibujos de un flow a otro dentro de la misma ventana. `Ctrl+D` sigue duplicando en el sitio.
7. `Esc` vuelve a Seleccionar y quita la selección.

Los dibujos comparten coordenadas y zoom con las cajas, se ven en el minimapa y **se guardan con el flow** (campo `drawings` del `.flow.json`, solo presente si hay dibujos). Viajan con export/import, GitHub Flows y MCP; el CLI los ignora. Con el panel cerrado, o en modo Seleccionar, las cajas funcionan exactamente igual que siempre (los dibujos quedan debajo de ellas).

> Ejemplo listo: `examples/pizarra-anotada.flow.json`.

> **5.31.1:** si un flow se escribe a mano (un agente, un script) y sus dibujos no llevan `id`, la app se lo pone al abrirlo; antes esos dibujos solo salían en el minimapa. Para crear dibujos desde un agente,

> mejor la tool del MCP `whiteboard_update`.

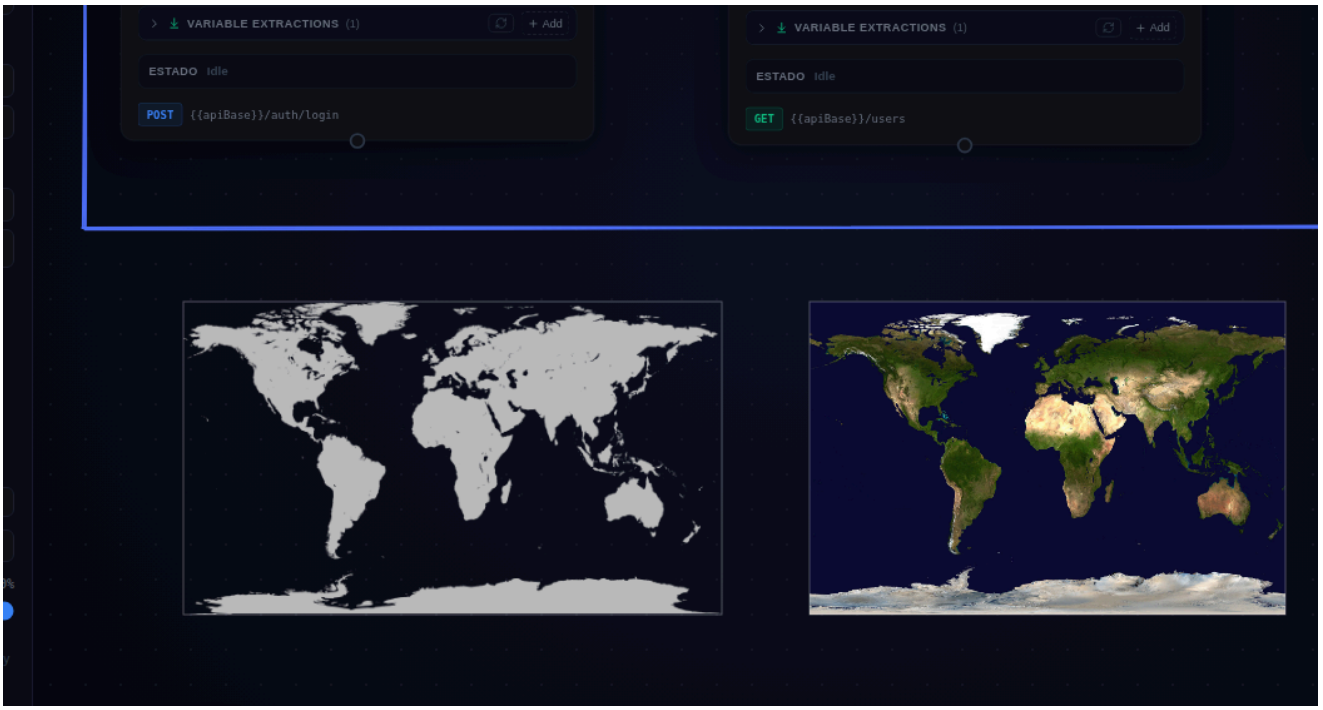
Sticky notes en la pizarra 🆕 (5.29)

Notas adhesivas para anotar el flow, con **fecha de creación** y **color a elegir**:

- **Crear**: herramienta **Sticky note** del panel Pizarra (tecla `N` o `9`). Un **clic** crea una nota de 200×160; **arrastrar** dibuja una caja a medida (mínimo 100×80).
- **Color**: en **Estilo** hay 6 pasteles (amarillo por defecto, rosa, verde, azul, naranja y lila). El elegido se aplica a las notas nuevas; con notas seleccionadas las recolorea todas en un solo paso de deshacer.
- **Fecha**: cada nota muestra `dd/mm/aaaa` del día en que se creó. Editar el texto **nunca** la cambia.
- **Editar**: doble clic sobre la nota abre un cuadro de texto multilínea; sale con `Esc` o clic fuera. Una nota vacía no se borra.
- **Mover, redimensionar, copiar/pegar, borrar y deshacer** funcionan como en el resto de dibujos. El texto se ve siempre en tinta oscura, legible sobre los 6 colores.
- Se guarda en `drawings` como elemento `type: "sticky"` con `text`, `color` y `createdAt`; los flows antiguos se abren igual. Viaja en PDF, captura y «Compartir enlace». El MCP la crea con `whiteboard_update` (`type: "sticky"`, `color`, `createdAt`; por defecto amarillo, hoy y 200×160). La vista móvil `/m` no pinta la pizarra.

Imágenes y GIF en la pizarra 🆕 (4.50)

La pizarra admite **imágenes (PNG/JPG) y GIF animados** como un dibujo más:



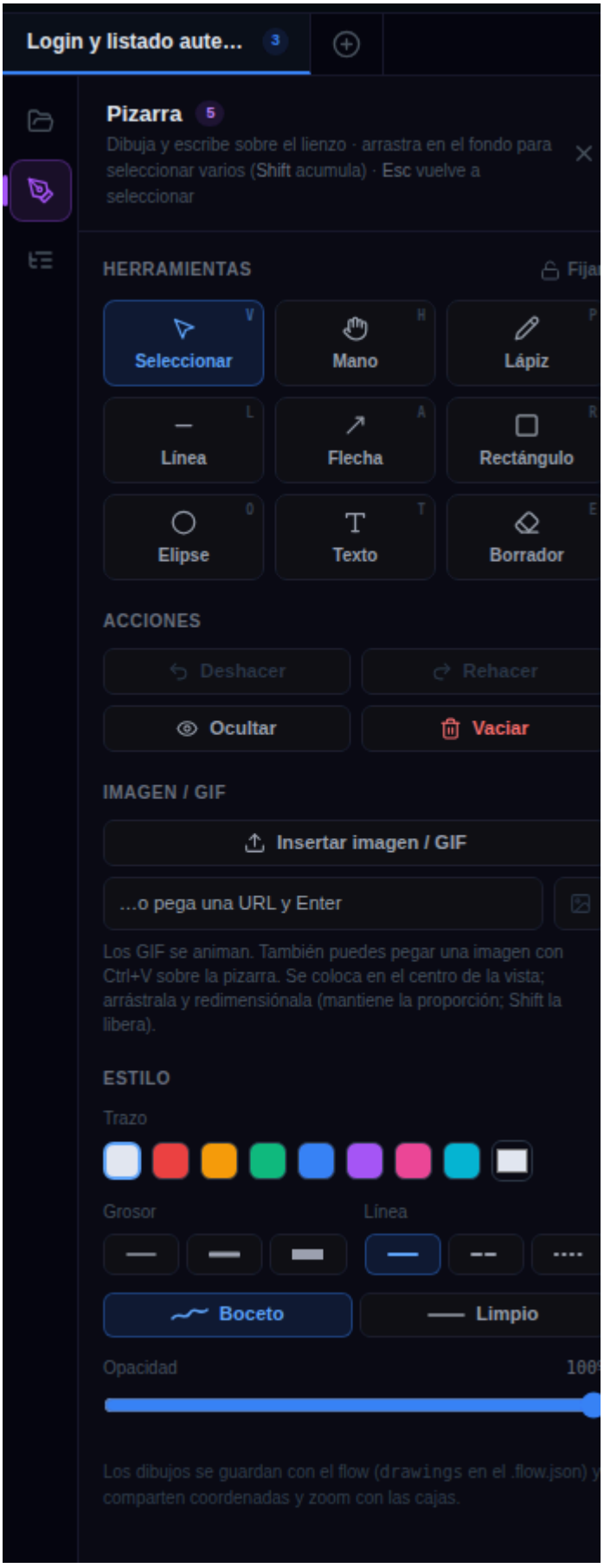
- **Panel Pizarra ▸ Imagen / GIF**: «Insertar imagen / GIF» abre un selector de fichero (la imagen se guarda en `flows/assets/whiteboard/` y viaja con el proyecto), o pega una URL y pulsa Enter.
- **Ctrl+V** con una imagen en el portapapeles (una captura de pantalla, una imagen copiada de la web) la inserta directamente — con el panel abierto y tras haber hecho clic en la pizarra.

- Se coloca en el centro de la vista (máx. 480 px de ancho, proporción natural) y queda seleccionada: arrástrala, redimensiónala (mantiene la proporción; **Shift** la libera), duplícala, cópiala/pégala, tráela al frente o envíala al fondo, cambia su opacidad desde el estilo de la selección. Los GIF se animan en el lienzo.
- Se guardan en el `.flow.json` como `drawings` de tipo `image` (`imageSrc`); el MCP las añade con `whiteboard_update` (`type: "image"`, `imageSrc`; ancho/alto medidos si no se indican).

Variables vivas en la pizarra 🆕 (4.51)

Los **textos** de la pizarra (y la URL de las **imágenes**) resuelven `{{variables}}` al pintarse: escribe «Brent: {{brent}} $ {{semaforoBrent}}» junto a un nodo y el rótulo cambia con cada Run Flow. Si la variable no existe se ve el marcador tal cual. Al editar el texto se trabaja con el original (con los marcadores), y una imagen cuya URL lleve una variable (`/flow-assets/whiteboard/semaforo-{{estado}}.gif`) cambia de imagen según el estado. Combinado con los scripts «después» (04) convierte la pizarra en un panel de estado.

Fuentes del texto 🆕 (4.35)

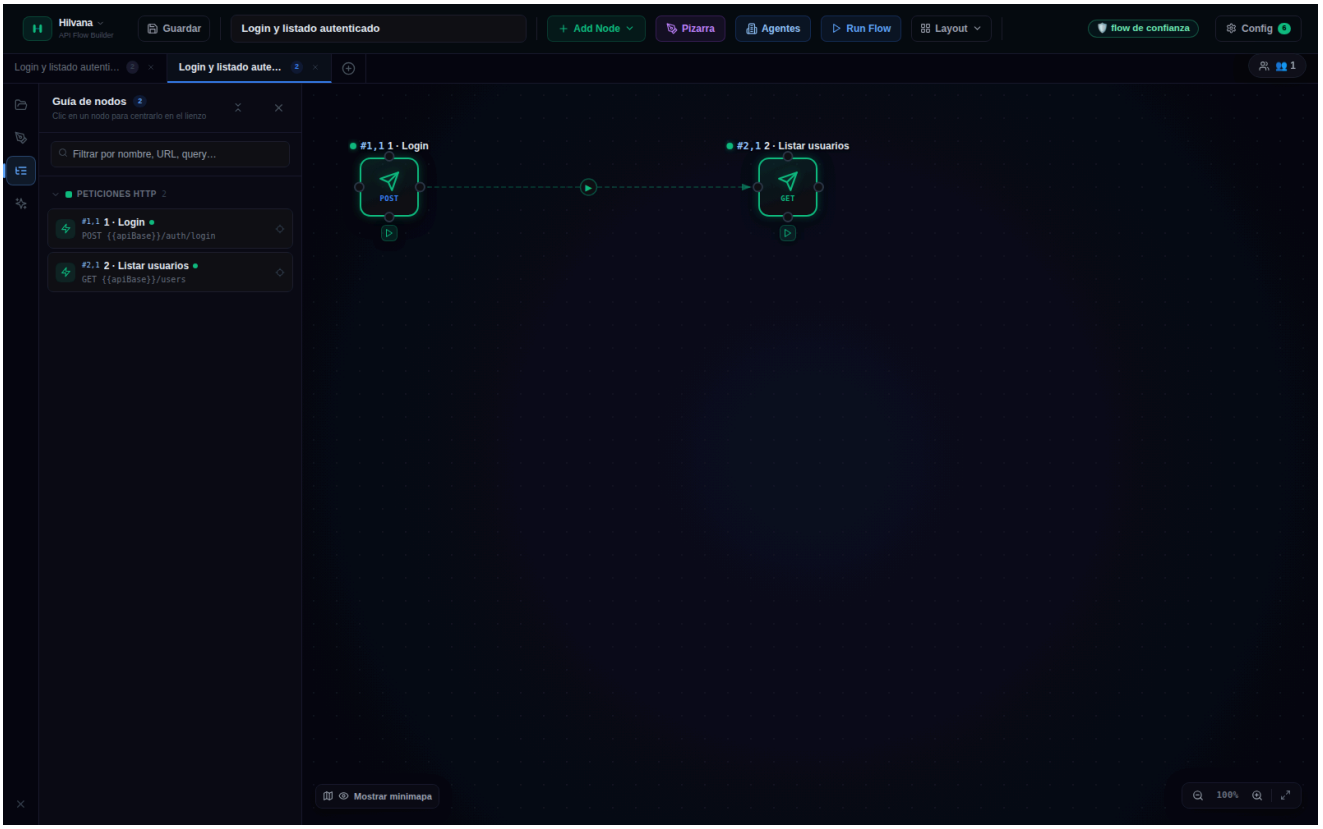


Con la herramienta **Texto** (o un texto seleccionado) el panel muestra una cuadrícula de **10 fuentes**, cada botón pintado con la suya y una **vista previa** con la fuente, tamaño y color actuales. Las ocho manuscritas van **empaquetadas con la aplicación** (no dependen de la red ni de las fuentes del sistema, también dentro de Docker):

Botón	Fuente	Para qué
Manuscrita (por defecto)	Patrick Hand	letra clara a mano, la de siempre (los flows antiguos la heredan)
Boceto	Caveat	cursiva rápida, tipo apunte
Excalidraw	Excalifont	la fuente real de Excalidraw
Indie	Indie Flower	trazo fino y letras redondas
Rotulador	Permanent Marker	títulos gruesos, como rotulador de pizarra
Esbozo	Cabin Sketch	letras «a lápiz» con tramado
Arquitecto	Architects Daughter	letra técnica de plano
Nota	Gloria Hallelujah	nota adhesiva
Normal / **Código**	sistema / monoespaciada	texto limpio y fragmentos de código

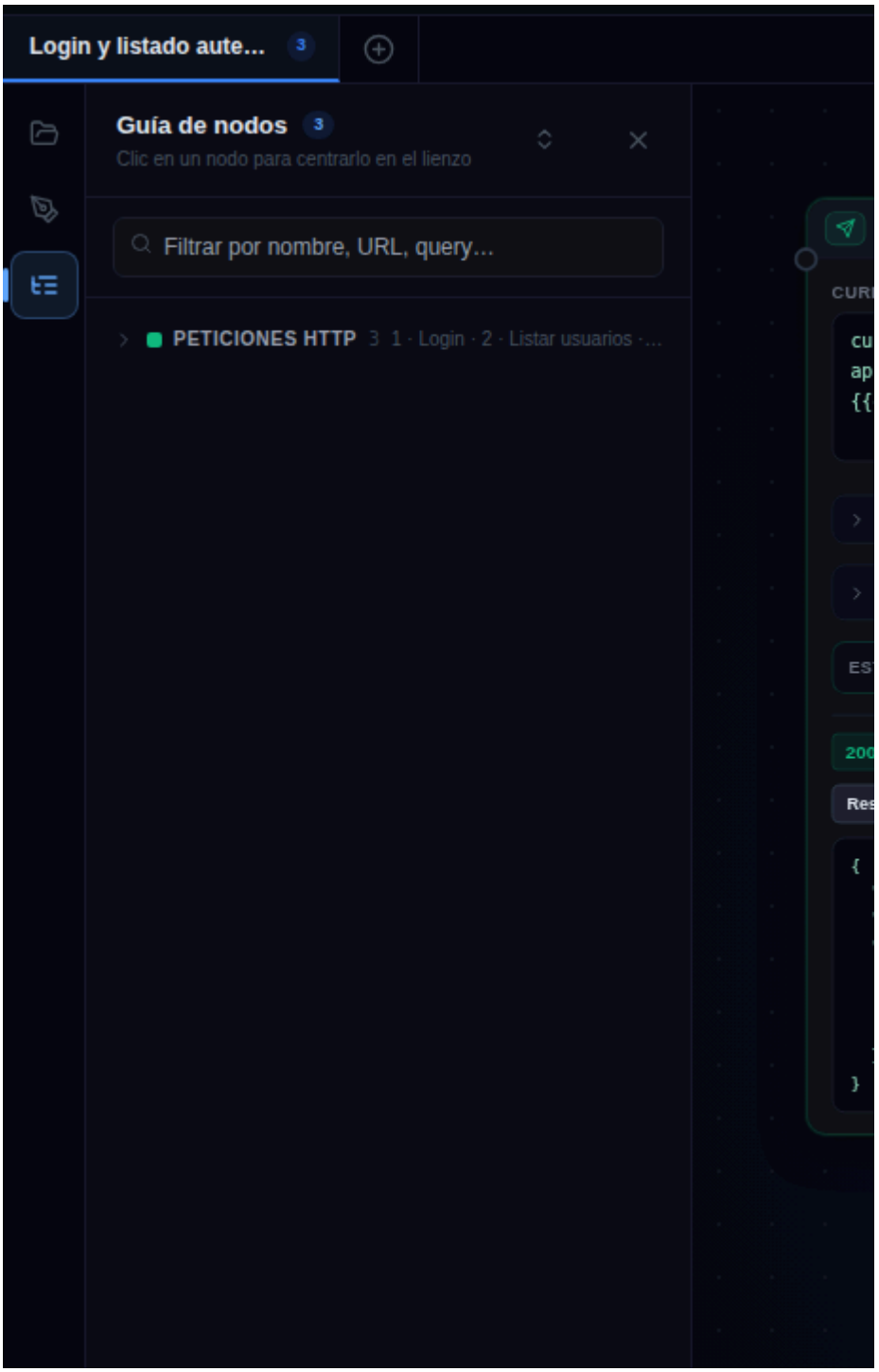
En el `.flow.json` el campo `font` del texto vale `hand`, `sketch`, `excali`, `indie`, `marker`, `draft`, `architect`, `note`, `sans` o `mono` (formato); el MCP `whiteboard_update` acepta los mismos valores y, si no se indica `w`/`h`, mide el texto con su fuente para que se pueda seleccionar y mover.

Guía de nodos 🆕 (4.24)



Layout ▸ Guía de nodos (o el icono de lista de la barra lateral) abre un índice de **todas las cajas del flow** agrupadas por tipo — Peticiones HTTP (método + URL), Consultas SQL (motor + query), Notas (texto / Mermaid / Captura) y Web (URL) — con su punto de estado, nº de orden y 📌. Tiene filtro de texto por nombre, URL o query. **Clic en un nodo → el lienzo se centra en él y la caja se resalta** un par de segundos. Imprescindible en flows de 20+ cajas.

Desde la 4.29 los **grupos se pliegan**: clic en la cabecera del grupo lo pliega/despliega (plegado muestra el recuento y los nombres en una línea); **Alt+clic, Shift+clic o doble clic** deja **solo ese grupo** abierto; el botón junto a la ✕ pliega/despliega todos. Se recuerda entre sesiones.



Ocultar conectores 🆕 (4.24)

Layout ▸ Ocultar conectores quita las líneas de conexión del lienzo para una vista limpia (o para dibujar tus propias flechas en la pizarra). Las conexiones siguen existiendo y la línea provisional al conectar dos cajas se sigue viendo; el mismo item pasa a **Mostrar conectores** para volver. Es un ajuste de vista: no se guarda en el flow ni afecta a la ejecución.

Variables usadas en la caja (4.23)

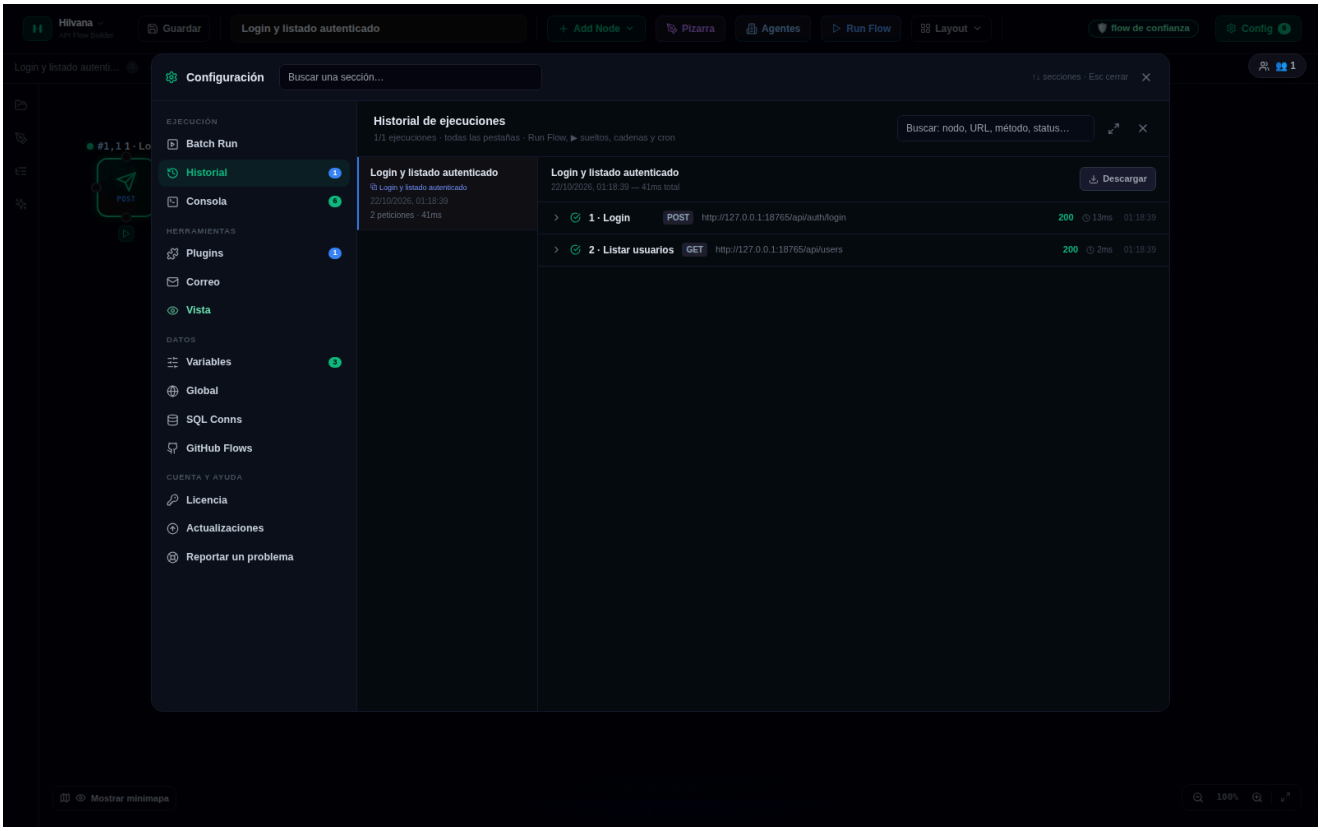
Cada caja **request** y **SQL** tiene una franja plegable **VARIABLES USADAS** que lista las `{{variables}}` de su curl, query o conexión con su valor actual, **editable in situ**: lo que escribes va a las variables de entorno del flow (y pisa el valor runtime si lo había). La cabecera avisa de las que **no tienen valor** y oculta las que parecen secretos.

📌 Fijar posición (4.23)

Cada caja (request, SQL, nota, web) tiene un candado en su cabecera: fijada, **nada la mueve** — ni colapsar/expandir (resolución de solapes), ni Collapse/Expand All, ni alinear, ni el auto-layout, ni el arrastre. Se guarda en el `.flow.json` (`pinned`).

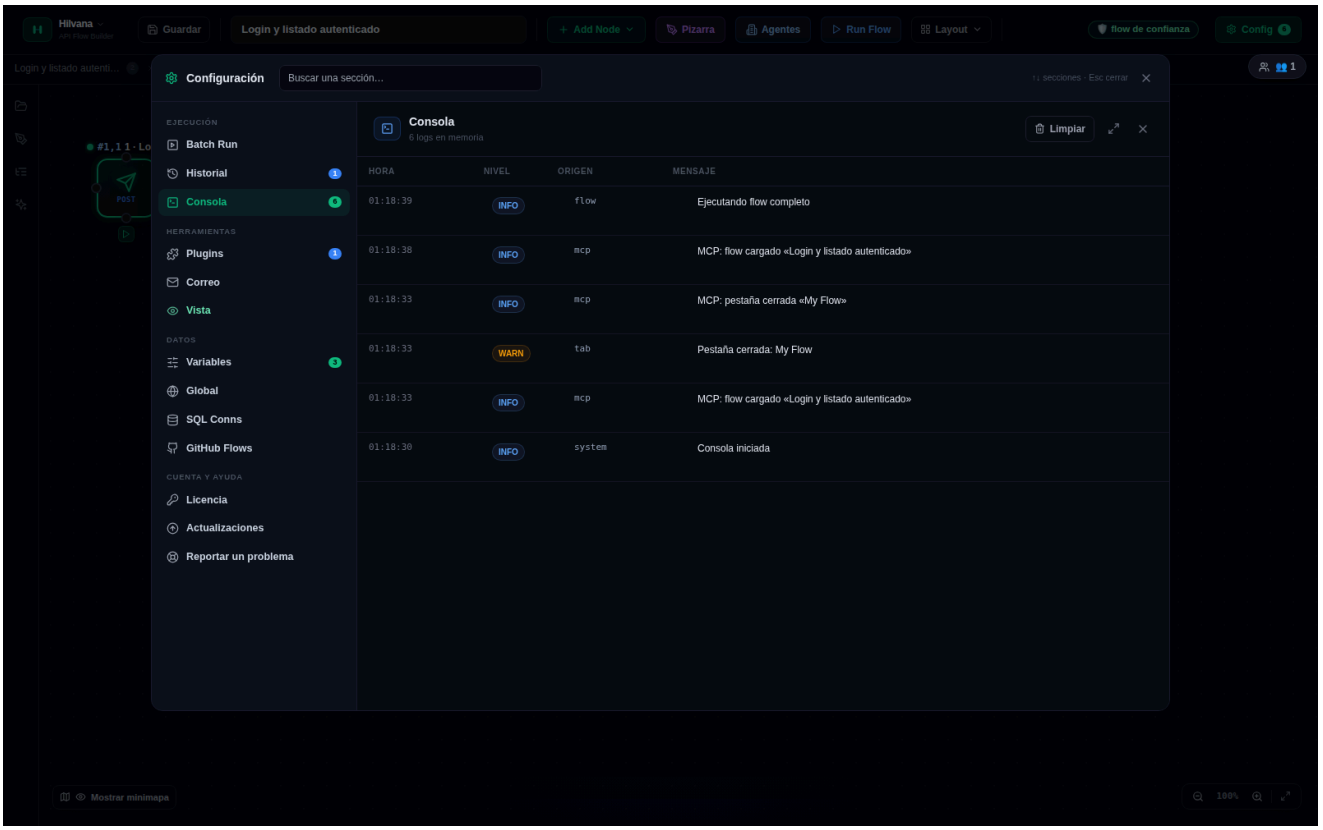
Historial

Cada **Run Flow** queda registrado: por nodo, la petición enviada (método, URL, headers, body) y la respuesta completa con duración. Es tu caja negra para «¿qué se envió exactamente hace 10 minutos?».



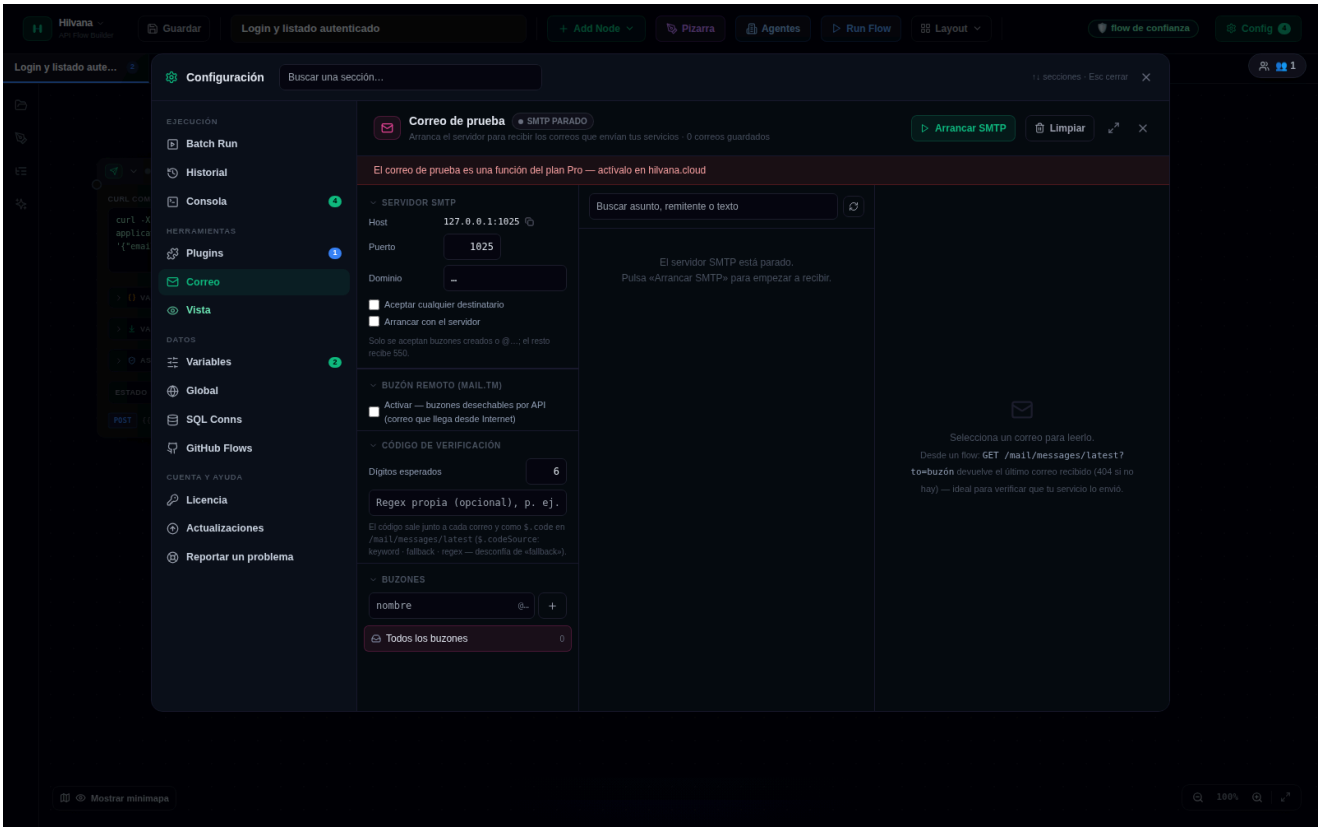
Consola

El log de la propia aplicación: nodos añadidos, ejecuciones, capturas, errores de parseo, comandos del MCP… Con niveles (info/success/warn/error) y filtro. Si algo «no hace nada», aquí suele estar el porqué.



Correo de prueba — SMTP embebido 🆕 (4.36)

Config → Correo abre un modal (como la Consola) con un **servidor de correo de mentira**: tus servicios le envían los correos de verdad (SMTP normal) y tú los ves aquí, sin crear cuentas reales ni que nada salga a Internet.



1. **Arrancar SMTP** (o marca «Arrancar con el servidor» para que arranque solo). Escucha en el puerto **1025** (`FLOW_MAIL_PORT`), sin TLS y con autenticación opcional: acepta cualquier usuario/contraseña.
2. **Buzones**: escribe `qa` y pulsa + → `qa@hilvana.local` (el dominio se edita en el panel, p. ej. `midominiotest.com`). Con **«Aceptar cualquier destinatario»** (por defecto) ni hace falta crearlos: toda dirección que reciba correo aparece sola en la lista con su contador. Si lo desmarcas, solo se aceptan los buzones creados y los del dominio; el resto recibe un `550`.
3. **Configura tu servicio**: host = la máquina de Hilvana, puerto `1025`, sin TLS (si tu librería insiste en STARTTLS, desactívalo: `ignoreTLS` / `secure:false` en nodemailer, `MAIL_ENCRYPTION=null` en Laravel, `EMAIL_USE_TLS=False` en Django…). En Docker publica el puerto: `-p 1025:1025` (o `flow:1025` desde otro contenedor de la misma red).
4. **Leer**: la lista se refresca sola cada 2 s; clic en un correo → remitente, destinatarios, fecha, tamaño, adjuntos (nombre/tipo/tamaño) y pestañas **HTML** (renderizado en un iframe aislado), **Texto** y **Origen** (el mensaje tal cual llegó). Búsqueda por asunto/remitente/texto, filtro por buzón, **Vaciar buzón** / **Limpiar**, borrar uno, borrar un buzón con sus correos.

Los correos se guardan en `flows/.mail/` (carpeta oculta: no sale en Proyecto ni en git; hasta 500, los más antiguos se descartan) y sobreviven al reinicio.

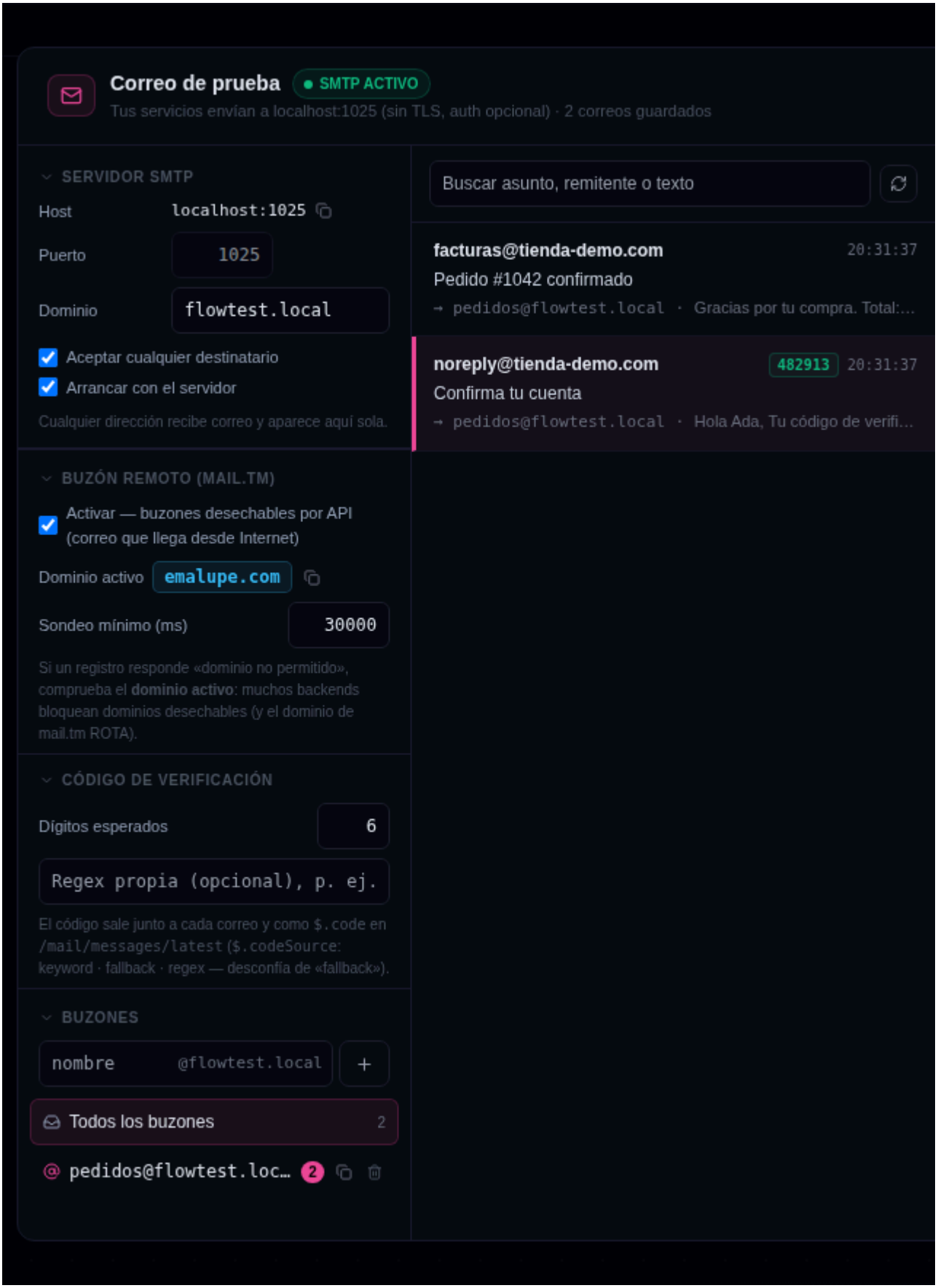
Comprobarlo desde un flow. El botón «Comprobar desde un flow» de cada buzón copia un `curl` listo para un nodo HTTP:

```
curl 'http://localhost:9998/mail/messages/latest?to=qa@hilvana.local&subject=Confirma'
# → { from, to, subject, text, html, attachments[], receivedAt, … } · 404 si no ha llegado nada
```

Así el flow «registro → el backend envía el correo → ¿llegó? → extraer `$.text` y sacar el código OTP o el enlace de confirmación → siguiente petición» se verifica de punta a punta. **Por MCP** (4.37):

`mail_server start` → `mail_address add` → acción bajo prueba → `mail_messages latest {to, subject}` — la IA lo hace sola, sin pestaña web (10 MCP). Filtros: `to`, `from`, `subject`, `since` (ms), `q`; `GET /mail/messages` lista, `DELETE /mail/messages?to=` vacía un buzón.

Extracción del código y buzones de Internet — mail.tm 🆕 (4.42)



El código, extraído. Cada correo sale con `code` y `codeSource` en `GET /mail/messages`, `GET /mail/messages/latest` y en el panel (badge verde junto al asunto):

- `keyword` — palabra clave (`código`, `code`, `otp`, `verificación`, `clave`, `pin`) + hasta 60 caracteres no numéricos + N dígitos. La vía fiable.
- `fallback` — primer grupo **aislado** de N dígitos si no hay palabra clave. Puede pescar basura (un año, un importe): desconfía.
- `regex` — si configuras una **regex propia** (manda el primer grupo de captura), útil para OTPs **alfanuméricos**, p. ej. `firma[^A-Z0-9]{0,40}([A-Z0-9]{6})`.

Se busca sobre el **texto plano** (si el correo solo trae HTML se deriva de él: un `<td width="600">` o un color hex no se cuelan). La longitud (6 por defecto) y la regex se configuran en el panel. Con esto, «registro → leer código → validar» son **3 nodos HTTP sin scripts**: el segundo hace `GET /mail/messages/latest?to=buzón` y extrae `$.code`.

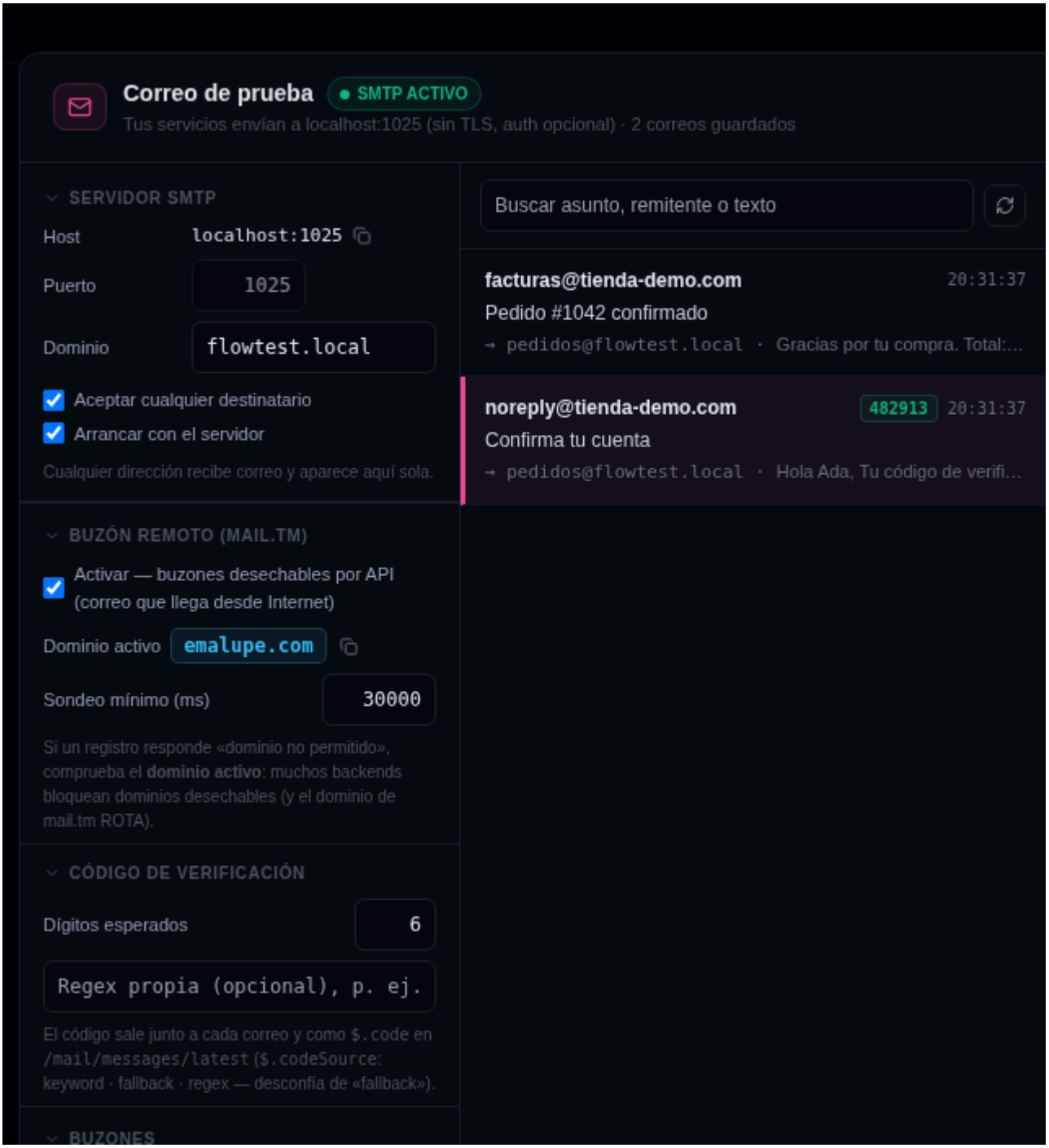
Buzones de Internet (mail.tm). El SMTP del 1025 solo sirve si el emisor puede abrir un TCP hasta tu máquina; cuando el correo lo envía un servicio en la nube, activa **Config → Correo ▸ Buzón remoto (mail.tm)** — conviven los dos, con **una sola lista** de correos (etiqueta `tm` en los remotos):

- **Dominio activo** bien visible — ¡rota!, y muchos backends bloquean listas de dominios desechables: si el registro responde «dominio no permitido», esto es lo primero que comprobar.
- **Crear buzón**: escribe el nombre y conmuta el sufijo `@…` al dominio de mail.tm — crea la cuenta real, guarda credenciales y token en `flows/.mail/config.json` (sobreviven al reinicio, sin variables de entorno) y **renueva el token** cuando caduca.
- **Auto-creación**: si un flow pregunta `latest?to=algo@<dominio-activo>` y el buzón no existe, se crea solo (el equivalente al «Aceptar cualquier destinatario» del SMTP). Mientras no llegue nada, `latest` sigue devolviendo **404** — dale margen con la pausa (`delayMs`) del conector.
- **Sondeo con cabeza**: mail.tm tiene rate limit — solo se consulta al pedir `/mail/*` (panel abierto o un flow preguntando) y como mucho una vez por intervalo (5 s por defecto, configurable). Panel cerrado y sin flows = **cero peticiones**.
- Borrar el buzón **borra también la cuenta remota**.

Por MCP: `mail_server` acepta `mailtmEnabled`, `mailtmPollMs`, `codeLength`, `codeRegex`; `mail_address add` acepta `provider: mailtm`; `mail_messages latest` devuelve `code` / `codeSource` ya extraídos.

Secciones plegables 🆕 (4.43)

La columna izquierda del panel ya son cuatro secciones **plegables** — clic en la cabecera (chevron) de **Servidor SMTP**, **Buzón remoto (mail.tm)**, **Código de verificación** o **Buzones**:



- Plegada, cada cabecera resume lo importante: el host del SMTP (o «parado»), el **dominio activo** de mail.tm (u «off»), los dígitos esperados del código (o «regex») y el número de buzones.
- Plegar **Buzones** oculta solo el alta de buzón nuevo; la lista de buzones sigue visible.
- El estado se recuerda entre sesiones (localStorage): configura una vez, pliega, y el panel queda para lo que se usa a diario — buzones y correos.

¿Y si el servicio que envía corre en otra máquina?

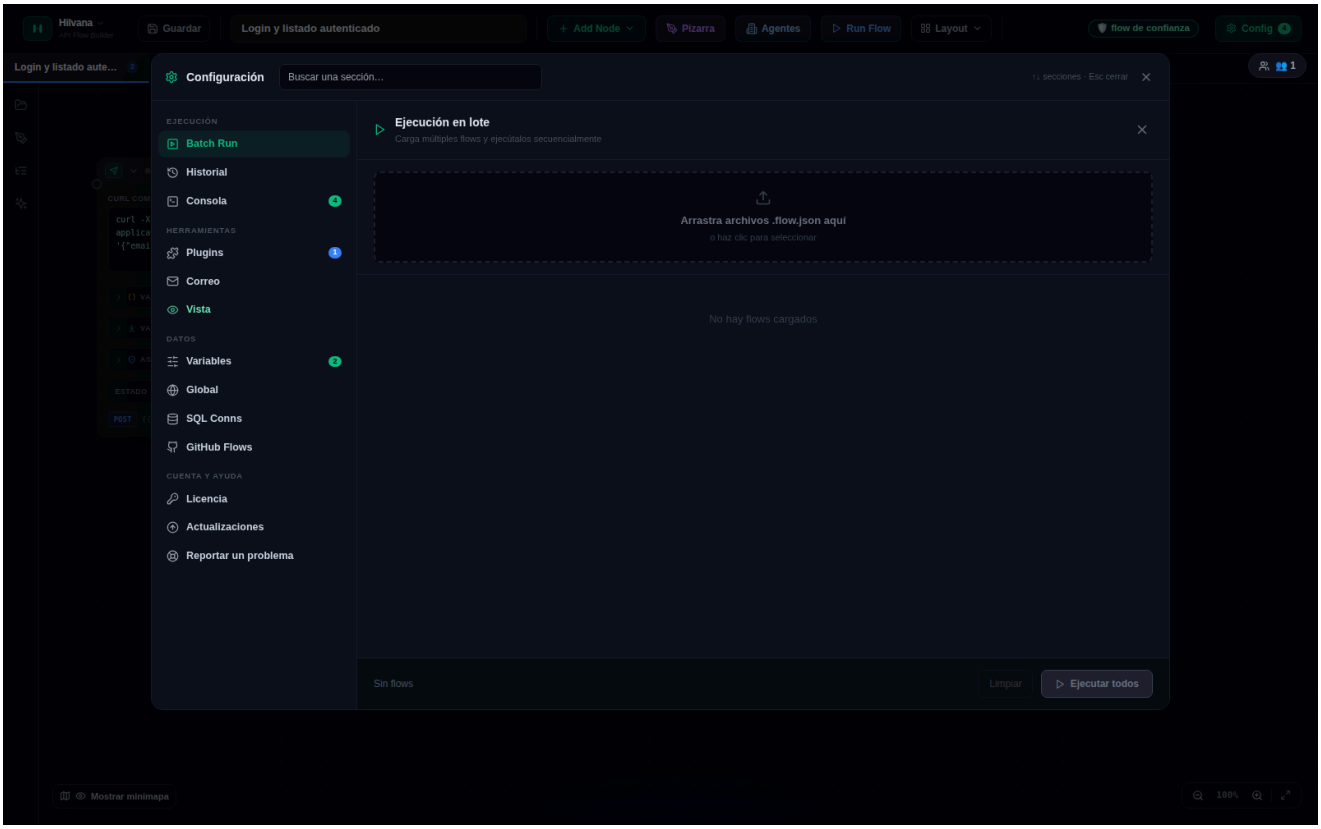
Llega igual: el SMTP escucha en todas las interfaces (`0.0.0.0:1025`); solo hace falta que esa máquina pueda abrir una conexión TCP al puerto 1025 de la que corre Hilvana.

Dónde está el servicio	Host SMTP a configurar	Qué tener en cuenta
Misma LAN / VPN	IP de tu máquina (`192.168.x.x`), puerto `1025`	Publica el puerto en Docker (`-p 1025:1025`) y ábrelo en tu firewall (`sudo ufw allow 1025/tcp`)
Otro contenedor de la misma red Docker	`flow:1025` (nombre del contenedor)	No pasa por el host ni necesita `-p`
Servidor en la nube / otra red sin ruta hacia ti	`localhost:1025` **en el servidor** + túnel inverso desde tu máquina: `ssh -R 1025:localhost:1025 usuario@servidor`	O `ngrok tcp 1025` / Cloudflare Tunnel y usas el host:puerto que te den
Hilvana desplegada en un servidor	`<servidor>:1025`	Publica el 1025 y ábrelo en el firewall / security group **solo a las IPs de tus servicios**: es un buzón abierto sin auth real

Prueba de conectividad desde la otra máquina antes de tocar el servicio: `nc -zv <host> 1025` — si contesta con el saludo SMTP (`220 …`), ya está. El puerto es `1025` y no `25` a propósito (el 25 lo bloquean muchos proveedores y requiere root); cámbialo en el panel, por MCP o con `FLOW_MAIL_PORT` .

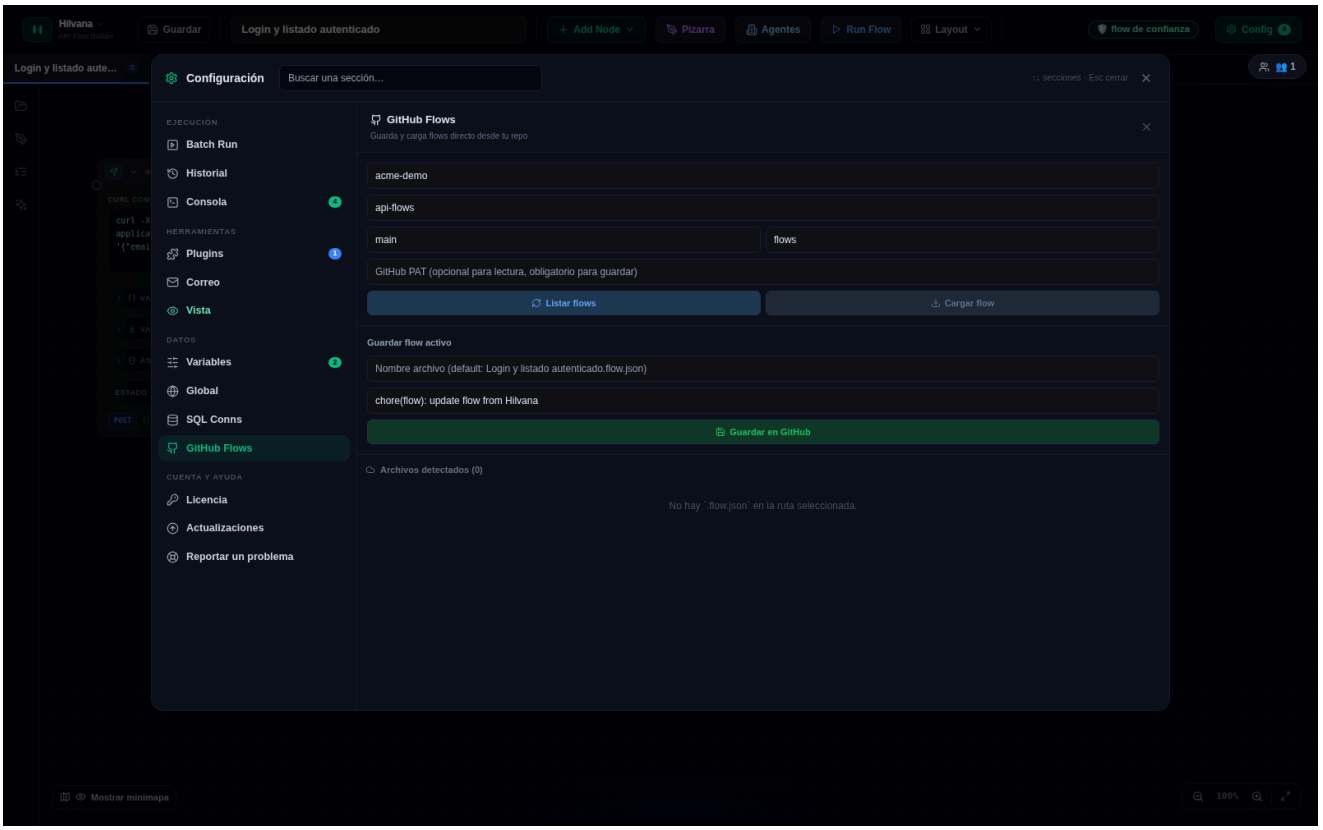
Batch Run

Ejecuta **varios flows en lote** (los seleccionas de tus pestañas o de archivos) y muestra el resultado de cada uno. La versión CI de esto es `flow-run --dir` (09 El CLI flow-run).



GitHub Flows

Conecta con un repo de GitHub (token + owner/repo/rama/carpeta) para **guardar y cargar flows directamente del repo** — el equipo comparte flows sin pasarse archivos.



Export / Abrir / guardado

- Todo se **autoguarda en localStorage** del navegador al momento: cierra y abre tranquilo.
- **Export** descarga el `.flow.json` del flow activo (estado limpio: sin respuestas ni contraseñas de runtime). Es el formato que consumen el CLI, el Batch, GitHub Flows y el MCP.
- **Abrir** (en la barra de pestañas) importa un `.flow.json` en una pestaña nueva.
- El indicador ámbar **«Sin guardar»** junto al nombre avisa de cambios no exportados (los dibujos de la pizarra también cuentan).

Collapse All · Expand All · Auto Layout · Alinear

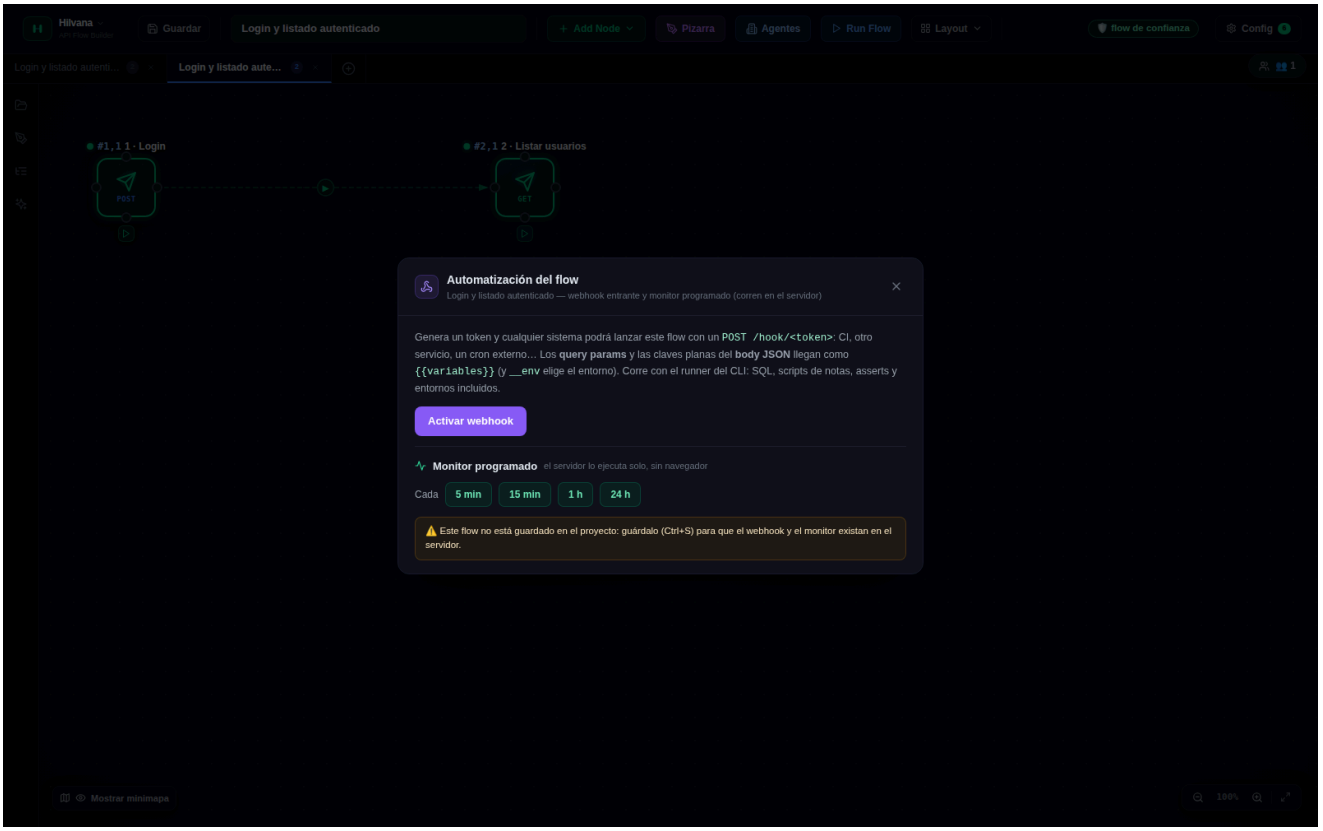
- **Reset**: limpia respuestas y estados (no borra nodos ni conexiones).
- **Collapse All**: colapsa todas las cajitas **y las acerca** — escala las distancias al ritmo al que encogen las cajas, **conservando tu disposición** (no reordena).
- **Expand All**: expande todo y **restaura las posiciones exactas** previas al Collapse All. Si expandes una cajita suelta, las de debajo se **empujan** para no pisarse.
- **Auto Layout**: reordena por el grafo — la cadena `next` en columnas, y los nodos colgados con conexión informativa (`none`) como satélites bajo su padre. Ordena **todos** los tipos de nodo.
- **Alinear en fila / en columna**: coloca los nodos según su **nº de orden** (campo `#` de cada cabecera; los que no tienen número van al final, en orden de lectura).
- Las cajas 📌 **fijadas** no se mueven con ninguna de estas acciones.

Cron

Los nodos Request, SQL, Web y Nota tienen pestaña **Cron**: ejecución periódica del nodo (cada X min/h) mientras la pestaña esté abierta. El badge con cuenta atrás aparece en la cabecera del nodo.

Webhook entrante 🆕 (5.3)

Una URL secreta que **ejecuta el flow en el servidor** al recibir un POST — desde CI, otro servicio o un cron externo, sin navegador.



Menú del logo → **«Webhook entrante…»** → *Activar webhook* genera el token y te da la URL (`POST /hook/<token>`). El webhook **se publica al guardar el flow** en `flows/` (Ctrl+S): el servidor lo lee del disco.

- Los **query params** y las claves planas del **body JSON** llegan como `{{variables}}` (el body gana); `__env=prod` elige el entorno.
- Ejecuta con el runner del CLI: SQL, scripts de notas, **asserts** y entornos incluidos — y el mismo candado de plan (en trial responde 402).
- Respuesta: **200** si todo verde, **422** con el detalle por nodo (incluidos los asserts que fallaron), **404** si el token no existe. El token es una credencial: *Regenerar* si se filtra.

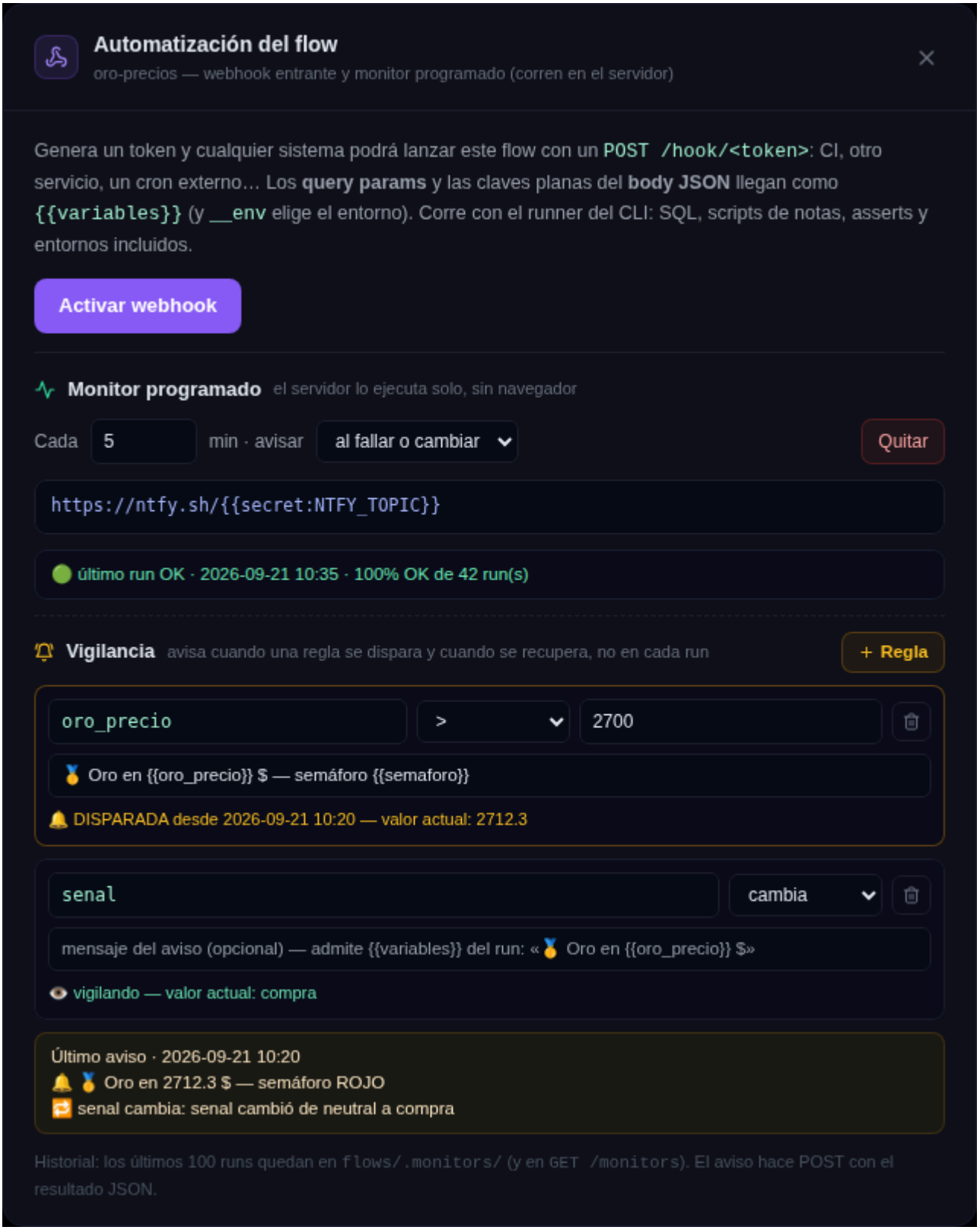
```
curl -X POST 'https://tu-flow/hook/abc123…' \
  -H 'content-type: application/json' -d '{"apiBase":"https://staging.mi-api"}'
```

Monitor programado 🆕 (5.4)

En el mismo modal **«Automatización…»** del menú del logo, bajo el webhook: elige un intervalo (5 min / 15 min / 1 h / 1 día, o el que tecleas) y el **servidor ejecuta el flow solo, sin navegador abierto** — la diferencia clave con el cron por nodo. Guarda los últimos 100 runs en `flows/.monitors/` (estado en vivo en el propio modal y en `GET /monitors`) y, si pones un `notifyUrl`, hace POST con el resultado **al fallar o cambiar de estado** (o en cada run): apunta a un webhook de Slack/Telegram/n8n… o al `/hook/` de otro flow para reaccionar en cadena. Se publica al guardar el flow (Ctrl+S).

Vigilancia: reglas y avisos 🆕 (5.17)

Un monitor sin reglas solo sabe decir «el flow falló». Con **reglas** vigila un **dato**: el servidor ejecuta el flow cada N minutos, mira las variables con las que termina el run (las **extracciones** y los **scripts «después»**) y te avisa **solo cuando algo cambia de estado** — no en cada ejecución.



1. Extrae el dato a una variable (JSONPath en el Request, columna en el SQL) o calcúlalo en un script «después» de una nota (`return Number(vars.oro_precio) > Number(vars.resistencia) ? 'ROTURA' : 'RANGO'`).
2. **Automatización… ▸ Monitor programado ▸ + Regla**: variable (el campo sugiere las que produce el flow), operador y valor. Operadores: `>` `≥` `<` `≤` `=` `≠`, **contiene** / **no contiene** y **cambia** (avisa cuando el valor es distinto al de la ejecución anterior). Los números se entienden aunque vengan como `2.650,40` o `2650.4 USD`.

3. Opcional: un **mensaje** propio con `{{variables}}` del run — «🥇 Oro en {{oro_precio}} $».
4. Pon un `notifyUrl` y **guarda** (Ctrl+S): la vigilancia vive en el servidor.

Cuándo avisa. 🔔 cuando una regla **se dispara** (la condición pasa a cumplirse), ✅ cuando **se recupera** (deja de cumplirse) y 🔁 cuando un valor vigilado con «cambia» es distinto. Un precio que lleva tres horas por encima del nivel manda **un** aviso, no treinta y seis. Si en una ejecución el dato no llega (la API falló), la regla **conserva su estado** y el aviso que recibes es el del fallo.

Qué recibe el `notifyUrl`. Un POST JSON con `text` y `message` (el mismo resumen legible, una línea por alerta — lo que esperan Slack, Telegram, Mattermost, ntfy o n8n), `alerts` (`kind: fired | recovered | changed`, `message`, `variable`, `value`), `rules` (estado de cada regla) y el resultado del run. La URL admite `{{secret:NOMBRE}}` (Variables ▸ 🔒 Credenciales): el token de tu bot no se guarda en el `.flow.json`.

```
https://ntfy.sh/{{secret:NTFY_TOPIC}}
https://api.telegram.org/bot{{secret:TG_BOT}}/sendMessage?chat_id=123456
https://hooks.slack.com/services/{{secret:SLACK_HOOK}}
```

El modal enseña el estado de cada regla (🔔 **disparada desde…**, 🟢 en calma, 👁️ vigilando, ❔ sin dato) y el **último aviso**. El historial solo guarda el valor de las variables vigiladas, nunca el resto. Una IA puede montarlo todo con la tool MCP `flow_monitor` (y consultar `status` sin navegador).

Colaboración en vivo 🆕 (5.5 · exclusiva Business)

Dos (o más) personas con **el mismo fichero de `flows/` abierto** se ven editar en tiempo real:

- **Presencia**: chip flotante 👥 **N** (arriba a la derecha) con quién está conectado y en qué flow (verde pulsante si alguien comparte el tuyo); tu nombre es editable desde el propio chip y las filas del panel Proyecto muestran un punto de color por cada compañero que tenga ese fichero abierto.
- **Edición sincronizada por nodos**: mueves una caja, cambias un curl, dibujas en la pizarra… y el resto lo ve al instante, con una etiqueta «✏️ nombre» sobre el nodo tocado. Cada uno conserva SUS respuestas y ejecuciones (se comparte el documento, no el run).
- **Aviso de pisada**: si dos tocáis lo mismo a la vez, gana el último **y el que pierde se entera siempre** — toast «⚠️ X sobrescribió tu cambio en “nodo”» + línea en la Consola, con la salida a mano: **Ctrl+Z recupera tu versión** (y se la reenvía al resto).
- **Guardado coordinado**: cuando un compañero guarda (Ctrl+S), tu pestaña adopta el fichero y queda limpia — sin conflictos de «cambió en disco».

Disponible con **plan Business** (cloud) o licencia team/enterprise (self-hosted); sin él, la función ni aparece. El fichero sigue mandando: la colaboración sincroniza canvases vivos, quien guarda escribe.

Chat del flow 🆕 (5.30 · Business)

En el chip 👥 de la colaboración en vivo hay una pestaña 💬 **Chat**: hablan quienes tienen abierto **el mismo flow**, y la conversación es el registro de ese flow.

- **Queda guardada con el flow**: en un fichero junto a él (`login.flow.json` → `login.chat.jsonl`). Al abrir el flow ves todo el historial; si lo mueves, renombras o borras, su

chat va con él. No viaja al compartir un enlace ni al exportar.

- **Quién escribe**: en el **cloud** apareces con tu usuario de login (no se puede cambiar). En un **Docker propio** el nombre es editable y se propone a partir del email de la cuenta vinculada; sin vincular ni licencia, el chat no está disponible.
- **Enter** envía y **Shift+Enter** hace salto de línea. Texto plano, hasta 2000 caracteres. Los mensajes no se editan ni se borran.
- **Registro por seguridad**: como avisa el propio chat, cada mensaje se registra también en el servicio de Hilvana y se conserva **12 meses**.

Deshacer y rehacer del canvas 🆕 (5.5 · todos los planes)

Ctrl+Z / Ctrl+Y (o Ctrl+Shift+Z) deshacen y rehacen los cambios **estructurales** del flow activo: mover/crear/borrar nodos, conexiones, pizarra, variables… (50 pasos por pestaña). Las respuestas y resultados se conservan al deshacer. Dentro de un campo de texto sigue mandando el undo nativo, y si tu último clic fue en la pizarra, el atajo deshace la pizarra (como siempre).

Panel Proyecto compacto 🆕 (5.5)

Las filas de fichero son ahora **una sola línea estilo árbol** (icono + nombre + los mismos botones, en fantasma): cabe más del doble de proyecto en pantalla. Los detalles (nº de nodos, dibujos, fecha) viven en el tooltip; el JSON inválido se marca con un chip rojo `!JSON`. Y desde la 5.5.0 los documentos `.md` del proyecto se abren **en cualquier plan** (el tour de la galería incluido).

Carpetas privadas por miembro 🆕 (5.6 · cloud)

En una organización del cloud, cada miembro tiene su carpeta **`privado/<usuario>/`** 🔒 dentro del workspace compartido: solo su dueño la ve en el panel Proyecto y solo él puede abrir, guardar o borrar lo que hay dentro (los demás reciben un 403 — ni siquiera aparece en sus listados). El resto del árbol sigue siendo el espacio común del equipo de siempre. En self-hosted no aplica: sin cuentas, tu máquina es toda tuya.

Guardar con ámbito (5.7): el modal «Guardar como…» (Ctrl+Shift+S) trae en el cloud un selector 👥 **Compartido** / 🔒 **Privado** — al elegir Privado el flow va a tu `privado/<usuario>/` sin teclear la ruta — y muestra las carpetas existentes como **chips clicables** (filtradas por ámbito) en vez de una caja de texto a ciegas. La ruta final y su aviso se actualizan en vivo.

Límite de flows por plan (5.7): Trial 3 · Pro 100 (+packs de 100) · **Business 1.000** (10× Pro). Al llegar al tope, guardar un flow NUEVO devuelve un aviso (los existentes se siguen editando). Los flows de fábrica (bienvenida y `ejemplos/`) no cuentan.

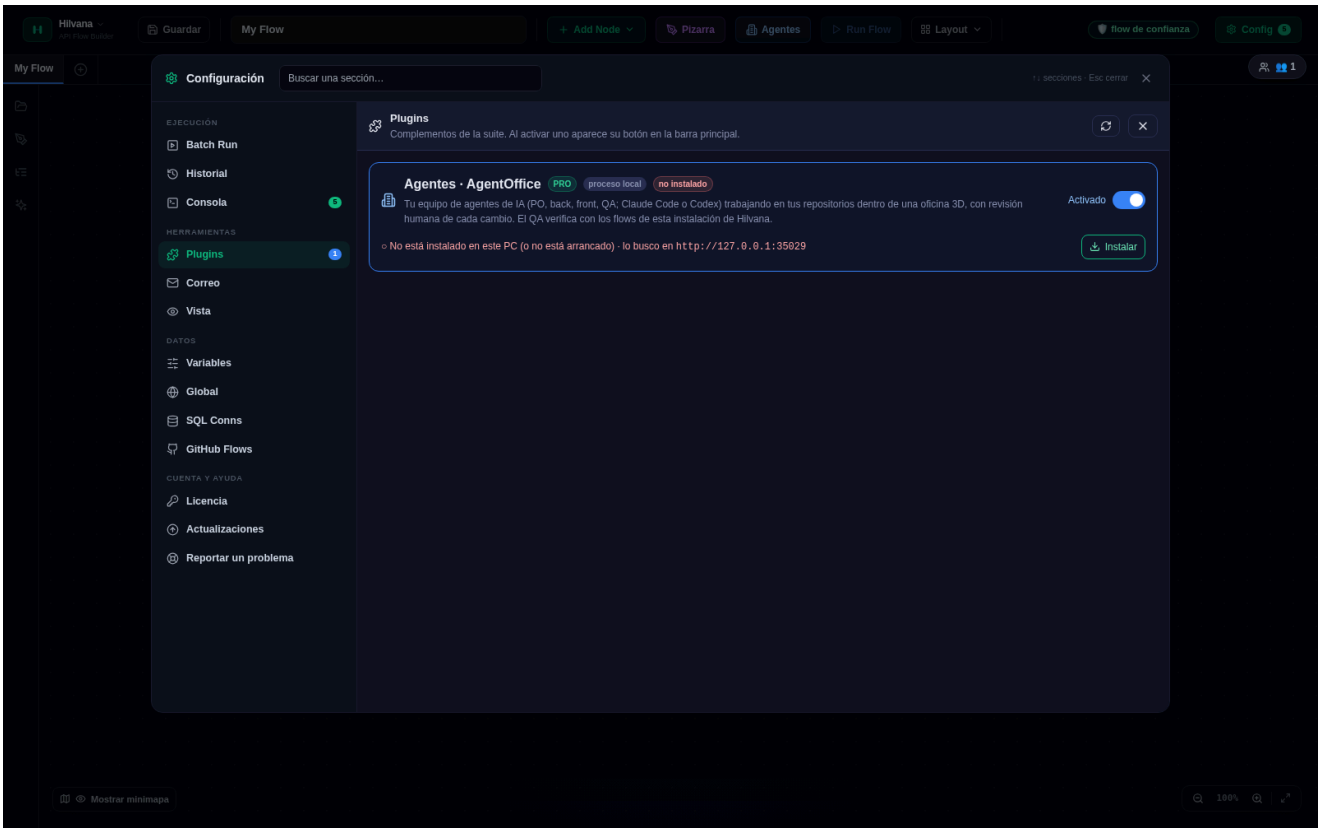
AgentOffice 🆕

Todo lo de AgentOffice (instalación y actualización, Resumen, Oficina 3D, Tareas, Equipo, Ajustes sección a sección, opciones de entrega, modos de desarrollo web/Blender, burbuja del agente, Guía, marketplace y móvil) está en el **capítulo 15 · AgentOffice**, con una captura por pantalla.

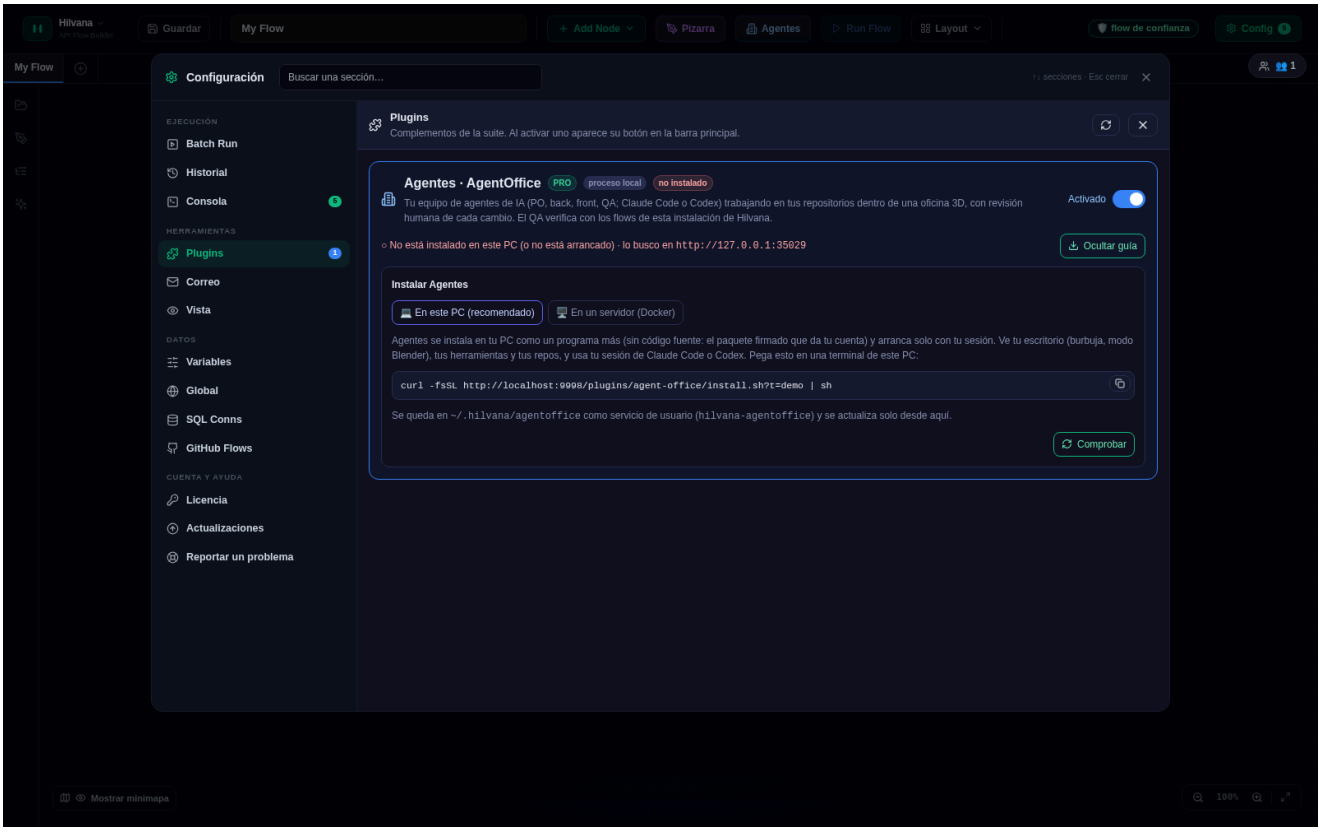
Aquí solo queda lo que pertenece al panel de plugins de Hilvana: **Config ▸ Plugins** lista los plugins instalados y permite activar o desactivar AgentOffice (con el plugin desactivado, `/agents` responde 403).

Instalar AgentOffice desde Plugins (6.1.0 – 6.2.1)

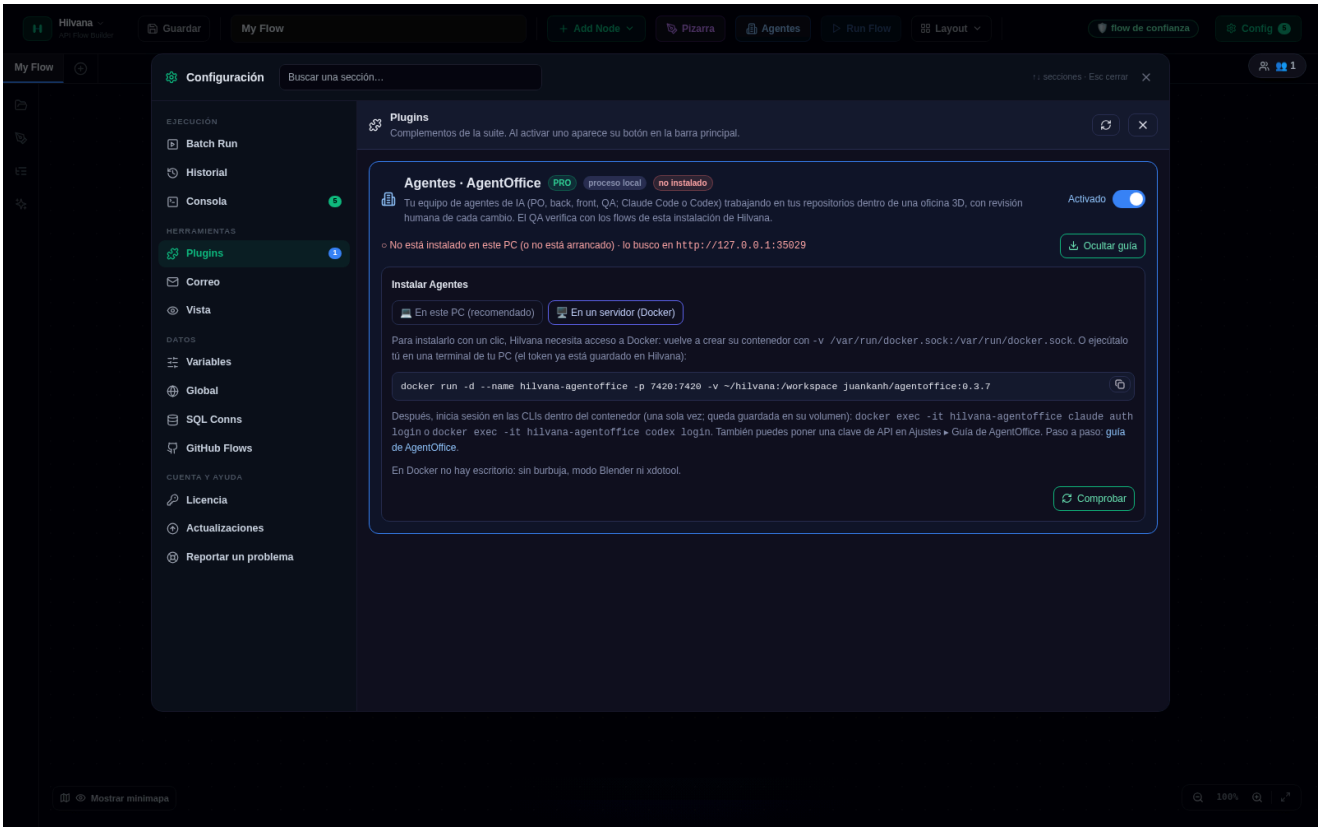
La tarjeta **Agentes · AgentOffice** (etiquetas PRO y «proceso local») lleva un interruptor **Activado / Desactivado** a la derecha. Según el estado muestra una cosa u otra:

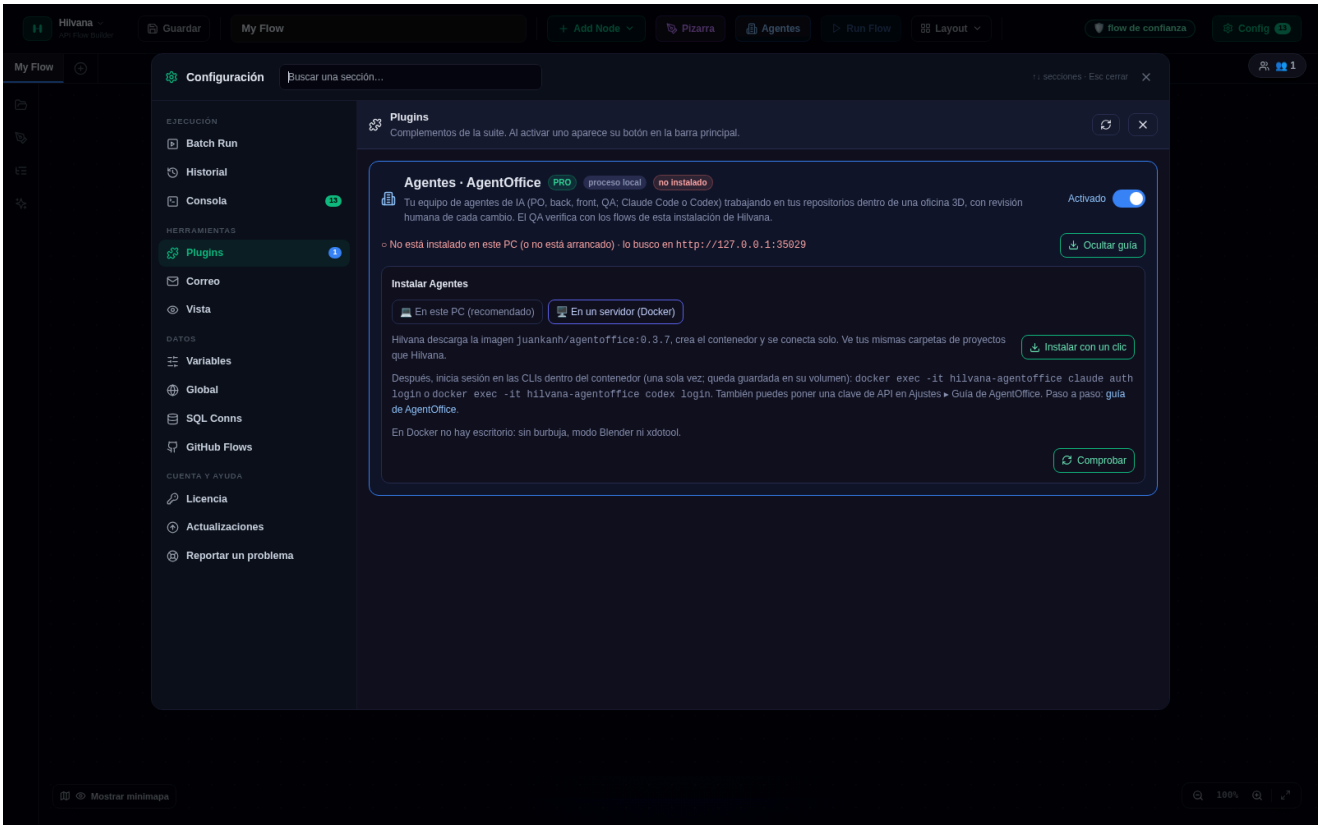


Cuando AgentOffice está activado pero no instalado, su tarjeta dice ○ **No está instalado en este PC (o no está arrancado)** y enseña la dirección donde lo busca (por defecto `http://127.0.0.1:7420`) y el botón verde **Instalar** (pasa a «Ocultar guía»). Si ya corre, la tarjeta muestra ● **En marcha** con proyectos, agentes y cuántos trabajan, y el botón **Abrir**. La guía tiene dos pestañas y un botón **Comprobar** que vuelve a mirar si ya responde.

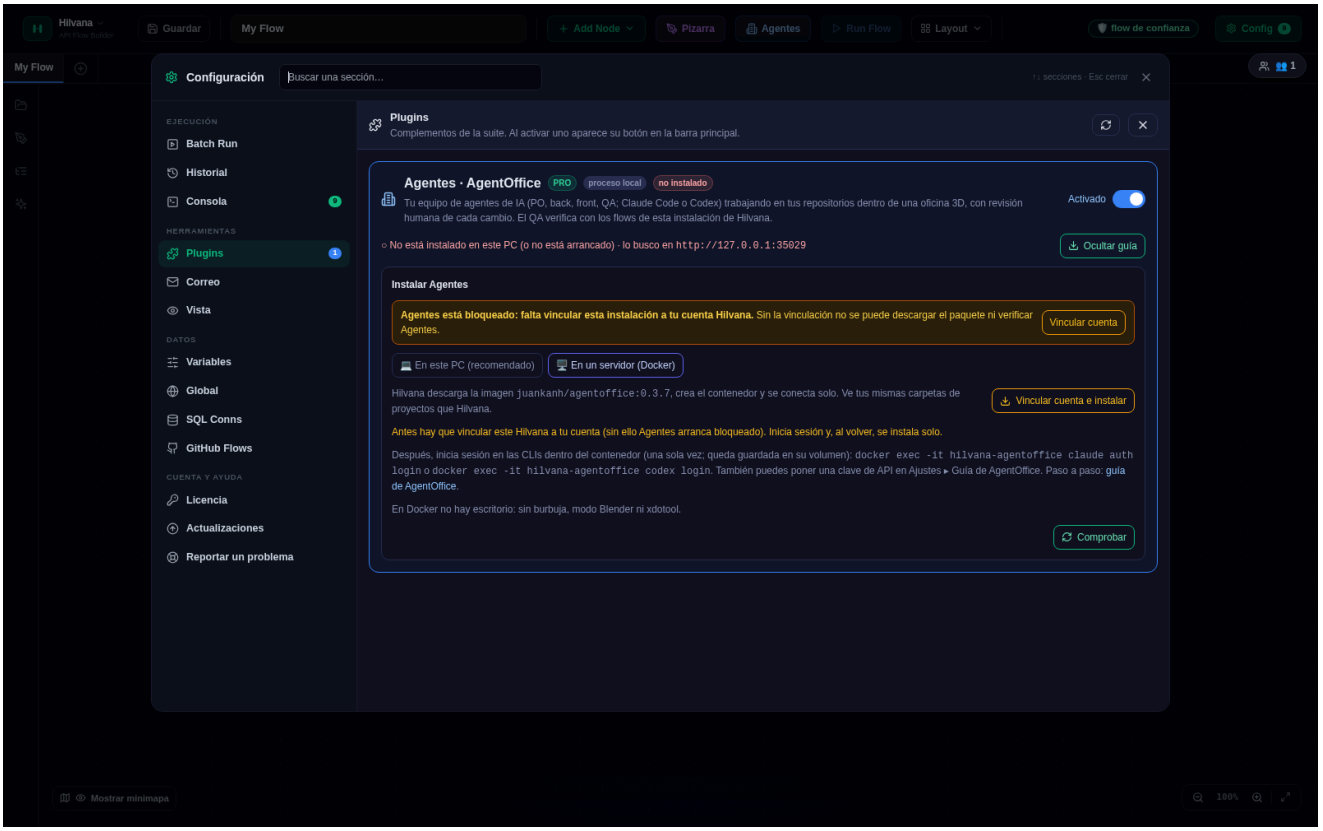


- 💻 **En este PC (recomendado)** — es la pestaña por defecto en Linux. Enseña una orden `curl … | sh` con botón de copiar: pégala en una terminal de este PC. Instala el paquete firmado que da tu cuenta, lo deja en `~/.hilvana/agentoffice` como servicio de usuario `hilvana-agentoffice`, lo empareja con este Hilvana y se actualiza solo desde aquí. Al ser nativo ve tu escritorio (burbuja de los agentes, modo Blender), tus herramientas y tu sesión de Claude Code o Codex. Mientras espera aparece «⏳ Esperando a que arranque en …» y, al responder, «✓ Conectado». En Windows y macOS la pestaña avisa de que el instalador nativo es solo para Linux: usa Docker.
- 🖥️ **En un servidor (Docker)** — para una máquina sin escritorio. Si este Hilvana tiene acceso a Docker, el botón **Instalar con un clic** descarga la imagen `juankanh/agentoffice`, crea el contenedor `hilvana-agentoffice` y se conecta solo; muestra el progreso («⏳ …», «✓ Instalado …» o «✗» con el motivo) y, si ya existe, el botón pasa a **Reinstalar**. Si Hilvana no ve Docker, explica que hay que recrear su contenedor con `-v /var/run/docker.sock:/var/run/docker.sock` y da el `docker run` completo para copiar, con el token ya preparado. Después hay que iniciar sesión en las CLIs dentro del contenedor, una sola vez: `docker exec -it hilvana-agentoffice claude auth login` (o `codex login`); también vale una clave de API en Ajustes ▸ Guía de AgentOffice. En Docker no hay escritorio: sin burbuja, modo Blender ni xdotool. En el cloud AgentOffice lo gestiona Hilvana y no hay instalación.



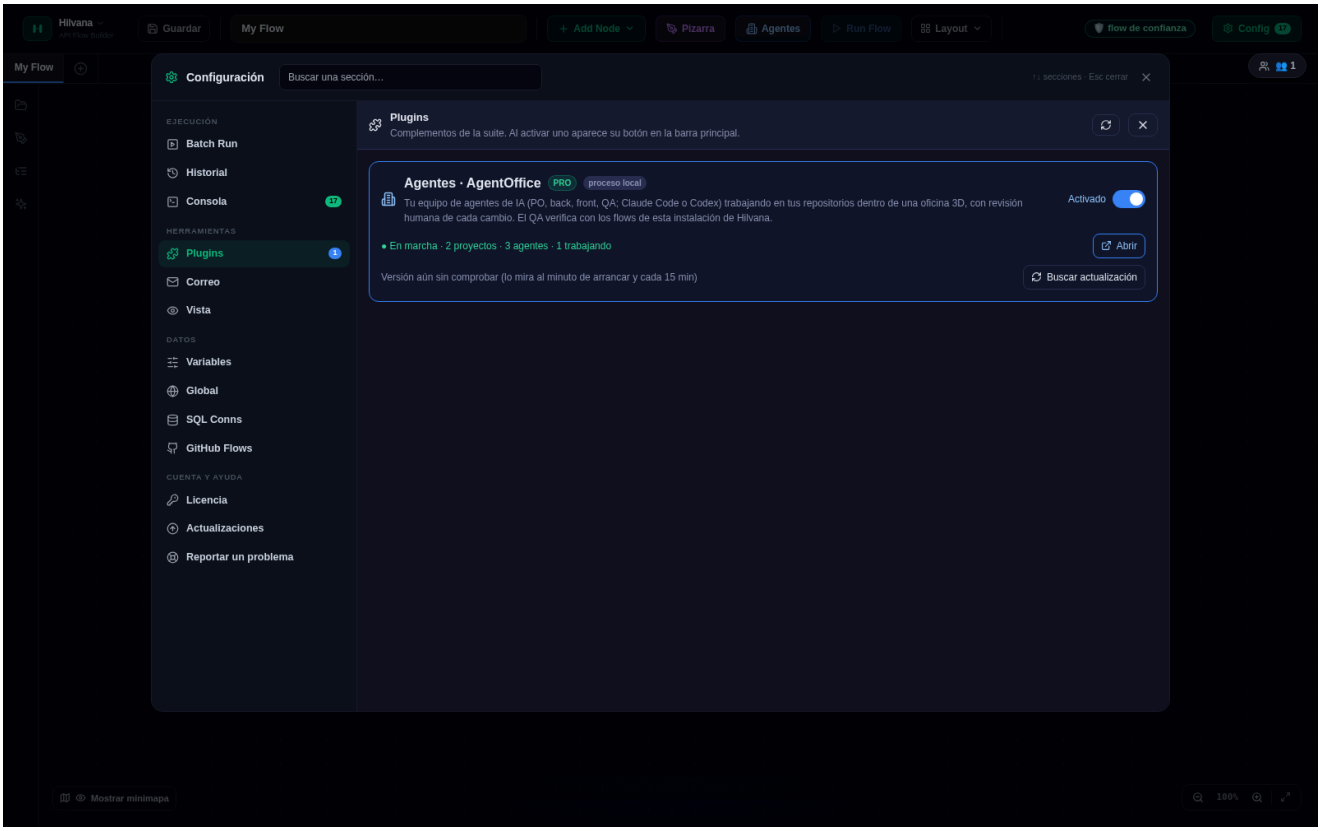


Vincular la cuenta primero.



AgentOffice solo se descarga y se verifica con este Hilvana vinculado a tu cuenta. Si no lo está, la guía abre con un aviso ámbar («AgentOffice está bloqueado: falta vincular esta instalación a tu cuenta Hilvana») y botón **Vincular cuenta**; en la pestaña Docker el botón de instalar se convierte en **Vincular cuenta e instalar**. Te lleva a Config ▸ Licencia; al iniciar sesión y volver a Plugins, la instalación en Docker sigue sola, sin otro clic. Si AgentOffice ya está instalado y arrancado pero sin vincular, la tarjeta muestra el mismo aviso.

Ya instalado y en marcha. La tarjeta pasa a «● En marcha · N proyectos · N agentes · N trabajando» con el botón **Abrir** (carga AgentOffice dentro de Hilvana) y, debajo, la línea de versión con **Buscar actualización**. Cuando hay una versión nueva aparece «⬆ Versión nueva …» y el botón pasa a **Actualizar** (la etiqueta «⬆ actualización» también se ve junto al nombre); si falta reiniciar tras actualizar, la tarjeta lo avisa.



Errores típicos: «Comprobando…» largo (Hilvana no alcanza el puerto 7420), el motivo en rojo cuando el servicio no responde (comprueba que `hilvana-agentoffice` esté en marcha) y el aviso de Docker cuando falta el socket.

⌨️ 09 · El CLI flow-run

> Revisado para Hilvana **6.2.1** (herramienta de terminal, sin pantalla propia).

Los mismos `.flow.json` del canvas, ejecutados **desde terminal**: para CI, cron del sistema, o baterías de regresión. Sin navegador.

```
cd hilvana
npm run flow:run -- flows/api-store.flow.json    # un flow
npm run flow:batch -- flows                      # todos los de una carpeta (recursivo)

# o directamente / instalado global con npm link:
node cli/run-flow.js --flow flows/mi.flow.json --var apiBase=http://localhost:8080
flow-run /ruta/a/flows
```

- Ejecuta los nodos en el **mismo orden topológico** que la web, con extracciones e interpolación — paridad total.
- **Exit code 1 si algo falla** → válido como test de CI.
- Ejecuta también los **sqlNodes** (perfiles: `--sql-connections <file>` o el `sql-connections.json` más cercano al flow).

Flags

Flag	Qué hace
`--flow <file>` / `--dir <dir>`	Un flow o una carpeta entera (ordenada por nombre)
`--var clave=valor`	Añade/sobrescribe variables (repetible)
`--timeout <ms>`	Timeout por petición (defecto 180000)
`--continue-on-error`	Sigue con el resto de nodos tras un error
`--skip-sql-nodes`	Ignora los sqlNodes (¡un batch toca BBDD reales!)
`--sql-connections <file>`	Perfiles SQL
`--out <file-o-dir>`	Log JSON de la ejecución
`--report-root <dir>` / `--no-report`	Carpeta de informes (defecto `resumen/`) o desactivarlos
`--env <nombre>` 🆕 5.3	Entorno con nombre: aplica `environments[nombre]` sobre las variables (sin la flag, vale el entorno activo guardado en el flow)
`--data <fichero>` 🆕 5.3	Data-driven: CSV (primera fila = nombres de variable) o JSON array — el flow corre una vez por fila con sus columnas como variables; columna `name` etiqueta el caso. Requiere `--flow`
`--no-localhost-rewrite`	No reescribir localhost→host.docker.internal (dentro de Docker)

> **`-F con ;type=`** (5.27): en una petición multipart, un campo de texto con tipo (`-F 'meta={"a":1};type=application/json'`) envía ya la cabecera `Content-Type` de esa parte, como hace curl (lo necesitan p. ej. los `@RequestPart` JSON de Spring). La web sigue sin enviar multipart: esos flows ejecútalos con el CLI.

Informes

Cada ejecución deja en `resumen/<fecha-hora>/` un **markdown** por flow con el detalle petición/respuesta de cada nodo, más un índice. Los informes son la referencia de «qué pasó» sin tener la web abierta.

> **Consejo: Los flows como tests** El patrón que usamos en el Core y la tienda: una batería de flows por API en `flows/`, y `flow-run --dir flows` en CI. Si un endpoint rompe el contrato, el flow falla y el informe te dice exactamente dónde.

Asserts, data-driven y CI 🆕 (5.3)

- Los **asserts por nodo** (definidos en la app — 02 del manual) se evalúan también en el CLI: un assert KO marca el nodo como fallido, se imprime bajo la caja (`✗ assert: $.rates.USD exists — real: (sin valor)`) y el proceso **sale con código ≠ 0** — listo para el pipeline.
- **Data-driven**: `flow-run --flow api.flow.json --data casos.csv` ejecuta el flow una vez por fila; el resumen agrega los casos (`Summary: 1/3 flows passed`) y cada fallo sale con su nombre de caso.
- **En CI**: el runner va dentro de la imagen — `docker run --rm -v "$PWD/flows:/app/flows" juankanh/hilvana node cli/run-flow.js --dir flows` (función **Pro**: pasa tu licencia con `-e FLOW_LICENSE=…` desde los secrets del CI). El paquete npm del runner (`flow-run` + `flow-bridge`, el puente cloud → tu máquina) **aún no está publicado**.

Todos los flags 🆕 (5.38.1)

Entrada: `--flow <f>` (un flow) · `--dir <d>` (lote, recursivo) · `--help`. Datos y variables: `--var k=v` (sobrescribe variables) · `--env <entorno>` (entorno de `Flow.environments`) · `--data <fichero>` (data-driven: una pasada por fila). Ejecución: `--timeout` (por defecto 180 s) · `--continue-on-error` · `--no-localhost-rewrite` (en Docker `localhost` → `host.docker.internal`) · `--skip-sql-nodes` · `--skip-info-scripts`. SQL: `--sql-connections <fichero>` (perfiles; por defecto `flows/sql-connections.json`). Salida: `--out` · `--report-root` · `--no-report` (informes en `resumen/`). Exit 1 si algo falla.

🤖 10 · El MCP embebido

> Revisado para Hilvana **6.2.1** (servidor MCP en `/mcp`, sin pantalla propia).

> **5.13** — Las mismas herramientas mueven el **asistente de IA embebido** (13): un chat dentro de la app con tu clave de Claude u OpenAI que las llama en proceso, sin necesidad de Claude Code ni de configurar el MCP.

Hilvana lleva un **servidor MCP dentro** (endpoint `/mcp`): cualquier cliente MCP —Claude Code, Claude Desktop— puede **construir, editar y ejecutar flows en tu canvas mientras miras**. La IA pinta las cajitas en directo en la última pestaña del navegador que esté conectada (la «controladora»).

Conectarlo a Claude Code

```
# contra el dev (npm run dev)
claude mcp add --transport http hilvana http://localhost:3001/mcp

# contra el contenedor Docker
claude mcp add --transport http hilvana http://localhost:9998/mcp
```

Y en una conversación: *«créame un flow que haga login en mi API y liste los usuarios»* — verás aparecer las cajitas, conectarse y ejecutarse solas.

Las 39 tools, por grupos

Grupo	Tools	Para qué
Observar	`bridge_status`, `flow_state`, `console_read`, `runs_read`, `sql_connections_list`	Ver pestañas, nodos (posición, orden, 📌, notas, web, dibujos, fichero enlazado, sin guardar, `settings` del flow 🆕 4.48), consola e historial
Pestañas	`flow_create`, `tab_select`, `tab_rename`, `tab_close`, `flow_overwrite`	Crear, activar, renombrar, cerrar o reemplazar pestañas
Construir	`node_add_request`, `node_add_sql`, `node_add_info`, `node_add_web`, `node_update`, `node_delete`, `nodes_connect`, `connection_update`, `connection_delete`, `variables_set`, `global_variables_set`	Crear/editar cualquier tipo de nodo (HTTP, SQL, nota/Mermaid/captura con scripts, web), conexiones y variables
Ejecutar	`flow_run`, `node_run`, `flow_reset`	Como pulsar Run Flow / el ▶ de un nodo (devuelve resultados) / Reset. `flow_run` ejecuta antes los scripts JS de las notas

Grupo	Tools	Para qué
Lienzo 🆕 4.26	`node_focus`, `canvas_layout`, `whiteboard_update`	Centrar y resaltar un nodo, ordenar el lienzo (auto / fila / columna / **cuadrícula** por `cell {col,row}` 🆕 4.28 / separar solapes 🆕 4.33 / colapsar / expandir / fijar-liberar todas 🆕 4.29), **`view_settings`** (tamaño de nodos, modo compacto, separación 🆕 4.33), **`flow_background`** (imagen de fondo del flow 🆕 4.49) y dibujar en la pizarra
Proyecto `flows/`	`flow_files_list`, `flow_open`, `flow_save`, `flow_file_delete`, `flow_file_read`	Lo mismo que el panel Proyecto: listar con metadatos, abrir enlazado al fichero, guardar como Ctrl+S (la pestaña queda enlazada), borrar, leer
Correo de prueba 🆕 4.37	`mail_state`, `mail_server`, `mail_address`, `mail_messages`	Arrancar/parar/configurar el SMTP de prueba (08 Paneles), crear buzones y leer los correos recibidos (`latest` con filtros `to` / `subject`) para verificar que tu servicio los envió y sacar el OTP/enlace. No necesitan pestaña web
Vigilancia 🆕 5.17	`flow_monitor`	Programar un flow en el servidor (cada N min, sin navegador) con reglas sobre sus variables y aviso cuando una se dispara o se recupera (08)

> `flow_overwrite` acepta un `.flow.json` completo — también con nodos Captura y con los dibujos de la pizarra (`drawings`): una IA puede cargarte una documentación entera, anotada, en la pestaña.

Seguridad

- Por defecto solo acepta clientes **locales** (localhost).
- `FLOW_MCP_TOKEN=xxx` exige `Authorization: Bearer xxx`.
- `FLOW_MCP_ALLOW_LAN=true` abre el acceso a la LAN (usa el token si lo haces).

> **Aviso: La pestaña controladora** El puente controla **la última pestaña conectada**. Si tienes abiertas la web del dev (5173) y la del contenedor (9998), cada servidor tiene su propio puente — asegúrate de a cuál apunta tu cliente MCP, o la IA pintará «en la otra».

Conectar tu IA al espacio del cloud 🆕 (5.8)

Además del MCP local (`localhost:9998/mcp`), si tienes cuenta en **app.hilvana.cloud** puedes conectar tu asistente a **tu espacio del cloud**:

1. En tu cuenta → 🤖 **MCP (IA)** → **Generar token MCP** (se muestra una vez).
2. Copia el comando ya montado y pégalo en tu terminal: `claude mcp add --transport http flow-cloud https://app.hilvana.cloud/mcp --header "Authorization: Bearer <tu-token>"`

A partir de ahí la IA construye y ejecuta flows en tu org del cloud igual que en local — respetando tus **carpetas y credenciales privadas**. El token es personal (Pro o Business), se lista con su último uso en

tu cuenta y en el admin, y se revoca cuando quieras. Máximo 5 por usuario.

Las tools, por grupos 🆕 (5.38.1)

- Observar: `bridge_status`, `flow_state`, `console_read`, `runs_read`.
- Pestañas/flows: `flow_create`, `flow_overwrite`, `flow_reset`, `tab_select`, `tab_rename`, `tab_close`, `flow_background`, `view_settings`.
- Nodos: `node_add_request`, `node_add_sql`, `node_add_info`, `node_add_web`, `node_update`, `node_delete`, `node_focus`.
- Conexiones y variables: `nodes_connect`, `connection_update`, `connection_delete`, `variables_set`, `global_variables_set`, `sql_connections_list`.
- Ejecutar/canvas: `flow_run`, `node_run`, `canvas_layout`, `whiteboard_update` (pizarra y sticky), `flow_monitor` (vigilancia).
- Ficheros de `flows/`: `flow_save` (409 salvo `force`), `flow_open`, `flow_files_list`, `flow_file_read`, `flow_file_delete`.
- Correo: `mail_state`, `mail_server`, `mail_address`, `mail_messages`.

🐳 11 · Docker y despliegue

> Revisado para Hilvana **6.4.0** · capturas: 117.

Ejecutar

```
docker run -d --add-host=host.docker.internal:host-gateway \
  -p 9998:3001 -p 1025:1025 --name hilvana juankanh/hilvana:6.4.0
# web + API + MCP en http://localhost:9998 · 1025 = SMTP de prueba (Config ▸ Correo, 4.36)
```

Desde el móvil

La misma instalación sirve una **vista móvil** en `http://tu-servidor:9998/m` (5.11): ejecutar flows, ver el resultado y leerlos como documentación, sin descargar el editor. Ver 12 La vista móvil.

Construir

```
cd hilvana   # repo con código fuente
docker build -t juankanh/hilvana:<version> .
```

Peculiaridades dentro del contenedor

- **localhost se reescribe** a `host.docker.internal` automáticamente (flows que apuntan a APIs de tu máquina siguen funcionando). Desactivable con `FLOW_REWRITE_LOCALHOST=false` o `--no-localhost-rewrite` en el CLI.
- El **CLI va dentro**: `docker exec flow node cli/run-flow.js --dir flows`.
- El **MCP va dentro**: `claude mcp add --transport http hilvana http://localhost:9998/mcp`.
- **`flows/` es el proyecto** (4.25): con `-v /tu/carpeta:/app/flows` (o `FLOW_FLOWS_DIR` para otra ruta), lo que guardas con Ctrl+S desde la web queda en tu disco, con tu usuario como dueño, listo para git y para el CLI.

Chrome dentro de la imagen (desde la 4.5.0)

La imagen trae **Chromium + puppeteer-core**: el botón **Capturar** del nodo Captura, el **modo Live** del nodo Web y el crawl de `flow-explore` (`docker exec flow node cli/explore-web.js https://mi-web.com`) funcionan dentro del contenedor sin instalar nada. La única excepción es `flow-explore --manual` / `--headful`, que abre un Chrome visible → necesita pantalla, así que se ejecuta desde el repo local.

Webs detrás de un login/SSO (Cloudflare Access, etc.)

El modo directo (iframe) no puede renderizar webs que responden `X-Frame-Options: DENY` — eso es el navegador, no Hilvana. El modo Captura/Live sí, pero el Chrome del server necesita tus credenciales; pásalas por entorno:

```
environment:
  # Cookie de sesión (p. ej. CF_Authorization de Cloudflare Access)
  - FLOW_CAPTURE_COOKIES=CF_Authorization=<valor>
  # O cabeceras (p. ej. un service token de Cloudflare Access)
  - FLOW_CAPTURE_HEADERS={"CF-Access-Client-Id":"<id>","CF-Access-Client-Secret":"<secreto>"}
```

También se aceptan por petición (`POST /capture` y `POST /capture-session/start` admiten `{ "headers": {...}, "cookies": "k=v" }`). Las llamadas registradas en los flows siguen enmascarando `authorization` / `cookie`: las credenciales nunca se escriben. Extras: `FLOW_CHROME` (ruta del binario) y `FLOW_CHROME_ARGS` (flags).

Escape hatch sin montar CAs (ya desde la 4.5.0): `FLOW_CHROME_ARGS=--ignore-certificate-errors` desactiva la validación TLS de **todas** las capturas — aceptable en un entorno de INT contra un MITM corporativo conocido, no como default. Nota técnica: `update-ca-certificates` a secas NO arregla Chromium (usa su BBDD NSS, no el store del sistema); por eso el entrypoint carga `/certs` en ambos y loguea `[certs] CA corporativa cargada (system + NSS): <CN>`.

Desde la **4.6.0** suele bastar con loguearte una vez en el propio Live (OTP incluido): el **perfil de Chromium persiste** en `flows/.chrome-profile` y el login sobrevive al cierre por inactividad y a restarts. `FLOW_CHROME_PROFILE=off` lo desactiva; `FLOW_SESSION_IDLE_MS` ajusta el cierre por inactividad (el perfil no se borra al expirar) y `POST /capture-session/logout` lo borra del todo — el logout real del SSO. ⚠ Con el volumen `./flows:/app/flows`, ese perfil (cookies vivas) queda en tu disco: no lo compartas.

Dominios internos y Local Network Access (desde la 4.9.0)

Dos trampas de red corporativa que producían **fallos mudos**:

- **Local Network Access (Chromium ≥138)**: una web pública que llama a su API en la red privada (p. ej. `app.empresa.es` → `api.internal.empresa.es`) queda bloqueada **sin prompt** en headless: la app carga, el click llega, y el fetch muere con `net::ERR_BLOCKED_BY_LOCAL_NETWORK_ACCESS_CHECKS` (visible en la pestaña Network). Desde la 4.9.0 esos checks van **desactivados** en el navegador de captura; `FLOW_CHROME_LNA_CHECKS=on` restaura el comportamiento estándar de Chromium.
- **DNS interno**: si el DNS corporativo vive en la VPN del host (resolv.conf apuntando a 127.0.0.x), Docker cae a DNS público y los dominios internos no resuelven dentro del contenedor. Solución:

```
dns:
  - 10.0.0.53   # el DNS corporativo que resuelve *.internal.empresa.es
```

CAs corporativas (desde la 4.6.0)

Si tu red intercepta TLS con una CA propia (Zscaler, WARP…), monta tus certificados:

```
    volumes:
      - /usr/local/share/ca-certificates:/certs:ro
```

Al arrancar, el contenedor los carga en el store del sistema, en `NODE_EXTRA_CA_CERTS` (peticiones del server/CLI y drivers SQL) y en la BBDD NSS de Chromium (captura/Live). Monta con volumen, **no** con `docker cp`: los `.crt` que son symlinks (p. ej. `managed-warp.crt → managed-warp.pem`) llegarían rotos — se ignoran con un aviso en los logs.

Licencia y prueba de 14 días 🆕 5.1

Hilvana se prueba **14 días gratis con todas las funciones** desde el primer arranque de la instalación. Pasado ese periodo, **ejecutar flows** (los nodos Request y SQL, en la web y en el CLI) requiere una **licencia de pago** o una **cuenta en la nube** — se consiguen en hilvana.cloud. Ver y editar tus flows nunca se bloquea.

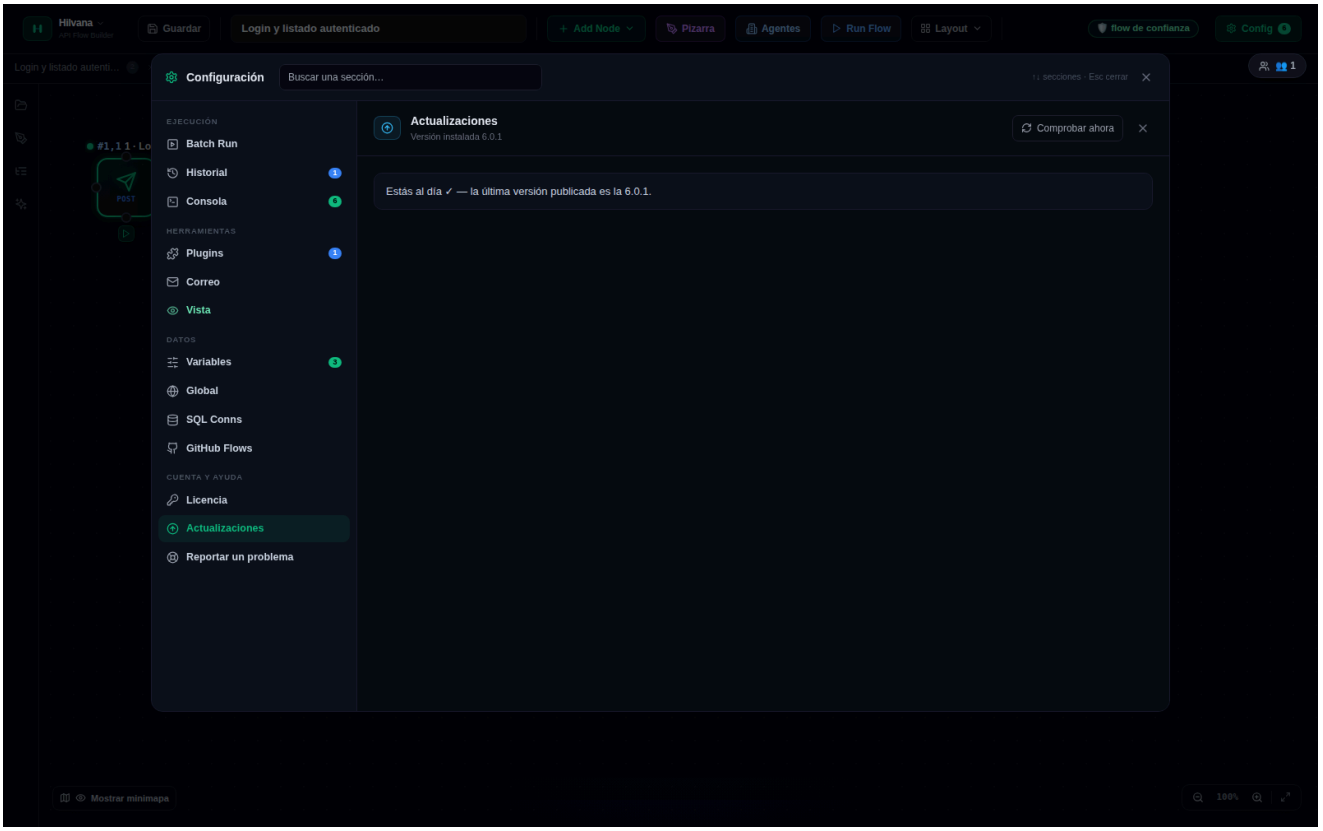
- **Activar una licencia**: en la app, **Config ▸ Licencia**, pega tu clave (o arranca el contenedor con `-e FLOW_LICENSE=<clave>`, o deja el fichero `<flows>/.license`).
- **Validación online**: con una licencia activa, la instalación la comprueba periódicamente contra el servidor del autor (firma + estado de la licencia); una licencia revocada deja de funcionar. Necesita salida a Internet, con **7 días de gracia** sin conexión.
- **Variables**: `FLOW_TRIAL_DAYS` (14 por defecto), `FLOW_LICENSE_SERVER` (por defecto el servicio del autor), `FLOW_LICENSE_GRACE_DAYS` (7).

> Las imágenes **≤ 5.0.1** mantienen su licencia anterior (uso personal gratuito). El cambio aplica desde la **5.1.0**.

Actualizaciones 🆕 5.22

Cuando hay una versión publicada más nueva que la tuya, aparece **«⬆ x.y.z»** junto a **Config**. Al pulsarlo se abre **Config ▸ Actualizaciones** con las novedades y el comando para actualizar:

```
docker pull juankanh/hilvana:<versión>
docker stop flow && docker rm flow
docker run … juankanh/hilvana:<versión>   # con los MISMOS -p, -v y -e de siempre
```



Actualizar con un clic (opcional)

Si quieres que el botón **«Actualizar a x.y.z ahora»** haga todo solo, arranca el contenedor con **las dos** cosas:

```
docker run -d … \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -e FLOW_SELF_UPDATE=on \
  -v /ruta/a/tus/flows:/app/flows \
  juankanh/hilvana:6.4.0
```

- Descarga la versión nueva y **recrea el contenedor con la misma configuración** (puertos, carpetas, variables, redes y reinicio). Tus flows y ajustes no se tocan.
- Si la versión nueva **no arranca en 2 minutos, vuelve sola a la anterior**. La anterior queda guardada (parada) como `<nombre>-anterior` por si quieres volver a mano: `docker rm -f flow && docker rename flow-anterior flow && docker start flow`.
- Solo instala versiones estables publicadas y más nuevas que la tuya.
- ⚠️ El socket de Docker da a Hilvana control de tu Docker: actívalo solo en máquinas tuyas. Sin `/app/flows` montado el botón no aparece (perderías los flows).
- La comprobación consulta Docker Hub cada 6 h; sin Internet simplemente no avisa. Para apagarla: `-e FLOW_UPDATE_CHECK=off`.
- En **Hilvana en la nube** no aparece: las versiones las instala el equipo de Hilvana.

Versiones

Versión	Qué trae
4.2.0	MCP embebido (18 tools)
4.3.0	Nodo Mermaid
4.4.0	Documentar webs: nodo Captura, modo Live del nodo Web, CLI flow-explore, Auto Layout para todos los nodos, maximizar el nodo Web (Chrome solo desde el código fuente)
4.5.0	Chromium dentro de la imagen: Captura, Live y flow-explore (crawl) funcionan en Docker; credenciales para webs tras SSO con `FLOW_CAPTURE_COOKIES` / `FLOW_CAPTURE_HEADERS`
4.6.0	Perfil de Chromium persistente (el login del Live sobrevive a idle/restart), CAs corporativas desde `/certs`, fallos de navegación visibles en el canvas, sonda de iframes que nombra al host bloqueante y ofrece Live, barra de URL + pegar en Live
4.7.0	Teclado directo sobre la vista Live (Ctrl+V pega el OTP), errores de certificado explicados en pantalla, sesiones de perfil concurrentes, `FLOW_SESSION_IDLE_MS`, `POST /capture-session/logout` borra el perfil
4.8.0	Pestañas **Consola** y **Network** en el nodo Web: la sesión Live enseña los `console.*` /errores de la página y todo su tráfico (sin headers ni cuerpos)
4.9.0	Local Network Access desactivado en el navegador de captura (apps internas: el fetch a la API privada ya no muere mudo), aviso LNA/PNA en pantalla y pista de DNS corporativo
4.12.1	Webcam falsa y generación de videomock de DNI para QA; viewport Live configurable y sincronizado con la vista y los clics
4.13 – 4.21	Toolbar compacta (Add Node ▾, Layout ▾, modal Config), Expand All, cajas colapsadas compactas, Collapse All sin reordenar, anti-solapes, Vista previa/Texto en notas, minimapa oculto por defecto
4.22.0	Nº de orden por nodo + Alinear en fila/columna
4.23.0	Scripts JS de las notas ejecutados al Run Flow (web, MCP y CLI; `--skip-info-scripts`), 📌 fijar cajas, Variables usadas en cada caja, tool MCP `tab_close`
4.24.0	**Pizarra** estilo Excalidraw sobre el lienzo (`drawings` en el flow), barra lateral de iconos con un panel a la vez, **Guía de nodos**, Ocultar conectores, Maximizar Mermaid, tool MCP `node_add_info` (20 tools)
5.15.1	🛡️ **Sandbox de los scripts de nota en el navegador** + flows de confianza (pregunta antes de ejecutar, chip 🔒/🛡️ en la barra) — 04; imagen con parches de Alpine y sin `npm`
6.4.0 (recomendada)	🏢 **AgentOffice 0.3.13–0.3.17 al día en el manual** (Mesa de trabajo, modo «aplicación», dos memorias, ayuda ⓘ y asistentes, revisión que no se queda parada) y scripts de datos de demo — 15
6.3.1	🤝 **AgentOffice en Docker usa el MCP de Hilvana**: agentes QA/docs y modo 🛠 Taller entran en `/mcp` con el token de emparejamiento (antes 403); el Taller en Docker usa tu sesión y memoria de Claude Code — 15
6.3.0	📄 **El flow en PDF desde el móvil**: la ficha de un flow en `/m` trae **Ver el flow en PDF** (el mismo PDF del canvas que el escritorio, lo genera el servidor; Pro) — 12. **Hilvana Admin** solo con el admin del cloud
6.2.1	🔗 **AgentOffice en un servidor, sin quedarse bloqueado**: si la instalación no está vinculada a tu cuenta Hilvana, el botón de Docker pasa a **Vincular cuenta e instalar** y, al volver de Config ▸ Licencia ya vinculado, instala solo. Antes creaba un AgentOffice que no arrancaba («Vincula Hilvana a tu cuenta»)
6.2.0	💻 **AgentOffice nativo en tu PC** (por defecto): Config ▸ Plugins ▸ Instalar ▸ En este PC da una orden de terminal que lo instala emparejado con Hilvana —Node oficial, paquete firmado sin fuentes,

Versión	Qué trae
	servicio de usuario— con escritorio (burbuja, modo Blender), tus herramientas y tu sesión de claude/codex; Linux por ahora. **En un servidor**, el contenedor de un clic (ahora con git-lfs y viendo el workspace de Hilvana) (AgentOffice)
6.1.0	🐳 **AgentOffice con un clic**: Config ▸ Plugins ▸ Instalar descarga la imagen `juankanh/agentoffice`, crea su contenedor y lo conecta solo (sin descargar paquetes ni recrear Hilvana; necesita montar el socket de Docker). Si al Guía de AgentOffice le falta `claude`, usa otro proveedor configurado o avisa claro (AgentOffice)
6.0.0	🧵 Versión de mantenimiento: marca Hilvana y base para «Iniciar sesión con Google» (aún inactiva); sin cambios de comportamiento en la app
5.39.0	📖 Versión de mantenimiento del manual (ampliación pantalla a pantalla y su revisión); sin cambios de comportamiento en la app
5.38.3	🧠 AgentOffice 0.3.3: recuerdo por agente, «que siga con…», burbuja a demanda, sesión viva entre tareas, un Blender por lado (15)
5.38.2	📖 Manual pantalla a pantalla con PDF y capítulo 15 AgentOffice; AgentOffice 0.3.2 (chat como respuesta, dictado, voz, nueva ronda, modo web) (15)
5.38.1	🛠 AgentOffice 0.3.1: burbuja que se queda al terminar (aprobar/devolver), plegado mínimo, Blender reabierto, fusión autosuficiente (08)
5.38.0	🛠 Modos de desarrollo de AgentOffice (burbuja por lado, Blender) y burbuja-chat con capturas y boceto (08)
5.37.0	📱 App móvil: adjuntos al crear tareas, ficha de «Equipo» y atrás de la hoja de detalle (12)
5.36.0	📦 Paquete firmado de AgentOffice verificado de punta a punta (08) y admin del cloud en el móvil (12)
5.35.0	🤖 AgentOffice: opciones de entrega (08) y app Android de administración versionCode 9 (12)
5.34.0	📦 AgentOffice por **paquete firmado** (sin `git clone`) (08)
5.33.0	🏢 La guía ⬇ **Instalar** de AgentOffice avisa del **acceso anticipado** (repo privado) (08)
5.32.0	📱 App Android **de administración** (variante `admin`, se instala junto a la normal) (12)
5.31.1	🖌️ la pizarra pinta los dibujos sin `id` (flows escritos a mano)
5.31.0	➕ botón «Nuevo proyecto» en AgentOffice, también en móvil (12)
5.30.4	📄 documentos (`.md`, `.pdf`, scripts, `.csv`) en `/m` y 🎤 dictado en AgentOffice desde la app Android (12)
5.30.3	🙈 carpetas ocultas del panel Proyecto guardadas en el servidor y respetadas en `/m` (12)
5.30.2	📱 app Android con botón 🤖 hacia AgentOffice (vista móvil) y lista de `/m` en árbol de carpetas (12)
5.30.1	■ botón Parar para Run Flow y la cadena del ▶; Reset también para la ejecución
5.30.0	💬 chat de cada flow en la colaboración en vivo (Business), guardado con el flow; usuario del login en el cloud
5.29.0	🗒️ sticky notes en la pizarra: fecha de creación y 6 colores a elegir
5.28.0	📊 los `.csv` / `.tsv` del proyecto se abren como tabla (o texto) en el panel Proyecto
5.27.1	⚙️ Ajustes de AgentOffice casi a pantalla completa, por grupos, con punto en las secciones con cambios y **Guardar** fijo; arregla «Proyecto» pegada debajo de todas
5.27.0	🔑 estado de la conexión del asistente de IA (probar, reconfigurar, desconectar); panel de AgentOffice abierto al recargar; CLI `-F ;type=`

Versión	Qué trae
5.26.0	🔌 conexiones en vivo por WebSocket: sin bloqueos con varias ventanas abiertas
5.25.1	🔌 menos conexiones por pestaña (el Guía no se queda colgado con varias pestañas) · proxy de Agentes robusto a reinicios
5.25.0	🧩 Instalar / actualizar AgentOffice desde Config ▸ Plugins · 🛒 Marketplace con pestaña Privado
5.24.1	☁️ espera automática mientras despierta el espacio del cloud · 🛒 marketplace de AgentOffice con Hilvana en Docker
5.24.0	🔐 Aviso y conversión de credenciales en claro al subir al cloud o compartir · ☁️ gestión de ficheros del cloud desde el panel Proyecto
5.23.0	🛒 Marketplace Hilvana (puerta `/account-link/marketplace/*` hacia tu cuenta) · tests unitarios del servidor
5.22.0	⬆️ Aviso de versión nueva y actualizar con un clic (opcional) · Configuración con menú lateral
5.21.3	🖥️ Funciona abierto por la IP de un servidor sin https (antes, página en blanco)
5.21.2	📬 Mejoras internas del correo de prueba
5.21.1	💾 Botón Guardar siempre visible en la barra; «Guardar como…» con pie fijo y lista de carpetas con scroll
5.21.0	🗂️ Panel Proyecto a medida: scripts visibles como código, destino de las carpetas enlazadas, ancho ajustable y ocultar carpetas
5.20.2	🗑️ Borrado de documentos por `DELETE /workspace/file`; la pestaña se cierra solo si era del disco local
5.20.1	🗂️ Ordenar el proyecto: carpetas nuevas, arrastrar y soltar, mover/renombrar, borrar carpetas vacías y documentos `.md` / `.pdf` — 08
5.19.4	🗂️ El workspace no lista los git worktrees enlazados (copias de los agentes): sin flows repetidos en el panel Proyecto ni en `flow_files_list` — 08
5.19.0–5.19.3	✅ Asserts de status 4xx/5xx esperados y con `{{variables}}` (02); plugin Agentes con contexto, Ajustes y voz; `flow_open` / `flow_save` respetan el disco
5.18.0	🧩 Gestor de plugins con AgentOffice (Pro) y workspace con enlaces simbólicos — 08
5.17.0	🔔 **Vigilancia en el servidor**: reglas sobre las variables del run en el monitor programado, aviso solo al dispararse o recuperarse, `{{secret:X}}` en `notifyUrl`, tool MCP `flow_monitor` — 08
5.16.0	🔎 **Notas a pantalla completa**: botón Maximizar junto a «Copiar» y «Maximizar nota» en el menú de la caja — 04. El panel Proyecto recuerda el entorno 💻 Local / ☁️ Cloud entre recargas (fix 5.15.2)
5.15.0	🛡️ Primera versión del sandbox en el navegador y del modelo de confianza por fichero
5.14.3	🔒 `npm audit` a cero y tope de memoria de los runners (`FLOW_RUNNER_HEAP_MB`)
5.14.2	🖼️ Imágenes de cualquier ruta del proyecto en el visor de documentos `.md` (`GET /workspace/image`) — 08
5.14.1	🔒 **Endurecimiento de seguridad**: sandbox de los scripts de nota en el servidor + cortafuegos de salida (SSRF) en el cloud — ver docs/seguridad.md
5.14.0	🚨 **Reportar un problema** desde la app (escritorio y móvil): error/idea/pregunta con captura y datos técnicos → bandeja del equipo, respuesta por email y «Mis reportes» (Pro/Business); el asistente de IA lleva la **skill de flows de serie**; galería `scripts/`

Versión	Qué trae
5.13.1	🤖 IA en el **móvil** (pestaña ✨ IA en `/m`), «Entrar con Anthropic», conexión desde el navegador, Ctrl+V de capturas, «Guardar como…» ▸ ☁️ Cloud (12, 13)
5.13.0	🤖 **Asistente de IA** embebido con tu clave de Claude u OpenAI: construye, explica, prueba y arregla flows en el canvas en directo, con confirmaciones, parar, deshacer turno, comandos, adjuntos y coste (13)
5.12.0	🔗 **Compartir enlace**: foto pública del canvas en `GET /share/<id>` (se ve como el PDF, sin app ni cuenta); con el Docker vinculado se publica en el cloud; caducidad opcional, visitas, cuota y gestión desde el modal, la cuenta y el admin (01)
5.11.0	📱 **Vista móvil** en `/m`: ejecutar flows, ver resultados y leerlos como documentación desde el teléfono (12)
5.10.0	🗂️ **El panel Proyecto apunta al entorno que gobierna** (disco local o tu espacio del cloud)
5.9.1	🔧 **Modo vinculado más limpio**: chip «☁️ org ● plan» en la barra superior (sin franja de aviso) y refresco inmediato del panel Proyecto / chips PRO al conmutar licencia ↔ cuenta cloud
5.9.0	☁️ **Sincronizar flows local ↔ cloud** (Docker vinculado): traer/subir flows entre tu disco y tu espacio del cloud desde el botón ☁️ del panel Proyecto
5.8.1	🔁 **Interruptor licencia ↔ cuenta cloud** (elige cuál gobierna cuando tienes las dos, sin mezclar)
5.8.0	🤖 Token MCP por usuario (conecta tu IA al cloud) · 🐳 vincular Docker a la cuenta (hereda el plan) · credenciales con ámbito 🔒 privado / 👥 equipo
5.5.0 – 5.7.0	👥 Colaboración en vivo (Business) + Ctrl+Z del canvas · credenciales cifradas `{{secret:X}}` · carpetas privadas por miembro · Business = 1.000 flows
5.4.0	**Monitores programados** server-side con historial y notifyUrl (modal Automatización)
5.3.0	**Asserts + entornos + webhook + data-driven + OpenAPI** — CI con exit ≠ 0, `--env` / `--data`, `POST /hook/<token>` y spec → flow
5.1.0	**Prueba 14 días + licencia validada online** — tras la prueba, ejecutar flows requiere licencia o cuenta cloud (hilvana.cloud); revocable en remoto. Ver/editar nunca se bloquea
5.0.1	**Fix de seguridad** — la imagen deja de incluir flows del proyecto; solo trae `bienvenida.flow.json` (onboarding limpio). Usa siempre ≥ 5.0.1
5.0.0	**Licencias por token (JWT)** — Config ▸ Licencia: plan Free/Commercial/Team/Business, activación de la clave, verificación offline RS256
4.51.3	**Licencia propietaria** (uso personal gratis, comercial de pago; `LICENSE` en la imagen)
4.51.2	**Scripts «después» con `vars`** · 4.51.2: el PDF exportado incluye la imagen de fondo y las imágenes de la pizarra también desde Docker (4.51.1: el MCP conserva `when` al crear notas con `node_add_info`) (conmutador antes/después por script, corren al acabar Run Flow con todas las extracciones; `{{variables}}` en textos e imágenes de la pizarra; MCP `when`)
4.50.0	**Imágenes y GIF en la pizarra** (panel Imagen / GIF, Ctrl+V; `flows/assets/whiteboard/`; MCP `whiteboard_update` type `image`)
4.49.1	**Imagen de fondo del flow** (Config ▸ Vista): mapa/plano/diagrama detrás del lienzo, en coordenadas de los nodos, guardado en `settings.background`; tool MCP `flow_background` (38 tools)
4.48.0	**Configuración por flow**: Vista, ocultar conectores, estilo de pizarra y zoom/posición del lienzo guardados en el `.flow.json` (`settings`), distintos en cada pestaña
4.47.0	**Copiar/cortar/pegar en la pizarra** (Ctrl+C/X/V + botones en el panel, pegado con desplazamiento acumulado) y **mover el grupo arrastrando desde el hueco dentro de la selección** (ya no se pierde

Versión	Qué trae
	la selección al coger una figura por dentro)
4.46.0	**Selección múltiple en la pizarra**: rectángulo de selección arrastrando sobre el fondo (Shift acumula) para mover o retocar varios dibujos a la vez
4.45.0	**Los PDF de `flows/` en el panel Proyecto**, abiertos como pestañas de documento con el visor nativo (chip PDF)
4.44.0	**Exportar el canvas a PDF** desde el menú del logo (una página del tamaño exacto del flujo, con la pizarra, aunque no quepa en pantalla)
4.43.0	**Panel de Correo con secciones plegables** (cabeceras con chevron y resumen; estado recordado)
4.42.0	**El correo extrae el código** (`$.code` / `$.codeSource` : keyword · fallback · regex propia; texto plano con HTML limpiado; longitud configurable) y **proveedor remoto mail.tm** junto al SMTP (buzones por API para correo desde Internet, dominio activo visible, auto-creación, tokens renovados, sondeo con throttle solo bajo demanda)
4.41.0	**Árbol de carpetas en el panel Proyecto**: subcarpetas anidadas dentro de su padre (nombre corto, indentación), plegar oculta todo el subárbol, contador = ficheros del subárbol, plegado recordado
4.40.0	**Los `.md` se abren como pestañas** (no en modal): el documento ocupa el área del canvas como un flow más, chip **MD** en la pestaña, solo lectura (Ctrl+S nunca lo pisa), filas del panel con ir-a/cerrar
4.39.0	**▶ Ejecutar un flow desde un documento**: cada enlace `[[flow]]` del visor Markdown lleva un botón verde ▶ que abre ese flow y lanza su Run Flow completo, cerrando el visor para ver la ejecución en el canvas
4.38.0	**Documentos Markdown en el panel Proyecto**: los `.md` de `flows/` (notas e informes de `flow-explore`) se listan con título y chip **MD** y se abren en un visor de solo lectura (tablas, código, `<details>` , imágenes de `assets/` , enlaces `[[flow]]` al canvas); `GET /workspace/file` ; MCP y CLI sin cambios
4.37.0	**El MCP controla el correo de prueba (37 tools)**: `mail_state` , `mail_server` , `mail_address` , `mail_messages` — arrancar el SMTP, buzones y verificar correos (OTP/enlace) sin pestaña web; guía de red cuando el servicio que envía corre en otra máquina (LAN, red Docker, túnel `ssh -R`)
4.36.0	**Correo de prueba (Config → Correo)**: SMTP embebido en el puerto `1025` (sin TLS, auth opcional) con buzones `@midominiotest.com` para capturar los correos que envían tus servicios sin cuentas reales; panel tipo Consola con HTML/texto/origen/adjuntos, búsqueda y filtro por buzón; persistencia en `flows/.mail/` ; `GET /mail/messages/latest?to=` para verificarlo desde un flow. `-p 1025:1025` , `FLOW_MAIL_PORT` , `FLOW_MAIL_AUTOSTART` , `FLOW_MAIL_DOMAIN`
4.35.0	**10 fuentes para el texto de la pizarra** (8 manuscritas empaquetadas: Manuscrita, Boceto, **Excalidraw**/Excalifont, **Indie** Flower, Rotulador, Esbozo, Arquitecto, Nota + Normal y Código) con cuadrícula y vista previa; MCP `whiteboard_update` las acepta y mide los textos sin `w/h` . **Las filas bajan en bloque cuando una card crece** al ejecutarse (Vista ▸ «Evitar solapes al soltar o al crecer una caja»)
4.34.0	**Cadenas entre cualquier tipo de nodo** (▶ sigue next/on_error/parallel hacia request, SQL, nota o web), **Run Flow ejecuta también los SQL**, **pausa por conector** (`delayMs` , popover en la flecha; web, CLI y MCP); fix caja SQL colapsada
4.33.0	**MCP con control total** (33 tools): `view_settings` , `global_variables_set` , `tab_rename` , `connection_update` , `sql_connections_list` , `canvas_layout separate` ; `flow_state` con globales, perfiles SQL y vista
4.32.0	**Variable extractions** plegables con el diseño de «Variables usadas», estado `n/n ✓` / `n sin valor` y botón ↻ **re-extraer** de la última respuesta sin repetir la llamada

Versión	Qué trae
4.31.0	**Activar / desactivar nodos** desde el menú del icono (se saltan en Run Flow, cadenas, cron, MCP y CLI; conexiones recorridas; `disabled` en el flow)
4.30.0	**Cabecera de las cajas simplificada** (icono-menú · plegar · estado · título · ▶; el resto en el menú del icono); fix MCP `node_add_sql` (`order` / `pinned` / `cell`)
4.29.0	Botón **Copiar** en la vista previa de las notas, **Layout ▸ Pin All / Unpin All**, **guía de nodos plegable** (MCP `canvas_layout pin_all
4.28.2	Campo `#` = `columna,fila` (1,1 arriba a la izquierda) y **Layout ▸ Alinear en cuadrícula**; `cell` en el flow y en el MCP (`canvas_layout grid`); 4.28.1: **Auto Layout respeta las celdas**; 4.28.2: **fix Run Flow** (flechas `next` desde notas/SQL/web ya no bloquean el request destino, ciclos al final, scripts de notas en el Historial)
4.27.0	**Config ▸ Vista**: tamaño de los nodos, **modo compacto** (caja con icono + título con estado + ▶, tooltip al pasar el ratón, clic abre la card) y **separación entre nodos** (mínima configurable, apartar vecinas al soltar, «Separar nodos solapados ahora»)
4.26.0	`flows/` **como proyecto**: panel Proyecto, abrir/guardar con **Ctrl+S** en el fichero, «Guardar como…», detección de cambios en disco, ficheros con el dueño del host desde Docker; **enlaces** `[[flow#nodo]]` entre flows en las notas; `FLOW_FLOWS_DIR` para apuntar el proyecto a otra carpeta

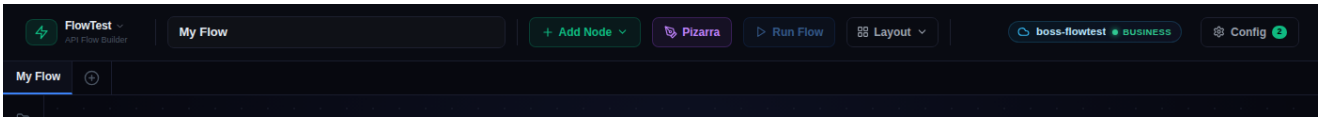
Vincular esta instalación a tu cuenta cloud 🆕 (5.8)

¿Tienes una **suscripción en app.hilvana.cloud** y también corres Hilvana self-hosted (Docker)? En vez de gestionar una clave de licencia, tu Docker puede **heredar el plan de tu cuenta**:

1. En tu cuenta cloud → 🔌 **Máquina local** → 🐳 **Vincular Docker** → **Generar token de vinculación**.
2. En tu Docker: **Config ▸ Licencia ▸ «🐳 Vincular con mi cuenta cloud»** → pega el token. (O arranca con `-e FLOW_ACCOUNT_TOKEN=flowlink_…`.)

Tu instalación heredará el plan de la org (Pro o Business) sin claves RS256. El **workspace sigue en tu disco local** — esto solo hereda el plan, no mueve datos. Tope de instalaciones vinculadas: **Pro 2 · Business = asientos × 2**. Se desvincula desde el mismo panel; el admin lista las instalaciones vinculadas de cada cuenta.

Mientras la vinculación gobierna, la barra superior muestra un **chip discreto** «☁️ *org* ● PLAN» a la izquierda de **Config** (5.9.1) en lugar de la franja de aviso: verde con tu plan, ámbar con los días que quedan si la org está en prueba, rojo si la prueba terminó y «verificando…» hasta la primera validación. Clic en el chip → Config ▸ Licencia. Al conmutar entre licencia y cuenta cloud (o al vincular, desvincular, activar o quitar la licencia), el panel Proyecto (botón ☁️) y los chips PRO se actualizan al instante, sin recargar la página.



Sincronizar flows local ↔ cloud 🆕 (5.9)

> **5.13** — También desde **«Guardar como…»** (Ctrl+Shift+S): con el Docker vinculado, el modal ofrece 💻 **Local / ☁️ Cloud · tu org**. Elige Cloud, nombre y carpeta (con 🔒 Privado si tu plan lo tiene) y el flow se guarda directamente en tu espacio del cloud; la pestaña queda enlazada a esa copia (el siguiente Ctrl+S va allí) y el fichero local no se toca. Si ya existe con ese nombre, avisa antes de sobrescribir.

Con el Docker **vinculado**, el panel **Proyecto** muestra un botón ☁️: ábrelo para **traer** flows de tu espacio del cloud a tu disco o **subir** los locales al cloud. Dos columnas (Local / En tu cloud) con ⬆️ Subir y ⬇️ Traer por flow; si el destino ya existe te pregunta antes de sobrescribir. Respeta tus **carpetas privadas** (cada uno ve las suyas y las compartidas del equipo). El **token de vinculación se queda en el servidor** — el navegador nunca lo ve. La sincronización **copia** el fichero, no fusiona cambios (en la subida gana tu versión local).

📱 12 · La vista móvil

> Revisado para Hilvana **6.3.0** (vista `/m`).

Desde la **5.11**, Hilvana trae una segunda cara pensada para el teléfono: entra en **`http://tu-servidor:9998/m`** y tienes tus flows para **ejecutarlos, ver el resultado y leerlos como documentación**. Editar sigue siendo cosa del escritorio.

No es la aplicación de siempre encogida: es una **app aparte**. El editor (canvas, pizarra, arrastrar cajas) no se descarga en el móvil, así que abre rápido incluso con datos.

> Si entras a `http://tu-servidor:9998` desde un teléfono, te lleva solo a `/m` **antes** de descargar el editor. Para ver el escritorio en un móvil o una tablet, usa `?desktop=1` (se recuerda) o el enlace «Abrir la versión de escritorio» del pie.

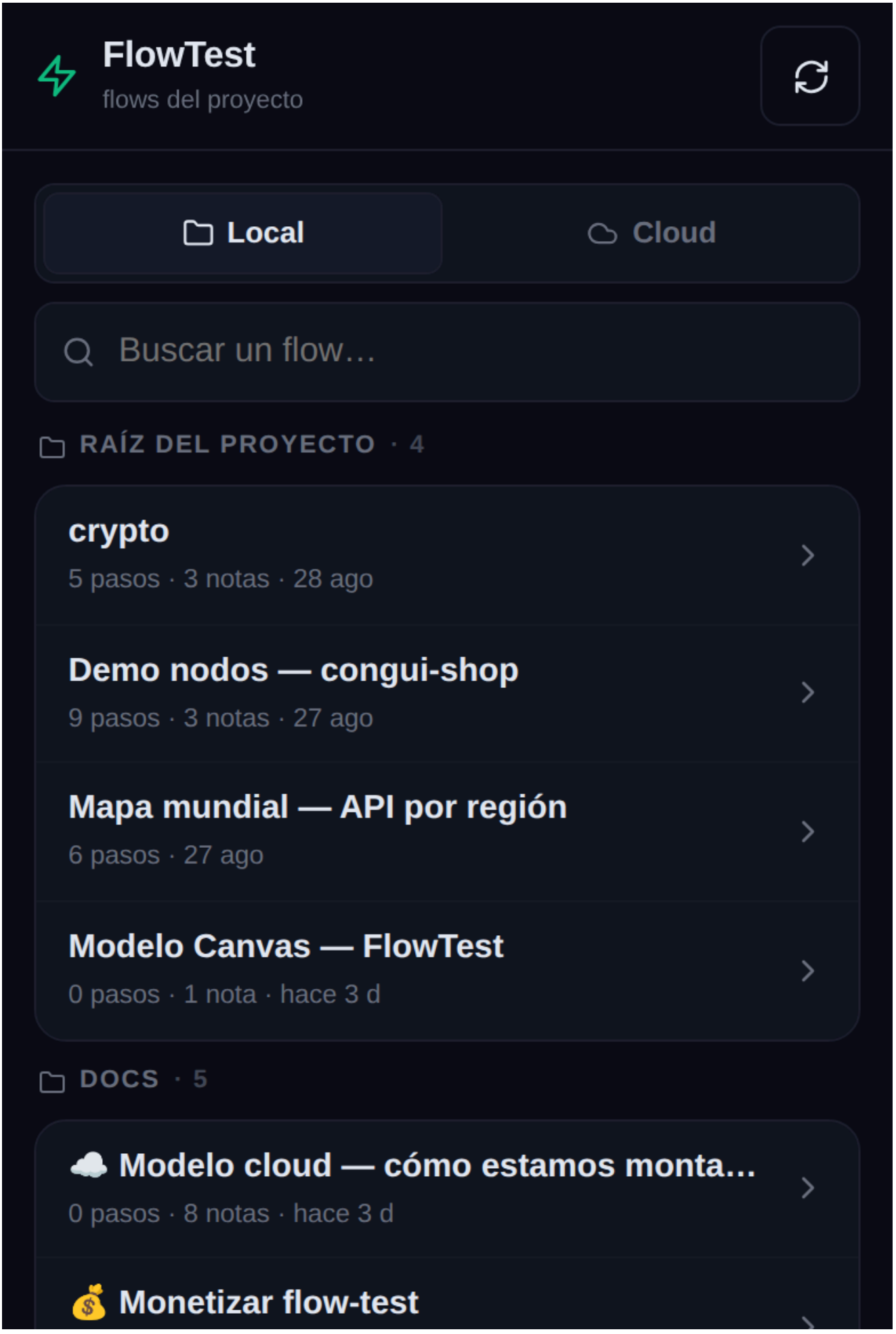
Llegar desde el teléfono

El móvil tiene que poder ver la máquina donde corre Hilvana. En una red doméstica u oficina basta la IP del servidor:

```
# en la máquina del contenedor
hostname -I | awk '{print $1}'   # p. ej. 192.168.1.50
# en el teléfono: http://192.168.1.50:9998/m
```

Añádela a la **pantalla de inicio** y se abre como una aplicación, sin barras del navegador.

La lista


FlowTest
flows del proyecto
Local
Cloud
Buscar un flow…
RAÍZ DEL PROYECTO · 4
crypto
5 pasos · 3 notas · 28 ago
Demo nodos — congui-shop
9 pasos · 3 notas · 27 ago
Mapa mundial — API por región
6 pasos · 27 ago
Modelo Canvas — FlowTest
0 pasos · 1 nota · hace 3 d
DOCS · 5
☁️ Modelo cloud — cómo estamos monta…
0 pasos · 8 notas · hace 3 d
💰 Monetizar flow-test

Los flows del proyecto, en un **árbol de carpetas** y con buscador. Cada fila dice cuántos pasos tiene, si lleva notas y cuándo se tocó por última vez.

Desde la 5.30.2 cada carpeta muestra solo su nombre, con sangría dentro de la suya, y a la derecha el total de flows que hay en ella y en sus subcarpetas. De entrada todas están plegadas, para ver el proyecto de un vistazo; las que abres se recuerdan. Una carpeta que solo contiene otra se junta con ella en una fila («almadance / flows»). Al buscar se abre todo para que salgan los resultados.

Desde la 5.30.3 la lista **no enseña las carpetas que ocultaste** en el panel Proyecto del escritorio (08): esa lista se guarda en el servidor y la comparten los dos. Al final aparece «N carpetas ocultas en el escritorio · mostrar» para verlas atenuadas un momento.

Desde la 5.30.4 la lista enseña también los **documentos** del proyecto, igual que el panel Proyecto: `.md`, `.pdf`, scripts (`.js`, `.ts`, `.py`, `.sh`, `.sql`…) y datos `.csv` / `.tsv`, después de los flows de cada carpeta y con su icono. Al tocarlos se abren en solo lectura con los mismos visores del escritorio: el Markdown renderizado (sus `[[enlaces]]` abren el flow), el script con su código y el CSV como tabla. El **PDF** se abre con el visor del teléfono (botón «Abrir el PDF», función Pro).

Si tu instalación está **vinculada a una cuenta cloud**, arriba aparece el selector 💻 **Local** / ☁️ **Cloud** y la lista arranca en el entorno que gobierna la instalación, igual que el panel Proyecto del escritorio.

Documentación de un flow

Economía Global 🌍

en esta instalación

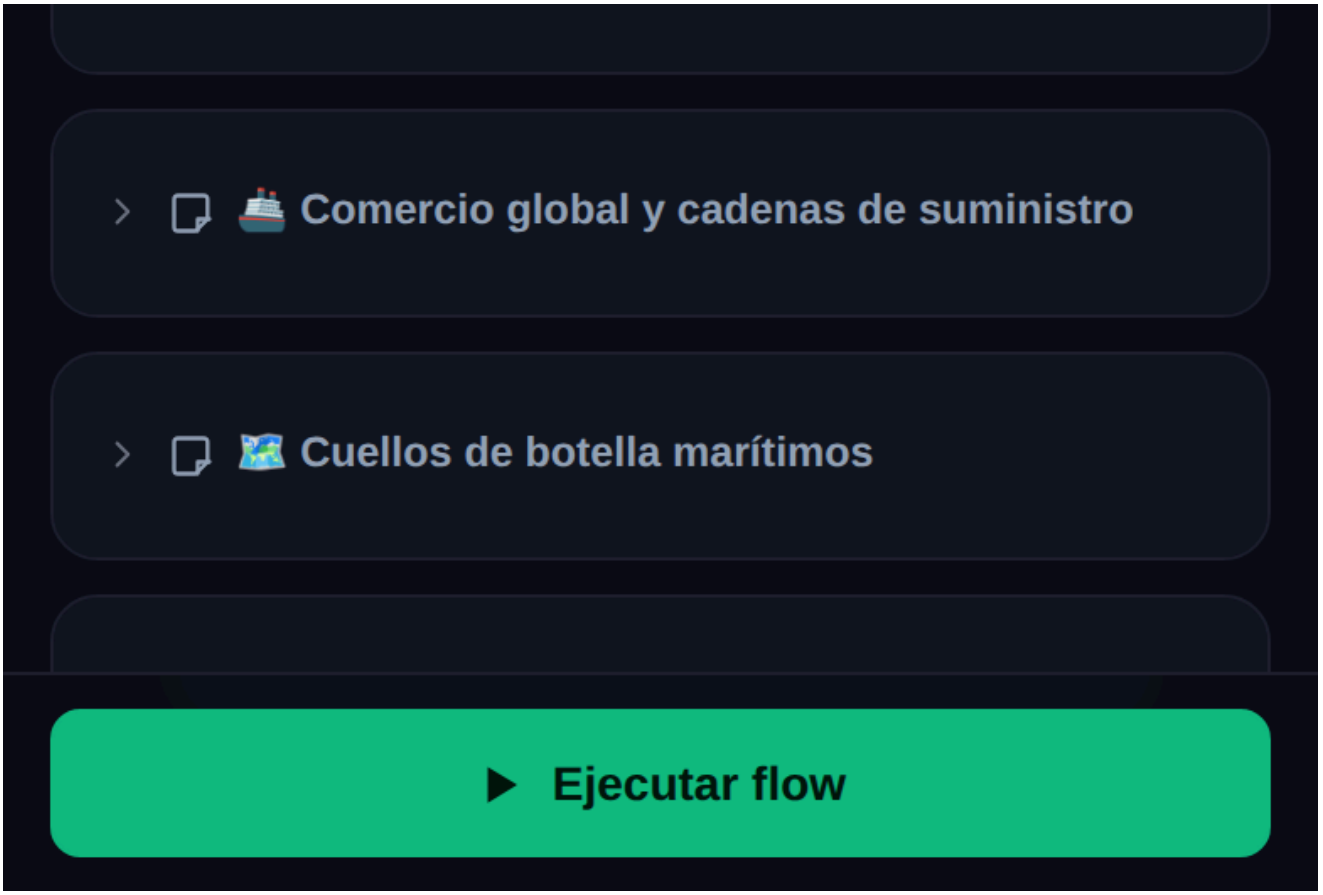
38 pasos · 160 notas · 2 web

docs/economia-global.flow.json

Documentación | Ejecución

NOTAS · 160

- 📖 Cómo leer esta pizarra
- 🛢️ Energía: petróleo y gas
- ⛏️ Metales y minerales críticos
- 🌾 Alimentos y agrícolas
- 📈 Qué mueve el precio de una materia prima



La pestaña **Documentación** enseña el flow como se lee, no como se dibuja:

- **Notas** con su texto, sus diagramas Mermaid o sus capturas. En un flow con muchas notas llegan plegadas y se abren al tocarlas.
- **Pasos en orden de ejecución**, numerados, con su método, su dirección y los asserts que tiene cada uno. Es el mismo orden en que los va a lanzar el runner.
- **Variables** y el entorno activo.

Sirve para consultar en una reunión qué comprueba un flujo, o para enseñárselo a alguien sin abrir el portátil.

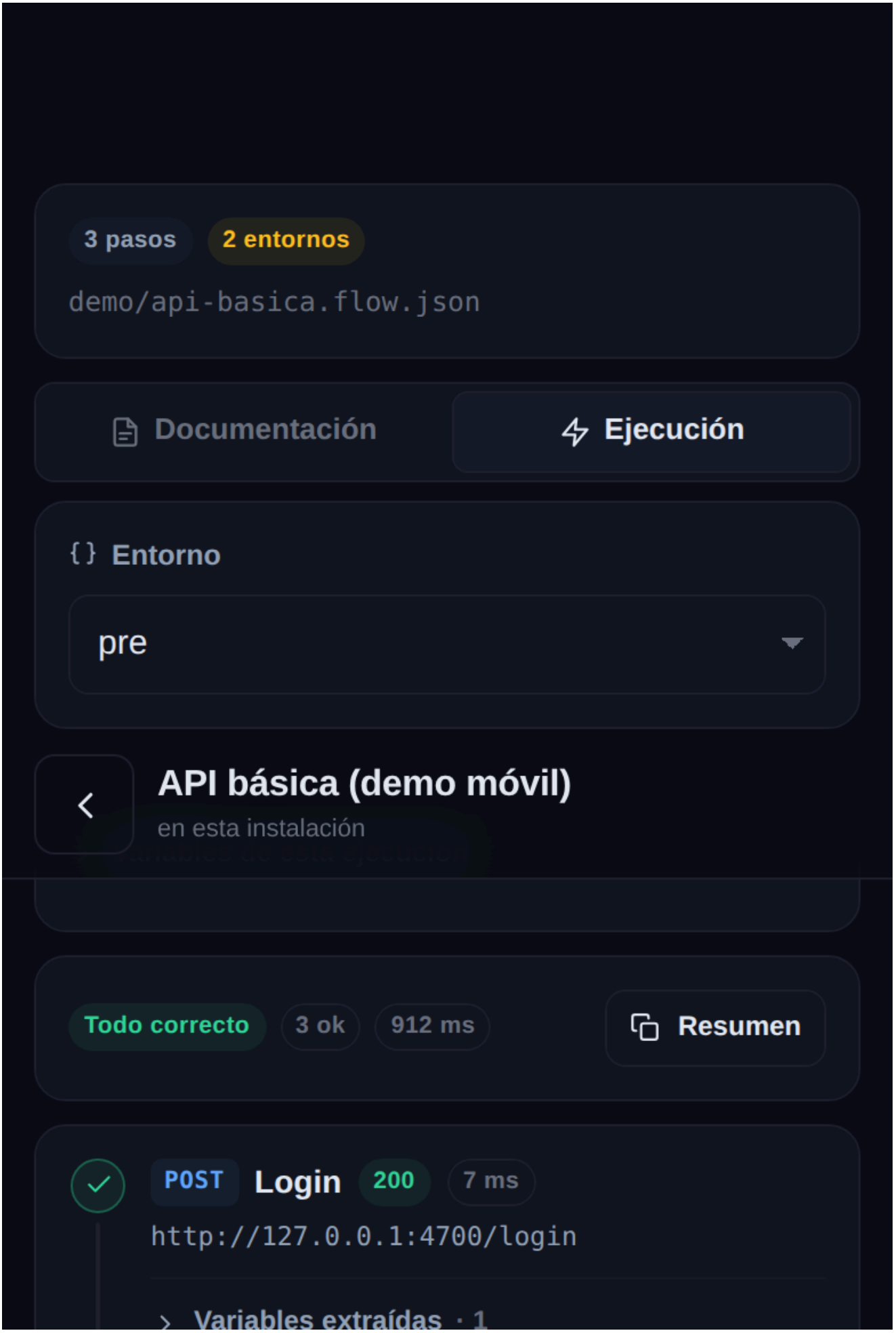
El flow en PDF 🆕 (6.3.0)

En la ficha de un flow, bajo su ruta, el botón **Ver el flow en PDF** genera el mismo PDF que **Exportar a PDF** del escritorio: el canvas entero (nodos, conexiones y pizarra) en una sola página del tamaño exacto del contenido.

- Lo genera el **servidor**: abre el flow con su Chromium, como si lo tuvieras en el escritorio, y lo imprime. Tarda unos segundos; no hace falta tener el escritorio abierto.
- El PDF se abre con el **visor del teléfono** (un navegador dentro de una app no pinta PDF), donde puedes ampliarlo, guardarlo o compartirlo.
- Solo para los flows de **esta instalación** (💻 Local); los del cloud se abren primero en el escritorio o se traen al disco.
- Es una función del plan **Pro**, como en el escritorio; en prueba el botón se cambia por un aviso.

- Necesita la imagen Docker (o la app compilada con `npm run build`): en modo desarrollo responde «PDF de un flow solo con la app compilada».

Ejecutar y ver el resultado


3 pasos
2 entornos
demo/api-basica.flow.json
Documentación
Ejecución
Entorno
pre
API básica (demo móvil)
en esta instalación
Todo correcto
3 ok
912 ms
Resumen
POST
Login
200
7 ms
http://127.0.0.1:4700/login
Variables extraídas · 1

La pestaña **Ejecución** lo lanza en el servidor, con el mismo motor que usan el CLI, los webhooks y los monitores. Antes de ejecutar puedes:

- elegir **entorno** (dev, pre, pro… los que tenga el flow),
- **cambiar variables solo para esa ejecución**, sin tocar el fichero.

Mientras corre ves una **barra de progreso**, la **salida del runner en directo** y los pasos que ya han terminado marcados en verde o en rojo. Al acabar, cada nodo muestra su código, el tiempo, los asserts, las variables extraídas y el cuerpo de la respuesta (plegado, con botón de copiar). El botón **Resumen** copia el resultado en texto para pegarlo donde haga falta.

Si te sales de la pantalla a mitad, la ejecución se corta en el servidor: no quedan procesos sueltos.

Lo que no hace

- **No se edita.** Para tocar un flow, el escritorio.
- **Ejecutar requiere plan Pro**, igual que el CLI y los webhooks. Durante la prueba de 14 días la documentación se ve entera y el botón de ejecutar queda desactivado.
- Un flow que vive **en tu cloud** se puede leer, pero para ejecutarlo con el motor de esta instalación el botón te ofrece **traerlo** a su disco primero.
- Los documentos `.md` y `.pdf` del proyecto, y el borrado de ficheros, siguen siendo del escritorio.

AgentOffice en la app Android 🆕 (5.30.2)

La app Android muestra un **botón redondo** 🤖 abajo a la derecha. Al tocarlo abre **AgentOffice** del mismo servidor en una vista pensada para el teléfono, y el botón pasa a ⚡ para volver a Hilvana. También se abre manteniendo pulsado el icono de la app ▸ **Agentes**.

- **Resumen:** tareas en curso, en revisión y por hacer; el equipo del proyecto con lo que hace cada uno; la cuota de Claude y Codex; y los demás proyectos.
- **Bandeja:** las preguntas de los agentes (con sus botones de respuesta), las tareas para revisar con ✓ **Aprobar** / ↩ **Devolver** y las propuestas. La pestaña lleva un número con lo pendiente.
- **Tareas:** filtros (activas, en curso, revisión, fallidas, backlog, hechas), detalle de cada una con su resumen y sus cambios, **Reintentar** las fallidas y ＋ **Nueva tarea**.
- **Guía:** el chat con el Guía. Con el 🎤 le hablas y lo dictado se envía.

🎤 **Dictado (5.30.4):** en la app, los campos de AgentOffice llevan un 🎤 que abre el reconocedor de voz del teléfono (en español) y escribe lo que dices. En ＋ **Nueva tarea**, **«🎤 Dictar la tarea»** te deja contarla con tus palabras y la pasa por ✨ **Redactar con IA**, que propone título, descripción y quién la hace; la revisas y pulsas **Crear**. También hay 🎤 al devolver una tarea y al responder una pregunta. (En un navegador por HTTP no hay micro: ahí se dicta con el 🎤 del teclado.)

Funciona con la instalación de Hilvana de tu red (el PC o un servidor) que tenga AgentOffice como plugin (08); en el cloud no hay AgentOffice. Si el AgentOffice es anterior y no tiene la vista móvil, se abre su web completa. Mantener pulsado el botón y arrastrarlo lo cambia de altura.

＋ Nuevo proyecto 🆕 (5.31.0)

Ya no hace falta ir al escritorio para crear un proyecto de AgentOffice:

- **En el escritorio** el botón ＋ **Nuevo proyecto** está siempre visible, con texto, junto al selector de proyectos.
- **En el móvil** (vista de AgentOffice) lo tienes en el **Resumen** y como última opción del selector de proyectos de arriba (＋ **Nuevo proyecto…**).

La hoja del móvil pide solo dos cosas: el **nombre** y la **carpeta del workspace** (se propone a partir del nombre y se crea si no existe). Se crea sin repositorio y con el equipo y el coordinador de serie; el repositorio y el resto de ajustes se añaden después desde la web. Avisa si el nombre está vacío o ya existe, o si falta la carpeta; al crear, el proyecto queda seleccionado y se abre su Resumen.

App Android de administración 🆕 (5.32.0)

Además de la app normal, `android/` compila una segunda variante de administración (`io.hilvana.mobile.admin`, icono naranja), para administrar tu propio despliegue: WebView con sesión persistente y botón flotante ☰ con las entradas del admin del cloud, Hilvana cloud `/m`, AgentOffice y tu propio servidor. Se instala **a la vez** que la app normal.

- Las URLs se fijan al compilar en `android/local.properties` (una URL por entrada del menú ☰), obligatoriamente `https://`; nunca contraseñas ni tokens.
- Solo navega por HTTPS a esos hosts; el login lo haces tú (autorrelleno desactivado).

- Compilar: `./gradlew assembleAdminDebug` desde `android/`; instalar con `adb install -r`.
- ⚠️ Sin compilar ni probar aún en dispositivo; el admin del cloud todavía no tiene diseño móvil propio.

Desde la 6.3.0 la app de administración solo abre el **admin del cloud** (entra directa en 🚨 Reportes); los servidores de Hilvana (tu PC, tu servidor y el cloud) se abren con la app normal. El botón ☰ se queda con «🖤 Admin del cloud» y «🔑 Contraseñas guardadas…», y tras entrar con usuario y contraseña la app ofrece guardarlos cifrados para rellenarlos la próxima vez.

versionCode 9 (5.35.0): la app puede **guardar la contraseña de cada panel** (🔑): se cifra en el teléfono con el Keystore de Android (AES-GCM), una por origen, se rellena el formulario **sin enviarlo** y se puede olvidar desde el menú ☰ o marcar «nunca» para un origen. ⚠️ No compilado ni probado en dispositivo en esta documentación.

Admin del cloud en el móvil 🆕 (5.36.0)

El panel de administración del cloud (`hilvana-accounts`) se adapta a pantallas de hasta **768 px**:

- **Barra superior** fija con ☰, la sección actual como título y ↻ recargar. El menú lateral pasa a un **cajón** deslizable: se cierra con ☰, tocando fuera o al elegir una sección.
- Los **contadores** ocupan el ancho completo, y los paneles de doble columna pasan a una.
- El subtítulo de cada panel se oculta tras un botón ⓘ; en escritorio no existe y el subtítulo se ve como siempre.
- Las **tablas** tienen scroll horizontal propio: la página nunca se desborda.
- Botones, campos y selectores miden **≥ 44 px** de alto y los campos usan 16 px para que Android no haga zoom.

Adjuntos, Equipo y botón atrás en AgentOffice móvil 🆕 (5.37.0)

Novedades de `/agents/m.html` (y de la app Android, que la abre con el botón flotante 🤖) — - **Adjuntar al crear una tarea.** La hoja ＋ **Nueva tarea** tiene los botones 📷 Foto, 🖼 Galería y 📎 Archivo, y admite pegar imágenes o ficheros en el título o la descripción. Hasta 10 adjuntos de 25 MB cada uno; las fotos se reducen a 1600 px (JPEG) antes de subir y cada adjunto aparece como chip con miniatura y ✕. El agente ve las imágenes y recibe los demás ficheros por su ruta. Los ejecutables (`.sh`, `.exe`…) se rechazan. Cancelar la hoja descarta los adjuntos.

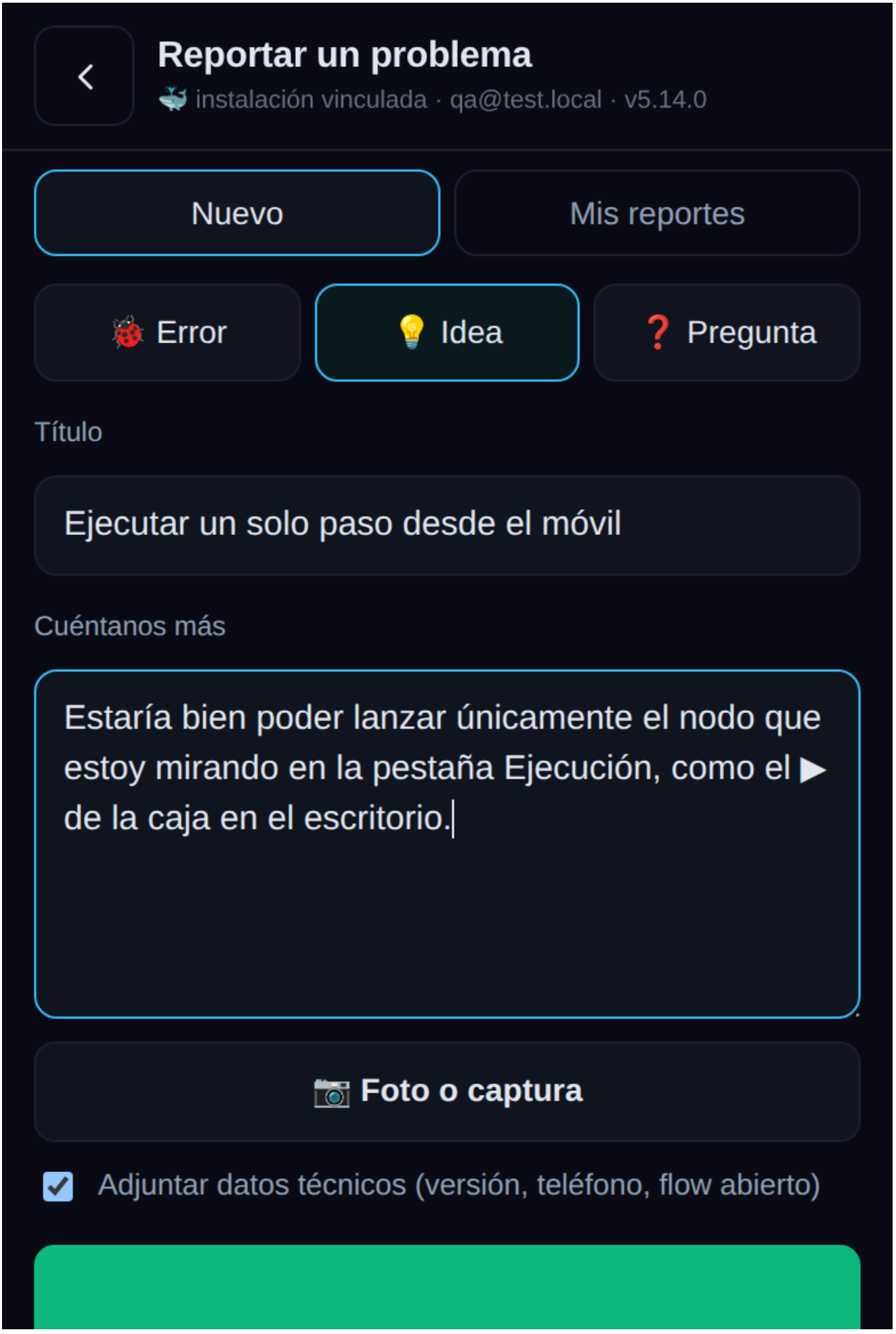
- **App Android.** El WebView abre un diálogo **Cámara / Galería / Ficheros** (selección múltiple); el permiso de cámara solo se pide si eliges cámara. Aplica a la app normal y a la de administración.
- **Equipo.** En el **Resumen**, cada fila de «Equipo · proyecto (n)» se puede tocar y abre una hoja con modelo, proyectos, skills/MCP/scripts, rol, tareas y prompt del rol (lo vacío no se pinta); ■ **Parar** aparece solo si trabaja.
- **Atrás.** En la app Android, el botón atrás del sistema cierra primero la hoja de detalle del Resumen en vez de salir de la pantalla.

La pestaña ✨ IA 🆕 (5.13.1)

El detalle de un flow tiene una tercera pestaña, ✨ **IA**: el mismo asistente del escritorio (13) desde el teléfono. Con cada mensaje la IA recibe un resumen del flow que estás leyendo (pasos en orden de ejecución, variables, asserts), así puedes pedirle que te lo explique, que te diga qué mejorarías o que revise el último fallo. Sin el escritorio abierto **no edita nada** (no hay canvas): explica y propone. Si tienes el escritorio abierto en el mismo servidor, las herramientas actúan sobre esa pestaña y lo ves llegar allí, con las mismas confirmaciones (Sí/No) y el botón de parar. La clave, el proveedor y el modelo son los del servidor: se configuran en el escritorio (panel ✨ ▸ ⚙). Función Pro.

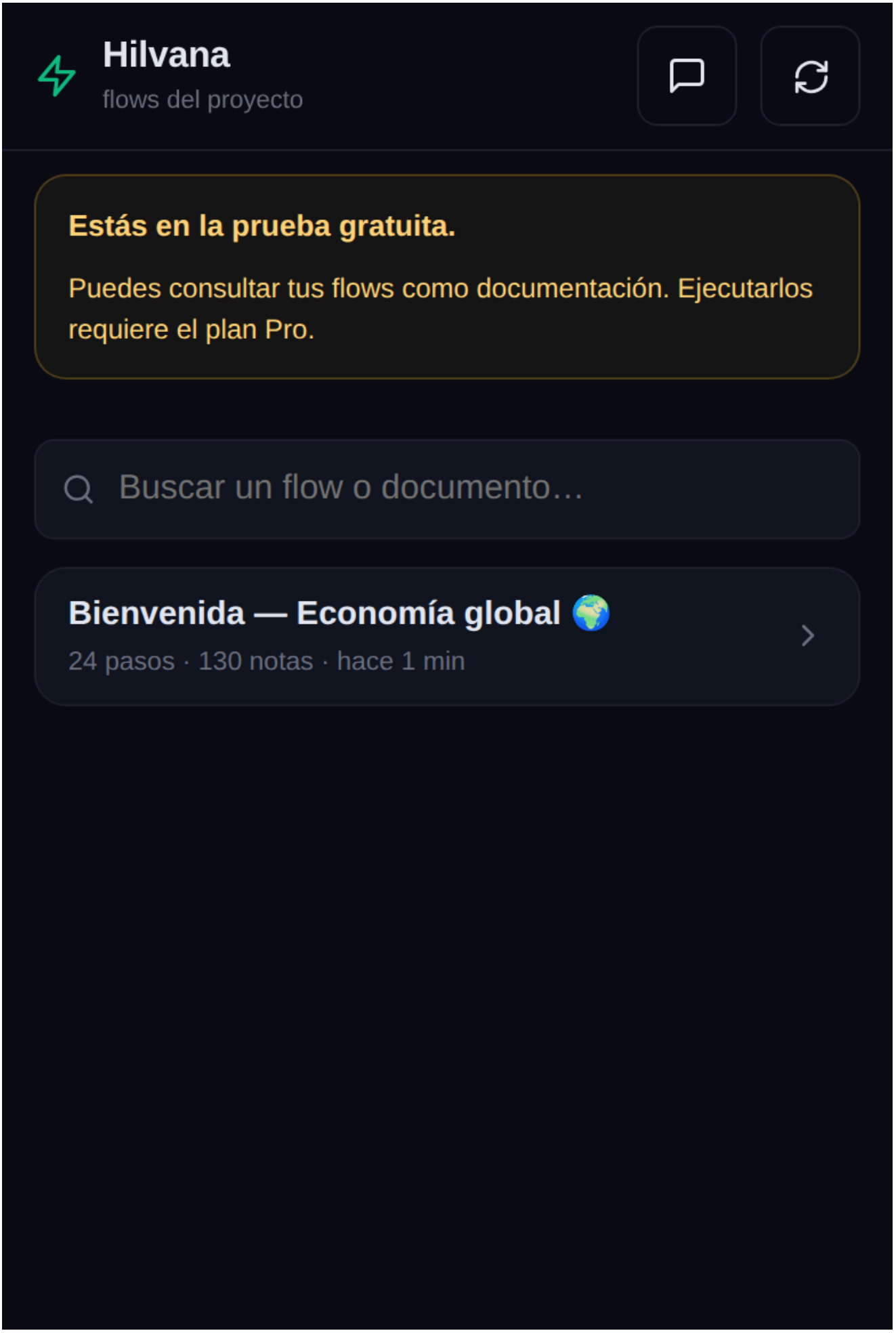
Reportar un problema desde el móvil 🆕 (5.14)

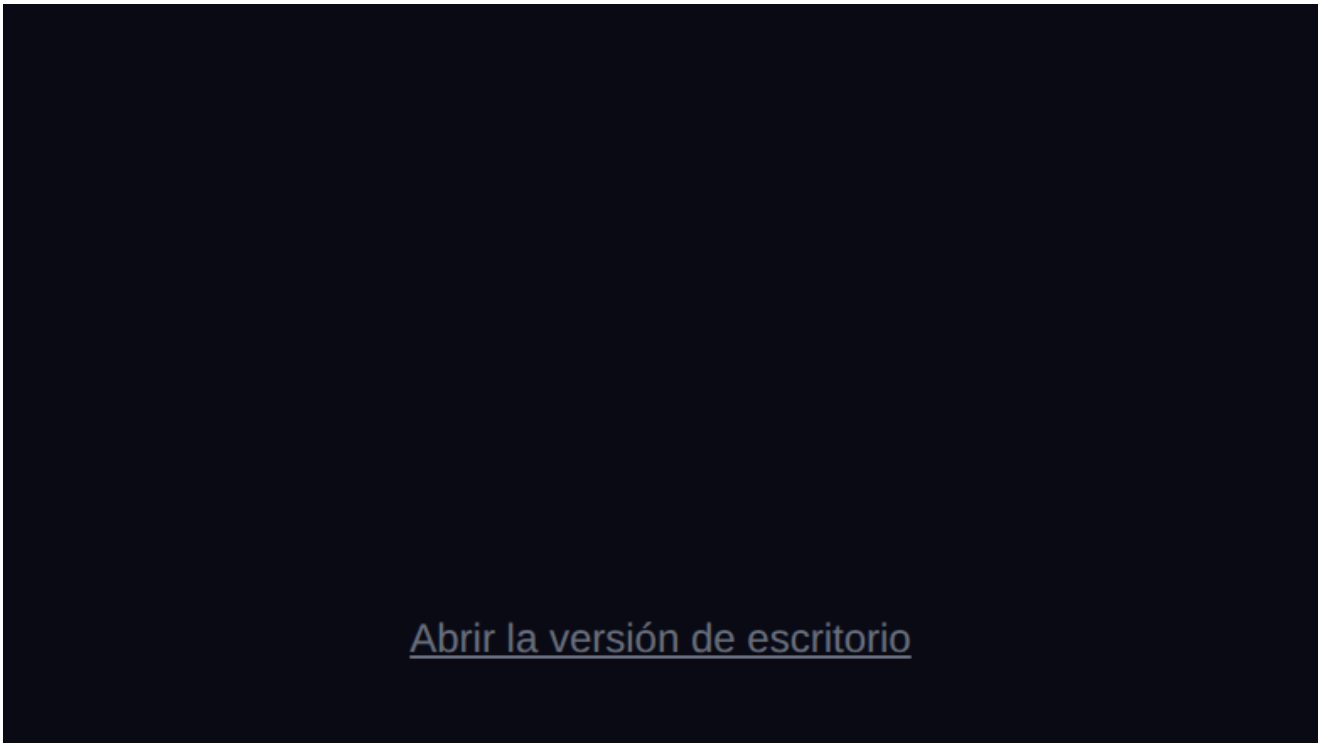
El botón 💬 de la barra superior (en la lista y dentro de un flow) abre el mismo formulario que en el escritorio: tipo (error / idea / pregunta), título, descripción, **foto de la cámara o de la galería** y datos técnicos opcionales (versión, teléfono, flow abierto). La pestaña «Mis reportes» muestra el estado y la respuesta del equipo. Es una función Pro/Business; en la prueba el formulario se ve pero no envía. Detalles en 14.


Reportar un problema
instalación vinculada · qa@test.local · v5.14.0
Nuevo
Mis reportes
Error
Idea
Pregunta
Título
Ejecutar un solo paso desde el móvil
Cuéntanos más
Estaría bien poder lanzar únicamente el nodo que estoy mirando en la pestaña Ejecución, como el ▶ de la caja en el escritorio.
Foto o captura
Adjuntar datos técnicos (versión, teléfono, flow abierto)

La lista, pantalla a pantalla 🆕 (5.38.1)

Lista de flows con buscador («Buscar un flow o documento…»), botón 💬 (chat) y 🔄 (recargar), y enlace «Abrir la versión de escritorio». En prueba gratuita avisa: «Puedes consultar tus flows como documentación. Ejecutarlos requiere el plan Pro.»


Hilvana
flows del proyecto
Estás en la prueba gratuita.
Puedes consultar tus flows como documentación. Ejecutarlos requiere el plan Pro.
Buscar un flow o documento…
Bienvenida — Economía global 🌍
24 pasos · 130 notas · hace 1 min

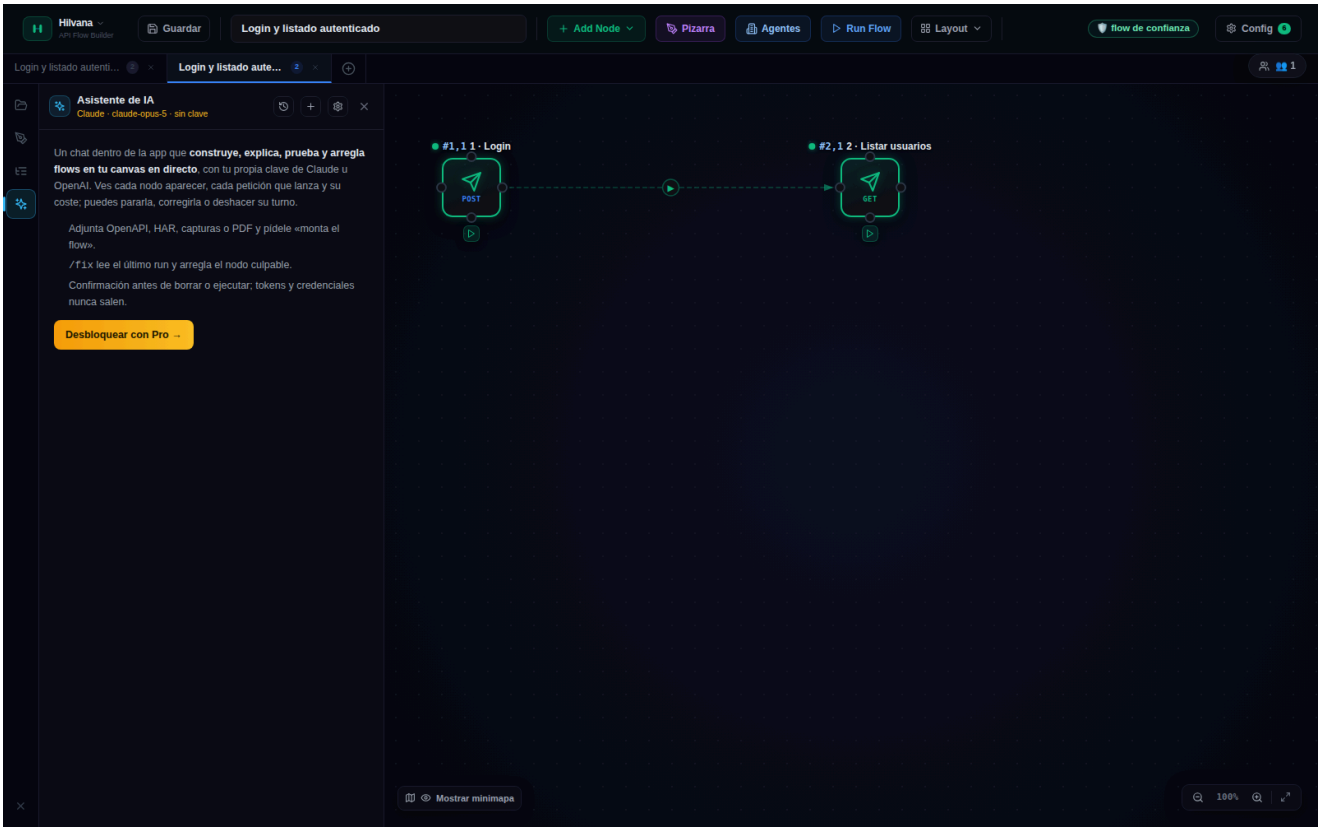


Nota sobre la 115 (GitHub Flows): en prueba gratuita la sección es Pro, así que la captura 115 muestra el aviso «GitHub Flows es del plan Pro» (La 114 es el panel Conexiones SQL: Importar/Exportar, perfil PostgreSQL/MySQL/Oracle, JDBC URL opcional, host/puerto, base, usuario/contraseña, «Añadir conexión».)

13 · El asistente de IA 🆕 (5.13)

> Revisado para Hilvana **6.2.1** · capturas: 104.

Un chat dentro de la app que **construye, explica, prueba y arregla flows en tu canvas en directo**, con tu propia clave de **Claude** (Anthropic) u **OpenAI**. No es un chat al lado del producto: cada cosa que la IA decide hacer pasa por las mismas herramientas que usa el MCP embebido, así que ves los nodos aparecer, las conexiones dibujarse y las respuestas llegar mientras habla. Tú puedes pararla, corregirla o deshacer su turno.

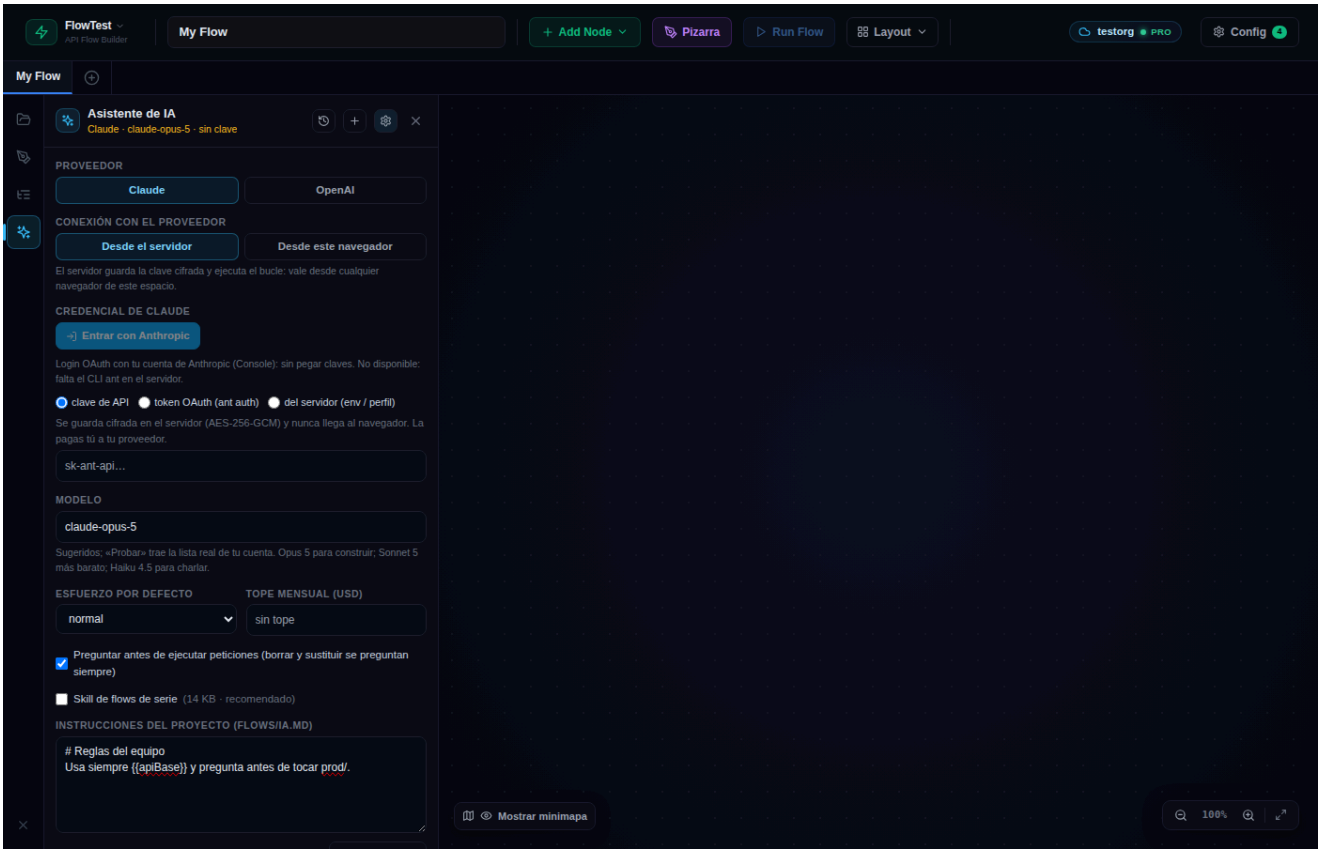


Se abre con el icono ✨ de la barra lateral izquierda. Es una función **Pro**; en la prueba de 14 días el panel se ve pero el chat pide pasar a Pro.

Qué sabe de serie: la skill de flows 🆕 (5.14)

El asistente lleva **incorporada la skill de flows** — el mismo método que usan Claude Code y cualquier cliente MCP (`skills/flows/SKILL.md` del repo público): cómo se construye un flow bien hecho (nodo a nodo, celdas y `canvas_layout`, asserts y extracciones, `{{apiBase}}` y credenciales `{{secret:X}}`, verificación con `flow_run` / `runs_read`) y, desde la 5.14, qué pueden hacer los scripts de nota, incluidos los que dibujan botones y páginas. Va en el prompt de cada turno para **todos los usuarios del asistente** (es una función Pro/Business); con la caché del proveedor cuesta muy poco. Se puede apagar en ⚙ Ajustes («Skill de flows de serie») si prefieres un prompt mínimo.

Instrucciones del proyecto desde Ajustes



Debajo del interruptor hay un editor de **`flows/IA.md`**: las convenciones de tu equipo (entornos, nombres, qué no tocar) que la IA lee en cada chat detrás de la skill. Hasta 30 KB; vaciarlo borra el fichero. En el cloud el fichero es de la organización (lo ven todos los miembros); en una instalación propia vive en la carpeta de flows.

Configurar: tu clave, tu proveedor

⚙ **Ajustes** dentro del panel:

- **Proveedor**: Claude u OpenAI. Cada uno con su clave; puedes tener las dos y cambiar cuando quieras (cada conversación se queda con el proveedor con el que empezó).
- **Clave de API**: se guarda **cifrada en el servidor** (AES-256-GCM, la misma clave maestra de las credenciales) y **nunca llega al navegador**. La pagas tú a tu proveedor: Hilvana no cobra por el uso de la IA. «Probar conexión» valida la clave y trae la lista real de modelos de tu cuenta.
- **«Entrar con Anthropic»** (sin claves): botón en los ajustes que abre la página de login de Anthropic en una pestaña nueva; entras con tu cuenta, la página te enseña un código, lo pegas en el panel y listo. Por debajo el servidor usa el CLI oficial `ant` de Anthropic (`ant auth login --no-browser`), que viene en la imagen Docker; la sesión (con refresco automático) se queda en el servidor, en un directorio propio del workspace o **privado del miembro** en el cloud, y antes de cada turno pide un token fresco. Es el login de la Console (API), facturado a esa organización; no sirve para gastar una suscripción de claude.ai. «Quitar» hace `logout` y borra la sesión.
- **Tipo de credencial** (Claude): además de la clave de API, se admite un **token OAuth** de `ant auth print-credentials --access-token` (empieza por `sk-ant-oat…`; va como Bearer con la

cabecera OAuth de Anthropic y caduca, así que cuando falle con 401 hay que pegar otro) o **«del servidor»**: no se guarda nada y el servidor usa sus propias credenciales (`ANTHROPIC_API_KEY`, `ANTHROPIC_AUTH_TOKEN`, el perfil de `ant auth login` del usuario que lo ejecuta; en Docker, pásalas con `-e`). OpenAI admite clave de API o «del servidor» (`OPENAI_API_KEY`). No existe un «Entrar con Claude / ChatGPT» que gaste la suscripción de consumo: los proveedores solo abren la API con clave.

- **Modelo**: sugeridos `claude-opus-5` (el mejor construyendo flows), `claude-sonnet-5` (más barato), `claude-haiku-4-5` (para charlar) y los `gpt-5` / `gpt-4.1` de OpenAI; o cualquiera de tu lista.
- **Esfuerzo** por defecto (rápido / normal / profundo / máximo): cuánto piensa el modelo antes de actuar. Se puede cambiar por mensaje desde el chat.
- **Preguntar antes de ejecutar**: pedir confirmación antes de `flow_run` / `node_run`. Borrar o sustituir se pregunta siempre.
- **Tope mensual** en dólares: al alcanzarlo el chat se para hasta que lo subas o cambie el mes.

En el **cloud**, un miembro puede usar la **clave de la org** (la comparte todo el equipo) o marcar **«solo mía»**: su clave privada manda sobre la compartida y solo la ve él.

Estado de la conexión 🆕 (5.27)

Arriba de los Ajustes, cada proveedor enseña **el mismo estado** — **Sin configurar**, **Conectado**, **Error** o **Caducado** —, el **modo** con el que conecta (clave de API, token OAuth, sesión de login, credenciales del servidor o desde el navegador) y un mensaje en español. El punto de color junto a «Claude» y «OpenAI» repite ese estado.

- **Probar conexión** (o **Reintentar** si hay error) prueba la credencial guardada o la que acabas de escribir y dice **qué falla en concreto**: no hay credencial, la clave no es válida, ha caducado, no hay red, no queda cuota o no tienes permiso.
- **Reconfigurar** vacía el campo y te lleva a escribir una credencial nueva.
- **Desconectar** borra la credencial de ese proveedor (solo de ese; pregunta antes) y lo deja «Sin configurar».

Ningún mensaje ni estado lleva la clave: se enmascara siempre.

Conexión desde el navegador

En ⚙ Ajustes ▸ **Conexión con el proveedor** puedes elegir **«Desde este navegador»**: la clave se guarda en el **localStorage de tu navegador**, no en el servidor, y las llamadas a Claude u OpenAI salen directamente de tu navegador (Anthropic lo permite con una cabecera específica; OpenAI también). El servidor nunca ve la clave y **no necesita salida a Internet**: solo prepara el contexto (system prompt, herramientas, foto del flow), ejecuta cada herramienta que pide la IA (validación, puente al canvas, redacción y auditoría siguen ahí), anota el uso y guarda la conversación.

Úsalo cuando el servidor no tenga Internet (red corporativa), cuando no quieras dejar tu clave en un servidor compartido, o para pasar por un proxy propio (campo «URL base»). A cambio, solo funciona en ese navegador (en otro equipo, o si borras los datos del sitio, hay que pegar la clave otra vez), no vale para el resto del equipo ni desde el móvil, y la clave queda tan protegida como el propio navegador. Todo

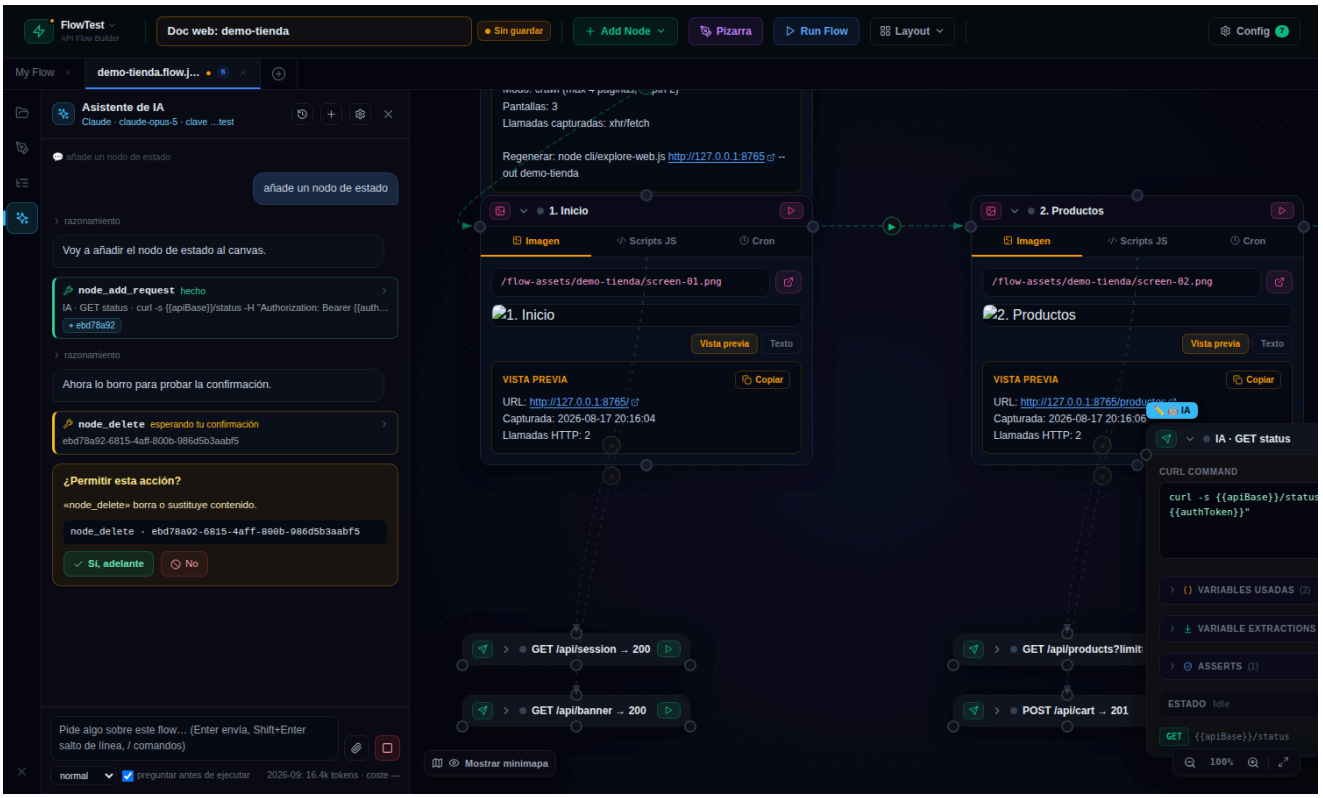
lo demás es igual: mismas herramientas, confirmaciones, parar, deshacer turno, coste y tope mensual, y la conversación aparece en el historial desde cualquier navegador.

Cómo trabaja

En cada mensaje la IA recibe una **foto compacta del flow** (nodos con nombre, método, URL, celda y estado; conexiones; nombres de variables) y la lista de **lo que has cambiado tú a mano desde su último paso**. Por eso puedes editar el canvas mientras trabaja: en el siguiente paso lo ve y no lo pisa.

Después actúa con las **mismas 39 herramientas del MCP** (`node_add_request`, `nodes_connect`, `variables_set`, `flow_run`, `canvas_layout`…). Todo se retransmite en streaming al panel:

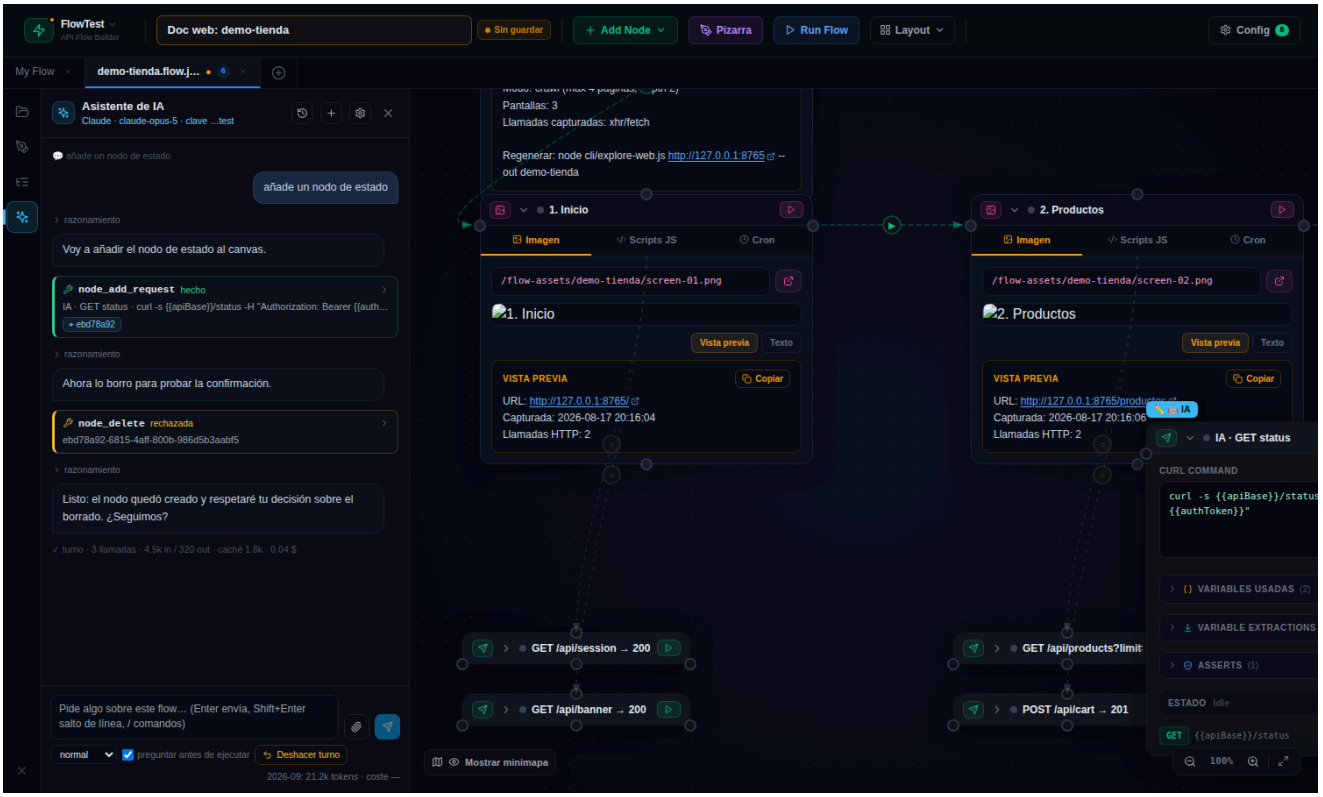
- el **razonamiento resumido** (plegado, «razonamiento») y el texto a medida que lo escribe;
- cada **tool** como una tarjeta: nombre, el input **escribiéndose en directo**, estado (escribiendo → ejecutando → hecho/error) y el resultado plegable; los ids de los nodos tocados salen como chips: clic para centrar el canvas en ese nodo;
- sobre el nodo, una marca **«🤖 IA»** unos segundos, igual que las marcas de colaboración;
- al final del turno, una línea con **llamadas, tokens (entrada, salida, caché) y coste** del turno.



Confirmaciones. Antes de borrar o sustituir (`node_delete`, `connection_delete`, `flow_overwrite`, cerrar pestaña, `flow_reset`, limpiar la pizarra, borrar correos…) aparece una tarjeta ámbar con la acción exacta y dos botones. Con «Preguntar antes de ejecutar» activo, lo mismo antes de lanzar peticiones o SQL de verdad. Si dices que no, la IA lo sabe («el usuario ha rechazado esta acción») y te pregunta qué prefieres. Si no contestas en 5 minutos o cierras el panel, cuenta como no.

Parar (■) corta la respuesta en cuanto termina la tool en curso; lo ya hecho en el canvas se queda. **Deshacer turno** restaura el canvas a como estaba antes del último mensaje (entra en la pila de Ctrl+Z,

así que Ctrl+Y lo rehace).



Comandos rápidos

Escribe `/` en el chat:

Comando	Qué hace
`/explain`	Explica el flow paso a paso sin tocar nada
`/docs`	Añade al canvas una nota Mermaid con el esquema y una nota con objetivo, variables y cómo ejecutarlo
`/test`	Convierte el flow en un test: asserts por request y extracciones para encadenar
`/fix`	Lee el último run y la consola, explica el fallo, corrige el nodo culpable y, con tu permiso, lo vuelve a ejecutar
`/run`	Ejecuta el flow y cuenta el resultado real nodo a nodo
`/clean`	Ordena el canvas por celdas y separa solapes

Puedes añadir detalle detrás: `/fix el login devuelve 401`.

Adjuntos y contexto

Arrastra ficheros al panel, usa 📎 o **pega una captura con Ctrl+V** directamente en el chat: **OpenAPI/Swagger** (`.json`, `.yaml`), **HAR**, `.md`, `.txt`, `.csv`, `.curl`, **PDF** (Claude lo lee nativo, con citas) e **imágenes** PNG/JPG/GIF/WebP (una captura de Postman, un error en pantalla, un diagrama). Las imágenes llegan al modelo como imagen de verdad, tanto en Claude como en OpenAI; el texto y el

YAML, como documentos. Hasta 20 MB por mensaje. Pídele «monta el flow de esta spec» o «este HAR es el login, reprodúcelo».

Crea `flows/IA.md` con las normas de tu equipo (entornos, nombres, qué no tocar, cómo se autentica la API) y la IA lo lee en todas las conversaciones.

Seguridad y coste

- A la IA **nunca le llegan credenciales en claro**: las cabeceras `Authorization` / `Cookie` / `x-api-key`, los `Bearer …` y las claves con pinta de `sk-…` de los resultados se sustituyen por «oculto» antes de enviarse; los `{{secret:X}}` y `{{variables}}` viajan como placeholders. Cuando necesite un token te pedirá que lo crees en Variables ▸ Credenciales.
- Las conversaciones, los adjuntos, el uso (`usage.jsonl`) y la auditoría (`audit.jsonl`: quién ejecutó qué tool) viven en `flows/.ai/` del servidor. En el cloud, **Mi cuenta ▸ 🤖 Asistente de IA** muestra tu consumo del mes y el del equipo, y el administrador ve por org el uso por persona y modelo y las últimas acciones; ni las claves ni los chats salen del contenedor.
- Cada llamada anota tokens y coste (precios de Anthropic por modelo; OpenAI muestra tokens y «—» en coste). El tope mensual corta el chat, no el resto de la app.

En el móvil 🆕 (5.13.1)

La vista móvil (`/m`) tiene la pestaña ✨ **IA** en cada flow: mismo asistente, con un resumen del flow como contexto. Sin escritorio abierto solo lee y propone; con el escritorio abierto edita ese canvas. Ver 12.

Límites y consejos

- Máximo 40 pasos (llamadas) por mensaje; si se queda corto, dile «continúa».
- Con la app cerrada no hay canvas: el chat funciona pero las tools fallan con aviso. Ten la pestaña abierta.
- Un flow grande consume más tokens por mensaje (la foto del flow va en cada uno); la caché del proveedor abarata las repeticiones.
- El asistente usa el mismo puente que el MCP: si tienes Claude Code conectado al `/mcp` a la vez, los dos mandan sobre la misma pestaña.

14 · Reportar un problema 🆕 (5.14)

> Revisado para Hilvana **6.2.1** · capturas: 118.

Desde la app puedes mandar al equipo de Hilvana un **error, una idea o una pregunta**, con captura y datos técnicos, sin salir a ningún formulario externo. Es una función de los planes **Pro y Business**: en la prueba gratuita la entrada se ve con el chip `PRO` y explica cómo pasar a Pro.

Dónde está

- **Escritorio**: botón **Config** (arriba a la derecha) ▸ **Reportar un problema**.
- **Móvil** (`/m` y la app Android): el botón 💬 de la barra superior, tanto en la lista como dentro de un flow.

Qué se envía

1. **Tipo**: 🐞 Error · 💡 Idea · ❓ Pregunta.
2. **Título** y **descripción** (para un error: qué ha pasado, qué esperabas y cómo reproducirlo).
3. **Captura** (opcional): «Capturar el canvas» hace una foto del flow visible tal cual lo ves (el mismo motor que Exportar PDF); «Adjuntar imagen» o **Ctrl+V** pega cualquier imagen; en el móvil, foto de la cámara o de la galería. Se reduce a 1800 px para que pese poco.
4. **Datos técnicos** (opcional, marcado por defecto): versión de la app, navegador, tamaño de pantalla, flow abierto (nombre, fichero, nº de nodos) y las últimas 60 líneas de la Consola. «Ver qué se envía» lo muestra antes de enviar.

> **Privacidad.** El servidor tapa cabeceras `Authorization`, tokens tipo `sk-…`, contraseñas y `{{secret:X}}` antes de que salga nada; las credenciales del servidor nunca viajan. El navegador no habla con el servidor de cuentas: el reporte sale de tu instalación con su propia credencial.

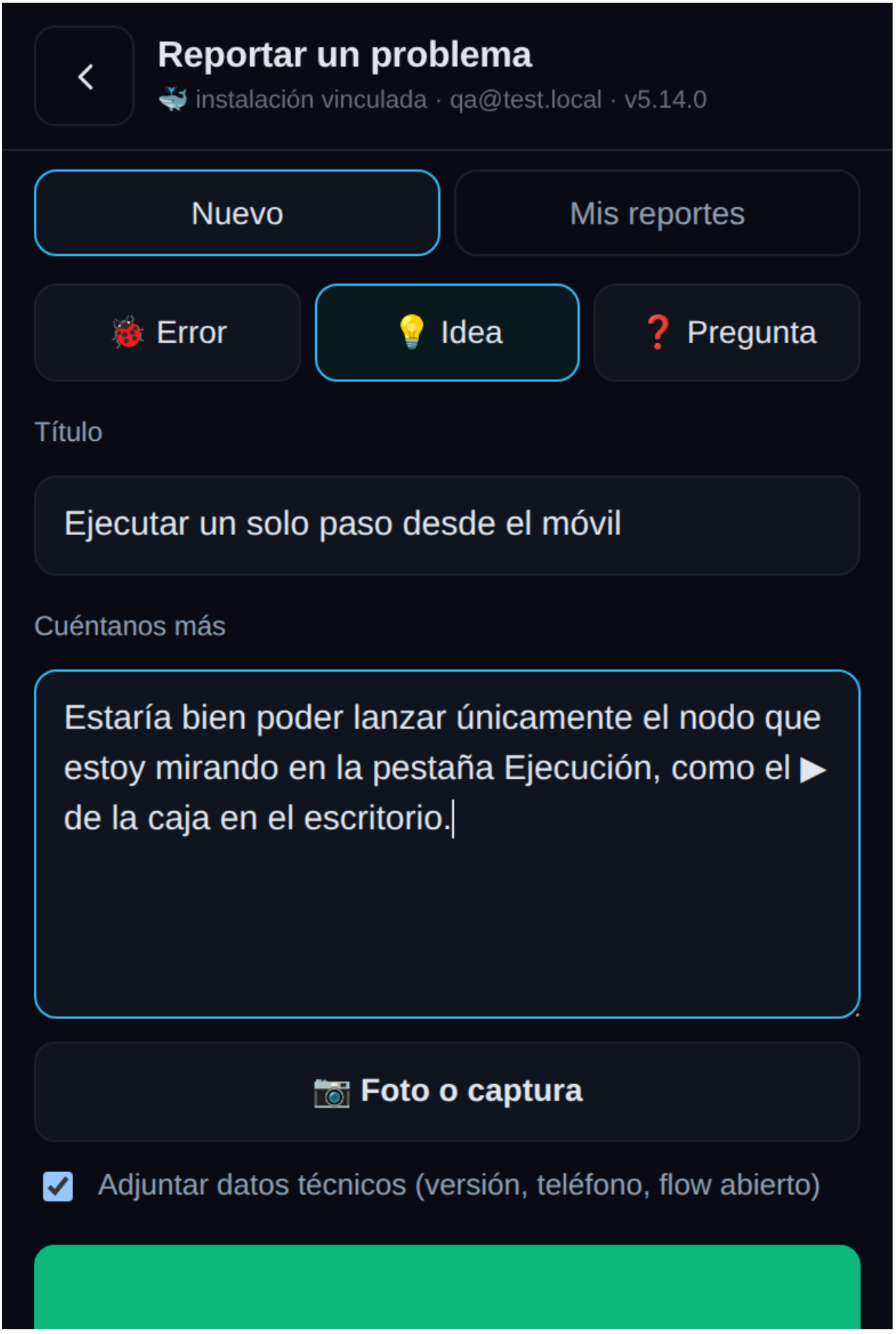
Cómo llega

Instalación	Vía
Cloud (`tuorg.app.hilvana.cloud`)	El contenedor de tu organización lo entrega con tu usuario.
Docker vinculado a tu cuenta cloud	Con el token de vinculación (el reporte lleva el dueño del token).
Docker con licencia	Con la licencia RS256 (el titular).
Sin identificar (prueba local)	No se puede enviar: vincula o activa una licencia en Config ▸ Licencia.

Si el servidor de cuentas no responde (sin red, mantenimiento), el reporte se guarda en la **bandeja de salida** de tu instalación y se reintenta solo cada 5 minutos; en «Mis reportes» aparece como *pendiente de envío* con el botón «Reintentar ahora».

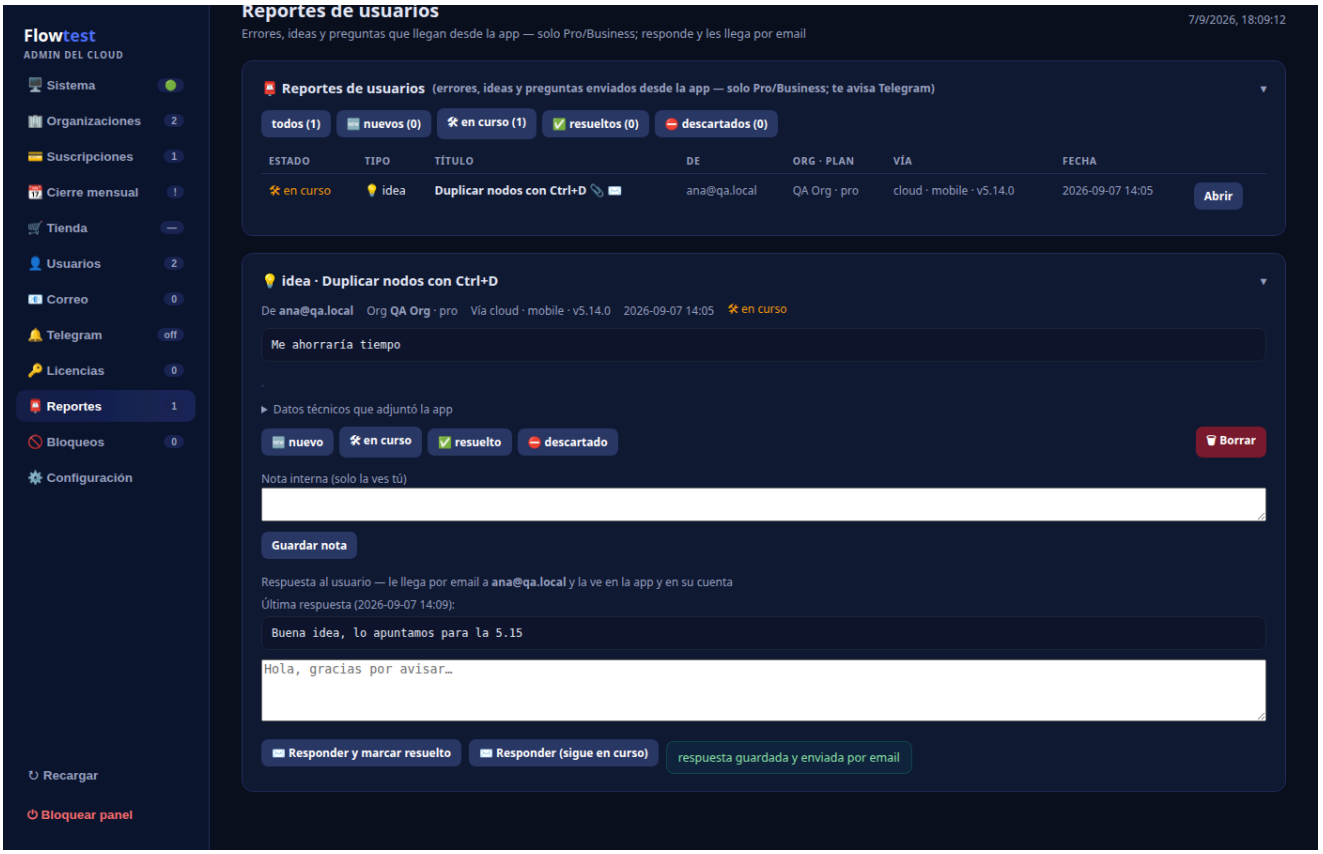
Mis reportes y la respuesta

La pestaña **«Mis reportes»** del mismo modal (y de la pantalla móvil) lista lo que has enviado desde esa instalación con su estado — *recibido · en curso · resuelto · descartado* — y la **respuesta del equipo** cuando la hay. La respuesta te llega también **por email** y la ves en tu cuenta de hilvana.cloud (**account.html ▸ 📮 Mis reportes**), junto a la captura que adjuntaste.


Reportar un problema
instalación vinculada · qa@test.local · v5.14.0
Nuevo
Mis reportes
Error
Idea
Pregunta
Título
Ejecutar un solo paso desde el móvil
Cuéntanos más
Estaría bien poder lanzar únicamente el nodo que estoy mirando en la pestaña Ejecución, como el ▶ de la caja en el escritorio.
Foto o captura
Adjuntar datos técnicos (versión, teléfono, flow abierto)

Para el administrador (hilvana-accounts)

En el admin aparece la sección 📮 **Reportes** con un contador de pendientes: filtros por estado, detalle con la captura, los datos técnicos y la consola, nota interna, cambio de estado y **responder** (le llega por email al usuario y aparece en su app). Cada reporte nuevo avisa por Telegram (categoría `feedback`). Las capturas viven en `data/feedback/`.



Preguntas rápidas

- **¿Cuánto puede pesar?** La imagen hasta 6 MB (ya reducida); la descripción hasta 20 000 caracteres.
- **¿Lo ven los demás miembros de mi organización?** No: en la app cada uno ve los suyos (en el cloud, por usuario); el administrador de Hilvana los ve todos.
- **¿Puedo borrar uno?** Escríbelo en un reporte nuevo o responde al email; el equipo lo retira.